

Symbolic Algorithms for the Analysis of Quantitative Systems

Proefschrift

ter verkrijging van
de graad van doctor aan de Universiteit Leiden,
op gezag van rector magnificus prof.dr. S. de Rijcke,
volgens besluit van het college voor promoties
te verdedigen op dinsdag 13 oktober 2026
klokke 10:00 uur

door
Sebastiaan Olivier Brand
geboren te Oegstgeest
in 1994

Promotores: Dr. A.W. Laarman
Prof.dr. T.H.W. Bäck

Promotiecommissie: Prof.dr. M.M. Bonsangue
Prof.dr. V. Dunjko
Dr. A.V. Kononova
Dr. J. Helsen (Centrum Wiskunde & Informatica)
Prof.dr. J.-M. Lagniez (Université d'Artois)

Copyright © 2026 Sebastiaan Brand.



IPA dissertation series 2026-14

ISBN 978-94-6537-820-6 (paperback)

Printing: Ridderprint | www.ridderprint.nl

The work in the thesis has been carried out under the auspices of the research school IPA (Institute for Programming research and Algorithmics).

The work in this thesis was supported by the NEASQC project, funded by the European Union's Horizon 2020, Grant Agreement No. 951821.

Cover design: Artistic interpretation of decision diagrams as bouldering routes. In bouldering, a route is also called a *problem*, and successfully climbing a route is called *solving* a problem. Illustration by Charlotte Brand.

Contents

Acknowledgements	vii
1 Introduction	1
1.1 Formal system analysis	1
1.2 Challenges and research questions	5
1.3 Contributions	7
1.4 Thesis outline	9
2 Preliminaries	11
2.1 (Non)deterministic and quantitative systems	11
2.2 Decision diagrams	16
2.3 Encoding transition systems in decision diagrams	19
3 A decision diagram operation for reachability	21
3.1 Introduction	22
3.2 Preliminaries	23
3.3 Decision diagram operation for reachability	30
3.4 Empirical evaluation	35
3.5 Conclusion	40
3.6 Appendix	40
4 Q-Sylvan: a parallel DD package for quantum computing	43
4.1 Introduction	44
4.2 Quantum computing and EVDDs	45
4.3 EVDD implementation	46
4.4 Use cases	50
4.5 Empirical evaluation	53

4.6	Appendix	57
5	Numerical errors in quantitative system analysis with DDs	67
5.1	Introduction	68
5.2	Preliminaries	70
5.3	Error analysis	75
5.4	Case study	80
5.5	Conclusion	85
5.6	Appendix	86
6	Lossy encoding of continuous Bayesian networks in decision diagrams	91
6.1	Introduction	92
6.2	Preliminaries	93
6.3	Methodology	95
6.4	Empirical evaluation	100
6.5	Conclusion	104
6.6	Appendix	105
7	Quantum graph-state synthesis with SAT	109
7.1	Introduction	110
7.2	Preliminaries and problem definition	112
7.3	SAT encoding	116
7.4	Empirical evaluation	119
7.5	Appendix	123
8	A divide-and-quantum algorithm for SAT	127
8.1	Introduction	128
8.2	Preliminaries	129
8.3	Divide-and-quantum for PPSZ	135
8.4	Space-efficient quantum subroutines for PPSZ	145
8.5	Conclusion	158
9	Conclusions	161
9.1	Contributions	161
9.2	Future work	164
	Bibliography	169

Summary	191
Samenvatting	193
List of publications	195
Curriculum Vitae	197

Acknowledgements

First, I want to thank my supervisor, **Alfons**, for guiding me and for always being available for in-depth technical discussions, even as the number of PhD students in our group grew. I also want to thank my promotor, **Thomas**. Even though our research fields are somewhat different, we ended up finding three papers' worth of overlap.

I also wish to thank **Tim**. While we had some interesting research collaborations during your time in Leiden, I am perhaps most grateful for your supervision and collaboration during my master's project at QuTech. The great work environment you and many others at QuTech helped create played an important role in my decision to pursue more research, which ended up taking the form of this PhD.

While I sometimes joked that doing a PhD doesn't leave a lot of room for hobbies (which might be true), bouldering has been a constant for me throughout a large part of my academic journey. Thank you, **Dalia**, **Bernhard**, and **Steef**, for your wonderful company, and your help with solving bouldering problems. I hope I'll have the opportunity to share the great and sophisticated bouldering advice "just go up" with you again sometime in the future.

I am also grateful to the people of the Theory group. Research is never done in isolation, and it was a great pleasure to be part of this group. Thank you, **Lieuwe**, **Dimitrios**, **Arend-Jan**, **Alejandro**, **Tanjona**, **Berend**, **Jingyi**, **Luc**, **Dalia**, **Steef**, **Jan**, **Muhammad**, **Joon**, **Liam**, **Marc**, and others. While I valued our technical discussions, brainstorming sessions, and research collaborations, I am maybe most appreciative of our simple lunch and coffee breaks together (where a little bit too often, research was still the topic of conversation ;)). I also enjoyed our (somewhat sporadic) social outings, and I would urge you to organize these more frequently.

Thank you, **Richard**, for your work on Q-Sylvan. Floating-point values in decision diagrams have been a bit of a headache for me since the beginning of my PhD, and although many open questions remain, I am glad we managed to shed light on some

of them.

Thank you, **Maarten**, for our collaboration on the BN-DD project. I think this project ran for much longer and into many more hurdles than either of us expected. However, in the end, we managed to get not one but two papers out of this project, in large parts thanks to your efforts. And of course, thank you **Anna**, **Jacob**, and **Diederick** for your work on this project as well.

I want to thank my sister for the cover design of this thesis, and for a small amount of sibling rivalry. Trying to one-up you in terms of academic achievements might have contributed slightly to my decision to pursue a PhD. And finally, I want to thank my parents for being incredibly supportive, not just during this PhD, but throughout my entire academic journey.

Chapter 1

Introduction

1.1 Formal system analysis

In today’s increasingly automated and interconnected world, many critical parts of our infrastructure, such as power plants [210, 215], train networks [21, 90], and water management systems [229, 99], are governed by computer-controlled systems. These systems are responsible for ensuring the safe and efficient operation of services that modern society heavily depends on. A malfunction in any of these systems, such as a train ignoring a signal or a flood barrier failing to close in time, can have severe consequences for human life, the environment, or the economy. As these systems grow in complexity and scale, ensuring their correctness, reliability, and resilience becomes increasingly challenging.

Traditional testing methods, while useful, can fall short when it comes to uncovering all potential system failures, as they do not exhaustively verify a system’s behavior. In contrast, formal system analysis offers a rigorous framework for specifying, modeling, and verifying the behavior of these systems, and cannot only be used to detect issues, but can also provide guarantees on the safety and reliability of systems.

The description of a system does not need to be limited to the behavior it can or cannot exhibit, but can also include additional information, such as the probability with which an event occurs, or how much time something takes. We call such systems *quantitative* systems because they can be used to reason not just about whether a system works (qualitative), but about how well it works (quantitative). Quantitative systems are useful to study, as many real-world systems contain quantitative components.

Transition systems. Formal system analysis requires systems to be described in a formal (i.e., mathematically precise) way. A broad class of such formal descriptions are *transition systems*. Many systems, both those that arise from computer programs as well as systems that model real-world behavior, can be described by transition systems. In their most general sense, transition systems are sets of states and sets of rules, so-called transition relations, that describe how one is allowed to transition between states. In this thesis, we focus on transition systems that have discrete states, such as the state of a bit being 0 or 1, or the state of a coin being heads or tails, rather than continuous states, such as the position of a particle.

A prominent example of transition systems that have a large impact in the real world are computer programs. For such transition systems, the states are given by the values held by the variables in the program, in combination with a counter that keeps track of where we are in the program execution (the program counter), and the transition relation is defined by the program code itself.

For any given transition system, we can ask (and try to answer) questions about its properties or its behavior. In the analysis of programs, especially those in safety-critical domains, we want to verify that the program functions according to its specifications and that bad behavior does not occur, which is called model checking. For example, in a setting where multiple programs are running simultaneously, we want to rule out the possibility of deadlocks, i.e. situations where two programs are stuck, both waiting to receive a message from the other one. Or when compiling a program to run on specific hardware, it is useful to check if the compiled version is equivalent to the source code.

Such queries about the behavior of systems are often computationally hard to answer because the size of the state space can grow exponentially with the system's description; for example, the rules of chess, including the initial configuration of its 32 pieces arranged over 64 squares, can be easily understood and remembered by a human, and yet this simple description gives rise to an estimated 10^{44} configurations of a chessboard. This is sometimes referred to as the *state-space explosion* problem. We will shortly discuss how so-called symbolic algorithms can help tackle this problem, but first we introduce quantitative transition systems.

Quantitative transition systems. When modeling systems, it can often be useful to not just capture whether a particular transition exists, but also to label transitions with quantitative values, such as time, costs or rewards, or the probability with which a transition occurs. In this thesis, we specifically focus on transitions that are labeled with probabilities and the probability-like values that occur in quantum

Table 1.1: What we mean with *(non)deterministic* and *quantitative* systems.

Name in this thesis	State space	Transitions
(Non)deterministic system	discrete, finite	Boolean (unlabeled)
Quantitative system	discrete, finite	probabilistic/quantum

systems. We use the term “quantitative systems” as an overarching term to refer to both probabilistic and quantum systems, although there are also types of quantitative systems that are not discussed in this thesis (such as timed systems). Conversely, we will refer to systems that do not have quantitative labels on their transitions as *(non)deterministic* transition systems. For both of these types of systems, we consider the number of states to be discrete and finite. To emphasize these terms, we restate them in Table 1.1.

Transition systems that are labeled with probabilities are useful for modeling and analyzing the behavior of real-world systems that have a degree of randomness to them. For example, in the safety analysis of power plants, the probability of a critical failure might be computed from the probabilities that individual components and safeguards fail. Probabilistic transition systems are also used in artificial intelligence. For example, Markov decision processes [180], which are transition systems that combine both probabilistic and nondeterministic transitions, are frequently used in reinforcement learning [220].

Quantum mechanical systems can often be described in ways that are very similar to probabilistic systems, such as generalizations of classical transition systems [183] and quantum versions of Markov chains [237]. For a large part, the interest in the analysis of quantum systems in computer science is motivated by the field of quantum computing. In quantum computing, quantum mechanics is leveraged to perform computations in a way that is theoretically much faster than regular computers can. Similar to how we wish to analyze the correctness or reliability of deterministic and probabilistic programs, we want to analyze the behavior of quantum programs as well.

The analysis of quantitative systems tends to be equally hard or often harder than that of (non)deterministic systems. Quantitative systems still suffer from the state space explosion problem that (non)deterministic systems suffer from, but additionally require keeping track of values such as probabilities or quantum amplitudes.

Symbolic algorithms. As mentioned, the analysis of both (non)deterministic and quantitative transition systems is often computationally hard because even a small

description of a system can give rise to an exponential number of states. Symbolic algorithms and data structures aim to tackle this problem by encoding the systems and their inputs “symbolically”.

Let us give a concrete example: we are given a logic circuit that takes as input bit strings $\vec{x}, \vec{y} \in \{0, 1\}^n$ and claims to output a bit string $\vec{z}$ that equals $\vec{x} + \vec{y}$ (modulo 2^n). As a step towards proving that the circuit indeed implements addition correctly, we are tasked to prove that when $\vec{x}$ and $\vec{y}$ represent odd numbers, the output $\vec{z}$ represents an even number. Instead of verifying this property for all 2^{2n} possible inputs—which would be $\sim 10^{38}$ inputs for two 64-bit numbers—a symbolic approach to this problem might be this: let $*^{n-1}0$ represent all bit strings of length n that ends in a 0 (even numbers), and $*^{n-1}1$ all bit strings of length n that ends in a 1 (odd numbers). If the circuit correctly implements addition, we should be able to show that $*^{n-1}1 + *^{n-1}1 = *^{n-1}0 \pmod{2^n}$.

Many problems regarding the analysis of transition systems are computationally hard, meaning that any algorithm that tackles such a problem cannot always be efficient. Symbolic algorithms often achieve their efficiency through *heuristics*, which are design choices in the algorithms that enable them to solve many practical instances efficiently, without providing guarantees of solving the problem efficiently for all instances. However, in practice we see that symbolic algorithms can quickly solve many real-world problem instances.

In this thesis we focus on two particular symbolic methods: decision diagrams (DDs) [37] and satisfiability (SAT) [31]. Decision diagrams are a data structure that can represent mathematical objects such as functions or sets concisely. DDs are especially useful because they can not only represent these objects in a compressed way, but also allow for the manipulation of the underlying objects without decompressing them. Similar to homomorphic encryption [4], this type of compression can be referred to as “homomorphic compression”. DDs were first created to deal with Boolean problems [37], but numerous types of DDs have since been created that can deal with quantitative (pseudo-Boolean) problems as well [91, 92, 222, 231, 201, 195, 160, 230, 236, 232, 214]. In SAT-based methods [31], a problem is expressed in logic formulas, which are then given to an algorithm that specializes in finding assignments to the variables of these formulas such that the formulas evaluate to true. Given a formula, a SAT-solving algorithm uses heuristics to efficiently search through the space of all possible solutions. While in some cases SAT-solvers cannot do better than iterating over exponentially many possible solutions, in many practical cases they are able to efficiently recognize parts of the solution space that contain no valid solutions. In doing

so, SAT-solvers can find solutions in search spaces with sizes well over 10^{3000} [114].

1.2 Challenges and research questions

While symbolic algorithms have been successful in the analysis of (non)deterministic and quantitative systems, there are still open challenges. We discuss several of these challenges, and also formulate the research questions that are tackled in this thesis.

Scaling of symbolic algorithms. There are limits to how far symbolic algorithms can be scaled up when applied to increasingly large problems. For both DD-based and SAT-based methods, we discuss challenges towards which improvements may be made.

The first challenge regarding the scalability of DD-based methods is that they are often bottlenecked by memory constraints. When given a compact description of a transition system, e.g. the source code of a computer program, and the task of analyzing which states are reachable and which are not, DDs can be used to represent both the transition rules as well as sets of states of this system. While the set of reachable states of a system is often exponentially large, DDs can, in many cases, represent these sets efficiently. When exploring the state space of a transition system, the DD that represents the found states starts small, and increases in size as more states are found. However, in many practical applications, the DD representation eventually decreases again as all reachable states are found. The bottleneck here is not the representation of all reachable states, but the representation of some intermediate set of states. This problem has been successfully tackled by the “saturation” algorithm [48] for cases where the transition relation of a system can be partitioned into multiple smaller relations, but this algorithm is not applicable to monolithic transition systems.

The second scalability challenge for DDs is that DD computations are difficult to parallelize. And while efficient parallel implementations of various types of DDs have been created [81], attempts to parallelize edge-valued decision diagrams [222], which have been popular in the analysis of quantum systems [242], have yielded limited success [120, 148]. In recent years, the improvements in computing hardware that allow sequential computations to be performed faster have slowed down. This makes it increasingly important to rely on parallel computation wherever possible.

For SAT-based approaches, the first scalability challenge is how to encode problems efficiently. While SAT is very generally applicable—every NP-problem can, in principle, be encoded in SAT in polynomial time and space—not every way of encoding a particular problem in SAT yields the same practical performance. There are many dif-

ferent techniques for improving the efficiency of SAT encodings, although what works well in practice can often depend on domain-specific properties of the problem that is being encoded [32].

The second challenge for scaling SAT-based methods is the efficiency of the SAT solvers themselves. Over the past few decades, great strides have been made in the efficiency of SAT solvers and the heuristics they use [30]. With the advent of quantum computing, it is natural to ask to what extent speedups from quantum computing can be combined with state-of-the-art SAT-solving algorithms.

We summarize improving the efficiency of both DD and SAT-based approaches in the following research question.

Research question 1. *How can we scale up symbolic algorithms for the analysis of both (non)deterministic and quantitative systems?*

Handling floating-point values. The second challenge is handling floating-point values in decision diagrams. When describing quantitative systems, it is often difficult to avoid the use of floating-point numbers. However, floating-point computations are notoriously inexact. For example, the computation $0.1 + 0.2$ can yield 0.30000000000000004 instead of 0.3 . While it is important to consider the effects of these floating-point errors on the correctness of the computation [107], in DDs, floating-point errors are the source of another problem as well: DDs achieve their compactness by recognizing redundancies or equivalences. However, floating-point imprecision can prevent equivalences from being recognized, making the DD bigger than it should be if all computations were exact [241]. This leads us to pose our second research question.

Research question 2. *How do we handle floating-point values in decision diagrams?*

Discretization. For most of this thesis, we consider systems that have discrete states, either with Boolean or quantitative transitions. These systems can be described by Boolean or pseudo-Boolean functions, which allows them to be encoded in decision diagrams or SAT formulas. However, in various domains, such as probability theory, components of states can be continuous. For example, a probability distribution over n continuous variables is given by a function of the form $\mathbb{R}^n \rightarrow \mathbb{R}$. When analyzing such distributions, we either need methods that can natively handle continuous domains, or we need to discretize the distributions. The benefit of the latter is that, if it can be done without great loss of precision or a large computational overhead, we can

reuse existing methods that work well for discrete systems. While more granular discretizations often yield more accurate results, they also come with an increase in computational and space complexity. Here, the compact representations offered by DDs can be valuable in allowing for high-quality discretizations, while also limiting the computational and space requirements. This leads us to our final research question.

Research question 3. *How can decision diagrams aid in the efficient discretization and analysis of continuous domains?*

1.3 Contributions

Towards Research question 1. The chapters that contribute towards answering Research question 1 are Chapters 3, 4, 7 and 8.

In Chapter 3, we address a major challenge in reachability analysis with decision diagrams. As discussed in the previous section, when using DDs to explore the state space of a transition system, the intermediate DDs can grow very large, even when the DD that represents all reachable states is small. In this chapter, we present a new DD algorithm for reachability that aims to avoid these large intermediate DDs, similar to the saturation algorithm [48]. Saturation has been considered the state-of-the-art for DD-based reachability since its creation over two decades ago, however, it is only applicable to systems where the transition relation can be partitioned into multiple smaller relations. On the other hand, our new algorithm, REACHDD, can act on a single monolithic transition relation. We create versions of REACHDD for two types of DDs: binary decision diagrams (BDDs) and multi-valued decision diagrams (MDDs), and parallelize these as well. In our empirical evaluation against saturation on a large set of benchmarks, we find that REACHDD often outperforms saturation, especially on transition relations with low locality.

In Chapter 4, we provide an efficient parallel implementation of edge-valued decision diagrams (EVDDs), a type of DD that has shown a lot of success in the simulation and analysis of quantum algorithms. As discussed, efficient parallelization of DDs is challenging, and while parallel speedups have been obtained for DDs used in classical applications, attempts to parallelize operations for EVDDs have yielded only limited success. In this chapter, we provide a parallel implementation of EVDDs whose parallel performance significantly improves on previous results. We also implement two use cases for these EVDDs: simulation and equivalence checking of quantum circuits, and provide a comprehensive empirical evaluation.

In Chapter 7, we tackle a particular problem in quantum information by encoding it in SAT. The problem concerns deciding whether particular quantum states called “graph states” can be transformed into each other using only local operations, and providing the particular sequence of transformations if it exists. In our empirical evaluation, we find that by using domain-specific knowledge to over-constrain the SAT formulas, the SAT-solving time can be improved by several orders of magnitude.

In Chapter 8, we combine the best-known symptomatic algorithm for SAT—PPSZ [178]—with Grover’s quantum search algorithm [103] in both a pure quantum algorithm, as well as a hybrid quantum-classical approach. This approach combines Grover’s $O(\sqrt{2^n})$ quantum speedup with PPSZ’s $\sim O(2^{0.386n})$ classical complexity. In this chapter, we also provide detailed specifications of space-efficient quantum subroutines that are required to achieve speedups over classical PPSZ in the hybrid quantum-classical setting.

Towards Research question 2. The main contribution towards Research question 2 is given in Chapter 5, where we provide a detailed analysis of the errors caused by floating-point imprecision in multi-terminal binary decision diagrams (MTBDDs). In this chapter we specifically look at the matrix-vector multiplication algorithm because it is the function that computes successor states for both probabilistic and quantum systems, and thus a key function for analyzing such systems. We prove that numerical errors in the MTBDD matrix-vector multiplication algorithm can be kept small under certain conditions, and discuss how an MTBDD implementation can meet these conditions. Additionally, we provide a case study of the numerical errors in the simulation of quantum circuits, which shows that the extent of numerical errors in practice varies greatly between instances.

Chapter 4 also provides a discussion on floating-point errors in EVDDs. Here we find that certain methods for normalizing the values on the edges are more sensitive to numerical errors than others.

Towards Research question 3. We address Research question 3 in Chapter 6. In this chapter, we focus on the discretization of continuous Bayesian networks that we then encode in decision diagrams. We investigate the trade-off between the quality of the discretization and the resulting DD size, and compare the computational cost of the DD approach against that of traditional Bayesian network inference algorithms. We give a comprehensive empirical evaluation of a range of discretization techniques and their effect on the discretization quality and the DD size. We also corroborate that

encoding BNs in DDs can significantly improve the computational cost of inference queries over algorithms that act on the BNs directly.

Open-source software. For every chapter of this thesis that includes any empirical component (Chapters 3 to 7), we provide well-documented, publicly available source code, as well as instructions on how to reproduce the empirical results. Additionally, for Chapters 3, 4 and 7 the resulting tools can also be used on their own.

- In Chapter 3 we provide implementations of a new reachability algorithm for two types of decision diagrams; BDDs and MDDs. The implementation makes use of the DD-package Sylvan [81], and can either be used with DDs generated by the model checking tool LTSmin [133], or can be used with Sylvan directly. The implementation, along with usage instructions, can be found at <https://github.com/System-Verification-Lab/reach-dd/>.
- In Chapter 4 we implement a parallel decision diagram package for quantum computing, named Q-Sylvan. We also provide implementations of quantum circuit simulation and equivalence checking, both of which accept quantum circuits in the standard QASM [58] format. The implementation, along with usage instructions, can be found at <https://github.com/System-Verification-Lab/q-sylvan>.
- In Chapter 7 we present an implementation of finding transformations of quantum graph states by encoding the problem in SAT. It can search for transformations that either only make use of local operations (single-qubit Clifford gates and measurements), or allow for limited two-qubit gates as well. The implementation, along with usage instructions, can be found at <https://github.com/System-Verification-Lab/graph-state-synthesis>.

1.4 Thesis outline

The remainder of this thesis is structured as follows, with an overview of the domains and methods of the research chapters given in Table 1.2. Chapter 2 provides general preliminaries on transition systems and decision diagrams. In Chapter 3 we begin our investigation in the classical, discrete domain. This chapter introduces a new decision diagram operation that computes reachable states for (non)deterministic transition systems. In Chapter 4 we move into the quantum domain, with a parallel decision

Table 1.2: Chapter overview

Chapter	Domain	System type		Method
		States	Transitions	
Chapter 3	classical	discrete	unlabeled	DDs
Chapter 4	quantum	discrete	quantitative	DDs
Chapter 5	quantum + probabilistic	discrete	quantitative	DDs
Chapter 6	probabilistic	continuous	N/A	DDs
Chapter 7	quantum	discrete	unlabeled	SAT
Chapter 8	SAT	N/A	N/A	quantum

diagram implementation that can be used for quantum circuit simulation and equivalence checking. In Chapter 5 we present an analysis of floating-point errors in decision diagrams that are used in the analysis of probabilistic or quantum systems. In Chapter 6 we explore the encoding of continuous Bayesian networks in decision diagrams and investigate the trade-off between the quality of the discretization and the resulting decision diagram size. In Chapter 7, we move away from decision diagrams and present a SAT-based method for solving a circuit synthesis problem in quantum networking. In Chapter 8, we instead use a quantum algorithm to improve classical SAT-solving. Finally, in Chapter 9, we conclude this work by revisiting the posed research questions and providing suggestions for future research directions.

Chapter 2

Preliminaries

Throughout this thesis, every chapter comes with its own preliminaries that provide the mathematical background relevant to that chapter in particular. In this chapter we show how the different types of transition systems that we considered in this thesis are related to one another, and we do the same thing for decision diagrams.

2.1 (Non)deterministic and quantitative systems

A transition system is a set of states $\mathbb{S}$, and a transition relation $R \subseteq \mathbb{S} \times \mathbb{S}$ that describes the transitions between states. In this thesis, we consider two types of transition systems. For both types of systems, we consider the state space $\mathbb{S}$ to be discrete and finite. However, we distinguish between systems that have unlabeled transitions, and systems where the transitions can be labeled with continuous values, such as probabilities or quantum probability amplitudes. We will refer to the first as (non)deterministic systems, and to the latter as quantitative systems, which we split up into probabilistic and quantum systems.

(Non)deterministic systems. We start by looking at the simplest, non-trivial example: a system with two states. Because it is mathematically convenient, we shall name these states 0 and 1.



2.1. (Non)deterministic and quantitative systems

For a system that has N states, we relate an N -sized “one-hot” vector $|x\rangle$ to each state $x \in \{0, N-1\}$, that is, a vector with 1 at index x , and 0’s everywhere else. For our two-state system, this gives

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \text{ and } |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

This notation, called bra-ket or Dirac notation, comes from quantum mechanics, however, for our purposes is useful to describe classical systems as well. The notation $\langle 0| = \begin{pmatrix} 1 & 0 \end{pmatrix}$ and $\langle 1| = \begin{pmatrix} 0 & 1 \end{pmatrix}$ represents the transpose of these vectors.¹

A transition, for example from state $|0\rangle$ to $|1\rangle$, can be visualized as



and is given by a matrix

$$R_{0 \rightarrow 1} = |1\rangle\langle 0| = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}$$

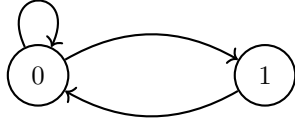
A description, such as this matrix, of one or more transitions is called a transition relation. From a state $|x\rangle$ in a (non)deterministic transition system, we can obtain the successor state(s) through a Boolean version of matrix-vector multiplication with the transition relation, where addition is replaced by logical or ($\vee$) and multiplication by logical and ($\wedge$). For example, for the matrix above, if we take $|0\rangle$ as the initial state, we obtain $|1\rangle$ as its successor.

$$R_{0 \rightarrow 1} |0\rangle = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} (1 \wedge 0) \vee (0 \wedge 0) \\ (1 \wedge 1) \vee (0 \wedge 0) \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} = |1\rangle$$

A transition system for which each state has at most one outgoing transition, such as $R_{0 \rightarrow 1}$ above, is called deterministic. A system with at least one state with multiple

¹In the more general case where we involve complex numbers, which we will see later in quantum systems, $\langle x| = |x\rangle^\dagger = (|x\rangle^*)^\top$ is the complex conjugate transpose of $|x\rangle$, where the complex conjugate (denoted with $*$) flips the signs of the imaginary components of the complex numbers.

outgoing edges, as shown below, is called nondeterministic.



$$R_{ND} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$$

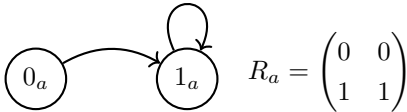
2

A transition relation R is reversible if there exists an R^{-1} such that $RR^{-1} = R^{-1}R = I$, where I is the identity matrix. For example, $X = |1\rangle\langle 0| + |0\rangle\langle 1|$ is reversible, with $X^{-1} = X$, while R_{ND} is not. While classical transition systems are often not reversible unless they are specifically designed to be, we will see later in this section that quantum systems are mostly described by reversible transition relations.

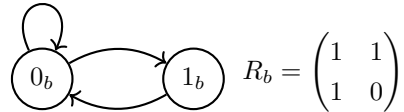
One particular method of defining a larger system is to define several smaller systems, and then consider the behavior of these smaller systems running in parallel. The parallel composition of transition systems can be computed using the tensor product ($\otimes$), which is also called to Kronecker product in the context of matrices specifically. Given an $n \times m$ matrix A and a $p \times q$ matrix B , their tensor product is given by the following $np \times mq$ matrix.

$$A \otimes B = \begin{pmatrix} a_{0,0}B & a_{0,1}B & \dots & a_{0,m-1}B \\ \vdots & \vdots & \ddots & \vdots \\ a_{n-1,0}B & a_{n-1,1}B & \dots & a_{n-1,m-1}B \end{pmatrix}$$

As an example, consider the following two transition systems R_a and R_b .

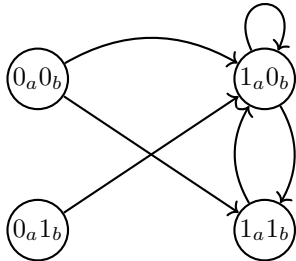


$$R_a = \begin{pmatrix} 0 & 0 \\ 1 & 1 \end{pmatrix}$$



$$R_b = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$$

Their parallel composition is given by $R_a \otimes R_b$, which is visualized below.



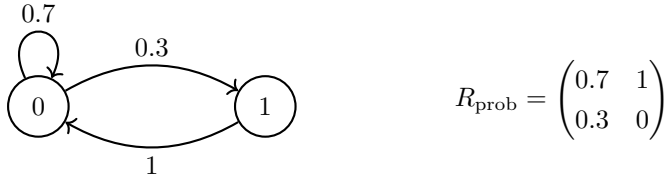
$$R_a \otimes R_b = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

2.1. (Non)deterministic and quantitative systems

The system $R_a \otimes R_b$ is called separable because it can be written as the tensor product of smaller systems. The behavior of such a system can be analyzed by analyzing the smaller systems separately. However, by making small modifications, for example, adding a transition $|0_a 0_b\rangle \rightarrow |0_a 1_b\rangle$, the system becomes non-separable.

Combining the behavior of multiple smaller systems in a single larger system quickly gives rise to an “explosion” of the state space. The tensor product of n two-state systems yields a system of 2^n states, and its transition relation is described by a $2^n \times 2^n$ matrix. When we wish to analyze the behavior of such systems, algorithms and data structures are needed that can deal with these large state-spaces, which we discuss in Section 2.2.

Probabilistic systems. While the transition systems described above only specify whether a transition $|s\rangle \rightarrow |t\rangle$ is possible, we can also specify that transitions happen with certain probabilities. We also refer to these probabilities on the transitions as “weights”. A transition system where all transitions are weighted with (fixed) probabilities, such as the example given below, is also called a Markov chain.



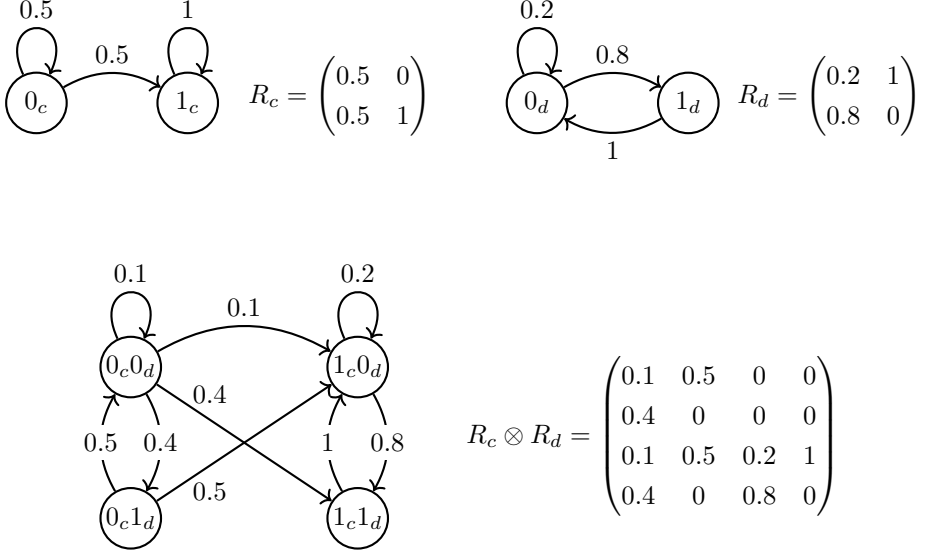
Probabilistic transition systems can be analyzed in much the same way as the discrete transition systems described above. Successor states are computed through (regular) matrix-vector multiplication, and multiple smaller systems can be combined into a larger system through their tensor product. An example of the computation of the successor states of $|0\rangle$ under R_{prob} is given below.

$$R_{\text{prob}} |0\rangle = \begin{pmatrix} 0.7 & 1 \\ 0.3 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0.7 \\ 0.3 \end{pmatrix} = 0.7 |0\rangle + 0.3 |1\rangle$$

As we see above, not only transitions but also states now have probabilities associated with them. For probabilistic systems, and even more so for quantum systems later on, it is common to refer to the vector $\begin{pmatrix} 0.7 & 0.3 \end{pmatrix}^\top = 0.7 |0\rangle + 0.3 |1\rangle$ as the *state vector*, or sometimes just the state, of the system. For a probabilistic system that has N states $\{|0\rangle, |1\rangle, \dots, |N-1\rangle\}$, we will refer to these as *basis states*, while the state vector is

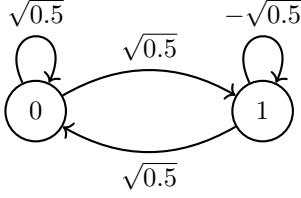
then a probability distribution over these basis states.

Combining probabilistic systems using the tensor product works in the same way as for discrete systems. Below we give an example.



The analysis of probabilistic systems suffers from the same state-space explosion problem that (non)deterministic systems do, however, it also comes with the additional hurdle of needing the deal with quantitative rather than just Boolean values in the state vectors and transition matrices.

Quantum systems. The mathematical description of quantum systems is similar to that of probabilistic systems, although there are some differences. First, while probabilities are real numbers ($\mathbb{R}$) in $[0, 1]$, quantum states and transitions can have complex ($\mathbb{C}$) values, which includes negative values. Second, while state vectors in probabilistic systems are ℓ_1 normalized (their values sum to 1), quantum state vectors are ℓ_2 normalized, i.e., the sum of all absolute values squared must equal 1. Additionally, in the same way a probabilistic transition matrix must preserve the ℓ_1 norm of a state vector, quantum transition matrices must be ℓ_2 -norm preserving. Finally, quantum transitions, with the exception of measurements, must be reversible. Below is an example of a quantum transition system.



$$H = \begin{pmatrix} \sqrt{0.5} & \sqrt{0.5} \\ \sqrt{0.5} & -\sqrt{0.5} \end{pmatrix}$$

Just like probabilistic systems, successor states are computed using matrix-vector multiplication.

$$H |1\rangle = \begin{pmatrix} \sqrt{0.5} & \sqrt{0.5} \\ \sqrt{0.5} & -\sqrt{0.5} \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} \sqrt{0.5} \\ -\sqrt{0.5} \end{pmatrix} = \sqrt{0.5} |0\rangle - \sqrt{0.5} |1\rangle$$

The weights in a quantum state vector are also called *probability amplitudes*. When a state described by a vector $|\psi\rangle$ is measured, the probability of obtaining an outcome $|x\rangle$ is given by $|\langle x|\psi\rangle|^2$. For example, when measuring the state $|\psi\rangle = -\sqrt{0.7}|0\rangle + \sqrt{0.3}$, the probability of obtaining $|0\rangle$ is

$$|\langle 0|\psi\rangle|^2 = |-\sqrt{0.7}\langle 0|0\rangle + \sqrt{0.3}\langle 0|1\rangle|^2 = |-\sqrt{0.7} + 0|^2 = 0.7.$$

While the description of quantum systems is similar to that of probabilistic systems, some aspects of their analysis are computationally much harder. Just like for (non)deterministic and probabilistic systems, the analysis of quantum systems also suffers from the state-space explosion problem. However, the inclusion of negative weights makes it so that even simulating quantum systems (a task that is easy for probabilistic systems) becomes computationally hard.

2.2 Decision diagrams

Decision diagrams (DDs) are a class of data structures that are well suited to tackling the state-space explosion problem discussed above. Here we explain the three types of DDs that are used primarily in this thesis, and show how they relate.

Binary decision diagrams. Binary decision diagrams (BDDs) [37] are a data structure used to represent Boolean functions, i.e., functions of the form $f : \{0,1\}^n \rightarrow \{0,1\}$, in a compact and canonical form. One of the main benefits of BDDs (and

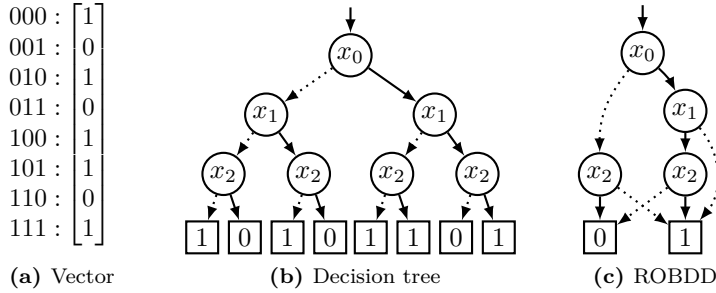


Figure 2.1: A Boolean vector (a) can be written as a decision tree (b) with its leaves the entries of the vector. Each path through the tree corresponds to a bit string (dashed = 0, solid = 1), which in turn corresponds to the index of that leaf value in the vector. An ROBDD (c) can be obtained by exhaustively merging all isomorphic subgraphs and removing nodes whose children are identical.

many other types of DDs) is that they not only offer a compressed representation of data, but also that this data can be manipulated without decompressing the DD. Similarly to how in cryptography “homomorphic encryption” [4] allows encrypted data to be manipulated without decryption, the type of compression DDs offer can be thought of as “homomorphic compression” [124].

Structurally, a BDD is a rooted, directed, acyclic graph with two types of nodes: terminal nodes (or leaf nodes) and non-terminal nodes (or internal nodes). Each internal node v is labeled with a variable $\text{var}(v) \in \{x_0, \dots, x_{n-1}\}$ and has two children: $v[0]$ (low), visualized with dashed edges, and $v[1]$ (high), visualized with solid edges. A BDD has two terminal nodes, one labeled with 1 (true) and another with 0 (false). An internal BDD node v can be understood as the Shannon decomposition of the function f^v it encodes:

$$f^v = (\bar{x}_i \wedge f_{|0}^v) \vee (x_i \wedge f_{|1}^v)$$

This decomposition can be read as “if $x_i = 0$ then $v[0]$, else $v[1]$ ”, where $v[0]$ and $v[1]$ correspond to the sub-functions $f_{|0}^v$ and $f_{|1}^v$ of f^v where x_j has been set to 0 and 1 respectively.

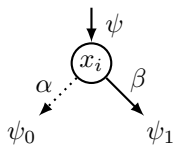
Commonly, the term BDD is used to refer specifically to reduced ordered BDDs (ROBDDs). An ordered BDD is a BDD where, on each path from the root to a

terminal node, variables are always encountered in the same order, and no variable appears more than once on a path. A is reduced by exhaustively applying the following two reduction rules: (1) merge isomorphic subgraphs, and (2) remove nodes whose low and high children are identical. Figure 2.1 shows how a vector can be compactly encoded in an ROBDD.

For all DDs discussed in these preliminaries, the input domain is Boolean (bit strings). However, in some cases, it might be more natural to label the states of a transition system with (tuples of) integers, rather than bit strings. While such domains can be translated to Boolean domains, there also exist types of DDs that can natively handle these domains, which we discuss in Chapter 3.

Multi-terminal binary decision diagrams. Both probabilistic and quantum systems as discussed in Section 2.1 can be described by pseudo-Boolean functions, i.e., functions of the form $\{0, 1\}^n \rightarrow \mathcal{D}$, with $\mathcal{D} = \mathbb{R}$ for probabilistic systems and $\mathcal{D} = \mathbb{C}$ for quantum systems. In order to encode pseudo-Boolean functions in DDs, we need DDs in which we can include real or complex values. The first type of DDs that allows for this are multi-terminal binary decision diagrams (MTBDDs), also referred to as algebraic decision diagrams (ADDs) [91, 92]. Where BDDs allow only for 0 and 1 leaf nodes, MTBDDs allow leaves to be labeled with values from any domain. The reduction rules and interpretation of an MTBDD are effectively the same as those for a BDD. Figure 2.2c shows an example.

Edge-valued decision diagrams. While BDDs can compactly encode tensor products of systems described by Boolean functions, MTBDDs cannot always compactly encode tensor products of systems described by pseudo-Boolean functions. This problem can be solved by moving the values from the leaf nodes onto the edges instead, in what are called edge-valued decision diagrams (EVDDs) [222]. Equivalent to the Shannon decomposition for Boolean functions, an EVDD node that represents a vector $|\psi\rangle$ can be understood as the following decomposition of that vector.



$$|\psi\rangle = |0\rangle \otimes \alpha |\psi_0\rangle + |1\rangle \otimes \beta |\psi_1\rangle$$

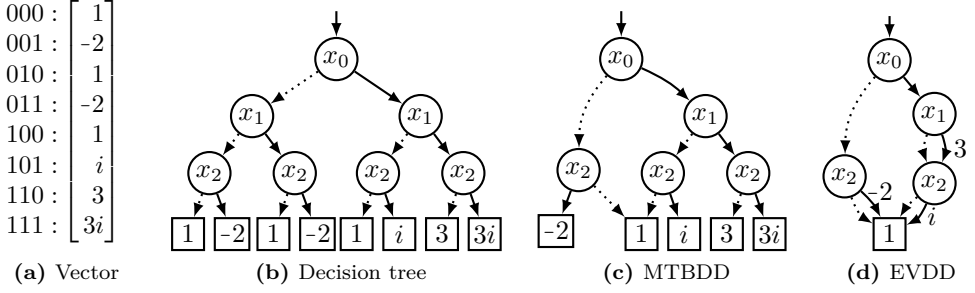


Figure 2.2: A real or complex vector (a) can be represented as a decision tree (b) with its leaves the entries of the vector. Each path through the tree corresponds to a bit string (dashed = 0, solid = 1), which in turn corresponds to the index of that leaf value in the vector. In MTBDDs (c), isomorphic nodes (nodes which represent the same vector) are merged, and redundant nodes (nodes for which both outgoing edges point to the same target) are removed. In EVDDs (d), values are stored on the edges instead of in the leaves. This allows two EVDD nodes to be merged when they represent the same vector up to a multiplicative factor.

By factoring out multiplicative factors α and β in this decomposition, two sub-functions $\psi_{|a}$ and $\psi_{|b}$ can now be represented by the same DD node if they are equal up to a multiplicative factor, i.e. when there exists some scalar α such that $\psi_{|b} = \alpha \cdot \psi_{|a}$. Figure 2.2d shows how this can yield significantly more compression than an MTBDD. In general, an EVDD can be exponentially more succinct than the corresponding MTBDD.

2.3 Encoding transition systems in decision diagrams

As discussed in Section 2.2, for any system with 2^n basis states, we can think of a state vector as a function $s : \{0, 1\}^n \rightarrow \mathcal{D}$, where $\mathcal{D} = \{0, 1\}$ for (non)deterministic systems, $\mathcal{D} = \mathbb{R}$ for probabilistic systems, and $\mathcal{D} = \mathbb{C}$ for quantum systems. For any $\vec{x} \in \{0, 1\}^n$, $s(\vec{x})$ returns the value of the state vector at index $\vec{x}$. Similarly, the transition relation of such a system, given by an $2^n \times 2^n$ matrix, can be seen as a function $R : \{0, 1\}^{2n} \rightarrow \mathcal{D}$, where, for $\vec{x}, \vec{x}' \in \{0, 1\}^n$, $R(\vec{x}, \vec{x}')$ returns the value in the transition matrix at column index $\vec{x}$ and row index $\vec{x}'$. A state vector s and a transition relation R can be encoded in DDs as shown in Figure 2.3.

Operations on DDs are typically defined recursively. For example, matrix-vector

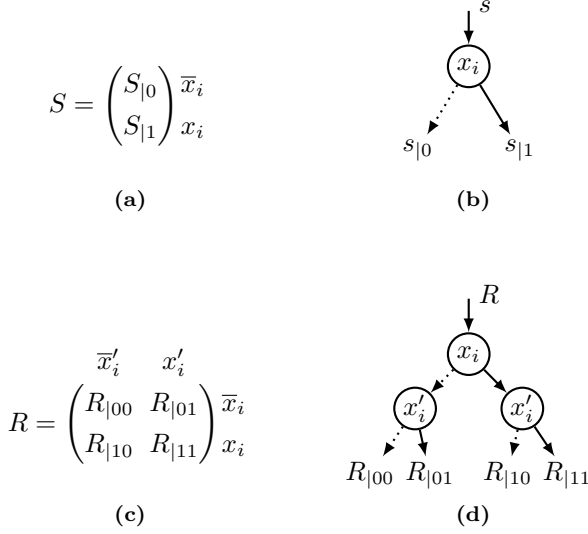


Figure 2.3: A 2^n -sized vector (a) can be recursively encoded in a DD by splitting the vector up into its top half (indices that start with a 0, corresponding to $\bar{x}_i$) and its bottom half (indices that start with a 1, corresponding to x_i) (b). A $2^n \times 2^n$ -sized matrix (c) can be recursively encoded in a DD by splitting it up into its quadrants, using variables $\bar{x}$ to index its rows, and variables $\bar{x}'$ to index its columns (d). The variables $\bar{x}$ and $\bar{x}'$ are interleaved in the DD variable order, such that a recursive DD algorithm can easily access the quadrants at each recursive level.

multiplication can be split up recursively as

$$Rs = \begin{pmatrix} R_{|00} & R_{|01} \\ R_{|10} & R_{|11} \end{pmatrix} \begin{pmatrix} s_{|0} \\ s_{|1} \end{pmatrix} = \begin{pmatrix} R_{|00}s_{|0} + R_{|01}s_{|1} \\ R_{|10}s_{|0} + R_{|11}s_{|1} \end{pmatrix}.$$

When s and R describe a vector and matrix of sizes 2^n and $2^n \times 2^n$ respectively, then $R_{|ij}s_{|j}$ represents matrix-vector multiplication of a vector and matrix of sizes 2^{n-1} and $2^{n-1} \times 2^{n-1}$. The terminal case, a 2^0 -sized vector and $2^0 \times 2^0$ -sized matrix, is simply scalar multiplication.

The description above is a fairly high-level illustration of matrix-vector multiplication with DDs. Throughout the next chapters, we describe various DD algorithms in more detail.

Chapter 3

A decision diagram operation for reachability

When computing fixpoints with decision diagrams (DDs) using a breadth-first approach, intermediate DDs often become significantly larger than the final DD that represents the fixpoint. The saturation algorithm, which can compute fixpoints for a partitioned transition relation and is generally considered the state-of-the-art DD approach in this setting, aims to avoid this bottleneck by computing fixpoints in a way that exploits the locality of the partitions. We present a new DD operation called REACHDD that computes fixpoints, and, in contrast to saturation, does not require a partitioning of the transition relation. We give sequential and parallel versions of REACHDD for both binary and multi-valued DDs. We implement these algorithms and experimentally compare their performance against saturation on 692 model checking benchmarks in different languages. The results show that the REACHDD operation often outperforms saturation, especially on transition relations with low locality. In a comparison between parallelized versions of REACHDD and saturation, we find that REACHDD obtains comparable speedups up to 16 cores, although it falls behind saturation at 64 cores. Finally, in a comparison with the state-of-the-art model checking tool ITS-tools, we find that REACHDD outperforms ITS-tools on 29% of models, suggesting that it can be useful as a complementary method in an ensemble tool.

This chapter has been published as: Sebastiaan Brand, Thomas Bäck, and Alfons Laarman. “A decision diagram operation for reachability”. In: *International Symposium on Formal Methods*. Springer. 2023, pp. 514–532.

3.1 Introduction

Reachability analysis. Model checking is an important technique for ensuring that systems work according to specification. A core task in model checking is reachability analysis [27, 51], i.e., computing forward or backward reachable states of a system. Typically, the state space of a program grows exponentially with the number of variables and threads. One method for dealing with this explosion is the use of symbolic methods such as decision diagrams. Decision diagrams [37] are directed, acyclic graphs that succinctly represent sets of states by leveraging the exponential growth in paths from a dedicated root node to a leaf. The data structure provides various efficient manipulation operations, such as logical disjunction and conjunction and image computation.

In this chapter, we present a new decision diagram operation for reachability.

Related work. While SAT-based methods for model checking have become increasingly popular, doing reachability analysis with decision diagrams is still an important component of many state-of-the-art model checking tools, as can be seen in the Model Checking Contest (MCC) [139, 228]. Symbolic reachability analysis with binary decision diagrams (BDDs) and other variants [93, 19, 132, 69, 234] is done by encoding both the initial system state S_{init} and its transition relation R in the diagram. The set of reachable states S is then iteratively computed using the image operation [155], denoted by $S.R$, starting from S_{init} . Since the order of exploration (e.g. breadth-first search, depth-first search, or other strategies) greatly influences the sizes of the intermediate decision diagrams, various exploration strategies, like saturation [48], chaining [190], and sweep-line [47], have been considered. These algorithms have in common the use of the image computation $S.R$ as their main operation.

Saturation stands out from other approaches, not only because it often performs better [49] and leading MCC tools use it in decision diagram based reachability [228], but also because it integrates the image computations into the traversal of the decision diagram of S . Saturation avoids redundant reconstructions by building the decision diagram for the reachable states bottom-up, eagerly *saturating* the bottom nodes by exhaustively applying all relevant transitions.

Contribution. We present three new decision diagram operations for reachability: REACHBDD and REACHMDD (for BDDs and MDDs respectively), as well as a parallel version of REACHBDD. These algorithms partially construct the decision diagram

of S from the bottom up, but unlike saturation do not require partial relations and can handle monolithic transition relations. An additional advantage of these new reachability operations is their relative simplicity in comparison with saturation.

We implement these new operations in the decision diagram package Sylvan [76], and experimentally compare them against saturation on a total of 692 problem instances from three model checking benchmark sets: DVE (BEEM [179]), Petri nets (MCC [140]), and Promela models [121, 25]. We find that our methods are competitive with saturation, and tend to outperform saturation on larger instances. The parallel speedups obtained by REACHBDD up to 16 cores are comparable to those achieved in a parallel version of saturation [78], although they fall behind on 64 cores. Aside from the comparison against saturation, we also compare REACHMDD against the state-of-the-art model checking tool ITS-tools [228], where we find that REACHMDD performs better than ITS-tools on 29% of models, and can therefore be useful as a complementary method in an ensemble tool.

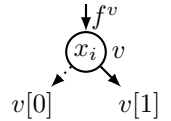
Outline. Section 3.2 discusses preliminaries. Section 3.3 explains the new reachability algorithms, and is then followed by an empirical evaluation of these algorithms in Section 3.4. Finally, Section 3.5 concludes this chapter.

3.2 Preliminaries

3.2.1 Binary decision diagrams

Binary decision diagrams (BDDs) [37, 216] are a data structure for representing Boolean functions $f(x_1, \dots, x_n)$, i.e., functions of type $f : \{0, 1\}^n \rightarrow \{0, 1\}$. Structurally, a BDD is a rooted, directed, acyclic graph with two types of nodes: terminal nodes with values $\{0, 1\}$ and non-terminal nodes v that have two children, $v[0]$ (*low*) and $v[1]$ (*high*), and a variable label $\text{var}(v) \in \{x_0, \dots, x_{n-1}\}$. Figure 3.1 shows examples of *ordered* BDDs, i.e., BDDs where on each path variable labels occur in a fixed order $x_0 \prec x_1 \prec \dots \prec x_{n-1}$.

A non-terminal node v with $\text{var}(v) = x_i$, shown right, can be read as the *Shannon decomposition* “if $x_i = 1$ then $v[1]$ else $v[0]$.” The function represented by the node v , call it f^v , is thus given by



$$f^v(x_1, \dots, x_n) \triangleq \begin{cases} x_i f^{v[1]} \vee \bar{x}_i f^{v[0]} & \text{if } v \notin \{0, 1\}, \\ v & \text{if } v \in \{0, 1\}. \end{cases} \quad (3.1)$$

3.2. Preliminaries

The definition in Eq. (3.1) shows that a conditioned sub-function $f_{|\vec{a}}^v$, as in Eq. (3.2) below, is represented by a decision diagram node, namely the one following the path $v[\vec{a}] \triangleq v[a_1][a_2] \dots [a_k]$, assuming the BDD is ordered and no variables are skipped.

$$f_{|a_1, \dots, a_k}^v(x_1, \dots, x_n) \triangleq f^v(x_1 = a_1, \dots, x_k = a_k, x_{k+1}, \dots, x_n) \quad (3.2)$$

The insight that BDDs exploit to realize succinct representations of commonly-encountered Boolean functions is that many sub-functions for different $\vec{a}, \vec{b} \in \{0, 1\}^*$ can be isotropic, i.e., $f_{|\vec{a}} = f_{|\vec{b}}$. Take $f_{|00} = f_{|11}$ in Fig. 3.1a. In the diagram, this means that the isomorphic subgraphs below the nodes representing $f_{|00}$ and $f_{|11}$ can be merged, as in Fig. 3.1c.

An ordered BDD is *reduced* when, in addition to isomorphic sub-graph merging, all *redundant nodes* (nodes v with $v[0] = v[1]$) are removed. Take $f_{|010} = f_{|011} = f_{|01} = 1$, in Fig. 3.1c. Removed redundant nodes can be reconstructed by recognizing that a variable is skipped on a path, as done Fig. 3.1b.

Reduced and ordered BDDs (ROBDDs) are canonical representations of Boolean functions, i.e., any two functions with the same truth table are uniquely represented by (the root node of) an ROBDD. Canonicity allows for equivalence checking in constant time through hashing of nodes v as tuples $\langle \text{var}(v), v[1], v[0] \rangle$ (provided that nodes $v[1], v[0]$ are already canonically represented, i.e., the BDD is built in a bottom-up fashion). This in turn allows efficient manipulation operations ($\wedge, \vee, \dots$) as in Section 3.2.3.

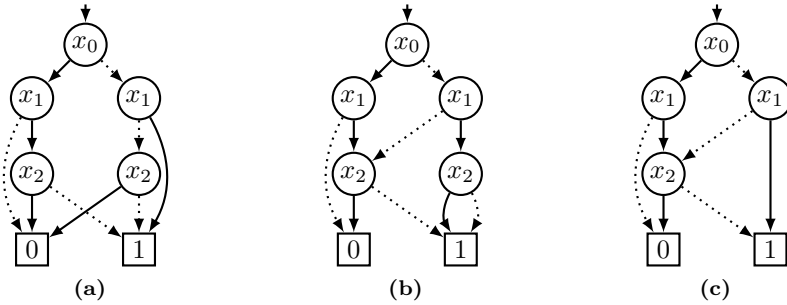


Figure 3.1: (Ordered) BDDs representing function $f = (x_0 \wedge x_1 \wedge \bar{x}_2) \vee (\bar{x}_0 \wedge \bar{x}_1 \wedge \bar{x}_2) \vee (\bar{x}_0 \wedge x_1)$. Dashed lines represent low branches ($v[0]$) and solid lines high branches ($v[1]$). Only the BDD in (c) is completely reduced. The BDD in (a) can be reduced to (c) by merging the two isomorphic nodes for x_2 , while the BDD in (b) can be reduced to (c) by removing the (right-most) redundant node x_2 .

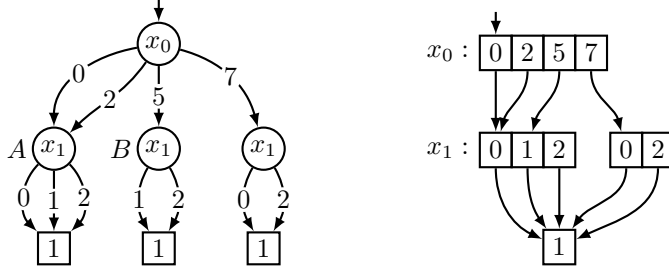


Figure 3.2: An MDD (left) and LDD (right) which both encode the set $\{\langle 0, 0 \rangle, \langle 0, 1 \rangle, \langle 0, 2 \rangle, \langle 2, 0 \rangle, \langle 2, 1 \rangle, \langle 2, 2 \rangle, \langle 5, 1 \rangle, \langle 5, 2 \rangle, \langle 7, 0 \rangle, \langle 7, 2 \rangle\}$. Arrays represent right-ward sibling chains in the LDD. To improve legibility, we omit edges pointing to the 0 terminal and replicate the 1 terminal for MDDs.

For conciseness, we will often consider quasi-ROBDD, which are ROBDDs where all redundant nodes are reconstructed (e.g. Fig. 3.1b). In many settings, this stronger definition does not lose generality, as quasi-ROBDD are also canonical, and at most a factor $\frac{n}{2}$ larger than an ROBDD for the same function [137]. At the same time, because for all quasi-ROBDD nodes v we have $f_a^v = v[\vec{a}]$, this assumption greatly simplifies algorithm representation and reduces cases in proofs. We denote with $level\ i$ the set of all nodes with the variable label x_i .

3.2.2 Multi-valued decision diagrams

Multi-valued decision diagrams (MDDs) [132, 201] are a generalization of BDDs for encoding functions $\mathcal{D}_1 \times \dots \times \mathcal{D}_n \rightarrow \{0, 1\}$, where $\mathcal{D}_i = \{0, 1, \dots, m-1\}$ for some m that can be different for each $\mathcal{D}_i$. Each MDD node with variable x_i has m outgoing edges, each with a label in $\mathcal{D}_i$. The interpretation of following an edge remains the same as for BDDs: for an MDD which encodes a function f and has root node v with $\text{var}(v) = x_i$, following an edge with label $a \in \mathcal{D}_i$ leads to an MDD which encodes the sub-function $f|_{x_i=a}$.

Similar to BDDs, MDDs are typically reduced by merging isomorphic sub-graphs. However, unlike ROBDDs, redundant nodes are usually not removed in MDDs, which means variables are never skipped on any path. So an MDD with $m = 2$ is a quasi-ROBDD. Figure 3.2 shows an example.

A list decision diagram (LDD) [34] is the Knuth transform of an MDD into a left-child right-sibling binary tree. Siblings (right-ward chains) are stored as (ordered) linked lists, which allows the reuse of common sibling suffixes, as shown in the example

in Figure 3.2 (e.g. $f_{|5}$ reuses a part of the siblings of $f_{|2} = f_{|0}$).

Finally, a BDD or MDD representing a function $f(x_1, \dots, x_n)$ can also be interpreted as a set of strings $\vec{a}$ of length n , according to the characteristic function $\{\vec{a} \mid f(\vec{a}) = 1\}$. So the BDDs in Figure 3.1 all represent $\{000, 010, 011, 110\}$.

3.2.3 Decision diagram operations

What makes BDDs and MDDs so useful, aside from their possible succinctness, is that many manipulation operations, such as disjunction (set union) and conjunction (set intersection), can be performed in polynomial time in the number of decision diagram nodes in the operands [37, 10]. While other operations that have been shown to be NP-complete [155], such as unbounded existential quantification and thus also image computation [155], are often still efficient in practice [40].

As an example, Algorithm 3.1 shows how to compute the union of two quasi-reduced diagrams A and B , i.e., $A \cup B$ (or in the functional interpretation: $A \vee B$). Because any BDD is defined by its root node, the arguments A and B are simply given as nodes. The algorithm first considers leaves as a base case, treating them according to the semantics of $\vee$. On lines 7 and 8, the function is called recursively on the children of the input BDDs, synchronizing on the low and high branches. The results from these recursive calls are then combined with the $\text{MAKENODE}(x, L, H)$ function which creates a *reduced BDD node* v with $v[0] = L$, $v[1] = H$ and $\text{var}(v) = x$. To ensure reduction, it returns $L (= H)$ when the node is redundant and looks up the tuple $\langle x, L, H \rangle$ in a *unique table*, as discussed in Section 3.2.1. For MDDs, we assume a function $\text{MAKENODE}(x, A_0, \dots, A_{m-1})$ which returns a (quasi-)reduced MDD node and, for notational convenience, takes $m + 1$ positional arguments: a variable x , and one MDD node A_i for each of its m children (some of which can be 0).

Lastly, it is important to realize that a decision diagram with $|V|$ nodes can have $\exp(|V|)$ paths from the root to a terminal node. To achieve polynomial runtimes, decision diagram operations use top-down dynamic programming (see lines 5 and 10). This ensures that different paths leading to the same (pairs of) nodes are caught by the cache, avoiding recomputation.

3.2.4 Encoding symbolic transition systems

A symbolic transition system is a tuple $(\vec{x}, \mathbb{S}, R, S_{\text{init}})$, where $\vec{x}$ is a tuple of Boolean variables $(x_0, \dots, x_{n-1})$, $\mathbb{S} = \{0, 1\}^n$ is the state space (including unreachable states), $R \subseteq \mathbb{S} \times \mathbb{S}'$ is a transition relation, and $S_{\text{init}} \subseteq \mathbb{S}$ is a set of initial states. The relation R

Algorithm 3.1: A BDD operation to compute the union of two BDDs.

```

1 def UNION( $A, B$ ) : ▷ For quasi-ROBDDs  $A$  and  $B$  on  $n$  variables.
2   if  $A = 0$  then return  $B$ 
3   if  $B = 0$  then return  $A$ 
4   if  $A = 1 \vee B = 1$  then return 1
5   if  $\text{res} \leftarrow \text{cache}[\text{UNION}, A, B]$  then return  $\text{res}$ 
6    $x \leftarrow \text{var}(A)$  ▷ By virtue of quasi reduction  $\text{var}(A) = \text{var}(B)$ .
7    $L \leftarrow \text{UNION}(A[0], B[0])$ 
8    $H \leftarrow \text{UNION}(A[1], B[1])$ 
9    $\text{res} \leftarrow \text{MAKENODE}(x, L, H)$ 
10   $\text{cache}[\text{UNION}, A, B] \leftarrow \text{res}$ 
11  return  $\text{res}$ 

```

is a constraint over variables $\vec{x}, \vec{x}'$, where $\vec{x}$ encodes the source states and $\vec{x}'$ (consisting of primed copies of $\vec{x}$) encodes target states. While R monolithically encodes the system's behavior, we also consider a local variant, discussed in the following example.

As an example we give a transition system which captures the dining philosophers problem. We have k processes (philosophers) and k resources (forks). Each process P_i , with $i \in \{1, \dots, k\}$, attempts to allocate two resources: a fork i on the left and a fork $j = ((i - 1) \bmod k) + 1$ on the right. If a fork is unavailable the philosopher waits until it becomes available. To describe the state space we use $3k$ Boolean variables: $\vec{x} = (a_1, \ell_1, r_1, \dots, a_k, \ell_k, r_k)$, where a_i ($\bar{a}_i$) indicates fork i is (not) available, ℓ_i ($\bar{\ell}_i$) indicates philosopher i does (not) hold a fork in their left hand, and r_i ($\bar{r}_i$) idem for their right hand. The starting state $S_{\text{init}} = (a_1, \bar{\ell}_1, \bar{r}_1, \dots, a_k, \bar{\ell}_k, \bar{r}_k)$.

To define R_i for $k > 1$ processes, we define three local relations R_i^m that implement picking up and putting down the left/right fork, and eating.

$$\begin{aligned}
 R_i^1 &= (a_i \oplus a'_i) \wedge (\ell_i \oplus \ell'_i) &> \text{pick up / put down left} \\
 R_i^2 &= (r_i \oplus r'_i) \wedge (a_j \oplus a'_j) &> \text{pick up / put down right} \\
 R_i^3 &= \ell_i \wedge \ell'_i \wedge r_i \wedge r'_i &> \text{eat (hold on to both forks)}
 \end{aligned}$$

Let $\text{support}(R_i^m)$ be the set of (primed and unprimed) variables in R_i^m . Notice that the support of each local relation contains a different subset of the target variables $\vec{x}'$. To ensure that the other target variables are not left unconstrained, we need to add the constraint $x \Leftrightarrow x'$ for all missing variables $x' \in \vec{x}'$. For instance, pick up / put

down left should be extended as:

$$(a_i \oplus a'_i) \wedge (\ell_i \oplus \ell'_i) \wedge (r_i \Leftrightarrow r'_i) \wedge \bigwedge_{j \neq i} (a_j \Leftrightarrow a'_j \wedge r_j \Leftrightarrow r'_j \wedge \ell_j \Leftrightarrow \ell'_j)$$

The same needs to happen when merging multiple processes R_i into a single global relation R , i.e. $R = \bigvee_{i=1}^k \mathcal{R}_i$, where $\mathcal{R}_i := R_i \wedge \bigwedge_{x' \in \vec{x}' \setminus \text{support}(R_i)} x \Leftrightarrow x'$. Section 3.2.5 discusses how extending local relations in this way can be avoided.

3

If a transition relation R is composed of R_i 's, where each $\text{support}(R_i)$ is a small subset of $\vec{x}, \vec{x}'$, we say that R has high locality. If R cannot be split up into such partial relations we say that R has low locality, or is *monolithic*.

3.2.5 Reachability with decision diagrams

For a transition system with n Boolean variables $\{x_0, \dots, x_{n-1}\}$, a single state is given by a bit string $s \in \{0, 1\}^n$. A set of states $S \subseteq \{0, 1\}^n$ can be encoded in a BDD. Likewise, a transition relation $R \subseteq \{0, 1\}^n \times \{0, 1\}^n$ can be encoded in a BDD with $2n$ variables. The variables are ordered by interleaving the source state variables $\vec{x} = (x_0, \dots, x_{n-1})$ with target state variables (primed copies: $x'_0, \dots, x'_{n-1}$), i.e., $(x_0, x'_0, \dots, x_{n-1}, x'_{n-1})$, as it is both convenient for the implementation of BDD algorithms, and it reduces the diagram size [155]. To compute the successors to an initial set of states S , we can then use the decision diagram operation $\text{IMAGE}(S, R)$ [155, 76]:

$$\text{IMAGE}(S, R) = (\exists x_1, \dots, x_n : (S \wedge R))_{[x'_0, \dots, x'_{n-1} := x_0, \dots, x_{n-1}]}, \quad (3.3)$$

where $[\vec{x}' := \vec{x}]$ indicates the relabeling of the target variables to source variables. The IMAGE operation can also be implemented for partial relations R_i , with the benefit that existential quantification only needs to happen over $\text{support}(R_i)$, rather than over all variables. The union of the image under each R_i separately equals the image under the global transition relation:

$$\begin{aligned} \exists \vec{x} : ((\mathcal{R}_1 \vee \dots \vee \mathcal{R}_k) \wedge S)_{[\vec{x}' := \vec{x}]} &= (\exists \vec{x}'_1 : R_1 \wedge S)_{[\vec{x}' := \vec{x}]} \vee \dots \vee \\ &(\exists \vec{x}'_k : R_k \wedge S)_{[\vec{x}' := \vec{x}]} \end{aligned}$$

where $\vec{x}'_i = \text{support}(R_i) \cap \vec{x}'$ and $\mathcal{R}_i$ is the extension of R_i as defined in Section 3.2.4.

With IMAGE , the reachable states can be computed by repeatedly applying R to a growing set of states until no new states are found, which we denote with $S.R^*$.

3.2.6 Saturation

Saturation [48] is a method for computing reachability that exploits locality of transitions. Aside from the initial states S_{init} , the algorithm takes as input several local transition relations R_i (see Section 3.2.4), ordered such that $\text{var}(R_i) \succeq \text{var}(R_{i+1})$.

To illustrate how saturation works, let us give an example. Say we have a global state space given by $\{x_0, \dots, x_3\}$, and three partial relations R_i with $\text{support}(R_1) = \{x_2, x'_2, x_3, x'_3\}$, $\text{support}(R_2) = \{x_1, x'_1, x_2, x'_2\}$, and $\text{support}(R_3) = \{x_0, x'_0, x_1, x'_1\}$. The “dependency matrix” below visualizes the dependencies of the partial relations on each of the variables.

$$\begin{array}{c} R_1 \quad R_2 \quad R_3 \\ \begin{array}{c} x_0 \\ x_1 \\ x_2 \\ x_3 \end{array} \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix} \end{array}$$

The saturation algorithm traverses the decision diagram of a set of states S and *saturates* the nodes from the bottom-up. What this means in the case of the example is that the partial relation R_1 is applied to all nodes v in S with $\text{var}(v) = x_2$, until v has converged. After saturating this node v the algorithm backtracks upwards in the decision diagram to a node w with $\text{var}(w) = x_1$, and saturates this node by exhaustively applying R_2 , while eagerly saturating new nodes created below w .

While breadth-first search (BFS) suffers from large intermediate diagram sizes [97], saturation often avoids this by ensuring that the lower levels of the decision diagram reach their final configuration early. Generally, this works well if the (average) bandwidth (the distance between the first and the last non-zero entry in each row) of the dependency matrix is low. This occurs, for example, in asynchronous systems where processes mainly modify local variables or communicate only with “neighboring processes” through channels or shared variables dedicated to neighboring pairs. This is, for example, the case in the dining philosophers example given in Section 3.2.4. Finding a variable order and organizing the partial relations such that both this bandwidth and sizes of the decision diagrams are minimized is generally hard (even finding an optimal variable order for a single BDD is NP-complete [35]), however good heuristics exist [6, 7, 11, 12, 158].

While originally proposed for MDDs, saturation has since been implemented for BDDs as well [78].

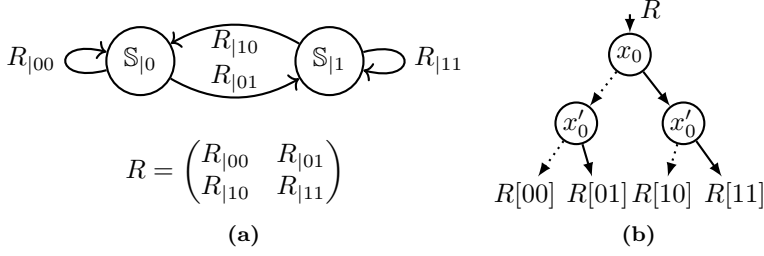


Figure 3.3: The state space $\mathbb{S}$ can be split up into states where the first variable equals 0, and states where the first variable equals 1 (a), and the transition relation R is split up accordingly. Because source and target variables are interleaved as usual, these partitions of R can be easily accessed in the BDD structure (b).

3.3 Decision diagram operation for reachability

3.3.1 For BDDs

We present an operation $\text{REACHBDD}(S, R)$, which computes the reachable states $S.R^*$ for BDDs. As is typical in BDD operations, our algorithm splits the computation into recursive calls on smaller, factored BDDs, after which the results are composed again in the backtracking step. The way we factor R is inspired by [154], where a BDD algorithm is given for computing the closure R^* of R (computing the closure R^* is generally much more expensive [154; §6], hence we want to compute $S.R^*$ directly for a given S). REACHBDD is given in Algorithm 3.2, and its correctness is discussed in Section 3.3.4 (and in further detail in Section 3.6).

The algorithm recurses on the low and high child of a BDD S . This splits the state space into $\mathbb{S}_{|0}$ (all states starting with a 0) and $\mathbb{S}_{|1}$. The relation R can be split up accordingly as shown in Figure 3.3. The self loops in this figure represent $S_{|0}.R^*_{|00}$ and $S_{|1}.R^*_{|11}$, and correspond to recursive calls to REACHBDD (lines 7 and 9). The results of these calls need to be propagated using image computation $S_{|i}.R_{|ij} = \text{IMAGE}(S[i], R[ij])$ (lines 8 and 10) until S has converged, so we incorporate a loop (line 6). For notational convenience we assume $S[0]$ and $S[1]$ are program variables to which we can assign new BDDs.

The base cases for the algorithm are as follows: If the set of initial states or the transition relation is empty ($S = 0$ or $R = 0$) there are no successors and the set of reachable states is the set of initial states. If the set of initial states contains all states ($S = 1$), or if R contains transitions from all states to all other states ($R = 1$) and S

is not empty, then all states are reachable.

With the decision diagram framework Sylvan [81], we can parallelize decision diagram operations through SPAWN/SYNC commands, which respectively fork and join light-weight tasks [81]. However, the order of (parallel) operations is something to take into account. In particular, lines 7 and 8 from Algorithm 3.2 are dependent, so cannot be executed in parallel. In order to parallelize REACHBDD, we change the order of calls in this loop and introduce REACHBDDPAR in Algorithm 3.3.

Algorithm 3.2: A BDD operation for computing reachability. Cache lookup and insert for dynamic programming after lines 5 and 10 are omitted.

```

1 def REACHBDD( $S, R$ ) :                                ▷ For quasi-ROBDDs  $S, R$  on  $n, 2n$  variables.
2   if  $S = 0$  then return 0
3   if  $R = 0$  then return  $S$ 
4   if  $S = 1$  then return 1
5   if  $R = 1 \wedge S \neq 0$  then return 1
6   while  $S$  did not converge do
7      $S[0] \leftarrow \text{REACHBDD}(S[0], R[00])$ 
8      $S[1] \leftarrow \text{UNION}(S[1], \text{IMAGE}(S[0], R[01]))$ 
9      $S[1] \leftarrow \text{REACHBDD}(S[1], R[11])$ 
10     $S[0] \leftarrow \text{UNION}(S[0], \text{IMAGE}(S[1], R[10]))$ 
11  return  $\text{MAKENODE}(\text{var}(S), S[0], S[1])$ 

```

Algorithm 3.3: REACHBDDPAR parallelizes the loop on lines 6 to 10 in Algorithm 3.2. The IMAGE and UNION calls are also parallelized [79].

```

6 while  $S$  did not converge do
7   SPAWN( REACHBDDPAR( $S[0], R[00]$ ) )                ▷ Spawn call as task (fork)
8    $S[1] \leftarrow \text{REACHBDDPAR}(S[1], R[11])$           ▷ Call directly
9    $S[0] \leftarrow \text{SYNC}$                                 ▷ Obtain task result (join)
10  SPAWN( IMAGE( $S[1], R[10]$ ) )                          ▷ Spawn call as task (fork)
11   $T_1 \leftarrow \text{IMAGE}(S[0], R[01])$                   ▷ Call directly
12   $T_0 \leftarrow \text{SYNC}$                                 ▷ Obtain task result (join)
13  SPAWN( UNION( $S[0], T_0$ ) )                            ▷ Spawn call as task (fork)
14   $S[1] \leftarrow \text{UNION}(S[1], T_1)$                   ▷ Call directly
15   $S[0] \leftarrow \text{SYNC}$                                 ▷ Obtain task result (join)

```

3.3. Decision diagram operation for reachability

3.3.2 Analysis

To provide some intuition for the complexity behavior of REACHBDD, we provide two cases: one where REACHBDD is exponentially faster than BFS, and one where REACHBDD reduces to BFS.

Ideal case. We give a concrete instance of a relation R and an initial set of states S_{init} for which REACHBDD performs exponentially better than a simple BFS. Consider a transition relation R which simply increases a (program) counter of n bits. This counts from a starting state $S_{\text{init}} = 0 = \langle 00 \dots 0 \rangle$ in steps of 1 to $2^n - 1 = \langle 11 \dots 1 \rangle$. As the state space is a line graph, the BFS algorithm discovers one new state every iteration, requiring $O(2^n)$ calls to the IMAGE function.

To illustrate the behavior of the REACHBDD algorithm, let us explicitly write all R_{ij} for $n = 3$:

$$R_{|00} = \left\{ \begin{array}{l} (\emptyset 00, \emptyset 01), \\ (\emptyset 01, \emptyset 10), \\ (\emptyset 10, \emptyset 11) \end{array} \right\} \quad R_{|11} = \left\{ \begin{array}{l} (\chi 00, \chi 01), \\ (\chi 01, \chi 10), \\ (\chi 10, \chi 11) \end{array} \right\} \quad R_{|01} = \{(\emptyset 11, \chi 00)\} \quad R_{|10} = \emptyset$$

While $R_{|00}$ and $R_{|11}$ represent different sets, the BDDs $R[00]$ and $R[11]$ are equal.

For all non-terminal cases Algorithm 3.2 does the following:

8	$S[0] \leftarrow \text{REACHBDD}(S[0], R[00])$	▷ computes states S_0^{all}
9	$S[1] \leftarrow \text{UNION}(S[1], \text{IMAGE}(S[0], R[01]))$	▷ generates 'seed' state S_1^{init}
10	$S[1] \leftarrow \text{REACHBDD}(S[1], R[11])$	▷ computes states S_1^{all}
11	$S[0] \leftarrow \text{UNION}(S[0], \text{IMAGE}(S[1], R[10]))$	▷ produces no new states

First, REACHBDD computes all reachable states that start with a 0, let us call these S_0^{all} . Next, the IMAGE call produces exactly one new state, $\langle 100 \dots 0 \rangle$, which will act as a “seed” state for the next REACHBDD call. Since the BDDs of S_0^{init} and S_1^{init} are equal, just as the BDDs $R[00]$ and $R[11]$, the second REACHBDD call can be looked up from cache. Finally, since $R[10] = \emptyset$, no new states will be added to S_0 , and both S_0 and S_1 will have converged.

We find two things: first, all the reachable states are found in a single loop iteration, and second, each call to REACHBDD only generates one recursive call to REACHBDD (because the second call can always be looked up from cache). Overall, REACHBDD only makes $O(n)$ recursive calls to itself and to the IMAGE function.

Due to the monolithic nature of the transition relation, saturation will behave like BFS in this case.

Bad case. Here we provide an instance for which REACHBDD reduces to breadth-first search. From an arbitrary transition relation R we create a new relation R' for which REACHBDD behaves like BFS. If R is a relation over $2n$ variables $\{x_1, x'_1, \dots, x_n, x'_n\}$, we let R' be a relation over $2(n+1)$ variables $\{x_0, x'_0, x_1, x'_1, \dots, x_n, x'_n\}$. Specifically, let $R' := x_0 \oplus x'_0 \wedge R$. The corresponding decomposition into sub-functions (as visualized in Figure 3.3) looks like

$$R' = \begin{pmatrix} 0 & R'_{|01} \\ R'_{|10} & 0 \end{pmatrix}.$$

For a relation like this, the loop on line 6 relies entirely on the image computation steps to expand the set of reachable states, while the recursive calls never add any states. This effectively turns REACHBDD into BFS.

3.3.3 MDD generalization

In this section, we generalize REACHBDD (Algorithm 3.2) from a BDD operation to an MDD operation (Algorithm 3.4, correctness discussed in Section 3.3.4 and Section 3.6). For simplicity, let us assume we have an MDD encoding a set of states of n variables, each of which takes values from the same domain $\mathcal{D} = \{0, 1, \dots, m-1\}$, and an MDD that encodes the transition relation, which has $2n$ variables (n source variables and n target variables). As shown below, the state MDD can be divided into m parts, and the relation MDD into m^2 parts, similar to how the BDDs are split up in Figure 3.3. For ease of notation we denote $m-1 = m'$:

$$S = \begin{pmatrix} S_{|0} \\ S_{|1} \\ \vdots \\ S_{|m'} \end{pmatrix} \quad R = \begin{pmatrix} R_{|00} & R_{|01} & \cdots & R_{|0,m'} \\ R_{|10} & R_{|11} & \cdots & R_{|1,m'} \\ \vdots & \vdots & \ddots & \vdots \\ R_{|m',0} & R_{|m',1} & \cdots & R_{|m',m'} \end{pmatrix}$$

Where the BDD algorithm iterates over four transition relations $R_{|ij}$, the MDD algorithm simply iterates over all m^2 relations $R_{|ij}$. When $i = j$, the transition relation $R_{|ij}$ contains transitions for which the first variable stays the same, and we can call $\text{REACHMDD}(S_{|i}, R_{|ij})$. For all cases where $i \neq j$ we use $\text{IMAGE}(S_{|i}, R_{|ij})$ instead.

We note that REACHBDD appears to not generalize so well to MDDs, in the following sense: for BDDs, Algorithm 3.2 has two REACHBDD calls and two IMAGE calls inside the loop. However, for MDDs we get $O(m)$ REACHMDD calls and $O(m^2)$ IMAGE

3.3. Decision diagram operation for reachability

Algorithm 3.4: An MDD operation for computing reachability. Cache lookup and insert for dynamic programming after lines 5 and 11 are omitted.

```

1  def REACHMDD( $S, R$ ) : ▷ For MDDs  $S, R$  on  $n, 2n$  variables.
2      if  $S = 0$  then return 0
3      if  $R = 0$  then return  $S$ 
4      if  $S = 1$  then return 1
5      if  $R = 1 \wedge S \neq 0$  then return 1
6      while  $S$  did not converge do
7          for  $i, j \in \mathcal{D}$  do
8              if  $i = j$  then
9                   $S[i] \leftarrow \text{REACHMDD}(S[i], R[ij])$ 
10             else then
11                  $S[j] \leftarrow \text{UNION}(S[j], \text{IMAGE}(S[i], R[ij]))$ 
12  return  $\text{MAKENODE}(\text{var}(S), S[0], \dots, S[m-1])$ 

```

calls every loop iteration. Thus, for MDDs, a larger part of the computation is no longer handled by recursive calls, but instead by image computations.

3.3.4 Correctness

In this section, we sketch the correctness proofs for both REACHMDD (Thm. 3.1) and REACHBDD (Cor. 3.1). A complete proof can be found in Section 3.6.

Theorem 3.1. *Given two MDDs S and R with n and $2n$ variables respectively, the algorithm $\text{REACHMDD}(S, R)$ (Alg. 3.4) computes all the reachable states $S.R^*$.*

Proof Sketch. The correctness of REACHMDD can be shown by means of algorithm transformation from breadth-first search. The algorithm for BFS (given below) directly follows from the definition $S.R^* = \bigcup_{k=0}^{\infty} S.R^k$, as shown by the Knaster-Tarski theorem [225].

```

1  while  $S$  did not converge do
2       $S \leftarrow \text{UNION}(S, \text{IMAGE}(S, R))$ 
3  return  $S$ 

```

The two main steps in the transformation are as follows: first, using the Shannon decomposition, the computation of $\text{IMAGE}(S, R)$ can be split up into $\text{IMAGE}(S[i], R[ij])$ calls for all $i, j \in \mathcal{D}$, the results of which can be combined with a MAKENODE function

as on line 12 of Algorithm 3.4. Second, since we are ultimately computing $S.R^*$, the calls $\text{IMAGE}(S[i], R[ij])$ can be replaced with $\text{REACHMDD}(S[i], R[ij])$ when $i = j$. The algorithm REACHMDD follows directly from these two steps. $\square$

Corollary 3.1. *Given two BDDs S and R , $\text{REACHBDD}(S, R)$ (Algorithm 3.2) computes all the reachable states $S.R^*$.*

Proof Sketch. The correctness of REACHBDD can be shown from Theorem 3.1 by taking an MDD with $\mathcal{D} = \{0, 1\}$. $\square$

3.4 Empirical evaluation

3.4.1 Experimental setup

We implemented the new algorithms in the decision diagram package Sylvan [76, 79, 81], using the task-based scheduling as described in Section 3.3.1. Instead of MDDs, Sylvan supports LDDs (see Section 3.2.2), which can be seen as a particular implementation of MDDs. For our experiments, we compare against the saturation procedure for BDDs and LDDs as implemented in Sylvan [78].¹

We use a number of existing benchmark sets to evaluate the performance of our algorithms. Specifically, we use the BEEM benchmark set, consisting of 300 instances of models in the DVE language, a benchmark set of over 300 Petri nets from the Model Checking Contest 2016 (MCC 2016) [140], and a small set of Promela models, compiled for the SpinS extension of LTSmin [25]. An overview is given in Table 3.1.

Table 3.1: Overview of used benchmarks.

Source	Type	#
BEEM [179]	DVE	300
MCC'16 [140]	Petri-nets	357
SPINS [25]	Promela	35

For these benchmarks we use the same experimental setup as used in [78]: we first use the model checker LTSmin [133] to generate BDDs and LDDs for the (partial) transition relations and initial set of states, which are exported to `.bdd` and `.ldd` files. In LTSmin, the partial transition relations are “learned” on-the-fly, during the

¹The implementation of our algorithms, along with the repeatable experiments can be found here: <https://github.com/sebastiaanbrand/reach-dd>.

3.4. Empirical evaluation

exploration [133], as opposed to directly building the transition relations from a modeling language like NuXMV [44]. The variables in these BDDs and LDDs have been reordered by LTSmin with Boost’s Sloan algorithm, since this reordering strategy has shown good results for saturation [158, 11, 12].

Unlike saturation, the REACHDD algorithms acts on a single monolithic transition relation. Therefore, as a preprocessing step for the REACHDD algorithms only, we merge the partial relations from LTSmin into a monolithic transition relation over all variables. The runtime of merging the partial relations is included in the total run times reported in Figure 3.4. This approach is slightly disadvantaging REACHDD because we could also change the setup to generate monolithic relations directly (as we do for Petri nets in the comparison with ITS-tools in Section 3.4.2), but using the setup from [78] allows us to make a direct comparison with parallel saturation from [78].

We limit the runtime of each algorithm to 10 minutes. The sequential benchmarks were performed on a machine with an AMD Ryzen 7 5800x CPU and 64 GB of available memory. The parallel benchmarks on a machine with 4 Intel Xeon E7-8890 v4 CPUs with 24 physical cores each (96 in total) and 2 TB of memory. Aside from the experiments that test parallelism specifically, all reported runtimes are for a single core.

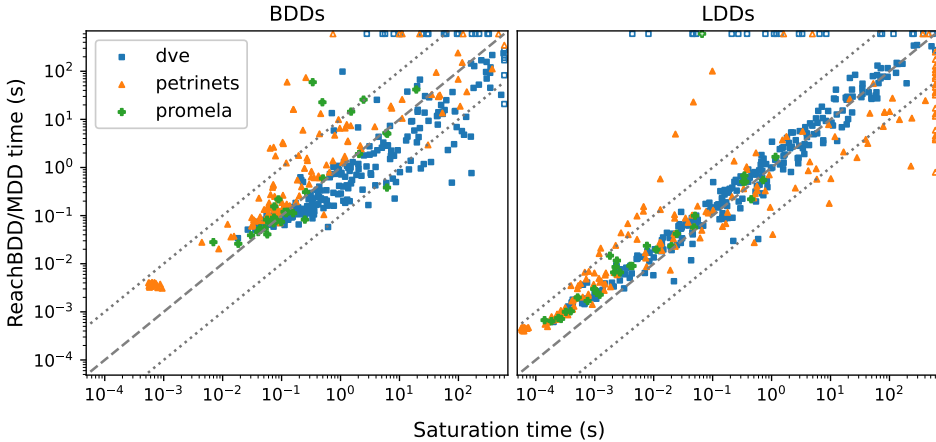


Figure 3.4: The runtime of finding all reachable states with BDDs (left) and LDDs (right) using REACHBDD and REACHMDD versus saturation. The dashed line indicates equal performance, while the dotted lines indicate a difference of factor $\times 10$ and $\times \frac{1}{10}$. Open markers indicate timeouts.

3.4.2 Results

Comparison with saturation. A comparison between the runtimes of saturation and REACHDD is given in Figure 3.4. In this discussion we differentiate between smaller models (runtimes ≤ 1 sec) and larger models (runtimes ≥ 1 sec). For BDDs, REACHDD outperforms saturation on a large number of bigger DVE models, but encounters timeouts as well. For LDDs, REACHDD appears competitive with saturation on DVE and Promela models, while the trend for the larger Petri net models shows REACHDD outperforming saturation by up to a factor 100. For both BDDs and LDDs saturation is often faster on the smaller instances. This is in part due to the fact that the relative overhead of merging the partial relations is greater for smaller instances.

Locality. As discussed in Section 3.2.6, saturation is known to work well on transition systems where the partial transition relations exhibit locality. To get insight into how locality affects our new algorithms relative to saturation, we define the *average relative bandwidth* as a metric for locality: For k partial relations $R_1, \dots, R_k$, sorted in an ascending order based on their first variable, we can define a $k \times k$ matrix M with entries m_{ij} such that $m_{ij} = 1$ if R_i and R_j share at least one variable, and 0 otherwise. We define the bandwidth of a row $m_{i,*}$ as the distance between the first and the last non-zero element in this row. The average bandwidth is then simply the average of the bandwidths of all the rows $m_{i,*}$. The average *relative* bandwidth is the average bandwidth divided by k . Note that this $k \times k$ matrix is different from (although related to) the $k \times n$ variable matrix shown in Section 3.2.6. The matrix M and the locality metric derived from it are independent of the variable order in the decision diagram.

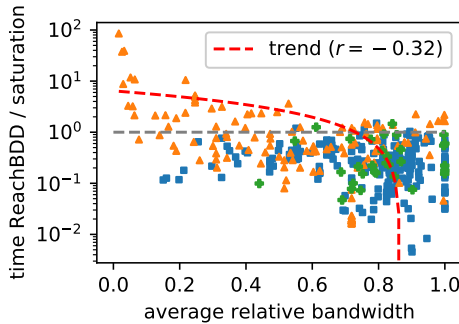


Figure 3.5: The effect of locality on the relative performance of REACHBDD. For REACHMDD, the correlation $r = -0.11$.

3.4. Empirical evaluation

Plotting the runtime of REACHBDD divided by the runtime of saturation against this average relative bandwidth (Figure 3.5), we see that there is a negative correlation. Although not extremely strong, this correlation shows that the benchmarks on which saturation outperforms our algorithms are predominantly the instances where the partial relations are relatively local, while on instances with less locality our algorithms have a greater edge over saturation.

3

Parallelism. Figure 3.6 shows the speedups obtained by REACHBDDPAR and the parallelized version of saturation from [78] on 16 and 64 cores. Table 3.2 gives the 95th, 99th and 99.5th percentile of the speedups. We see that for the 16 core runs, REACHBDDPAR is able to keep up with [78], although falling slightly behind. For the 64 core runs, while REACHBDDPAR falls behind [78] on the 99th percentile, it is still able to achieve a $\times 17$ speedup on in the 99.5th percentile, compared to [78]’s $\times 22$.

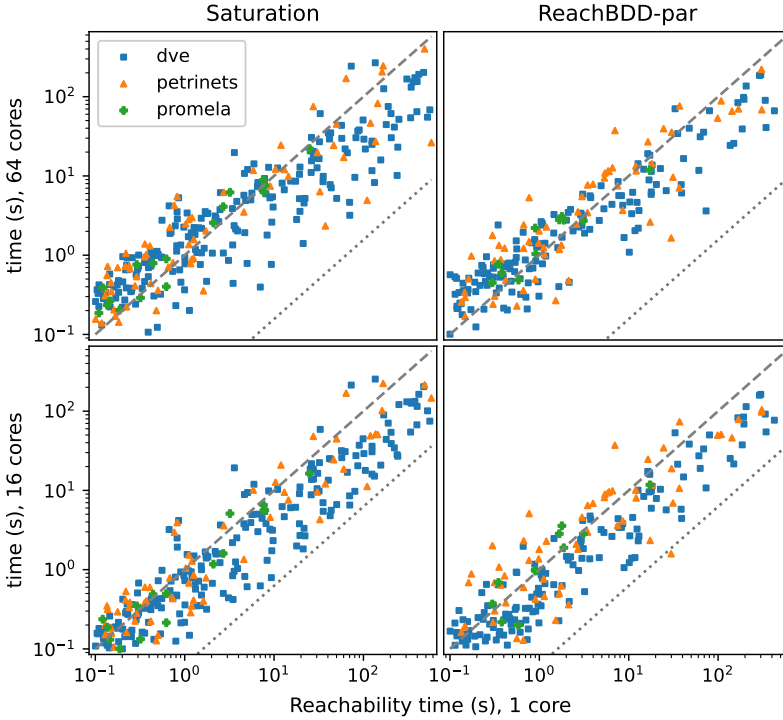


Figure 3.6: Parallel speedup for saturation (left column) and REACHBDDPAR (right column). The dotted diagonal lines indicate a speedup of a factor 16 (bottom row) and 64 (top row) relative to the single-core performance.

Table 3.2: Parallel speedups.

algorithm	cores	speedup		
		P ₉₅	P ₉₉	P _{99.5}
saturation [78]	16	×8.1	×11	×11
REACHBDDPAR	16	×6.6	×8.3	×8.8
saturation [78]	64	×8.7	×22	×22
REACHBDDPAR	64	×5.6	×9.1	×17

Comparison against ITS-tools. We also briefly compare how REACHDD performs against a state-of-the-art model checking tool. For this, we pick ITS-tools [228], the overall highest scoring tool in the Model Checking Contest 2021 [139]. Since here we compare against a different tool, as opposed to comparing algorithms within the same package, we need to slightly extend our setup. We add two things: first, we create a small program `pnml-encode` that builds the decision diagrams of the transition relations directly from the Petri net files. Second, we extend our LDDs with a (much simpler) version of homomorphisms that are also used in the set decision diagrams (SDDs) [57] that are a part of ITS-tools.

The results are given in Figure 3.7. While ITS-tools outperforms REACHMDD on average, there is a significant number of instances where ITS-tools gives timeouts and REACHDD does not. Including these timeouts, REACHDD is faster than ITS-tools on 29% of instances. This suggests that REACHDD could be useful as a complementary method in an ensemble tool, where a different method can be tried if the first one times out.

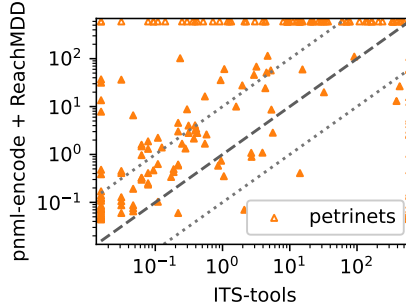


Figure 3.7: Comparison of REACHMDD against ITS-tools.

3.5 Conclusion

Summary. We presented two new reachability operations for decision diagrams: REACHBDD and REACHMDD. In contrast to other approaches, like saturation, these operations can act on a single monolithic transition relation. Similar to saturation, these new algorithms build the decision diagram for the reachable states (at least partially) bottom-up. One advantage of these operations is their simplicity. This simplicity allows us for example to more easily parallelize the decision diagram operations, as demonstrated for REACHBDDPAR. Empirical evaluation of REACHBDD and REACHMDD on a large number of benchmark sets shows that the new operations are competitive with saturation, and tend to outperform saturation on larger instances. Additionally, we find that REACHBDDPAR's peak parallel performance does not fall far behind that of saturation. Finally, our empirical results show that the REACHDD operations can solve 29% of instances faster than ITS-tools, which indicates REACHDD can be useful as a complementary algorithm.

Future work. Saturation still outperforms our algorithms on a number of instances. Many of these instances have a lot of locality, which is exactly the regime where saturation is expected to do very well. With further investigation, REACHBDD and REACHMDD could potentially be modified to perform better on such instances. The current bottleneck, as illustrated by our analysis, is the reliance on the standard IMAGE operation. Further integration of both operations could perhaps yield improvement.

3.6 Appendix

This section contains the correctness proofs for REACHBDD and REACHMDD. Since REACHBDD can be seen as a special case of REACHMDD where $\mathcal{D} = \{0, 1\}$, we first prove the correctness of REACHMDD in Theorem 3.1 and then show that this also holds for REACHBDD.

Normally the correctness of a decision diagram operation which implements a logical operation $\circ$ is derived by expanding the arguments via the Shannon decomposition and rewriting the logical formula for $\circ$ (see for example [39; Eq. 1]). However, to avoid introducing notation for the reflexive-transitive closure ($*$) in logical formulas, we instead choose to present a more intuitive proof on an algorithmic level.

Theorem 3.1. *Given two MDDs S and R with n and $2n$ variables respectively, the algorithm REACHMDD(S, R) (Alg. 3.4) computes all the reachable states $S.R^*$.*

Proof. Here we will show that REACHMDD can be derived by means of algorithm transformation from breadth-first search (BFS), which in turn follows directly from the definition of $S.R^*$.

Following the definition $S.R^* = \bigcup_{k=0}^{\infty} S.R^k$, as shown by the Knaster-Tarski theorem [225], the reachable states can be computed with the algorithm below.

```

while  $S$  did not converge do
   $S \leftarrow \text{UNION}(S, \text{IMAGE}(S, R))$ 
return  $S$ 

```

The IMAGE call can be split into calls on pairs $(S[i], R[ij])$, $\forall i, j \in \mathcal{D}$. This is the Shannon decomposition on an algorithm level.

```

while  $S$  did not converge do
  for  $i, j \in \mathcal{D}$  do
     $S[j] \leftarrow \text{UNION}(S[j], \text{IMAGE}(S[i], R[ij]))$ 
return  $\text{MAKENODE}(\text{var}(S), S[0], \dots, S[|\mathcal{D}| - 1])$ 

```

Because the loop is executed until S converges, the algorithm above is equivalent to the following algorithm.

```

while  $S$  did not converge do
  for  $i, j \in \mathcal{D}$  do
    if  $i = j$  then
      while  $S[i]$  not converged do
         $S[i] \leftarrow \text{UNION}(S[i], \text{IMAGE}(S[i], R[ii]))$ 
      else then
         $S[j] \leftarrow \text{UNION}(S[j], \text{IMAGE}(S[i], R[ij]))$ 
return  $\text{MAKENODE}(\text{var}(S), S[0], \dots, S[|\mathcal{D}| - 1])$ 

```

Now we can identify the inner while loop as a recursive call to REACHMDD based on its resemblance to the initial BFS procedure. In the following algorithm, we therefore replace the inner loop with a recursive call, inductively solving the problem on MDDs $S[i]$ and $R[ij]$ with $n - 1$ and $2(n - 1)$ variables respectively.

```

while  $S$  did not converge do
  for  $i, j \in \mathcal{D}$  do
    if  $i = j$  then
       $S[i] \leftarrow \text{REACHMDD}(S[i], R[ij])$ 
    else then
       $S[j] \leftarrow \text{UNION}(S[j], \text{IMAGE}(S[i], R[ij]))$ 
  return  $\text{MAKENODE}(\text{var}(S), S[0], \dots, S[|\mathcal{D}| - 1])$ 

```

Finally, we treat the terminal cases of REACHMDD. The following are cases where were at least one of S and R is a leaf node.

$$\text{REACHMDD}(S, R) = S.R^* = \begin{cases} 0 & \text{if } S = 0 \\ S & \text{if } R = 0 \\ 1 & \text{if } S = 1 \\ 1 & \text{if } R = 1 \text{ and } S \neq 0 \end{cases}$$

The terminal cases are derived as follows. If the set of initial states is empty ($S = 0$) then the set of reachable states is also empty. If the transition relation is empty ($R = 0$) then, because we consider the *reflexive*-transitive closure, the set of reachable states is the set of initial states. If the set of initial states contains all states ($S = 1$) then the set of reachable states also contains all states (because of reflexivity this holds even when $R = 0$). Finally, if the transition relation contains all transitions from any state to any other state ($R = 1$) then, if S is not empty, all states are reachable. $\square$

Corollary 3.1. *Given two BDDs S and R , $\text{REACHBDD}(S, R)$ (Algorithm 3.2) computes all the reachable states $S.R^*$.*

Proof. Filling in $\mathcal{D} = \{0, 1\}$ in REACHMDD and expanding the individual loop iteration we obtain the following.

```

while  $S$  did not converge do
   $S[0] \leftarrow \text{REACHBDD}(S[0], R[00])$ 
   $S[1] \leftarrow \text{UNION}(S[1], \text{IMAGE}(S[0], R[01]))$ 
   $S[0] \leftarrow \text{UNION}(S[0], \text{IMAGE}(S[1], R[10]))$ 
   $S[1] \leftarrow \text{REACHBDD}(S[1], R[11])$ 
  return  $\text{MAKENODE}(\text{var}(S), S[0], S[1])$ 

```

Since each step only adds elements to S , and the loop runs until S converges, the order of steps within the loop can be freely permuted, which gives us Algorithm 3.2. $\square$

Chapter 4

Q-Sylvan: a parallel decision diagram package for quantum computing

As physical realizations of quantum computers move closer towards practical applications, the need for tools to analyze and verify quantum algorithms grows. Among the algorithms and data structures used to tackle such problems, decision diagrams (DDs) have shown much success. However, an obstacle with DDs is their efficient parallelization, and while parallel speedups have been obtained for DDs used in classical applications, attempts to parallelize operations for quantum-specific DDs have yielded only limited success. In this chapter, we present an efficient implementation of parallel edge-valued DDs that makes use of fine-grained task parallelism and lock-free hash tables. Additionally, using these DDs, we implement two use cases: simulation and equivalence checking of quantum circuits. In our empirical evaluation, we find that our tool, Q-Sylvan, shows a single-core performance that is competitive with the state-of-the-art quantum DD tool MQT DDSIM on large instances, and moreover achieves parallel speedups of up to $\times 18$ on 64 cores.

This chapter has been published as: Sebastiaan Brand and Alfons Laarman. “Q-Sylvan: a parallel decision diagram package for quantum computing”. In: *International Symposium on Automated Technology for Verification and Analysis*. 2025.

4.1 Introduction

Quantum computing is an emerging technology that aims to provide computational speedups on problems in areas such as cryptography [213], finance [175], and optimization [108], as well as on problems in quantum physics [123] and quantum chemistry [43]. As the number of available qubits on quantum chips grows, and the field moves closer towards practical applications, so grows the need for tools to analyze and verify quantum circuits.

In this chapter, we focus on two tasks specifically: simulation and equivalence checking of quantum circuits. Although these are computationally hard problems [1, 125, 128, 224], much like problems in classical verification, heuristic algorithms and data structures can significantly help with the scalability of these tasks. One particular data structure that has seen a lot of success in the classical evaluation of quantum circuits is decision diagrams (DDs). While different types of decision diagrams have been proposed in the context of quantum computing [231, 195, 160, 242, 230, 236, 232, 214], edge-valued decision diagrams (EVDDs) [222] with complex edge values [160, 242] have been shown to be very useful in practice.

However, a particular obstacle with decision diagrams is that computations on them are hard to efficiently parallelize [77]. And although the decision diagram library Sylvan [81] has shown good speedups for DDs without edge values, the storage and handling of floating-point values in EVDDs adds additional complications. This has thus far yielded limited scalability of EVDDs for quantum computing applications, with speedups of up to $\times 3$ using 32 cores on the simulation of random circuits [120] and $\times 2$ – 3 using 16 cores on Grover circuits [148].

In this chapter, we present an efficient implementation of parallel EVDDs, building on the parallel decision diagram library Sylvan [81], which before now only supported decision diagrams without edge values, and we address several obstacles regarding floating-point values in DDs. We also provide implementations for two use cases: quantum circuit simulation and quantum circuit equivalence checking, both supporting the full set of standard quantum gates of Open QASM 2.0 [58]. We evaluate the performance of our resulting tool, Q-Sylvan,¹ against several other recent tools on a large set of benchmarks. We show that Q-Sylvan is capable of obtaining speedups of up to $\times 7$ on 8 cores and up to $\times 18$ on 64 cores, while also providing a single-core performance that is competitive with the state-of-the-art DD-based quantum circuit simulator MQT DDSIM [242] on large instances. Our contributions are summarized

¹Available online at <https://github.com/System-Verification-Lab/Q-Sylvan>.

as follows:

1. An efficient implementation of parallel EVDDs.
2. The implementation of two use cases using these EVDDs: simulation and equivalence checking of quantum circuits.
3. An evaluation against state-of-the-art tools on a large benchmark set.

4.2 Quantum computing and EVDDs

In this section, we briefly explain the necessary basics of quantum computing and the role of decision diagrams in this context.

While there are many interesting intricacies in the mathematics of quantum computing, for the purposes of this chapter, a high-level description of *quantum states* and *quantum gates* will be sufficient. The state of n quantum bits (qubits) can be described by a vector in $\mathbb{C}^{2^n}$, and can also be seen as a function $\psi : \{0, 1\}^n \rightarrow \mathbb{C}$. Similarly, a quantum gate (i.e. a linear transformation that maps states to states) that acts on n qubits can be described by a matrix in $\mathbb{C}^{2^n \times 2^n}$ (or a function $U : \{0, 1\}^{2^n} \rightarrow \mathbb{C}$). The effect of a gate on a state can be computed through matrix-vector multiplication. A *quantum circuit* is a sequence of quantum gates, and can be classically simulated through repeated matrix-vector multiplication.

To store these exponentially large vectors and matrices compactly, people have turned to decision diagrams, a data structure for which it is well known how to compute matrix multiplications [92]. In this chapter we use edge-valued DDs [222], which, in the context of quantum computing, have also been called quantum multi-valued DDs (QMDDs) [160, 242].

An EVDD is a rooted, directed, acyclic graph whose edges have values associated with them. For our purposes these values are complex numbers. Each node v in an EVDD has a variable $\text{var}(v) = x_i$, and two outgoing edges. For an EVDD that encodes an n -qubit state, we use the variables $\{x_0, \dots, x_{n-1}\}$. EVDDs are always ordered, i.e. on every path the variables are encountered in the same order $x_0 \prec x_1 \prec \dots \prec x_{n-1}$, although variables may be skipped. Every path through an EVDD corresponds to a single entry in the vector it encodes, with that value being equal to the product of the edge values on that path. As an example, the EVDD in Figure 4.1 encodes the vector

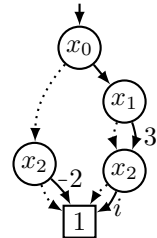


Figure 4.1:
EVDD example.

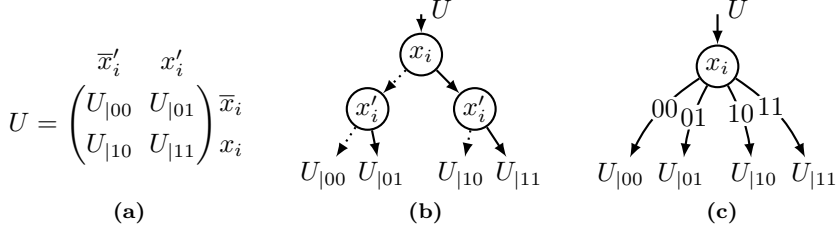


Figure 4.2: A $2^n \times 2^n$ matrix (a) can be recursively encoded in a DD with two children per node (b), as in EVDD [222], or four children per node (c), as in QMDD [160].

$\vec{\psi} = (1 \ -2 \ 1 \ -2 \ 1 \ i \ 3 \ 3i)^\top$. The value $\psi(110)$, for example, can be read from the EVDD by following the 1 (solid) edges for x_0 and x_1 , and the 0 (dashed) edge for x_2 , obtaining $1 \cdot 3 \cdot 1 = 3$. Algorithms on DDs are typically defined recursively. Two examples are given in Algorithms 4.1 and 4.2.

Figure 4.2 shows how DDs represent matrices. EVDDs do so by reinterpreting the matrix as a function $f(\vec{x}, \vec{x}')$, where $\vec{x}$ indexes rows and $\vec{x}'$ columns (Fig. 4.2a). The variables in $\vec{x}$ and $\vec{x}'$ are then interleaved to enable recursive descent on matrix quadrants (Fig. 4.2b). In contrast, a QMDD [160] is refined for matrices, as Figure 4.2c shows. Because a QMDD can be translated into an EVDD in linear time and vice versa, they are effectively the same data structure [87].

Other types of DDs have also been used within quantum computing, such as multi-terminal binary DDs (MTBDDs) [231, 195], context-free-language ordered BDDs (CFLOBDDs) [214], local invertible map DDs (LIMDDs) [232], and local invertible map tensor DDs (LimTDDs) [122]. EVDDs appear to strike a good balance between compression and practical efficiency: compared to MTBDDs, they benefit from better compression [242], while compared to LIMDDs and CFLOBDDs, they are practically more efficient on a wider array of circuits [214, 233].

4.3 EVDD implementation

Since our goal is to provide an efficient implementation of parallel EVDDs, we choose to implement these on top of the parallel decision diagram library Sylvan [81]. Sylvan contains implementations of several different types of decision diagrams, among which are multi-terminal binary decision diagrams (MTBDDs), list decision diagrams (LDDs), and zero-suppressed decision diagrams (ZDDs). However, Sylvan does not yet support any decision diagrams with edge values.

Algorithm 4.1: Vector addition with EVDDs. Here $\text{val}(A)$ denotes the edge value of the root edge of A , and $A[0]$ ($A[1]$) denotes the 0 (1) child of A .

```

1 def PLUS(EVDD  $A$ , EVDD  $B$ ) : ▷ assuming  $\text{var}(A) = \text{var}(B)$ 
2   if  $A$  and  $B$  are terminals then
3     return  $\text{val}(A) + \text{val}(B)$ 
4   if  $R \leftarrow \text{cache}[\text{PLUS}, A, B]$  then return  $R$  ▷ Memoization to avoid
▷ recomputing
5    $R_0 \leftarrow \text{PLUS}(\text{val}(A) \cdot A[0], \text{val}(B) \cdot B[0])$ 
6    $R_1 \leftarrow \text{PLUS}(\text{val}(A) \cdot A[1], \text{val}(B) \cdot B[1])$ 
7    $R \leftarrow \text{MAKENODE}(\text{var}(A), R_0, R_1)$ 
8    $\text{cache}[\text{PLUS}, A, B] \leftarrow R$ 
9   return  $R$ 

```

The diagram shows the addition of two EVDDs, A and B, to produce a result EVDD R. EVDD A has a root node x_i with two children, $A[0]$ and $A[1]$. EVDD B has a root node x_i with two children, $B[0]$ and $B[1]$. The result EVDD R has a root node x_i with two children, $A[0]+B[0]$ and $A[1]+B[1]$. The addition is represented by a plus sign between the two input EVDDs and an equals sign followed by the result EVDD.

Algorithm 4.2: Matrix-vector multiplication with EVDDs.

```

1 def MULTIPLY(EVDD  $M$ , EVDD  $V$ ) : ▷ assuming  $\text{var}(M) = \text{var}(V)$ 
2   if  $M$  and  $V$  are terminals then
3     return  $\text{val}(M) \cdot \text{val}(V)$ 
4   if  $R \leftarrow \text{cache}[\text{MULTIPLY}, M, V]$  then return  $R$ 
5    $R_{00} \leftarrow \text{MULTIPLY}(\text{val}(M[0]) \cdot M[00], V[0])$ 
6    $R_{01} \leftarrow \text{MULTIPLY}(\text{val}(M[0]) \cdot M[01], V[1])$ 
7    $R_{10} \leftarrow \text{MULTIPLY}(\text{val}(M[1]) \cdot M[10], V[0])$ 
8    $R_{11} \leftarrow \text{MULTIPLY}(\text{val}(M[1]) \cdot M[11], V[1])$ 
9    $R_0 \leftarrow \text{MAKENODE}(\text{var}(v), R_{00}, R_{10})$ 
10   $R_1 \leftarrow \text{MAKENODE}(\text{var}(v), R_{01}, R_{11})$ 
11   $R \leftarrow \text{PLUS}(R_0, R_1)$  ▷ using Algorithm 4.1
12   $\text{cache}[\text{MULTIPLY}, M, V] \leftarrow R$ 
13  return  $R \cdot \text{val}(M) \cdot \text{val}(V)$ 

```

4.3. EVDD implementation

Floating-point equality. A prominent hurdle in the implementation of DDs with real (or complex) edge values is handling floating-point values. Floating-point computations infamously do not always yield exact solutions, e.g. $0.1 + 0.2$ might give 0.30000000000000004 , and so using exact floating-point equality would often prevent the merging of nodes, which in turn prevents the decision diagrams from staying compact. Therefore, we would like to consider edges with edge values that are very close to be equivalent. Specifically, we consider two floating-point values a, b equivalent if $|a - b| < \delta$, with $\delta = 10^{-14}$, and we consider two complex values equivalent if the same inequality holds for both their real and imaginary components. Although this can introduce numerical errors, setting $\delta = 0$ has been shown empirically to allow for almost no node-merging [172], and thus setting δ to a small but non-zero value appears to be a necessary evil of decision diagrams with floating-point edge values. Alternative representations of real or complex values, such as algebraic representations, have their own shortcomings. For example, the algebraic representation proposed in [172] only works for a limited gate set (specifically $\{H, T, CX\}$). And even though this gate set is technically universal for quantum computing, more general single-qubit rotation gates, which appear in ubiquitous quantum algorithms such as quantum approximate optimization algorithms (QAOA), variational quantum eigensolvers (VQE), and the quantum Fourier transform (QFT), can only be approximated by it. Specifically, an m -gate quantum circuit that contains general single-qubit rotation gates can be approximated up to an error of ε by a circuit with $O(m \log(m/\varepsilon))$ gates [136, 73, 209].

Algorithm 4.3: Find-or-put a complex value c in a hash table. δ is a configurable variable with a default value of 10^{-14} .

```

1 def FINDORPUT( $c$ ) :
2    $d.r \leftarrow \text{ROUND}(c.r, \varepsilon)$ 
3    $d.i \leftarrow \text{ROUND}(c.i, \varepsilon)$                                 ▷ Round real and imaginary components
4    $\text{index} \leftarrow \text{HASH}(d)$                                     ▷ Compute hash based on rounded value
5   while not found or put do
6     if CAS( $\text{table}[\text{index}]$ , empty,  $c$ ) then                      ▷ Store unrounded value with
7        $\text{return index}$                                            ▷ atomic compare-and-swap
8     else then
9        $v \leftarrow \text{table}[\text{index}]$ 
10      if  $|c.r - v.r| < \varepsilon$  and  $|c.i - v.i| < \varepsilon$  then          ▷ If  $\varepsilon$ -close, return
11         $\text{return index}$                                            ▷ existing value
12      else then
13         $\text{index} \leftarrow \text{index} + 1$ 

```

Storing edge values. In order to efficiently recognize equivalent edge values, we store them in a hash table. To facilitate efficient concurrent access we use a hash table with atomic compare-and-swap operations based on [143]. As mentioned above, due to the imperfection of floating-point arithmetic we cannot simply use floating-point equality to determine if two edge values are equivalent. Instead, when storing a complex edge value c we hash a rounded version of c to compute a bucket index, in which we store the (non-rounded) value of c . If the bucket is already occupied we compare the absolute values of the real and imaginary components to check if the stored value is equivalent to the value we want to store. A pseudocode description of this procedure is given in Algorithm 4.3.

Normalizing edge values. In order to recognize equivalent nodes, they must be stored in a canonical form. To bring an arbitrary EVDD node into a canonical form, the edge values need to be normalized (Figs. 4.3a and 4.3b). Four different methods, specified in Figure 4.3c, have been implemented. Out of these the first three achieve canonicity by setting one of the edge values to 1, while the fourth is based on the way quantum states are normalized in quantum computing in general. These methods have been tested on the MQT Bench benchmark set [181] (discussed in Sec. 4.5) for circuits up to 30 qubits. For each run that terminated within the given timeout of 10 minutes we computed the ℓ^2 -norm (the sum of absolute values squared) of the output and checked if this equals 1 ($\pm 10^{-3}$) as it should for quantum states. We find that while

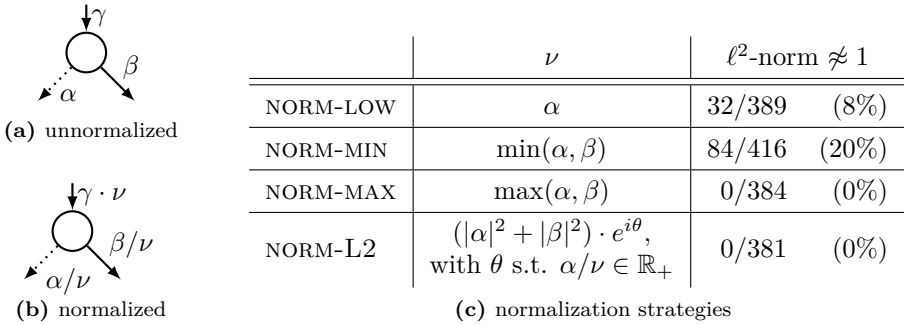


Figure 4.3: An arbitrary tuple of edge values $\langle \alpha, \beta, \gamma \rangle$ is not generally in a canonical form, e.g. $\langle 2, 6, 1 \rangle \equiv \langle 1, 3, 2 \rangle$. Such a tuple can be normalized by dividing α and β , and multiplying γ , by some choice of ν . In the example, normalizing both tuples by $\nu = \alpha$ yields $\langle \frac{2}{2}, \frac{6}{2}, 1 \cdot 2 \rangle = \langle 1, 3, 2 \rangle$ and $\langle \frac{1}{1}, \frac{3}{1}, 2 \cdot 1 \rangle = \langle 1, 3, 2 \rangle$. The table in (c) shows different choices for ν , along with an empirical evaluation of the numerical errors when simulating circuits from MQT Bench [181] and checking the ℓ^2 -norm of the output.

NORM-MAX and NORM-L2 did not have any issues, NORM-LOW and NORM-MIN suffered from a significant amount of numerical errors. This can be an indication that having larger values higher up in the decision diagram can increase issues with numerical instability, and it rules out NORM-LOW and NORM-MIN as viable methods. Between the remaining strategies we find that while NORM-L2 yields smaller decision diagrams on some instances, NORM-MAX is faster on most instances. This difference is likely due to the higher complexity of NORM-L2. We therefore choose to set NORM-MAX as the default, and have also used this for the evaluation in Section 4.5.

Parallelization. Sylvan makes use of the work-stealing framework Lace [80], which can be used for both intra- and inter-operational parallelism with its SPAWN (fork) and SYNC (join) commands. Since we aim to create efficient parallel quantum DD operations, we focus here on intra-operational parallelism, i.e. parallelism within a single (recursive) DD operation. As an example, consider the EVDD algorithm for vector addition, given in Algorithm 4.1. The algorithm contains two recursive calls (lines 5 and 6) that are typically executed sequentially. With Lace, these can be parallelized as follows:

5 SPAWN(PLUS(val(A) · $A[1]$, val(B) · $B[1]$))	▷ Spawn call as task (fork)
6 $R_0 \leftarrow$ PLUS(val(A) · $A[0]$, val(B) · $B[0]$)	▷ Call directly
7 $R_1 \leftarrow$ SYNC	▷ Obtain task result (join)

The spawned tasks are queued and can be executed by another thread, while the task on line 6 is executed by the current thread. After finishing its own task, the thread waits for the result of the spawned task.

Since all threads share the same hash table that stores unique nodes, a mechanism is required to protect against race conditions. Sylvan does this using a lock-free node table with atomic compare-and-swap operations, rather than locking (parts of) the table [77]. The same mechanism is also used for the table that stores the edge values (see Algorithm 4.3).

4.4 Use cases

We now briefly describe the implementation of our two main use cases: simulation and equivalence checking of quantum circuits. Implementations of both are available as command line programs and take as input circuits in the standard quantum circuit format Open QASM 2.0 [58], with support for the full set of quantum gates defined

by Open QASM’s “qelib1.inc”. Additionally, Q-Sylvan’s C interface can also be used directly, supporting many functions for creating and manipulating vectors and matrices used in quantum computing.²

Simulating quantum circuits with DDs is straightforward: first a DD is constructed for the initial all-zero state, i.e. a 2^n -sized vector $\begin{pmatrix} 1 & 0 & 0 & \dots & 0 \end{pmatrix}^\top$, after which for every gate in the circuit the state is updated through matrix-vector multiplication (Alg. 4.2). Q-Sylvan then allows for the final state to be either output directly, or to draw samples from it through simulated quantum measurements.

Our second use case is quantum circuit equivalence checking, which is defined as follows: given two n -qubit quantum circuits $U = \{U_1, \dots, U_m\}$ and $V = \{V_1, \dots, V_\ell\}$, where U_i and V_i are the individual gates composing the circuits, are U and V represented by the same matrices up to a global factor? I.e. does there exist some $c \in \mathbb{C}$ such that $U = cV$ (or equivalently we write $U \equiv V$)? There are a variety of ways to check quantum circuit equivalence. The naive method is to compute the full $2^n \times 2^n$ sized matrices U and V through multiplication of their individual gates. However, much like computing the composition of transition relations tends to be inefficient with DDs in classical model checking [154], the DDs resulting from multiplying quantum gates together tend to be much larger than those obtained from only updating states. However, as discussed below, when cleverly choosing the order in which the gates are multiplied, good results can still be obtained.

We implement two equivalence checking algorithms: “alternating” and “Pauli”. The alternating algorithm (see Algorithm 4.4) was proposed in [41], and the idea is to first rewrite $U \equiv V$ as $UV^\dagger \equiv I$, where I is the identity matrix, $UV^\dagger = U_m \dots U_2 U_1 V_1^\dagger V_2^\dagger \dots V_\ell^\dagger$, and $V_i^\dagger = (V_i^*)^\top$ is the conjugate transpose of V_i . This product can then be computed from the inside out, e.g. if $m = \ell$ the computation order would be $(U_m \dots (U_2 (U_1 V_1^\dagger) V_2^\dagger) \dots V_\ell^\dagger)$. If (w.l.o.g.) $m \geq \ell$ the algorithm takes $\frac{m}{\ell}$ gates from U for every gate from V . The motivation for this approach is that when U and V are identical, every step of the computation yields the identity matrix, and thus the computation remains easy. When U and V are equivalent but not identical, this approach can still heuristically yield matrices that have small DD encodings. The Pauli algorithm (see Algorithm 4.5) is based on [226; Thm.1], which notes that $U \equiv V \Leftrightarrow \forall j \in \{0, \dots, n-1\} (UX_j U^\dagger = VX_j V^\dagger) \wedge (UZ_j U^\dagger = VZ_j V^\dagger)$, where X_j and Z_j special matrices (specifically tensor products of identity and Pauli matrices) that have an efficient classical description. Similar to the alternating algorithm, the terms can be computed from the inside out, e.g. compute $UX_j U^\dagger$ as

²See <https://github.com/System-Verification-Lab/Q-Sylvan#Documentation>.

4.4. Use cases

Algorithm 4.4: “Alternating” quantum circuit equivalence checking algorithm, proposed in [41].

```

1 def CHECKEQUIVALENCE( $U, V$ ) :
    ▷  $U$ :  $n$ -qubit quantum circuit with gates  $U_1, \dots, U_m$ 
    ▷  $V$ :  $n$ -qubit quantum circuit with gates  $V_1, \dots, V_\ell$ 
2    $R \leftarrow I^{\otimes n}$ 
3    $i \leftarrow 1, j \leftarrow 1$ 
4   while  $i \leq \ell$  do
5        $R \leftarrow RV_i^\dagger$ 
6       while  $j \leq i \cdot m/\ell$  and  $j \leq m$  do           ▷ Assume w.l.o.g.  $m \geq \ell$ 
7            $R \leftarrow U_j R$ 
8            $j \leftarrow j + 1$ 
9        $i \leftarrow i + 1$ 
10  return  $R \stackrel{?}{\equiv} I^{\otimes n}$ 

```

Algorithm 4.5: “Pauli” quantum circuit equivalence checking algorithm.

```

1 def CHECKEQUIVALENCE( $U, V$ ) :
    ▷  $U$ :  $n$ -qubit quantum circuit with gates  $U_1, \dots, U_m$ 
    ▷  $V$ :  $n$ -qubit quantum circuit with gates  $V_1, \dots, V_\ell$ 
2   for  $P \in \{X, Z\}$  do
3       for  $j \in \{0, 1, \dots, n-1\}$  do
4            $U' \leftarrow UP_j U^\dagger$                        ▷ using alg. on line 9
5            $V' \leftarrow VP_j V^\dagger$                      ▷ using alg. on line 9
6           if  $U' \neq V'$  then
7               return false
8   return true

9 def COMPUTEUPUDAG( $U, P_j$ ) :
    ▷  $U$ :  $n$ -qubit quantum circuit with gates  $U_1, \dots, U_m$ 
    ▷  $P_j$ : Pauli matrix  $X_j$  or  $Z_j$ 
10   $R \leftarrow \text{CREATEEVDD}(P)$                              ▷ create EVDD  $I_0 \otimes \dots \otimes P_i \otimes \dots \otimes I_{n-1}$ 
11  for  $k = 1$  to  $m$  do
12       $M_k \leftarrow \text{CREATEEVDD}(U_k)$ 
13       $M_k^\dagger \leftarrow \text{CREATEEVDD}(U_k^\dagger)$ 
14       $R \leftarrow M_k R M_k^\dagger$                          ▷ computed with EVDD matrix mult.
15  return  $R$ 

```

$(U_m \dots (U_2(U_1 X_j U_1^\dagger) U_2^\dagger) \dots U_m^\dagger)$. The motivation here is that when U and V consist of a particular subset of quantum gates called Clifford gates this computation is provably efficient (i.e. polynomial time), and when they consist of more general gates there can still be heuristic benefits from the compression DDs provide. Ours is the first DD-based implementation of this algorithm.

4.5 Empirical evaluation

We evaluate Q-Sylvan against several state-of-the-art tools.³ The single and 8-core results were obtained on an AMD Ryzen 7 5800x CPU with 8 cores and 64 GB of memory. The 64-core results were obtained on a machine with two AMD EPYC 7601 CPUs with 32 physical cores each (64 in total) and 1 TB of memory.

4

4.5.1 Simulation

For testing simulation performance we use two sets of quantum circuits in the Open QASM 2.0 format. The first, MQT Bench [181], contains 22 types of quantum circuits for which the number of qubits can be arbitrarily scaled, as well as 6 types of non-scalable circuits with varying numbers of qubits. To obtain a greater variety of circuits, the second dataset we include is one generated by KetGPT [15], an instance of ChatGPT that was trained on the MQT Bench dataset, and consists of 1000 quantum circuits with varying numbers of qubits. The reported runtimes only include computing the final state vector. Simulating measurements given this state vector constitutes negligible overhead [242].

We first test Q-Sylvan’s single-core performance against the well-established EVDD-based quantum circuit simulator MQT DDSIM [242], shown in Figure 4.4. We find that while DDSIM outperforms Q-Sylvan on the smaller instances (presumably in part due to a more efficient initialization), on larger instances (where either tool takes ≥ 10 seconds) Q-Sylvan outperforms DDSIM on 61% of the MQT Bench circuits, and 30% of KetGPT circuits. A comparison against Quasimodo [214] (Section 4.6, Table 4.3) shows that, while CFLOBDDs can theoretically achieve much greater compactness than EVDDs, on most practical instances EVDDs outperform CFLOBDDs. We omit SliQSim [230], as it does not support this benchmark set due to a lack of rotation-gate support.

³The experimental setup, including instructions on how to reproduce the results, can be found online at <https://github.com/sebastiaanbrand/q-sylvan-benchmarks>.

4.5. Empirical evaluation

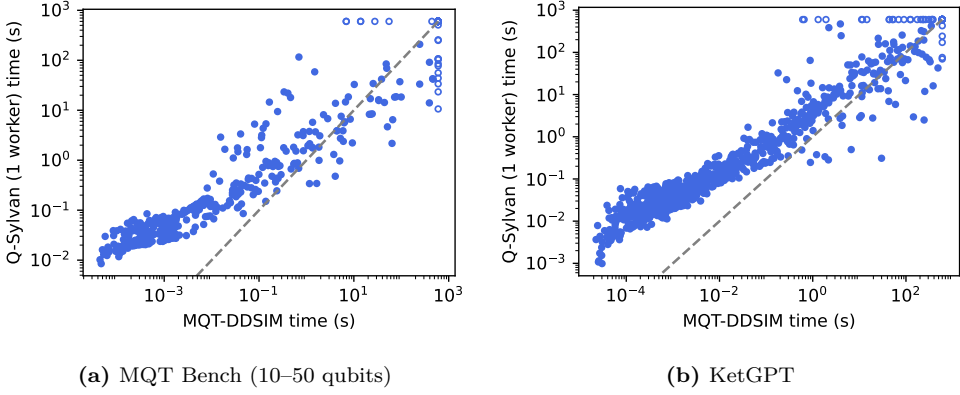


Figure 4.4: Q-Sylvan vs DDSIM. Both order variables according to the qubit ordering in the QASM file. Open markers indicate timeouts. For both plots, we verified the full state vector output of both tools up to 20 qubits. While DDSIM is faster on the smaller circuits, on circuits where either tool takes ≥ 10 seconds Q-Sylvan beats DDSIM in 61% of MQT Bench circuits, and 30% of KetGPT circuits.

Second, we evaluate Q-Sylvan’s parallel performance. Since, when comparing single-core performance, Q-Sylvan performs well on a significant fraction of larger circuits, we evaluate Q-Sylvan’s multi-core performance against its own single-core performance. As noted in [120], parallel speedups can greatly depend on the compactness of the decision diagrams. For example, it is easier to obtain speedups on a DD that is effectively a binary tree than it is to obtain speedups on a very compact DD. To evaluate this effect, we split the DDs resulting from the different benchmarks into three categories: *no sharing* (DDs which, for n qubits, have almost 2^n nodes), *high sharing* (DDs with less than $n \log n$ nodes), and *some sharing* (the DDs which fall in between these categories). Figures 4.5a and 4.5b show the distribution of decision diagram sizes for both benchmark sets. Obtaining speedups on the “no sharing” category would be the easiest, however these are instances where EVDDs offer little or no benefit. On the other hand, obtaining speedups on the “high sharing” category would be very difficult, since the DDs are so compact that little work remains that can be done in parallel. We are therefore mostly interested in speedups in the “some sharing” category.

The full parallel performance results are shown in Figures 4.5c to 4.5f, and a summary of the speedups at different percentiles is given in Table 4.1. We find that, on the “some sharing” instances, Q-Sylvan is able to obtain speedups of up to $\times 7.2$

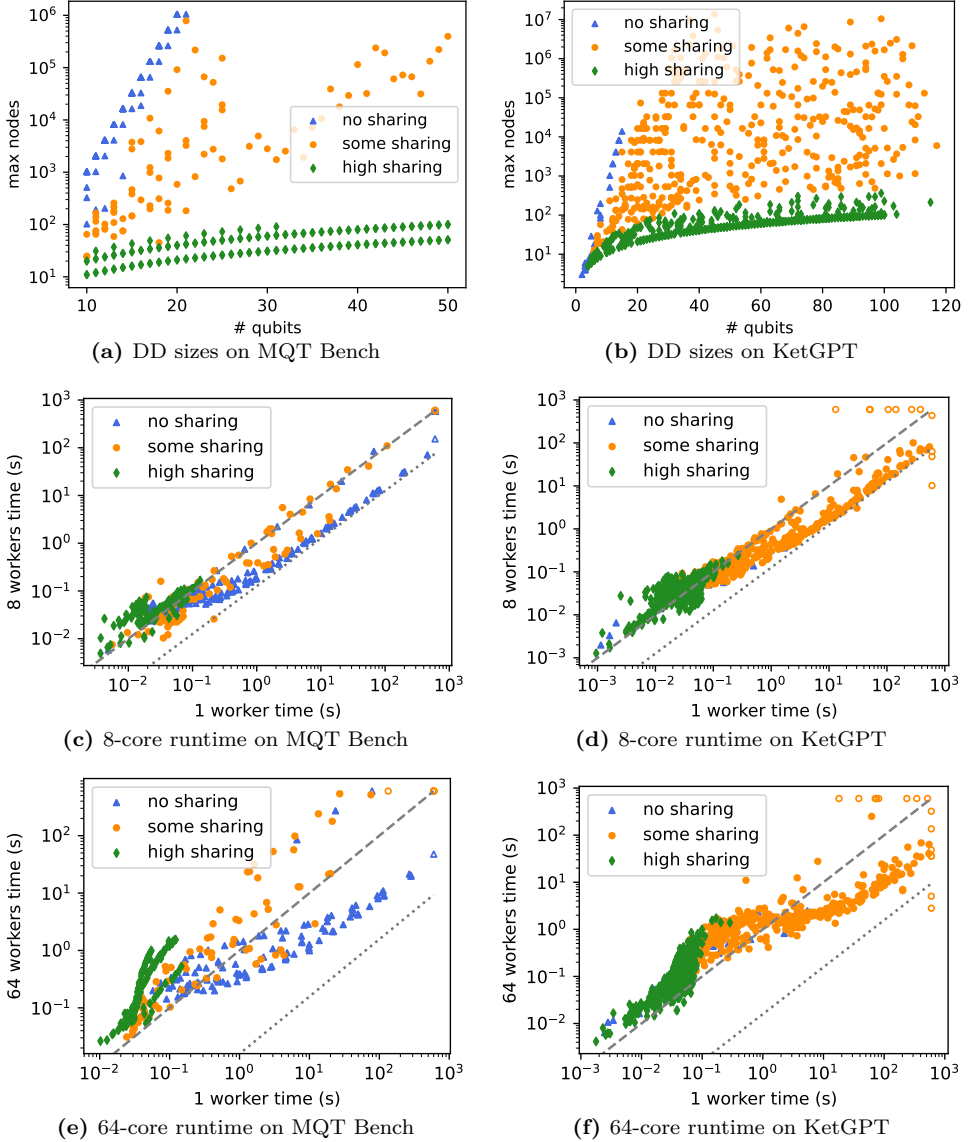


Figure 4.5: Parallel performance. Open markers indicate timeouts, dashed lines indicate equal performance, and dotted lines indicate a $\times k$ speedup for k -cores.

4.5. Empirical evaluation

Table 4.1: Speedup percentiles corresponding to the data in Figure 4.5.

dataset	sharing	8-core speedup			64-core speedup		
		P_{90}	P_{95}	P_{99}	P_{90}	P_{95}	P_{99}
MQT Bench	high sharing	$\times 1.1$	$\times 1.3$	$\times 1.7$	$\times 0.5$	$\times 0.5$	$\times 0.7$
	some sharing	$\times 3.6$	$\times 4.2$	$\times 5.8$	$\times 1.2$	$\times 2.9$	$\times 4.2$
	no sharing	$\times 6.0$	$\times 6.2$	$\times 6.4$	$\times 11$	$\times 13$	$\times 14$
KetGPT	high sharing	$\times 1.4$	$\times 1.9$	$\times 2.7$	$\times 0.7$	$\times 0.9$	$\times 1.2$
	some sharing	$\times 5.9$	$\times 6.4$	$\times 7.2$	$\times 8.3$	$\times 11$	$\times 18$
	no sharing	$\times 3.5$	$\times 4.5$	$\times 4.6$	$\times 1.0$	$\times 2.1$	$\times 2.6$

and $\times 18$ for 8 and 64 cores respectively. Unfortunately, we are unable to test directly against the two other parallel EVDD implementations [120, 148], as one [120] does not have an implementation available anymore (neither public nor private), while the other [148] hard codes two circuits but does not allow for the parallel simulation of arbitrary circuits. However, for comparison, the first [120] reported speedups of up to $\times 3$ using 32 cores on “no sharing” circuits, while the second [148] reported speedups of $\times 2$ – $\times 3$ using 16 cores on “some sharing” circuits.

On the MQT Bench dataset we find that while the 8 core speedups are promising, the 64 core results are less strong. This could in part be caused by a higher communication overhead, since the 64 cores are split between two separate CPUs. Additionally, it might be that larger benchmarks (not just in numbers of qubits, but in number of DD nodes) are required to show greater speedups for 64 cores. The underperformance of Q-Sylvan on the “no sharing” subset of the KetGPT circuits can be explained by the fact that those types of circuits are almost not present in the KetGPT dataset.

4.5.2 Equivalence checking

Next we compare our two equivalence checking implementations against two recent tools: MQT QCEC [41] and Quokka-Sharp [157]. We omit SliQEC [236] as it supports very few circuits due to a lack of rotation-gate support. MQT QCEC is a portfolio of different algorithms and makes use of both EVDDs as well a ZX-calculus, while Quokka-Sharp is based on weighted model counting. Both support multi-core computations. We use the benchmark set from [157], consisting of 78 pairs of equivalent circuits, and 146 non-equivalent pairs.

An overview of the number of completed benchmarks, as well as the speedups for 8-core computations, is given in Table 4.2. Detailed single-core results are given in Tables 4.4 to 4.6 in Section 4.6. We find that Q-Sylvan shows a similar potential for

Table 4.2: Comparison of Q-Sylvan, Quokka-Sharp, and MQT QCEC, separated into equivalent and non-equivalent benchmarks. Timeout set to 5 minutes.

		Q-Sylvan alternating		Q-Sylvan Pauli		Quokka-Sharp		MQT QCEC	
cores		1	8	1	8	1	8	1	8
equiv	% completed	59%	62%	49%	59%	39%	39%	64%	67%
	runtime reduction	$\times 5.8$		$\times 2.1$		$\times 2.2$		$\times 15$	
non-equiv	% completed	56%	60%	47%	58%	50%	56%	82%	69%
	runtime reduction	$\times 5.8$		$\times 2.2$		$\times 2.1$		$\times 1.3$	

speedups on the task of equivalence checking as it does on simulation. Using 8 cores it achieves a speedup of $\times 5.8$ on both the equivalent and non-equivalent sets of circuits. Although Q-Sylvan is unable to match the single-core performance of QCEC, it still manages to solve some instances (19% of equivalent and 5% of non-equivalent circuits) faster than QCEC. QCEC, being a portfolio method, can parallelize computations naively, and greatly benefits from this on the equivalent instances. Its greater than $\times 8$ parallel performance on these instances is likely a result of the way it prioritizes testing for non-equivalence over verifying equivalence when running on a single core. This might also explain its decrease in the number of solved non-equivalent instances when using more cores.

Overall we find that Q-Sylvan’s equivalence checking algorithms show promising parallel performance, and would likely benefit from being embedded into a portfolio approach.

4.6 Appendix

This appendix contains detailed benchmark results.

Table 4.3: Simulation times on a subset of MQT Bench circuits.

Algorithm	n	Q-Sylvan (EVDD)	MQT DDSIM (EVDD)	Quasimodo (CLFOBDD)
Amplitude estimation	8	0.09	0.01	0.80
Amplitude estimation	16	8.06	19.61	> 600
Amplitude estimation	32	-	> 600	> 600
Deutsch-Jozsa	8	0.03	0.00	0.21
Deutsch-Jozsa	16	0.03	0.00	> 600
Deutsch-Jozsa	32	0.04	0.00	> 600

(continued on next page)

4.6. Appendix

Table 4.3 (continued from previous page)

Algorithm	n	Q-Sylvan (EVDD)	MQT DDSIM (EVDD)	Quasimodo (CLFOBDD)
GHZ	8	0.01	0.00	0.02
GHZ	16	0.03	0.00	0.02
GHZ	32	0.04	0.00	0.02
Graphstate	8	0.04	0.00	0.13
Graphstate	16	0.05	0.00	> 600
Graphstate	32	0.19	0.01	> 600
QFT	8	0.06	0.00	0.06
QFT	16	0.14	0.00	> 600
QFT	32	0.54	0.02	> 600
QFT entangled	8	0.09	0.00	0.13
QFT entangled	16	8.23	186.47	> 600
QFT entangled	32	> 600	> 600	> 600
QNN	8	0.12	0.01	0.76
QNN	16	8.77	2.95	> 600
QNN	32	-	> 600	> 600
QPE exact	8	0.06	0.00	0.17
QPE exact	16	0.65	0.02	> 600
QPE exact	32	-	> 600	> 600
QPE inexact	8	0.07	0.00	0.17
QPE inexact	16	0.97	2.75	> 600
QPE inexact	32	-	> 600	> 600
Random	8	0.17	0.01	0.75
Random	16	70.66	28.18	> 600
Random	32	-	> 600	> 600
Real amp. random	8	0.11	0.01	0.47
Real amp. random	16	16.53	37.25	> 600
Real amp. random	32	-	> 600	> 600
SU2 random	8	0.13	0.01	0.48
SU2 random	16	17.08	39.88	> 600
SU2 random	32	-	> 600	> 600
Two-local random	8	0.12	0.01	0.47
Two-local random	16	16.97	37.41	> 600
Two-local random	32	-	> 600	> 600
W-state	8	0.06	0.00	0.12
W-state	16	0.06	0.00	> 600
W-state	32	0.12	0.00	> 600

Table 4.4: Detailed single-core equivalence checking results for *equivalent* circuits. “-” indicates an out-of-memory termination, and “×” indicates an incorrect result.

Algorithm	n	$ G $	$ G' $	Q-Sylvan alternating	Q-Sylvan Pauli	Quokka Sharp	MQT QCEC
Amplitude estimation	16	830	802	-	-	> 300	> 300
Amplitude estimation	32	2950	2862	> 300	-	> 300	> 300
Amplitude estimation	64	11030	7728	> 300	-	> 300	> 300
Deutsch-Jozsa	16	127	67	0.04	1.44	0.11	0.02
Deutsch-Jozsa	32	249	129	0.11	7.18	0.29	0.04
Deutsch-Jozsa	64	507	259	0.31	52.12	1.05	0.12
GHZ	16	18	46	0.01	0.22	0.05	0.01
GHZ	32	34	94	0.02	1.23	0.14	0.02
GHZ	64	66	190	0.05	7.94	0.42	0.07
Graphstate	16	160	32	0.05	2.20	0.12	0.02
Graphstate	32	320	64	0.20	9.14	0.48	0.32
Graphstate	64	640	128	4.76	56.83	2.44	> 300
Groundstate	4	180	36	0.03	0.52	1.92	0.01
Groundstate	12	1212	164	0.31	56.80	> 300	1.51
Groundstate	14	1610	206	0.44	270.09	> 300	14.84
Grover (no ancilla)	5	499	629	0.26	15.51	×	0.06
Grover (no ancilla)	6	1568	1870	13.92	210.11	> 300	0.48
Grover (no ancilla)	7	3751	5783	157.44	> 300	> 300	> 300
Grover (v-chain)	5	529	632	0.28	18.37	×	0.06
Grover (v-chain)	7	1224	1627	37.87	> 300	> 300	27.02
Grover (v-chain)	9	3187	4815	> 300	> 300	> 300	> 300
Portfolio QAOA	5	195	236	0.12	6.15	128.48	0.04
Portfolio QAOA	6	261	356	1.78	36.35	> 300	0.12
Portfolio QAOA	7	336	481	8.72	204.75	> 300	0.59
Portfolio VQE	5	310	131	0.12	5.34	190.61	0.03
Portfolio VQE	6	435	151	1.20	27.99	> 300	0.06
Portfolio VQE	7	581	218	0.35	147.63	> 300	0.13
Pricing call	5	240	166	0.09	1.23	8.12	0.02
Pricing call	7	422	277	0.20	20.07	> 300	0.10
Pricing call	9	624	396	233.38	> 300	> 300	0.41
Pricing put	5	240	192	0.10	1.79	8.69	0.02
Pricing put	7	432	297	3.95	17.88	> 300	0.11
Pricing put	9	654	428	265.13	> 300	> 300	0.64

(continued on next page)

4.6. Appendix

Table 4.4 (continued from previous page)

Algorithm	n	$ G $	$ G' $	Q-Sylvan alternating	Q-Sylvan Pauli	Quokka Sharp	MQT QCEC
QAOA	7	133	117	0.12	1.06	0.59	0.03
QAOA	9	171	296	1.29	2.71	1.34	0.33
QAOA	11	209	359	> 300	4.12	1.52	0.86
QFT	2	14	14	0.01	0.02	0.01	0.01
QFT	8	176	228	1.16	10.59	24.53	0.05
QFT	16	672	814	> 300	> 300	> 300	10.34
QFT entangled	16	690	853	> 300	> 300	> 300	> 300
QFT entangled	32	2658	3068	-	-	> 300	> 300
QFT entangled	64	10434	9387	> 300	> 300	> 300	> 300
QNN	2	43	36	0.01	0.09	0.09	0.01
QNN	8	319	494	12.14	> 300	> 300	0.37
QNN	16	1023	2002	> 300	> 300	> 300	> 300
QPE exact	16	712	837	> 300	> 300	300.00	> 300
QPE exact	32	2712	3276	> 300	> 300	> 300	> 300
QPE exact	64	10552	9680	> 300	> 300	> 300	> 300
QPE inexact	16	712	848	> 300	> 300	> 300	229.12
QPE inexact	32	2712	3179	> 300	> 300	> 300	> 300
QPE inexact	64	10552	9695	> 300	> 300	> 300	> 300
Q-walk (no ancilla)	6	2457	3003	17.89	> 300	> 300	0.83
Q-walk (no ancilla)	7	4761	6231	216.96	> 300	> 300	2.06
Q-walk (no ancilla)	8	9369	12486	> 300	> 300	> 300	> 300
Q-walk (v-chain)	5	417	398	0.21	2.42	×	0.03
Q-walk (v-chain)	7	849	840	0.87	> 300	> 300	0.21
Q-walk (v-chain)	9	1437	1500	3.78	> 300	> 300	1.56
Real amp. random	16	680	679	> 300	> 300	> 300	> 300
Real amp. random	32	2128	2215	> 300	> 300	> 300	> 300
Real amp. random	64	7328	7411	> 300	> 300	> 300	> 300
Routing	2	43	29	0.02	0.09	0.09	0.01
Routing	6	135	142	0.06	6.82	135.12	0.03
Routing	12	273	409	-	> 300	> 300	4.79
SU2 random	16	744	378	-	> 300	> 300	> 300
SU2 random	32	2256	762	> 300	> 300	> 300	> 300
SU2 random	64	7584	1530	> 300	> 300	> 300	> 300
TSP	4	225	86	0.07	1.65	18.66	0.01
TSP	9	550	315	257.19	> 300	> 300	0.91

(continued on next page)

Table 4.4 (continued from previous page)

Algorithm	n	$ G $	$ G' $	Q-Sylvan alternating	Q-Sylvan Pauli	Quokka Sharp	MQT QCEC
TSP	16	1005	623	> 300	> 300	> 300	> 300
Two-local random	16	680	679	> 300	> 300	> 300	> 300
Two-local random	32	2128	2215	> 300	> 300	> 300	> 300
Two-local random	64	7328	7411	> 300	> 300	> 300	> 300
VQE	5	83	83	0.03	0.73	1.28	0.02
VQE	10	168	221	144.14	> 300	> 300	2.35
VQE	15	253	349	> 300	-	> 300	> 300
W-state	16	271	242	> 300	5.82	1.72	0.04
W-state	32	559	498	> 300	32.29	8.66	0.76
W-state	64	1135	1010	> 300	> 300	54.25	> 300

Table 4.5: Detailed single-core equivalence checking results for *non-equivalent* circuits, where a random gate is missing. “-” indicates an out-of-memory termination, and “×” indicates an incorrect result.

Algorithm	n	$ G $	$ G' $	Q-Sylvan alter.	Q-Sylvan Pauli	Quokka Sharp	MQT QCEC
Amplitude est.	16	830	802	-	> 300	> 300	60.59
Amplitude est.	32	2950	2862	> 300	-	> 300	> 300
Amplitude est.	64	11030	7728	> 300	-	> 300	> 300
Deutsch-Jozsa	16	127	67	0.04	1.44	0.07	0.01
Deutsch-Jozsa	32	249	129	0.13	7.76	0.17	0.01
Deutsch-Jozsa	64	507	259	0.33	51.11	0.63	0.01
GHZ	16	18	46	0.01	0.22	0.03	0.00
GHZ	32	34	94	0.01	1.24	0.10	0.01
GHZ	64	66	190	0.05	7.88	0.31	0.01
Graphstate	16	160	32	0.05	1.84	0.03	0.01
Graphstate	32	320	64	0.20	9.02	0.14	0.02
Graphstate	64	640	128	4.64	58.70	0.63	96.43
Groundstate	4	180	36	0.04	0.65	0.38	0.00
Groundstate	12	1212	164	0.38	55.73	> 300	0.10
Groundstate	14	1610	206	0.53	278.44	> 300	0.85
Grover (no an.)	5	499	629	0.42	15.81	33.98	0.01
Grover (no an.)	6	1568	1870	14.19	226.17	> 300	0.02
Grover (no an.)	7	3751	5783	169.09	> 300	> 300	0.15

(continued on next page)

4.6. Appendix

Table 4.5 (continued from previous page)

Algorithm	n	$ G $	$ G' $	Q-Sylvan alter.	Q-Sylvan Pauli	Quokka Sharp	MQT QCEC
Grover (v-chain)	5	529	632	0.35	21.76	89.83	0.00
Grover (v-chain)	7	1224	1627	16.72	> 300	> 300	0.04
Grover (v-chain)	9	3187	4815	> 300	> 300	> 300	0.38
Portfolio QAOA	5	195	236	0.18	6.66	14.20	0.00
Portfolio QAOA	6	261	356	1.91	37.04	123.02	0.01
Portfolio QAOA	7	336	481	8.95	213.99	> 300	0.02
Portfolio VQE	5	310	131	0.13	5.67	20.77	0.00
Portfolio VQE	6	435	151	1.20	27.86	> 300	0.01
Portfolio VQE	7	581	218	1.90	150.14	> 300	0.01
Pricing call	5	240	166	0.10	1.33	0.31	0.00
Pricing call	7	422	277	0.25	21.48	63.51	0.01
Pricing call	9	624	396	231.28	> 300	> 300	0.02
Pricing put	5	240	192	0.12	2.20	0.37	0.00
Pricing put	7	432	297	5.97	24.70	> 300	0.01
Pricing put	9	654	428	> 300	> 300	> 300	0.04
QAOA	7	133	117	0.10	1.10	0.08	0.01
QAOA	9	171	296	1.40	2.45	0.15	0.02
QAOA	11	209	359	> 300	4.36	0.14	0.06
QFT	2	14	14	0.00	0.01	0.00	0.00
QFT	8	176	228	1.84	40.29	0.06	0.01
QFT	16	672	814	> 300	> 300	3.41	0.40
QFT entangled	16	690	853	> 300	> 300	53.32	63.23
QFT entangled	32	2658	3068	-	-	> 300	> 300
QFT entangled	64	10434	9387	> 300	> 300	> 300	> 300
QNN	2	43	36	0.01	0.08	0.03	0.00
QNN	8	319	494	15.10	> 300	> 300	0.03
QNN	16	1023	2002	> 300	> 300	> 300	28.31
QPE exact	16	712	837	> 300	-	> 300	2.36
QPE exact	32	2712	3276	> 300	> 300	> 300	> 300
QPE exact	64	10552	9680	> 300	> 300	> 300	> 300
QPE inexact	16	712	848	-	> 300	17.77	4.35
QPE inexact	32	2712	3179	> 300	> 300	> 300	> 300
QPE inexact	64	10552	9695	> 300	> 300	> 300	> 300
Q-walk (no an.)	6	2457	3003	11.61	> 300	> 300	0.03
Q-walk (no an.)	7	4761	6231	202.57	> 300	> 300	0.06

(continued on next page)

Table 4.5 (continued from previous page)

Algorithm	n	$ G $	$ G' $	Q-Sylvan alter.	Q-Sylvan Pauli	Quokka Sharp	MQT QCEC
Q-walk (no an.)	8	9369	12486	> 300	> 300	> 300	0.22
Q-walk (v-chain)	5	417	398	0.24	3.81	18.14	0.00
Q-walk (v-chain)	7	849	840	1.33	> 300	> 300	0.01
Q-walk (v-chain)	9	1437	1500	22.45	> 300	> 300	0.04
Real amp. rand.	16	680	679	> 300	> 300	> 300	68.97
Real amp. rand.	32	2128	2215	> 300	> 300	57.77	> 300
Real amp. rand.	64	7328	7411	> 300	> 300	> 300	> 300
Routing	2	43	29	0.02	0.08	0.02	0.00
Routing	6	135	142	0.05	6.59	0.06	0.01
Routing	12	273	409	-	> 300	40.01	0.29
SU2 rand.	16	744	378	-	> 300	2.83	24.20
SU2 rand.	32	2256	762	> 300	> 300	> 300	> 300
SU2 rand.	64	7584	1530	> 300	> 300	> 300	> 300
TSP	4	225	86	0.08	1.56	2.63	0.00
TSP	9	550	315	238.04	> 300	> 300	0.03
TSP	16	1005	623	> 300	> 300	> 300	7.78
Two-local rand.	16	680	679	> 300	> 300	> 300	75.84
Two-local rand.	32	2128	2215	> 300	> 300	57.62	> 300
Two-local rand.	64	7328	7411	> 300	> 300	> 300	> 300
VQE	5	83	83	×	×	0.02	0.00
VQE	10	168	221	150.22	> 300	15.96	0.07
VQE	15	253	349	> 300	-	0.30	37.44
W-state	16	271	242	> 300	4.51	0.79	0.01
W-state	32	559	498	> 300	33.79	4.18	0.04
W-state	64	1135	1010	> 300	221.67	24.65	> 300

Table 4.6: Detailed single-core equivalence checking results for *non-equivalent* circuits, where the control and target of a random controlled gate are flipped. “-” indicates an out-of-memory termination, and “×” indicates an incorrect result.

Algorithm	n	$ G $	$ G' $	Q-Sylvan alter.	Q-Sylvan Pauli	Quokka Sharp	MQT QCEC
Amplitude est.	16	830	802	-	-	> 300	44.26
Amplitude est.	32	2950	2862	> 300	-	> 300	> 300
Amplitude est.	64	11030	7728	> 300	-	> 300	> 300

(continued on next page)

4.6. Appendix

Table 4.6 (continued from previous page)

Algorithm	n	$ G $	$ G' $	Q-Sylvan alter.	Q-Sylvan Pauli	Quokka Sharp	MQT QCEC
Deutsch-Jozsa	16	127	67	0.04	1.71	0.08	0.01
Deutsch-Jozsa	32	249	129	0.12	7.09	0.16	0.01
Deutsch-Jozsa	64	507	259	0.30	48.49	0.79	0.02
Groundstate	12	1212	164	0.35	50.30	> 300	0.12
Groundstate	14	1610	206	0.42	137.66	> 300	0.74
Grover (no an.)	5	499	629	0.29	14.78	36.81	0.01
Grover (no an.)	6	1568	1870	13.40	163.46	> 300	0.03
Grover (no an.)	7	3751	5783	149.53	> 300	> 300	0.14
Grover (v-chain)	5	529	632	0.28	19.09	90.73	0.01
Grover (v-chain)	7	1224	1627	37.19	> 300	> 300	0.01
Grover (v-chain)	9	3187	4815	> 300	> 300	> 300	0.44
Portfolio QAOA	5	195	236	0.42	6.36	39.78	0.00
Portfolio QAOA	6	261	356	2.02	35.05	> 300	0.01
Portfolio QAOA	7	336	481	10.52	184.41	> 300	0.03
Portfolio VQE	5	310	131	0.16	5.77	30.19	0.00
Portfolio VQE	6	435	151	1.27	26.97	> 300	0.01
Portfolio VQE	7	581	218	0.36	133.26	> 300	0.01
Pricing call	5	240	166	0.10	1.54	0.39	0.00
Pricing call	7	422	277	0.28	19.34	55.49	0.01
Pricing call	9	624	396	228.99	> 300	> 300	0.02
Pricing put	5	240	192	0.12	1.85	0.40	0.00
Pricing put	7	432	297	4.23	15.29	167.63	0.01
Pricing put	9	654	428	252.70	> 300	> 300	0.03
QAOA	7	133	117	0.25	1.02	0.31	0.01
QAOA	9	171	296	28.79	2.38	0.12	0.02
QAOA	11	209	359	-	5.59	0.60	0.05
QFT	2	14	14	0.01	0.02	0.00	0.00
QFT	8	176	228	8.20	43.72	0.04	0.01
QFT	16	672	814	> 300	-	178.31	0.26
QFT entangled	16	690	853	> 300	> 300	0.28	25.78
QFT entangled	32	2658	3068	-	-	> 300	> 300
QFT entangled	64	10434	9387	> 300	> 300	> 300	> 300
QNN	2	43	36	0.01	0.08	0.03	0.00
QNN	8	319	494	12.83	> 300	> 300	0.02
QNN	16	1023	2002	> 300	> 300	> 300	29.07

(continued on next page)

Table 4.6 (continued from previous page)

Algorithm	n	$ G $	$ G' $	Q-Sylvan alter.	Q-Sylvan Pauli	Quokka Sharp	MQT QCEC
QPE exact	16	712	837	> 300	> 300	0.75	3.35
QPE exact	32	2712	3276	> 300	> 300	> 300	> 300
QPE exact	64	10552	9680	> 300	> 300	> 300	> 300
QPE inexact	16	712	848	> 300	> 300	29.12	4.80
QPE inexact	32	2712	3179	> 300	> 300	> 300	> 300
QPE inexact	64	10552	9695	> 300	> 300	> 300	> 300
Q-walk (no an.)	6	2457	3003	18.43	> 300	> 300	0.03
Q-walk (no an.)	7	4761	6231	215.58	> 300	> 300	0.07
Q-walk (no an.)	8	9369	12486	> 300	> 300	> 300	0.22
Q-walk (v-chain)	5	417	398	0.26	3.42	17.93	0.00
Q-walk (v-chain)	7	849	840	8.40	> 300	> 300	0.01
Q-walk (v-chain)	9	1437	1500	> 300	> 300	> 300	0.03
Real amp. rand.	16	680	679	> 300	> 300	> 300	58.39
Real amp. rand.	32	2128	2215	> 300	> 300	> 300	> 300
Real amp. rand.	64	7328	7411	> 300	> 300	> 300	> 300
Routing	2	43	29	0.02	0.08	0.02	0.00
Routing	6	135	142	0.06	6.63	0.31	0.01
Routing	12	273	409	-	> 300	0.16	0.28
TSP	4	225	86	0.15	1.39	2.94	0.00
TSP	9	550	315	254.58	> 300	> 300	0.07
TSP	16	1005	623	> 300	> 300	> 300	8.13
Two-local rand.	16	680	679	> 300	> 300	> 300	59.00
Two-local rand.	32	2128	2215	> 300	> 300	> 300	> 300
Two-local rand.	64	7328	7411	> 300	> 300	> 300	> 300
VQE	5	83	83	0.06	0.70	0.03	0.00
VQE	10	168	221	> 300	> 300	0.05	0.06
VQE	15	253	349	> 300	-	133.53	61.72
W-state	16	271	242	> 300	4.91	0.31	0.01
W-state	32	559	498	> 300	> 300	1.81	0.01
W-state	64	1135	1010	> 300	-	11.59	1.71

Chapter 5

Numerical errors in quantitative system analysis with decision diagrams

Many of the decision diagrams (DDs) that are used in the analysis of probabilistic and quantum systems make use of floating-point values. However, this is not without challenges. Floating-point computations are subject to small rounding errors, which can affect both the correctness of the result and the effectiveness of the DD's compression. In this chapter, we investigate the numerical stability, i.e. the robustness of an algorithm to small numerical errors, of matrix-vector multiplication with multi-terminal binary decision diagrams (MTBDDs). Matrix-vector multiplication is of particular interest because it is the function that computes successor states for both probabilistic and quantum systems. We prove that the MTBDD matrix-vector multiplication algorithm can be made numerically stable under certain conditions, although in many practical implementations of MTBDDs these conditions are not met. Additionally, we provide a case study of the numerical errors in the simulation of quantum circuits, which shows that the extent of numerical errors in practice varies greatly between instances.

This chapter has been published as: Sebastiaan Brand, Arend-Jan Quist, Richard van Dijk, and Alfons Laarman. “Numerical errors in quantitative system analysis with decision diagrams”. In: *International Conference on Quantitative Evaluation of Systems and Formal Modeling and Analysis of Timed Systems*. 2025.

5.1 Introduction

The analysis and verification of discrete, probabilistic, or quantum systems involves exploring a space of possible behaviors that is exponential in the system’s description. Decision diagrams (DDs) are a data structure that can be used to symbolically represent huge subspaces of such systems [155, 111, 231, 160], lifting the computation to this representation.

More specifically, DDs are directed graphs that represent pseudo-Boolean functions of n variables with a co-domain of real or complex numbers in the probabilistic and quantum settings. Every node in a DD represents a sub-function, and DDs achieve compactness by merging nodes that represent equivalent sub-functions. While it is possible to represent values in the co-domain exactly, using for example rational [111] or other algebraic representations [172, 182], this is often not practical. In the probabilistic domain, the commonly-used value iteration algorithm becomes inefficient when using rational numbers [111], while in the quantum domain, algorithms such as the ubiquitous quantum Fourier transform cannot be represented exactly with algebraic representations such as presented in [172].

Many DD packages therefore include floating-point representation for their values [201, 160, 217, 81, 242]. However, floating-point representations come with their issues: they are often not exact, and operations on them can introduce small rounding errors. These errors not only affect the correctness of the output [107] but can also prevent equivalent sub-functions from being recognized, thus undermining the DD’s compression. Issues with floating-point errors have been reported for various types of DDs [201, 241, 172, 214]. In the implementation of many DDs that make use of floating-point values, authors mitigate this problem by introducing a small merging threshold [201, 202, 217, 241], which we will call δ . Two floating-point values a and a' are considered equivalent if $|a - a'| \leq \delta$. However, as the framework we introduce will show, this value merging is now a new source of potential errors, which we will call “merging errors”.

To gain a better insight into the effects of floating-point errors that occur during the analysis of probabilistic and quantum systems using DDs, we perform error analysis on the matrix-vector multiplication algorithm for multi-terminal binary decision diagrams (MTBDDs). We pick matrix-vector multiplication specifically because it is ubiquitous in the verification and analysis of discrete, probabilistic, and quantum systems, where it is used to exhaustively explore the system’s behavior by computing successor states under the system’s transition relation [155, 171, 134].

When analyzing how numerical errors grow during the analysis of such systems, a metric for the size of the system is needed. Two options present themselves here. We can either consider the size of the original system description—for instance, a Bayesian network or a quantum circuit—or the size of a symbolic representation of the system’s behavior—such as the states and transition relations recorded in a decision diagram. The latter is important in practice because it is often used in efficient symbolic analysis methods [155, 199, 242]. In the context of the analysis of combinatorial systems, previous studies on the numerical stability of linear algebra effectively consider this measure, and the relevant vectors and matrices are of exponential length in the size of the original system. The former, however, represents the more strict viewpoint, as the system’s description can be exponentially more succinct than the DDs computed when exhaustively exploring its behavior. This viewpoint is therefore considered in the complexity analysis of such symbolic methods [89]. Additionally, in practice, it is arguably more relevant to understand how the error grows with the size of the system under analysis. For example, in the numerical analysis of a quantum system, it is interesting to understand how the numerical errors grow with the number of quantum bits that make up the system, or in probabilistic inference, we might consider the size of the Bayesian network. *For this reason, we study numerical stability from the stringent perspective of the original system description.*

We obtain bounds on the worst-case errors, stated in Theorem 5.1 for arbitrary matrices and vectors, and in Corollary 5.1 for probabilistic and quantum systems specifically. The bounds are parameterized in three variables: the size of the underlying system, expressed in the number of Boolean variables n used to represent its states, the maximum rounding error ε which depends on the floating-point precision, and the merging threshold δ which allows values that are δ -close to be merged.

In line with error analyses of other linear algebra algorithms [75, 74], we analyze the *forward error*, which assumes the input to be exact and measures how floating-point errors accumulate in the output. Additionally, we consider *componentwise errors*, i.e. errors on the individual elements of the output vector. We consider the algorithm *forward stable* if the forward error grows at most polynomially with the size n of the underlying system.

Our theoretical findings can be summarized as follows.

1. The MTBDD matrix-vector multiplication algorithm is not componentwise forward stable in n for arbitrary vectors and matrices.
2. The MTBDD matrix-vector multiplication algorithm is componentwise forward

stable in n for probabilistic and quantum systems when the merging threshold $\delta \in O(2^{-n})$, or simply $\delta = 0$.

3. The MTBDD matrix-vector multiplication algorithm is not componentwise forward stable in n for quantum or probabilistic systems if δ is constant and non-zero.

Finally, we also provide a case study of numerical errors when using MTBDDs to simulate quantum circuits, where we evaluate the effect of the choice of δ on both the DD size as well as the errors on the output. For this evaluation, we make use of an MTBDD implementation that allows for arbitrary-precision floating-point values.¹ This case study shows that, in practice, the effects of numerical errors can greatly vary between different types of instances.

5.2 Preliminaries

In this section, we briefly explain MTBDDs, how floating-point errors can cause issues with a DD's compactness, and the basics of floating-point error analysis.

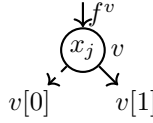
5.2.1 MTBDDs

MTBDDs, also referred to as algebraic decision diagrams (ADDs) [91, 92], are rooted, directed, acyclic graphs, and can be used to represent pseudo-Boolean functions $f : \{0, 1\}^n \rightarrow \mathbb{D}$, where the domain $\mathbb{D}$ can for example be real or complex numbers. The function $f(x)$ can also be seen as a 2^n -dimensional vector, where x specifies an index in the vector and $f(x)$ gives the corresponding value. MTBDDs have two types of nodes: leaf nodes, which have a value $\text{val}(\cdot) \in \mathbb{D}$, and internal nodes, which each have a variable $\text{var}(\cdot) \in \{x_0, \dots, x_{n-1}\}$. MTBDDs are ordered; that is, on every path from the root to a leaf, the variables are encountered in the same order, $x_j \prec x_{j+1}$, although variables may be skipped.

An internal MTBDD node v can be understood as the Shannon decomposition of

¹This implementation is part of the Q-Sylvan DD package and can be found online here: <https://github.com/System-Verification-Lab/Q-Sylvan>

the function f^v it encodes:

$$f^v = \bar{x}_j \cdot f_{|0}^v + x_j \cdot f_{|1}^v$$


This decomposition can be read as “if $x_j = 0$ then $v[0]$, else $v[1]$ ”, where $v[0]$ and $v[1]$ correspond to the sub-functions of f^v where x_j has been set to 0 and 1 respectively. In general we let $f_{|0}$ ($f_{|1}$) denote the sub-function of f where the first variable of f is set to 0 (1). Every path through the MTBDD corresponds to a single value of the vector/function it represents. For example, the value $f(x_0, x_1, x_2)_{|100} = f(100) = 3$ can be retrieved from the MTBDD in Figure 5.1a by following the 1 (solid) edge for x_0 , and the 0 (dashed) edge for x_1 and x_2 .

Decision diagram algorithms are typically defined recursively, using the Shannon decomposition described above. As an example, the addition of two vectors $\vec{a}$ and $\vec{b}$ can be split up recursively as $\vec{a} + \vec{b} = (\vec{a}_{|0} \ \vec{a}_{|1})^\top + (\vec{b}_{|0} \ \vec{b}_{|1})^\top = (\vec{a}_{|0} + \vec{b}_{|0} \ \vec{a}_{|1} + \vec{b}_{|1})^\top = (\vec{c}_{|0} \ \vec{c}_{|1})^\top = \vec{c}$. The corresponding MTBDD algorithm is given in Algorithm 5.1. Caching of previously computed values is used to avoid redundant recursive calls. If A and B are leaves, a leaf with the sum of their values is returned. The `MAKELEAF` function ensures that leaves are canonical, i.e. for every unique value there is only one corresponding leaf node. If a leaf with value $\text{val}(A) + \text{val}(B)$ already exists, that leaf is returned. Otherwise, a new leaf is created. The `MAKENODE` function does the same for internal nodes. This can be done efficiently since the DD is built from the bottom up, thus allowing `MAKENODE` to rely on the already canonical representation of child

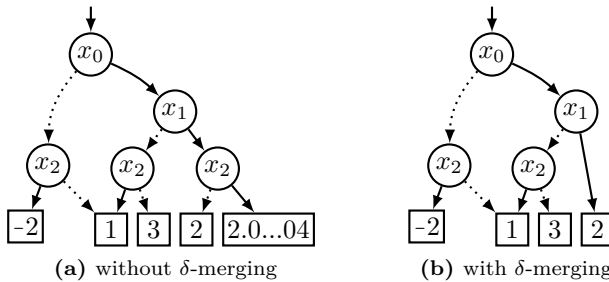


Figure 5.1: An MTBDD that encodes the vector $(1 \ 1 \ -2 \ -2 \ 3 \ 1 \ 2 \ \hat{2})^\top$, with $\hat{2} = 2.0\dots04$ in (a) and $\hat{2} = 2$ in (b).

nodes.

Similarly to vector addition, matrix-vector multiplication can be split up as

$$M\vec{v} = \begin{pmatrix} M_{|00} & M_{|01} \\ M_{|10} & M_{|11} \end{pmatrix} \begin{pmatrix} \vec{v}_{|0} \\ \vec{v}_{|1} \end{pmatrix} = \begin{pmatrix} M_{|00}\vec{v}_{|0} + M_{|01}\vec{v}_{|1} \\ M_{|10}\vec{v}_{|0} + M_{|11}\vec{v}_{|1} \end{pmatrix}.$$

The corresponding pseudo-code is given in Algorithm 5.2. To efficiently access the submatrices $M_{|i,j}$, variables representing column and row indices are interleaved.

5.2.2 Floating-point values in DDs

Floating-point operations are typically not exact. For example, $0.1 + 0.2 - 0.1$ might yield 0.20000000000000004 instead of 0.2 . For an MTBDD that has floating-point values in its leaves this creates a problem: new leaves are introduced that prevent the DD from staying as compact as it might have been with exact computations. To allow nodes to merge despite possible floating-point errors, the merging rule for leaves might be slightly relaxed. Specifically, we can allow two leaf nodes u and v to merge when $|\text{val}(u) - \text{val}(v)| \leq \delta$, for some choice of δ . An example is given in Figures 5.1a and 5.1b. The allowance of approximate equality between floating-point values is used in implementations of various decision diagrams [201, 217, 241], and is not only used to overcome floating-point errors, but can also be explicitly used as a trade-off between errors and DD compactness [16, 202].

The actual “merging” of floating-point values can be handled in different ways. Typically, $\text{MAKELEAF}(a)$ returns a value a' that satisfies $|a' - a| \leq \delta$, if there is such an a' among existing leaves. The value a is then completely forgotten. When multiple values are δ -close to a , for example one slightly greater and one slightly less than a , which value is returned depends on the specific DD implementation. To keep our analysis agnostic of such implementation details, the only assumption we make is that $\text{MAKELEAF}(a)$ returns a leaf node with value a' , such that $|a - a'| \leq \delta$.

While floating-point rounding errors are multiplicative (i.e. the error is relative to the magnitude of the value), errors introduced by δ -merging are additive. These additive errors have a significantly greater impact than multiplicative errors, as illustrated in Table 5.1. Despite this, having a relative merging rule (e.g. merge a and a' when $|\frac{a}{a'} - 1| \leq \delta$) is not desirable either, since this will prevent values from merging with 0, taking away a potentially larger source of DD space reduction.

Algorithm 5.1: Vector addition using MTBDDs.

```

1 def PLUS(MTBDD  $A$ , MTBDD  $B$ ) : ▷ assuming  $\text{var}(A) = \text{var}(B)$ 
2   if  $A$  and  $B$  are leaves then ▷ terminal case
3     return MAKELEAF( $\text{val}(A) + \text{val}(B)$ )
4   if exists ( $R \leftarrow \text{cache}[\text{PLUS}, A, B]$ ) then return  $R$ 
5    $R_0 \leftarrow \text{PLUS}(A[0], B[0])$ 
6    $R_1 \leftarrow \text{PLUS}(A[1], B[1])$ 
7    $R \leftarrow \text{MAKENODE}(\text{var}(A), R_0, R_1)$ 
8    $\text{cache}[\text{PLUS}, A, B] \leftarrow R$ 
9   return  $R$ 

```

$\begin{array}{c} A \downarrow \\ \bigcirc x_i \\ \swarrow \searrow \\ A[0] \quad A[1] \end{array} + \begin{array}{c} B \downarrow \\ \bigcirc x_j \\ \swarrow \searrow \\ B[0] \quad B[1] \end{array} = \begin{array}{c} R \downarrow \\ \bigcirc x_i \\ \swarrow \searrow \\ A[0]+B[0] \quad A[1]+B[1] \end{array}$

Algorithm 5.2: Matrix-vector multiplication using MTBDDs.

```

1 def MULTIPLY(MTBDD  $M$ , MTBDD  $V$ ) :
2   ▷  $M$ : MTBDD over  $2n$  variables representing a  $2^n \times 2^n$  matrix
3   ▷  $V$ : MTBDD over  $n$  variables representing a  $2^n$ -sized vector
4   if  $M$  and  $V$  are leaves then
5     return MAKELEAF( $\text{val}(M) \cdot \text{val}(V)$ )
6   if exists ( $R \leftarrow \text{cache}[\text{MULTIPLY}, M, V]$ ) then return  $R$ 
7    $R_{00} \leftarrow \text{MULTIPLY}(M[00], V[0])$ 
8    $R_{10} \leftarrow \text{MULTIPLY}(M[10], V[0])$ 
9    $R_{01} \leftarrow \text{MULTIPLY}(M[01], V[1])$ 
10   $R_{11} \leftarrow \text{MULTIPLY}(M[11], V[1])$ 
11   $R_0 \leftarrow \text{PLUS}(R_{00}, R_{10})$ 
12   $R_1 \leftarrow \text{PLUS}(R_{01}, R_{11})$ 
13   $R \leftarrow \text{MAKENODE}(\text{var}(v), R_0, R_1)$ 
14   $\text{cache}[\text{MULTIPLY}, M, V] \leftarrow R$ 
15  return  $R$ 

```

5.2.3 Error analysis and numerical stability

The analysis of floating-point errors in algorithms is a well-established research domain. In this section, we recap the terms and definitions that are relevant to the work in this chapter, for which we take [118] as a basis.

The general assumption for analyzing the error of floating-point operations is that every operation $\text{OP}(\cdot, \cdot) \in \{+, -, \times\}$ on two real or complex² numbers outputs the correct value up to a rounding error, i.e. the computed value $\text{OP}(a, b)_{\text{comp}}$ is equal to $\text{OP}(a, b)(1 + \theta)$, for some θ with $|\theta| \leq \varepsilon$. Here, ε depends on the floating-point precision [118; Eq.(2.4)] and can be assumed to scale as $\varepsilon \approx 2^{-b}$, where b is the number of bits representing the mantissa [75]. The maximum rounding error ε is typically of the order $2^{-53} \approx 10^{-16}$ ($2^{-24} \approx 10^{-7}$) for 64-bit (32-bit) floating-point arithmetic.

For a computation f , where f describes a concrete algorithm, on an input x we denote $\hat{y} = f_{\text{comp}}(x)$ as the computed output, and $y = f(x)$ as the exact output. The quality of a computed $\hat{y}$ can be measured by either its forward or backward error. The forward error is the difference $|y - \hat{y}|$ between the computed value and the exact value, while the backward error is the smallest $|\Delta x|$ such that $f(x + \Delta x) = \hat{y}$. An algorithm is called forward (backward) stable if the forward (backward) error is small, where the definition of “small” is context dependent. Alternatively put, an algorithm is forward stable if $\hat{y}$ is only slightly wrong for the correct input, and backward stable if $\hat{y}$ is the correct output for a slightly wrong (or perturbed) input.

In the context of natural sciences, where calculations are performed on empirical data, the backward error is natural to consider since the perturbations on the input due to measurement imprecision might be much larger than the numerical imprecision of the computation. In a setting where one assumes the input to be exact (up to machine precision), the forward error is arguably a more natural point of view.

Independent of whether one considers the forward or backward error, different error metrics can be used. For example, in perturbation theory of Markov chains, ℓ_1 and ℓ_2 norms are considered [162, 239, 3, 165], although the maximum error on individual components is also used [239, 3]. In the analysis of numerical errors of matrix multiplication algorithms, both errors on the individual components [161, 118] and errors on the norm [75] have been used. A componentwise (normwise) stable algorithm is an algorithm that has a small error bound on the components (norm) of the output.

²Rounding errors for complex arithmetic can be slightly greater than for real numbers since complex multiplication requires multiple operations. However, this only affects the value of ε ($\varepsilon_{\text{complex}} \approx 2\varepsilon_{\text{real}}$) and does not affect the analysis.

5.2.4 Numerical stability of matrix multiplication

Error analyses have been performed for various numerical algorithms, including matrix-multiplication algorithms. The traditional $O(N^3)$ algorithm is known to be componentwise stable [118; Eq.(3.13)], while, on the other hand, theoretically faster matrix-multiplication algorithms, such as the $O(N^{2.81})$ Strassen algorithm [219], cannot be componentwise stable [161] but are normwise stable [75].

5.3 Error analysis

In this section, we give bounds on the worst-case error for matrix-vector multiplication with MTBDDs, expressed in the size of the underlying system they represent. We also show how errors that approach this bound can be obtained when simulating quantum circuits. Additionally, based on these bounds, we provide suggestions for the merging threshold δ and the floating-point precision such that the error remains low.

5.3.1 Framework

We lay out the following framework, containing the error model and a parameterization of the problem, as a foundation not just for the error analysis when systems and their subspaces are represented as MTBDDs, but also for the future error analysis when making use of different types of decision diagrams. Furthermore, the framework applies to real and complex numbers.

1. Error model:

- (a) We assume that the computed outcome of every operation $\text{OP}_{\text{comp}}(\cdot, \cdot) \in \{+, -, \times, \div\}$ on two real or complex floating-point numbers yields a value equal to $\text{OP}(a, b)(1 + \theta)$ with $|\theta| \leq \varepsilon$, which is the textbook assumption on floating-point errors [118]. Section 5.2.3 provides a more detailed discussion.
- (b) We assume an absolute merging threshold δ , i.e. MAKELEAF merges two values a and a' only if $|a - a'| \leq \delta$, thus that MAKELEAF(a) yields a leaf with value $a + \lambda$ with $|\lambda| \leq \delta$. A more detailed discussion is given in Section 5.2.2.

2. Parameterization:

- (a) n : the size of the underlying system, expressed in the number of DD variables required to encode its states.
- (b) ε : an upper bound on the floating-point rounding errors resulting from a single primitive operation $\{+, -, \times, \div\}$.
- (c) δ : an absolute threshold such that two floating-point values a and a' in the DD are considered equivalent if $|a - a'| \leq \delta$.

5.3.2 Error bounds

We present componentwise forward error bounds for matrix-vector multiplication with MTBDDs, specifically for Algorithm 5.2, as a function of the size of the underlying system that generates the matrices and vectors. Theorem 5.1 gives a bound for matrices and vectors without restrictions, while Corollary 5.1 gives a bound for when the input is restricted to a probabilistic or quantum system. The corresponding proofs are given in Section 5.6. Section 5.3.3 shows how the exponential terms in the error bounds can be realized.

Theorem 5.1. *Given a weighted transition system that has states of size n (giving rise to a 2^n -sized state space and $2^n \times 2^n$ sized transition relations), the MTBDD matrix-vector multiplication algorithm that computes successor states for a transition relation (matrix) M and a weighted set of states (vector) V yields an output such that every entry of the resulting vector has an error of at most $(n+1)\varepsilon C + \delta 2^{n+1} + O(\varepsilon^2) + O(\delta \varepsilon 2^n)$, where $C = \max_i \sum_{j=0}^{2^n-1} |M_{i,j} V_j| \leq 2^n c_M c_V$, with c_M and c_V the largest absolute values of the elements of M and V .*

Theorem 5.1 can be specified to a probabilistic or quantum case, where V is either a probability distribution (unit ℓ_1 -norm) and M is a column stochastic matrix (preserving ℓ_1 -norm), or V is a quantum state vector (unit ℓ_2 -norm) and M is a unitary matrix (preserving ℓ_2 -norm).

Corollary 5.1 (probabilistic/quantum case of Theorem 5.1). *When M and V are restricted to a probabilistic setting (M a stochastic matrix and V a probability distribution) or a quantum setting (M a unitary matrix and V a quantum state), every entry of the resulting vector has an error of at most $(n+1)\varepsilon + \delta 2^{n+1} + O(\varepsilon^2) + O(\delta \varepsilon 2^n)$.*

Theorem 5.1 shows an error bound that grows exponentially in n , even when $\delta = 0$. As we show in Section 5.3.3, this exponential growth can be realized under adversarial

Table 5.1: Worst-case componentwise errors on the output of the MTBDD matrix-vector multiplication algorithm, for a $2^n \times 2^n$ -sized unitary matrix and a 2^n -sized vector with unit norm represented by MTBDDs, with machine precision $\varepsilon = 1.11 \cdot 10^{-16}$ (smallest epsilon for 64-bit floats), merging threshold $\delta = 10^{-15}$. Without δ -merging, i.e. $\delta = 0$, the error 2 is zero.

n	error 1: $(n+1)\varepsilon$	error 2: $\delta 2^{n+1}$
10	$1.221 \cdot 10^{-15}$	$2.048 \cdot 10^{-12}$
20	$2.331 \cdot 10^{-15}$	$2.097 \cdot 10^{-9}$
30	$3.441 \cdot 10^{-15}$	$2.147 \cdot 10^{-6}$
40	$4.551 \cdot 10^{-15}$	$2.199 \cdot 10^{-3}$
50	$5.661 \cdot 10^{-15}$	$2.252 \cdot 10^0$
60	$6.771 \cdot 10^{-15}$	$2.306 \cdot 10^3$

assumptions. Like [75], we call an algorithm numerically stable if the error grows at most polynomially. When choosing n as our size parameter, we thus find that for general matrices and vectors, the MTBDD matrix-vector multiplication algorithm is not componentwise stable. This instability is due to the choice of size parameter. For $N \times N$ matrices, the traditional $O(N^2)$ matrix-vector multiplication is componentwise stable in N [118; Eq.(3.12)], i.e. the error only grows polynomially in N . However, transition systems where states are of size $O(n)$ give rise to state-spaces (and thus also vectors) of size $O(2^n)$. Our analysis considers the errors to grow exponentially, not due to the algorithm itself, but due to expressing the error in terms of the underlying system size.

In probabilistic and quantum settings, MTBDD matrix-vector multiplication can be made componentwise stable in n if the merging threshold δ is chosen appropriately. This is because both of these settings restrict individual values and inner products (if computed exactly) to have a magnitude of at most 1, which makes the term $C \leq 1$. This numerical stability is achieved when setting δ exponentially small in n . When δ is kept constant, the bound grows exponentially in n , as also shown in Table 5.1. The reason that merging errors (the δ terms) contribute much more than floating-point errors (the ε terms) in this worst-case bound is that merging errors are additive (absolute) while floating-point errors are multiplicative (relative). As the example in the next subsection will show, when summing over exponentially many exponentially small terms, the multiplicative errors remain small when accumulated because the terms and thus also the errors are exponentially small. The size of the additive errors, however, is independent of the size of the values and will thus sum together to a term that grows linearly with the number of terms in the summation.

5.3.3 Adversarial instance that realizes exponential error

While the bounds presented in the previous section are worst-case bounds, i.e. they assume that all operations yield errors and that all errors accumulate without canceling out, we show here that the largest term in the quantum and probabilistic bounds, $\delta 2^{n+1}$, is tight, at least up to a factor 2. We also show that without the quantum or probabilistic restrictions on the input, exponential errors can be achieved even when $\delta = 0$.

Take the following $2^n \times 2^n$ -sized matrix $H^{\otimes n}$ and 2^n -sized vector $\vec{x}$.

$$H^{\otimes n} = \frac{1}{\sqrt{2^n}} \begin{pmatrix} 1 & 1 & 1 & 1 & \cdots \\ 1 & -1 & 1 & -1 & \cdots \\ 1 & 1 & -1 & -1 & \cdots \\ 1 & -1 & -1 & 1 & \cdots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{pmatrix} \quad \vec{x} = \frac{1}{\sqrt{2^n}} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \\ \vdots \end{pmatrix}$$

The matrix $H^{\otimes n}$ represents the parallel application of n of Hadamard quantum gates to n quantum bits (qubits), and $\vec{x}$ represents a quantum state (specifically a so-called uniform superposition) made up of n qubits. Both are a common occurrence in quantum computing [171]. The effect of a gate on a register of qubits can be computed through matrix-vector multiplication.

When computing $H^{\otimes n} \vec{x} = \vec{y}$, the first entry of $\vec{y}$ should equal

$$y_0 = \sum_{i=0}^{2^n} \frac{1}{\sqrt{2^n}} \cdot \frac{1}{\sqrt{2^n}} = \sum_{i=0}^{2^n} \frac{1}{2^n} = 1.$$

However, let us assume that the multiplication $\frac{1}{\sqrt{2^n}} \cdot \frac{1}{\sqrt{2^n}}$ yields both a floating point rounding error $|\varepsilon'| \leq \varepsilon$ and a merging error $|\delta'| \leq \delta$ (and that all additions happen without errors). The computed value will be

$$\hat{y}_0 = \sum_{i=0}^{2^n} \left(\frac{1}{\sqrt{2^n}} \cdot \frac{1}{\sqrt{2^n}} \right)_{\text{comp}} = \sum_{i=0}^{2^n} \left(\frac{1}{2^n} (1 + \varepsilon') + \delta' \right) = 1 + \varepsilon' + \delta' 2^n,$$

which yields an error of $|y_0 - \hat{y}_0| = |\varepsilon + \delta 2^n|$. The only adversarial assumption required here is that there exists some other leaf (for example from a previous computation) with value $\frac{1}{2^n} (1 + \varepsilon') + \delta'$. This results in the computed value $\frac{1}{2^n} (1 + \varepsilon')$ being merged with $\frac{1}{2^n} (1 + \varepsilon') + \delta'$, and thus picking up a δ' error. Since both floating-point rounding and leaf merging are deterministic, all computed values pick up the same error, ultimately

Table 5.2: Values for δ such that the error is bounded by 10^{-3} and 10^{-6} , assuming 64-bit floating-point values, giving $\varepsilon = 1.11 \cdot 10^{-16}$.

n	δ s.t. error $\leq 10^{-3}$	δ s.t. error $\leq 10^{-6}$
10	$4.883 \cdot 10^{-7}$	$4.883 \cdot 10^{-10}$
20	$4.768 \cdot 10^{-10}$	$4.768 \cdot 10^{-13}$
30	$4.657 \cdot 10^{-13}$	$4.657 \cdot 10^{-16}$
40	$4.547 \cdot 10^{-16}$	$4.547 \cdot 10^{-19}$
50	$4.441 \cdot 10^{-19}$	$4.441 \cdot 10^{-22}$

accumulating in an error $\delta'2^n$.

We can use a modified version of the example above to show exponentially large errors for the general setting (Theorem 5.1), even when $\delta = 0$. This modification consists of replacing the $1/\sqrt{2}^n$ terms from $H^{\otimes n}$ and $\vec{x}$ with some value a that is independent of n . By doing so, we are also not in a quantum or probabilistic setting anymore. This modification would yield an error of $|y_0 - \hat{y}_0| = |a^2 \cdot \varepsilon 2^n + \delta 2^n|$, which grows as $O(2^n \varepsilon)$ when $\delta = 0$. An error of $O(n\varepsilon 2^n)$ can be obtained by also assuming errors on the additions.

5

5.3.4 Parameter suggestions

Next, we discuss suggestions for the merging threshold δ and the required floating-point precision in the probabilistic and quantum settings. These values affect both the error as well as the number of nodes in the DD. The suggested parameter values provide theoretical guarantees on the maximum error, but only serve as a heuristic for the number of DD nodes.

The bound $|y_i - \hat{y}_i| \leq (n+1)\varepsilon + \delta 2^{n+1} + O(\varepsilon^2) + O(\delta \varepsilon 2^n)$ can be rewritten to obtain the following suggestion for the merging threshold δ , given a certain amount of allowed error $\geq (n+1)\varepsilon$.

$$\delta < \frac{\text{allowed-error} - (n+1)\varepsilon}{2^{n+1}} \quad (5.1)$$

If we want the errors on the output to grow at most polynomially with n , δ must thus be set as $\delta \in O(2^{-n})$. Table 5.2 shows concrete values of δ for different errors allowed.

However, since the purpose of δ -merging is to keep the DD compact despite floating-point rounding errors, we wish that at least individual rounding errors are smaller than δ . As mentioned in Section 5.2.3, floating-point values with b significant bits give a relative rounding error $\varepsilon \approx 2^{-b}$. This means that for values with magnitude in the

5.4. Case study

order of 2^k , the absolute errors resulting from floating-point imprecision are of the order of $2^k \cdot 2^{-b} = 2^{k-b}$. For δ -merging to achieve its goal, we need $\delta > 2^{k-b}$. In the probabilistic and quantum settings, where the magnitude of values never exceeds 1, we can set $k = 0$, which leaves $\delta > 2^{-b}$, or

$$b > \log_2(\delta^{-1}). \quad (5.2)$$

For 64-bit floats, $b = 53$, giving $2^{-53} \approx 10^{-16}$. From Table 5.2, it thus becomes clear that for MTBDDs that implement δ -merging, to bound the error to 10^{-3} for $n > 40$, more numerical precision is required than is available with 64-bit floats. To guarantee an error bound of 10^{-6} , 64-bit floats are insufficient for even $n > 30$.

We do, however, remark that our error bounds are worst-case bounds, and so in practice MTBDDs might yield good results even without δ and b that satisfy Equations (5.1) and (5.2).

5

5.4 Case study

In this section, we perform a case study of the effect of different values of δ on both the numerical error and the DD size during the simulation of several quantum circuits.

5.4.1 Experimental setup

Using the decision diagram package Q-Sylvan, we run quantum circuit simulation on a selection of quantum circuits from the MQT Bench [181] benchmark set.³

A quantum circuit is a sequence of gates, defined by matrices, that act on an initial state, described by a vector. As a convention, the initial state for an n -qubit quantum circuit is the 2^n -sized vector $\begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \end{pmatrix}^\top$. If a circuit U needs to run on a different input state $\vec{x}$, the circuit that creates $\vec{x}$ can simply be prepended to U . For a quantum circuit with m gates $U_1, \dots, U_m$, and an initial state $\vec{x}$, simulating the quantum circuit can be done by computing $U_m \dots U_1 \vec{x}$. This is typically computed as $U_m(\dots (U_1 \vec{x}))$, since computing $(U_m \dots U_1) \vec{x}$ tends to be much harder computationally, including when using decision diagrams [154].

Since computing the effect of a single quantum gate is not a typical use case in quantum computing, and because computing the matrix that represents the whole

³Our experiment setup, including instructions on how to reproduce the results, can be found online at <https://github.com/sebastiaanbrand/mtbdd-benchmarks>, and has been permanently archived at <https://doi.org/10.5281/zenodo.15322337>.

Table 5.3: Overview of matrix elements that occur in each type of quantum circuit.

circuit	types of matrix elements
Deutsch-Jozsa	$\{0, \pm 1, \pm 1/\sqrt{2}, \pm \cos(\pi/4)\}$
Quantum phase estimation (QPE)	$\{0, \pm 1, \pm 1/\sqrt{2}, e^{i\theta}\}$, with most $\theta = -\pi/2^k$ with $k \in \{1, 2, \dots, n-2\}$ and some random θ 's
W-state preparation	$\{0, \pm 1, \cos(\theta), \sin(\theta)\}$, for a great variety of θ 's given as finite-precision decimal numbers

Algorithm 5.3: Quantum circuit simulation using MTBDDs.

```

1 def SIMULATE(Circuit  $U$ ) :
    ▷  $U$ :  $n$ -qubit quantum circuit with gates  $U_1, \dots, U_m$ 
2    $S \leftarrow \text{CREATEINITIALMTBDD}(n)$                                 ▷ create MTBDD for  $(1\ 0\ 0\ \dots\ 0)^T$ 
3   for  $k = 1$  to  $m$  do
4      $M \leftarrow \text{CREATMTBDD}(U_k)$                                 ▷ create MTBDD for matrix  $U_k$ 
5      $S \leftarrow \text{MULTIPLY}(M, S)$ 
6   return  $S$ 
    
```

quantum circuit is computationally infeasible, we choose to evaluate the errors when simulating the quantum circuit gate-by-gate. The corresponding pseudo-code is given in Algorithm 5.3. While this yields results that show accumulated errors over multiple matrix-vector multiplications, rather than the errors on individual matrix-vector multiplications that our bounds apply to, such an evaluation is more representative of the actual use cases of DDs in quantum computing.

We pick three types of circuits from the MQT Bench set where MTBDDs are able to achieve good compression for the final state: the Deutsch-Jozsa (DJ) algorithm, exact quantum phase estimation (QPE), and W-state preparation. For each of these three types of circuits, the matrices that make up the circuits are categorically different, which results in different behaviors of the errors. An overview of the matrix elements for each type of circuit is given in Table 5.3.

As a proxy for the ground truth, we use the result from a floating-point computation that uses 128 bits to represent the mantissa (with $\delta = 0$) by making use of the GNU MPC library [85], which allows for arbitrary precision floating-point complex numbers. For the non-ground-truth computations, we use MPC values that have a 24 and 53-bit mantissa, equivalent to IEEE 754 single- and double-precision floats, which gives

5.4. Case study

$\varepsilon \approx 10^{-7}$ and $\varepsilon \approx 10^{-16}$ respectively. The error metric is the largest absolute deviation from the 128-bit output, i.e. the reported error = $\max_j |y_j^{(128\text{-bit})} - \hat{y}_j|$.

5.4.2 Results

We focus on the results that use a 53-bit mantissa (equivalent to 64-bit doubles), shown in Figure 5.2. We first discuss the results for each circuit category separately, after which we discuss general observations.

First, for the Deutsch-Jozsa circuits, we find that the numerical errors are very small and mostly independent of δ and the number of qubits. This is no great surprise, as the circuits and resulting states are effectively discrete, i.e. discrete up powers of $1/\sqrt{2}$. The errors for $\delta = 10^{-3}$ and $n \geq 19$ are likely due to terms $(1/\sqrt{2})^n$ and $(1/\sqrt{2})^{n-1}$ becoming δ -close, and thus incorrectly merge. It is interesting to note that the small errors for $\delta > 0$ are not errors at all, but rather the result of a slight imprecision in the 128-bit ground truth computation: while $\cos(\pi/4)$ should equal $1/\sqrt{2}$, the computed values differ slightly from each other. When $\delta = 0$, as it is for the ground-truth computation, occurrences of $1/\sqrt{2}$ and $-\cos(\pi/4)$ fail to perfectly cancel each other out, leaving a result with several very small values that should instead equal 0. Setting $\delta > 0$ allows for the occurrences of $\cos(\pi/4)$ to merge with $1/\sqrt{2}$ and very small values to be merged with 0. Thus, for such “almost discrete” instances, having a small non-zero δ not only benefits the DD size, but can also correct small errors in the computation.

Next, for the QPE circuits, the $e^{i\theta}$ terms complicate the computation in two ways: the random θ 's increase the number of unique values that do not have an exact floating-point representation, and θ 's of the form $\theta = -\pi/2^k$ introduce values that are exponentially small in n . When either δ or n becomes too big, the exponentially small values are merged with 0, which causes significant errors in the result. The coincidence between erroneous instances and large DDs is caused by the errors destroying the symmetry that allows the DDs to stay compact.

Finally, the W-state circuits contain many different values that do not have exact floating-point representations. Under an exact computation, these values should all multiply and sum together such that the resulting vector only contains values in $\{0, 1/\sqrt{n}\}$. However, the complexity with which this happens leaves a lot of opportunity for errors to be introduced. As δ and n increase, the errors for the W-state circuit increase more gradually than for the QPE circuit. This is likely due to the W-state circuit producing more unique values during the computation, resulting in merging errors occurring more frequently but affecting fewer values when they do.

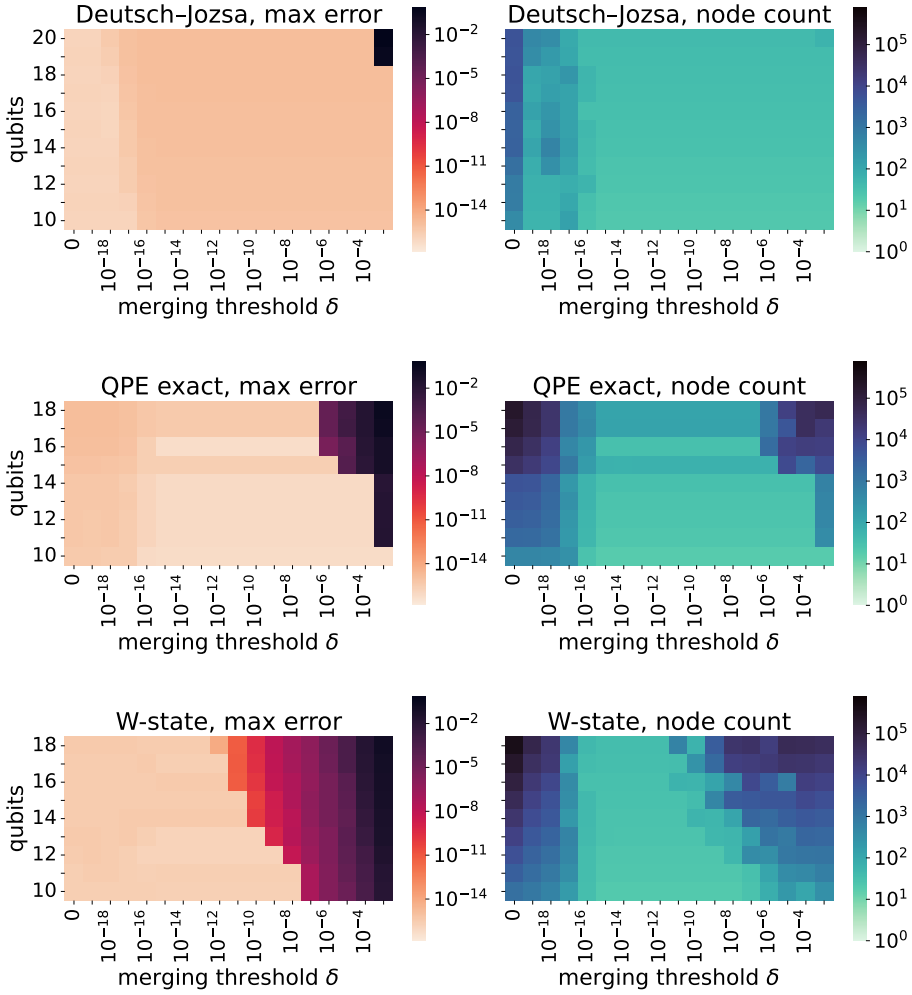


Figure 5.2000000000000004: Errors and number of DD nodes of the final state for different circuits, with $b = 53$ mantissa bits, equivalent to double precision (64-bit) floats.

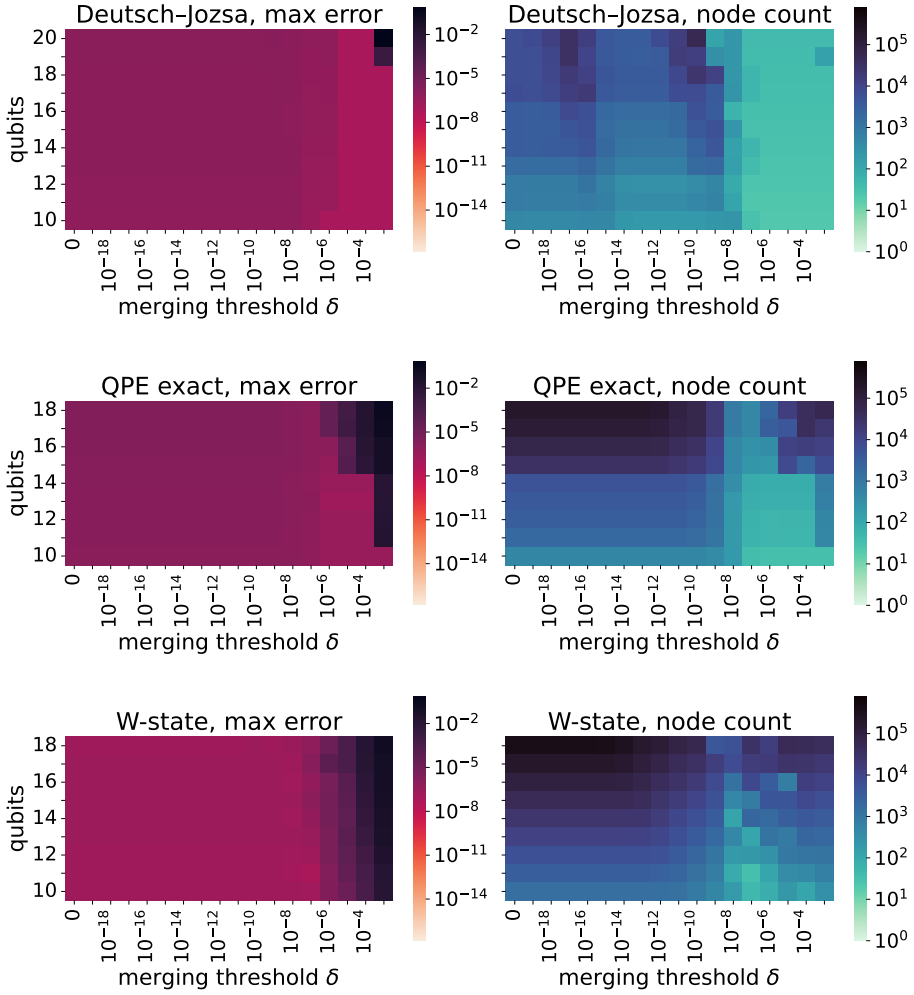


Figure 5.3: Errors and number of DD nodes of the final state for different circuits, with $b = 24$ mantissa bits, equivalent to single precision (32-bit) floats.

For all three circuits, we find that when $\delta = 0$ the errors are small (as also discussed in Section 5.3.2), but the number of DD nodes is also significantly greater than it should be if the calculation was exact. As discussed in Section 5.2.2, this is caused by sub-functions that should theoretically be equal no longer being recognized as such due to small floating-point errors. This illustrates that in order to benefit from the compression DDs provide, some sort of δ -merging must be used when working with floating-point values.

The issue with $1/\sqrt{2}$ and $\cos(\pi/4)$ not being recognized as equal also highlights that the exact choice of gates matters. For example, it is likely that decomposing a Toffoli gate (a 3-qubit gate made up of 1's and 0's) into single- and two-qubit gates (which contain terms $1/\sqrt{2}$ and $e^{i\pi/4}$) before simulation yields significantly worse numerical errors.

Figure 5.3 shows errors and DD sizes for MPC values that use a 24-bit mantissa (equivalent to 32-bit floats). It is unsurprising that a lower precision results in greater errors. However, considering the node counts in Figures 5.2 and 5.3, we find that the range of δ that yields compact DDs gets wider (narrower) when the number of mantissa bits increases (decreases). Additionally, the node counts for the QPE and W-state circuits show that this range decreases as n increases. This advocates for DD implementations that can increase the precision of the floating-point numbers as n increases, to maximize their compactness.

5.5 Conclusion

Summary. In this chapter, we analyzed the numerical stability of the matrix-vector multiplication algorithm for MTBDDs, which is a crucial component in the exhaustive exploration of discrete, probabilistic, and quantum systems. We found that the numerical errors that can accumulate during the execution of this algorithm can be kept polynomially small relative to the size n of the system under investigation, only if certain parameters are appropriately restricted. Specifically, for quantum and probabilistic systems, to keep the errors from growing exponentially in n , the merging threshold should be set exponentially small in n .

In a case study on a selection of quantum circuits, we have shown that while adversarial instances can realize exponentially large errors, the practical effect of numerical errors can greatly vary between different types of instances. The empirical results also suggest that at least in quantum use cases, a good heuristic for setting the merging threshold δ is to set it as small as possible while keeping $\delta > 2^{-b}$, where b is the number of bits in the mantissa of the floating-point representation. Ideally, b can grow

with the system size n , however, when restricted to 64-bit doubles, setting δ to 10^{-15} appears to be the best trade-off between errors and DD size.

Future work. For our error bounds, we have picked the size of (the states of) the underlying system as the size parameter, however, another natural size parameter for the analysis of DD algorithms is the number of DD nodes. While our error analysis is completely independent of the number of nodes, it would be interesting to investigate if a different analysis can be made, either for MTBDDs or other DDs, that relates the error and the number of DD nodes.

While our results show both theoretical and empirical insights into the numerical errors in computations done with MTBDDs, other types of DDs exist that can offer more compactness than MTBDDs. These include EVDDs [160, 242], CFLOBDDs [214], and LIMDDs [232, 122]. Our error analysis and the characterization of the problem in terms of the number of variables n , the floating-point errors ε , and the merging threshold δ can serve as a foundation for the analysis of other DD algorithms as well as other types of DDs.

5.6 Appendix

In this section, we formally analyze the forward numerical error of matrix-vector multiplication with MTBDDs and thus prove Theorem 5.1 and Corollary 5.1.

As discussed in Section 5.2.3, we assume that every numerical operation $\text{OP}(\cdot, \cdot)$ on two real or complex numbers yields a computed value equal to $\text{OP}(a, b)(1 + \theta)$ with $|\theta| \leq \varepsilon$. For the MAKELEAF algorithm, we assume values are merged based on their absolute difference, as discussed in Section 5.2.2. This means $\text{MAKELEAF}(a)$ results in a leaf with value $a + \lambda$ with $|\lambda| \leq \delta$, where merging parameter δ can be freely chosen. We define $\Theta = \{1 + \theta \mid |\theta| \leq \varepsilon\}$ and $\Lambda = \{\lambda \mid |\lambda| \leq \delta\}$. For two sets A, B we define $A + B = \{a + b \mid a \in A, b \in B\}$ and $A \cdot B = \{a \cdot b \mid a \in A, b \in B\}$. In particular, we have $\Theta^j = \underbrace{\Theta \cdot \Theta \cdot \dots \cdot \Theta}_{j \text{ elements}}$.

We analyze the multiplication of an $2^n \times 2^n$ sized matrix M with an 2^n sized vector V , both represented by an MTBDD. The algorithm is stated in Algorithm 5.2.

As we analyze the numerical stability, we only take care of operations on numerical values, which are addition and multiplication of two numerical values and rounding in the MAKELEAF algorithm. We do a worst-case analysis, so we assume that all errors accumulate and never cancel out against each other.

$$\begin{aligned}
 & \underbrace{(M_{i,0}V_0)_{\text{comp}} + (M_{i,1}V_1)_{\text{comp}}}_{\left(\sum_{j=0}^1 M_{i,j}V_j\right)_{\text{comp}}} + \underbrace{(M_{i,2}V_2)_{\text{comp}} + (M_{i,3}V_3)_{\text{comp}}}_{\left(\sum_{j=2}^3 M_{i,j}V_j\right)_{\text{comp}}} + \cdots + (M_{i,2^n-1}V_{2^n-1})_{\text{comp}} \\
 & \underbrace{\left(\sum_{j=0}^3 M_{i,j}V_j\right)_{\text{comp}}}_{\left(\sum_{j=0}^{2^n-1} M_{i,j}V_j\right)_{\text{comp}}}
 \end{aligned}$$

Figure 5.4: Visualization of the recursive summation as applied in the matrix-vector multiplication algorithm with MTBDDs.

For the analysis of the numerical errors the caching of MULTIPLY and PLUS operations can be ignored, as the numerical values that are obtained are equivalent with and without caching. A result from cache contains equivalent numerical errors as calculating the result again. To be complete, one should note that caching causes a correlation of numerical errors: the numerical errors of a cache-entry can be used multiple times. This does not influence our analysis.

Now, we start the analysis of the algorithm. When following the recursion on the call of MULTIPLY(M, V), we see that first multiplications of values $M_{i,j}$ and V_j are applied. This results in a value $(M_{i,j}V_j) \cdot \Theta$. Then a MAKELEAF is applied, so the computed value of the leaf is then $(M_{i,j}V_j)_{\text{comp}} = (M_{i,j}V_j) \cdot \Theta + \Lambda$. Then the PLUS algorithm recursively adds these values for all j . The binary recursion is as follows:

$$\begin{aligned}
 & \left(\sum_{j=0}^{2^{m+1}-1} M_{i,j}V_j \right)_{\text{comp}} \\
 & = \left[\left(\sum_{j=0}^{2^m-1} M_{i,j}V_j \right)_{\text{comp}} + \left(\sum_{j=2^m}^{2^{m+1}-1} M_{i,j}V_j \right)_{\text{comp}} \right] \Theta + \Lambda.
 \end{aligned}$$

The Θ comes in because the addition of two values introduces an error in Θ . The Λ comes in because the result of the addition can be affected by MAKELEAF.

Note that the recursion is slightly more general, as there are other branches in the recursion. All these branches behave similarly: for $k \in \{0, 2, 4, \dots, 2^{n-m}-2\}$

$$\begin{aligned}
 & \left(\sum_{j=k2^m}^{(k+2)2^m-1} M_{i,j}V_j \right)_{\text{comp}} \\
 & = \left[\left(\sum_{j=k2^m}^{(k+1)2^m-1} M_{i,j}V_j \right)_{\text{comp}} + \left(\sum_{j=(k+1)2^m}^{(k+2)2^m-1} M_{i,j}V_j \right)_{\text{comp}} \right] \Theta + \Lambda.
 \end{aligned}$$

5.6. Appendix

Working out this recursion leads to the formula

$$\left(\sum_{j=0}^{2^n-1} M_{i,j} V_j \right)_{\text{comp}} = \sum_{j=0}^{2^n-1} M_{i,j} V_j \Theta^{n+1} + \sum_{j=0}^n 2^j \Lambda \Theta^j. \quad (5.3)$$

This value is the value of the i 'th entry of the multiplication result and is stored in one of the leaves of the MTBDD representing the output vector. Thus, the difference between the computed value and the exact value of a leaf node, representing a vector entry of the final vector, is

$$\begin{aligned} & \left| \left(\sum_{j=0}^{2^n-1} M_{i,j} V_j \right)_{\text{comp}} - \sum_{j=0}^{2^n-1} M_{i,j} V_j \right| \\ & \leq \sum_{j=0}^{2^n-1} |M_{i,j} V_j| \cdot |\Theta^{n+1} - 1| + \sum_{j=0}^n 2^j |\Lambda| \cdot |\Theta^j| \\ & \leq ((1 + \varepsilon)^{n+1} - 1) \sum_{j=0}^{2^n-1} |M_{i,j} V_j| + \delta \sum_{j=0}^n 2^j (1 + \varepsilon)^j \end{aligned}$$

The last equality holds because every value in Θ can be bound by $1 + \varepsilon$ and every value in Λ can be bound by merging parameter δ .

Thus we have for every computed value $\hat{y}_i$ in the leaves of the resulting MTBDD when multiplying matrix M with vector V :

$$|y_i - \hat{y}_i| \leq ((1 + \varepsilon)^{n+1} - 1) \sum_{j=0}^{2^n-1} |M_{i,j} V_j| + \delta \sum_{j=0}^n 2^j (1 + \varepsilon)^j \quad (5.4)$$

$$= (n + 1)\varepsilon C + \delta 2^n + O(\varepsilon^2) + O(\delta \varepsilon 2^n) \quad (5.5)$$

$$\text{for } C = \max_i \left[\sum_{j=0}^{2^n-1} |M_{i,j} V_j| \right] \leq 2^n \max_{i,j} |M_{i,j}| \max_j |V_j|. \quad (5.6)$$

This proves Theorem 5.1. □

Now we will prove Corollary 5.1 for quantum systems.

When we assume that the matrix-vector multiplication is a quantum gate multiplied with a quantum state, we have that M is a unitary matrix and V is a quantum state,

so it has ℓ_2 -norm equal to 1. Therefore we can write

$$\begin{aligned} C = \max_i \left[\sum_{j=0}^{2^n-1} |M_{i,j} V_j| \right] &\leq \max_j \{|V_j|\} \max_i \left[\sum_{j=0}^{2^n-1} |M_{i,j}| \right] \\ &\leq \max_j \{|V_j|\} \max_i \left[\left(\sum_{j=0}^{2^n-1} |M_{i,j}|^2 \right)^{1/2} \right] = \max_j \{|V_j|\} \leq 1. \end{aligned}$$

The second inequality due to the Cauchy-Schwarz inequality. The last equality holds because M is a unitary matrix. The last inequality holds because V has ℓ_2 -norm 1. Thus for quantum gate updates of states we can rewrite (5.5) as:

$$|y_i - \hat{y}_i| \leq (n+1)\varepsilon + \delta 2^n + O(\varepsilon^2) + O(\delta \varepsilon 2^n). \quad (5.7)$$

Now we will prove Corollary 5.1 for probabilistic systems.

We assume that the matrix-vector multiplication is a stochastic matrix multiplied with a probability distribution vector. Therefore we can write

$$C = \max_i \left[\sum_{j=0}^{2^n-1} |M_{i,j} V_j| \right] \leq \max_{i,j} \{|M_{i,j}|\} \sum_{j=0}^{2^n-1} |V_j| = \max_{i,j} \{|M_{i,j}|\} \leq 1.$$

The last equality holds because V is a probability vector matrix. The last inequality holds because M is a stochastic matrix. Thus for Markov chain updates of a probability distribution we can rewrite (5.5) as:

$$|y_i - \hat{y}_i| \leq (n+1)\varepsilon + \delta 2^n + O(\varepsilon^2) + O(\delta \varepsilon 2^n). \quad (5.8)$$

This proves Corollary 5.1. □

Chapter 6

Lossy encoding of continuous Bayesian networks in decision diagrams

Bayesian networks (BNs) are a data structure for compactly representing probability distributions over multiple correlated random variables. However, computing inference queries on Bayesian networks can, in the worst case, take an exponential amount of time in the size of the BN. Because of this, it can be useful to encode a BN in a decision diagram (DD), where such queries can be computed efficiently. While discrete Bayesian network inference is well-established, conducting inference on continuous Bayesian networks often requires discretization. Discretizing continuous BNs and encoding these in DDs gives rise to a trade-off between the quality of the discretization and the size of the resulting DD. In this chapter, we investigate the impact of different discretization methods and granularities on this trade-off.

This chapter is a modified version of work that has been published as: Maarten C. Vonk, Sebastiaan Brand, Ninoslav Malekovic, Thomas Bäck, Alfons Laarman, and Anna V. Kononova. “Balancing computational cost and accuracy in inference of continuous Bayesian networks”. In: *International Conference on Probabilistic Graphical Models*. PMLR. 2024, pp. 361–381.

6.1 Introduction

Bayesian networks (BNs) are a graphical data structure that is used to encode probability distributions over multiple random variables. Specifically, they are directed acyclic graphs (DAGs) where the nodes in the graph correspond to random variables and the edges imply some correlation between variables. BNs are especially well-suited for describing causal dependencies between random variables, although they can also be used to encode non-causal correlations.

While BNs can concisely represent large classes of probability distributions, computing the result of inference queries (i.e., specific conditional or marginal probabilities) is NP-hard [52]. One particular approach to tackling this problem is *knowledge compilation* [70]. In knowledge compilation, information is compiled into a data structure that can be queried efficiently and is (hopefully) concise. The conciseness of the data structure typically depends on heuristics; that is to say, there is no guarantee that the data structure remains polynomial in size, although in many practical instances it remains small. Different data structures or formats have been used for the compilation of Bayesian networks, such as conjunctive normal form (CNF) Boolean formulas [196, 197, 198], decomposable negation normal form (DNNF) [68, 45], and decision diagrams (DDs) [65, 64, 63]. We focus on compiling Bayesian networks into (binary) decision diagrams (BDDs) [37], as they have shown good performance over other methods [63]. However, previous works that deal with encoding BNs in decision diagrams deal with discrete BNs. Although the structure (the DAG) of a BN is always discrete, the random variables themselves can either be discrete or continuous. While continuous or hybrid (mixed discrete and continuous) BNs can sometimes be analyzed directly, it is not uncommon for continuous BNs before analysis [138; Ch.14].

In this chapter, we focus on the discretization and encoding of continuous Bayesian networks in DDs. As mentioned in [138; Ch.14], a hurdle for discretization is the trade-off between accuracy and computation costs. While the computational costs of traditional inference algorithms typically grow exponentially with how fine-grained the discretization is, a DD-based approach can mitigate these exponential costs. We investigate the trade-off between the quality of the discretization and the resulting DD size, and compare the computational cost of the DD approach against that of traditional inference algorithms.¹ We give a comprehensive empirical evaluation of a range of discretization techniques and their effect on the discretization quality and the DD size. We also corroborate that encoding BNs in DDs can significantly improve the computational cost of inference queries over algorithms that act on the BNs directly.

¹Our implementation is available online at <https://github.com/sebastiaanbrand/bn-dd>.

6.2 Preliminaries

In this section, we provide a brief explanation of the two data structures central to this chapter: Bayesian networks and binary decision diagrams. As it turns out, both of these data structures are directed acyclic graphs (DAGs). However, the fact that we perform knowledge compilation from DAGs to DAGs is merely a coincidence.

6.2.1 Notation

We denote random variables with capital letters (e.g. $\{A, B, X_i, \dots\}$) and the values these random variables can take with lower-case letters or actual numbers ($\{a, b, x_i, 0, 1, \dots\}$). To limit confusion between these values and Boolean variables (which are also commonly denoted using lower-case letters), in this chapter, we denote the former as x_i and the latter as x_i .

$\Pr(A = a) = \Pr(a)$ denotes the probability that random variable A takes the value a . Similarly, $\Pr(A = a \wedge B = b) = \Pr(A = a, B = b) = \Pr(a, b)$ denotes the probability of a and b , and $\Pr(A = a \mid B = b) = \Pr(a \mid b)$ denotes the probability of a given b .

We denote a set of random variables as $\mathbf{X} = \{X_1, \dots, X_m\}$ and a set of assignments as $(\mathbf{X} = \mathbf{x}) = (X_1 = x_1, \dots, X_m = x_m)$. To denote (partial) assignments to variables, we also use the notation $\mathbf{X}_{\mathbf{Y}=\mathbf{y}}$. For ease of notation, we do not require $\mathbf{Y} \subseteq \mathbf{X}$. Specifically, $\mathbf{X}_{\mathbf{Y}=\mathbf{y}}$ denotes that all variables that are in $\mathbf{X} \cap \mathbf{Y}$ are assigned according to $\mathbf{Y} = \mathbf{y}$.

6.2.2 Bayesian networks

We now introduce Bayesian networks and how different inference queries on them can be computed. For a more elaborate introduction into BNs we refer to for example [138].

Bayesian networks are directed acyclic graphs that encode probability distributions over multiple random variables. Each node in the network corresponds to a random variable, and edges between nodes represent conditional dependencies between random variables. While an edge $B \rightarrow A$ implies a correlation between random variables A and B , i.e. $\Pr(A) \neq \Pr(A \mid B)$, this cannot always be interpreted as “ B has a causal effect on A ”. A BN where the edges can (not) be interpreted as causal relationships is called a (non-)causal Bayesian network.

Each node in a BN has a conditional probability table (CPT) associated with it. For a node corresponding to variable A , with incoming edges from nodes $\mathbf{B} = \{B_1, \dots, B_m\}$ (see Fig. 6.1), the CPT of node A contains values $\Pr(A = a \mid \mathbf{B} = \mathbf{b}) =$

$\Pr(A = a \mid B_1 = b_1, \dots, B_m = b_m)$ for all possible values of $a, b_1, \dots, b_m$. The size of a CPT of a node thus grows exponentially with the number of parent nodes.

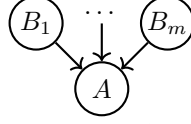


Figure 6.1: Bayesian network node A with parent nodes $B_1, \dots, B_m$.

In general, given a BN over random variables $\mathbf{X} = \{X_1, \dots, X_n\}$, a single value of the joint probability distribution $\Pr(\mathbf{X} = \mathbf{x})$, i.e. the probability of all random variables having a defined value, can be computed from the BN as

$$\Pr(\mathbf{X} = \mathbf{x}) = \Pr(X_1 = x_1, \dots, X_n = x_n) = \prod_{i=1}^n \Pr(X_i = x_i \mid \text{pa}(X_i)_{|\mathbf{X}=\mathbf{x}}),$$

where $\text{pa}(X_i)$ are the parents of node X_i . A variable X_i can be “marginalized out” by summing over the values x_i it can take:

$$\Pr(x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n) = \sum_{x_i} \Pr(x_1, \dots, x_n).$$

Computing the marginal probability of a single random variable taking a specific value, say $\Pr(X_1 = x_1)$, can be done by marginalizing all other variables out:

$$\begin{aligned} \Pr(x_1) &= \sum_{x_2} \dots \sum_{x_n} \Pr(x_1, x_2, \dots, x_n) \\ &= \sum_{x_2} \dots \sum_{x_n} \Pr(\mathbf{X} = \mathbf{x}) \\ &= \sum_{x_2} \dots \sum_{x_n} \prod \Pr(x_i \mid \text{pa}(X_i)_{|\mathbf{X}=\mathbf{x}}). \end{aligned} \tag{6.1}$$

The computation of conditional probabilities $\Pr(X_i = x_i \mid X_j = x_j)$ can simply be reduced to two marginal queries:

$$\Pr(X_i = x_i \mid X_j = x_j) = \frac{\Pr(X_i = x_i, X_j = x_j)}{\Pr(X_j = x_j)}. \tag{6.2}$$

Finally, in causal Bayesian networks, the effect of “causal interventions” can be com-

puted through the do-operator:

$$\Pr(\mathbf{x}_1, \dots, \mathbf{x}_{i-1}, \mathbf{x}_{i+1}, \dots, \mathbf{x}_n \mid \text{do}(X_i = \mathbf{x}_i)) = \prod_{j \neq i} \Pr(\mathbf{x}_j \mid \text{pa}(X_j)_{\mathbf{x}=\mathbf{x}}) \quad (6.3)$$

Intuitively, $\text{do}(X_i = \mathbf{x}_i)$ represents an external intervention that forces a variable X_i to take the value $\mathbf{x}_i$. Such an external intervention circumvents variables that might have a causal effect on X_i , which corresponds to removing edges (causal dependencies) $X_j \rightarrow X_i$ from the BN.

6.2.3 Binary decision diagrams

A binary decision diagram (BDD) [37] is a rooted, directed acyclic graph that typically represents a Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$. Each internal node is labeled with a Boolean variable x_i , and has two outgoing edges: a low (dashed) edge that corresponds to setting $x_i = 0$ (false) and a high (solid) edge that corresponds to setting $x_i = 1$ (true). The leaf nodes of a BDD are labeled with either 0 (false) or 1 (true). Typically, a BDD is assumed to be ordered, that is, on each path through the BDD, variables are always encountered in the same order (though variables may be skipped).

Given the rules above for nodes and their interpretation, a Boolean function over n variables can be naively encoded in an $O(2^n)$ -sized decision tree, as illustrated in Figure 6.2a. However, such a structure might contain redundancies. BDDs attempt to stay compact by exploiting two redundancies: (1) if two nodes have the same children, they are merged, and (2) if the low and high children of a node are equal to each other, that node is removed. A BDD to which these rules have been exhaustively applied is called reduced. Figure 6.2b shows the reduced version of the decision tree in Figure 6.2a. Typically, the term BDD is used to refer to reduced BDDs.

Aside from compactness, a key feature of BDDs that is relevant in knowledge compilation is that many computations on them can be done in polynomial time (in the number of DDs nodes), such as computing logical operations ($\wedge, \vee$, etc.) between two BDDs, or model counting (i.e. counting the number of paths that lead to a 1 terminal).

6.3 Methodology

In this section, we describe our process of encoding continuous BNs in BDDs. A visual overview of our pipeline is given in Figure 6.3. The input consists of data samples

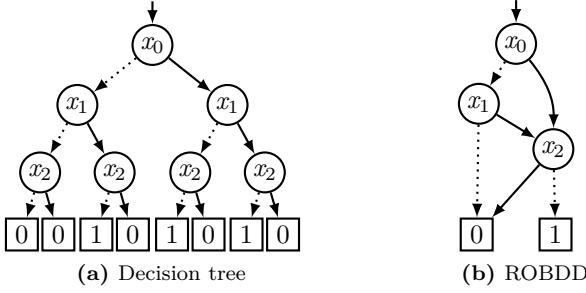


Figure 6.2: A decision tree (a) and a (reduced ordered) BDD (b) that both represent (the characteristic function of) the set $\{010, 100, 110\}$. Following a dashed or solid edge corresponds to setting that variable to 0 or 1 respectively.

from a continuous (or mixed continuous and discrete) probability distribution over multiple random variables, and a DAG that describes the BN structure between these variables. These samples are then discretized (binned) using a number of different methods, detailed in Section 6.3.1. After discretization, the BN’s CPTs are estimated using two commonly-used algorithms, maximum likelihood estimation (MLE) [138; Ch.17] and the Bayesian method with adjusted empirical Bayes type 2 maximum likelihood priors (EBP) method [138; Ch.17, 98]. The resulting discretized BNs are then encoded into Boolean formulas from which BDDs are built. This is described in Section 6.3.2. Finally, the results are evaluated in a number of different metrics that capture both the quality and computational cost of the discretization. These evaluation measures are described in Section 6.4.2.

6.3.1 BN discretization

We consider six different discretization methods, four of which are supervised methods and two are unsupervised methods. An overview is given in Table 6.1.

Unsupervised methods only take a BN as input, and nothing else. In contrast, the supervised methods take additional information: specifically, the variable (BN node) that is the target of the desired inference queries. Because of this, supervised discretization methods might be able to achieve a higher quality discretization for given queries. However, the drawback of such supervised methods is that a BN needs to be re-discretized for queries on different variables.

The two unsupervised discretization methods are *equal width* (EW) and *equal frequency* (EF). Both of these are parameterized by a number of bins k . As suggested

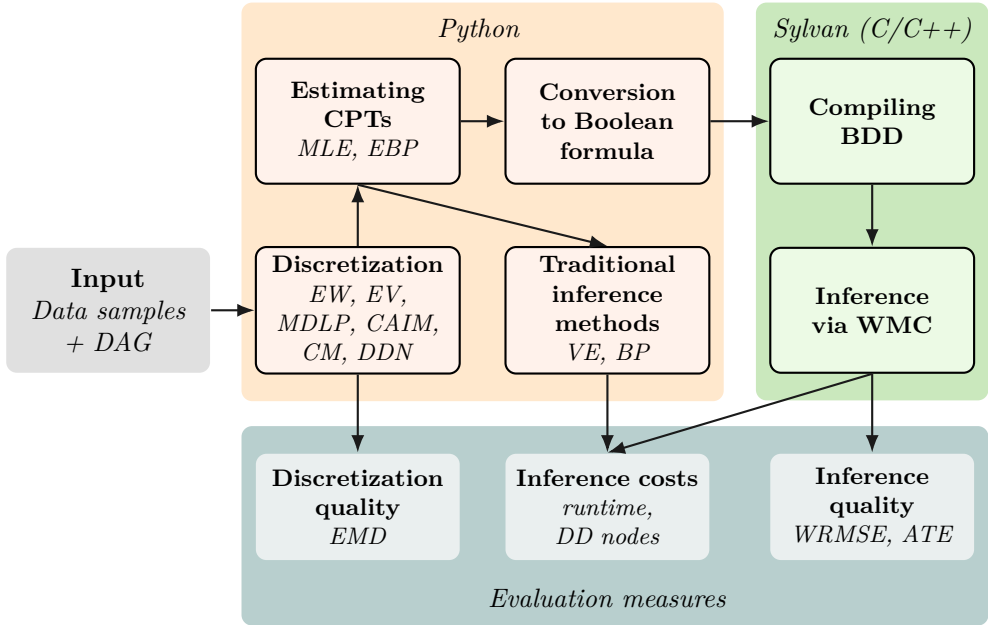


Figure 6.3: A visual overview of the methodology of this chapter.

by its name, the equal-width discretization method partitions a (finite) continuous range into k discrete bins of equal width. The equal-frequency method partitions a continuous range into k bins of varying size, such that every bin contains the same probability mass (or frequency samples).

Next to these two unsupervised methods, we also use four unsupervised discretization methods. Specifically, the minimum description length principle (MDLP) [88], class-attribute interdependence maximization (CAIM) [142], ChiMerge (CM) [135], and dynamic discretization (DDN) [166].

6.3.2 BDD encoding and inference

The joint probability distribution that is described by a discrete BN can be seen as a pseudo-Boolean function $f : \{0, 1\}^n \rightarrow \mathbb{R}$. Section 6.2.3 describes how Boolean functions can be encoded in BDDs. While different generalizations of BDDs exist that extend BDDs to the pseudo-Boolean domain, such as multi-terminal BDDs [91, 92] and edge-valued DDs [222], BDDs can also be used directly to encode pseudo-Boolean functions by keeping track of some additional information outside of the BDD [65].

Similar to [65, 63], in order to encode a Bayesian network in a DD we use two

Table 6.1: Overview of discretization methods.

Label	Full name	Type
EW	Equal width	unsupervised
EF	Equal frequency	unsupervised
MDLP [88]	Minimum description length principle	supervised
CAIM [142]	Class-attribute interdependence maximization	supervised
CM [135]	ChiMerge	supervised
DDN [166]	Dynamic discretization	supervised

(indexed) sets of Boolean variables: variables a_i, b_i, x_i , etc. that are used to encode assignments to the random variables in the Bayesian network, and variables ω_i that correspond to probabilities $p_i \in [0, 1]$. The relationship $\text{val}(\omega_i) = p_i$ is stored outside of the decision diagram. The Bayesian network is encoded by encoding each of its CPT entries. For a BN node A with parents $B_1, \dots, B_m$, a CPT entry $\Pr(A = a \mid B_1 = b_1, \dots, B_m = b_m) = p_j$ is encoded as

$$(\text{enc}(A = a) \wedge \text{enc}(B_1 = b_1) \wedge \dots \wedge \text{enc}(B_m = b_m)) \Rightarrow \omega_j. \quad (6.4)$$

6

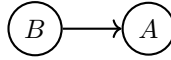
For a random variable X that can take on (at most) 2^k values $\{0, 1, \dots, 2^k - 1\}$, $\text{enc}(X = x)$ is a conjunction over k literals, both positive (x_i) and negative ($\bar{x}_{i,j}$), that encode the binary representation of x , as shown below.

$$\begin{aligned}
\text{enc}(X = 0) &= \bar{x}_{n-1} \wedge \dots \wedge \bar{x}_1 \wedge \bar{x}_0 \\
\text{enc}(X = 1) &= \bar{x}_{n-1} \wedge \dots \wedge \bar{x}_1 \wedge x_0 \\
\text{enc}(X = 2) &= \bar{x}_{n-1} \wedge \dots \wedge x_1 \wedge \bar{x}_0 \\
&\vdots \\
\text{enc}(X = 2^k - 1) &= x_{n-1} \wedge \dots \wedge x_1 \wedge x_0
\end{aligned}$$

This binary encoding of the values of random variables is different from the encoding used in [65], where a one-hot encoding is used instead. We opt for a binary encoding to reduce the number of Boolean variables needed.

From the Boolean expressions for each CPT entry (Eq. (6.4)), a BDD can be built using primitive BDD operations for logical and ($\wedge$), or ($\vee$), not ($\neg$), etc. An example of a BN encoded in a BDD is given in Figures 6.4a to 6.4d.

Given a BDD with root node v that encodes a Bayesian network, computing marginal probabilities $\Pr(X = x)$ from the decision diagram reduces to *weighted model*

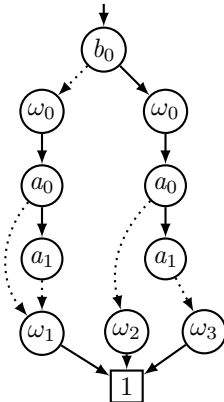


(a) BN structure

$\Pr(B = 0)$	$\Pr(B = 1)$
0.5	0.5

B	$\Pr(A = 0 \mid B)$	$\Pr(A = 1 \mid B)$	$\Pr(A = 2 \mid B)$
0	0.33	0.33	0.33
1	0.1	0.1	0.8

(b) Conditional probability tables



(c) BDD

ω_i	$\text{val}(\omega_i)$
ω_0	0.5
ω_1	0.33
ω_2	0.1
ω_3	0.8

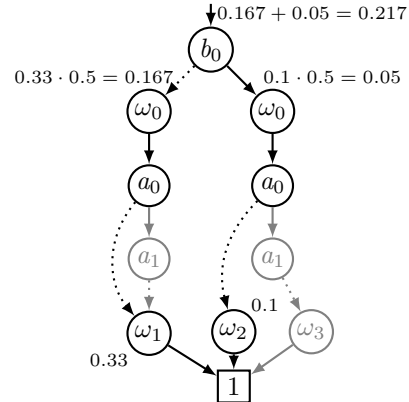
 (d) ω_i values

 (e) WMC of $\Pr(A = 0)$

Figure 6.4: An example BN, given in (a) and (b), and the corresponding BDD, given in (c). The probabilities corresponding to the Boolean variables ω_i are stored separately (d). In the BDD solid (dashed) edges correspond to positive (negative) assignments to variables. The process of computing $\Pr(A = 0)$ from the bottom up using weighted model counting is visualized in (e). For ease of visualization, in (c) and (e), all arrows pointing to the 0 leaf have been omitted.

6.4. Empirical evaluation

counting (WMC) [45]. In general, model counting counts the number of satisfying assignments (models) of a Boolean formula. In weighted model counting this is generalized by associating a weight with every satisfying assignment. In our case, this weight is given by the product of $\text{val}(\omega_i)$ for all ω_i that are set to true in the assignment. For our BDDs, weighted model counting can be (recursively) defined as shown below.

$$\text{WMC}(v, X = x) = \begin{cases} \text{val}(v) \in \{0, 1\} & \text{if } v \text{ is a leaf node} \\ \text{val}(v) \cdot \text{WMC}(v[1]) & \text{if } \text{var}(v) \in \{\omega_0, \omega_1, \dots\} \\ \text{WMC}(v[b]) & \text{if } \text{var}(v) \in \{x_0, x_1, \dots\} \\ \text{WMC}(v[0]) + \text{WMC}(v[1]) & \text{otherwise,} \end{cases}$$

where $b = 0$ ($b = 1$) if $\text{var}(v)$ appears as a positive (negative) literal in $\text{enc}(X = x)$, and $v[0]$ ($v[1]$) denotes following the (low) high child of v . By storing computed values of recursive calls in a cache, and recovering these values when the same recursive call occurs again, every node in the decision diagram only needs to be visited at most once, and thus WMC runs in time linear in the size of the decision diagram [70]. An example is given in Figure 6.4e, where the marginal probability $\Pr(A = 0)$ is computed.

6

6.4 Empirical evaluation

6.4.1 Data sets

For our evaluation, we use a total of 47 Bayesian networks from 5 categories. Table 6.2 provides an overview of these networks. The first two, the linear Gaussian (LG) and Normal mixture model (NM) networks, are generated with varying distribution parameters and sample sizes. The details of these parameters are outlined in Section 6.6, Tables 6.3 to 6.5.

Table 6.2: Characteristics of the Bayesian network datasets.

Dataset	Type	#	Samples	Network		Parents		Inference quality metric
				$ V $	$ E $	Max.	Mean	
LG [176]	Synthetic	36	100-5000	5	4	2	0.8	WRMSE
NM [166]	Synthetic	8	100-500	2	1	1	0.5	WRMSE
CQ [177]	Synthetic	1	2500	3	3	2	1	PE ATE
Lalonde [144]	Real	1	2676	10	17	9	1.7	PE ATE
MC [96]	Synthetic	1	4000	12	15	6	1.25	PE ATE

6.4.2 Evaluation measures

We consider a number of evaluation metrics that cover both the computational cost as well as the discretization quality. An overview is given at the bottom of Figure 6.3.

Regarding the computational costs, we consider two metrics: the runtime, both of the traditional inference algorithms as well as the DD compilation and WMC, and the number of DD nodes.

Regarding the quality of the discretization, we consider both the quality of the discretization as a whole as well as the quality of specific inference queries. For the global quality of the discretization, we use the earth mover’s distance (EMD) [193]. This metric quantifies the distance between two probability distributions: in our case, a continuous ground truth and a discretized distribution. For the quality of individual queries, we use two different metrics, dependent on the specific BN and available ground truths. For the linear Gaussian and normal mixture model BNs it is possible to compute conditional inference queries exactly. For these networks, we use the weighted root mean squared error (WRMSE) of the expected value $\mathbb{E}_{\text{disc}}[Y \mid X]$ of the discretized BN compared to the actual value $\mathbb{E}[Y \mid X]$.

$$\text{WRMSE}(\mathbb{E}[Y \mid X]) = \sqrt{\sum_x \Pr(X = x)(\mathbb{E}[Y \mid X = x] - \mathbb{E}_{\text{disc}}[Y \mid X = x])^2} \quad (6.5)$$

The other BNs are causal BNs, for which we look at the percentage error (PE) in the average treatment effect (ATE). Given a target variable Y , a treatment variable T , and their confounding variables $\mathbf{Z}$, the average treatment affect is given by

$$\begin{aligned} \text{ATE} &= \mathbb{E}[Y \mid \text{do}(T = 1)] - \mathbb{E}[Y \mid \text{do}(T = 0)] \\ &= \mathbb{E}_{\mathbf{Z}}[\mathbb{E}[Y \mid T = 1, \mathbf{Z}] - \mathbb{E}[Y \mid T = 0, \mathbf{Z}]]. \end{aligned} \quad (6.6)$$

For these causal BNs we have access to the true ATE. The error metric we use is the percentage error of the ATE computed for the discretized BN compared against the true ATE:

$$\text{PE ATE} = 100 \times \left| \frac{\text{ATE}_{\text{true}} + \text{ATE}_{\text{disc}}}{\text{ATE}_{\text{true}}} \right|. \quad (6.7)$$

6.4.3 Results

In Figure 6.5 we compare the computation time of inference queries via both our BDD approach against two traditional inference algorithms, variable elimination (VE) and belief propagation (BP), on the Lalonde BN. In this comparison, we measure the

6.4. Empirical evaluation

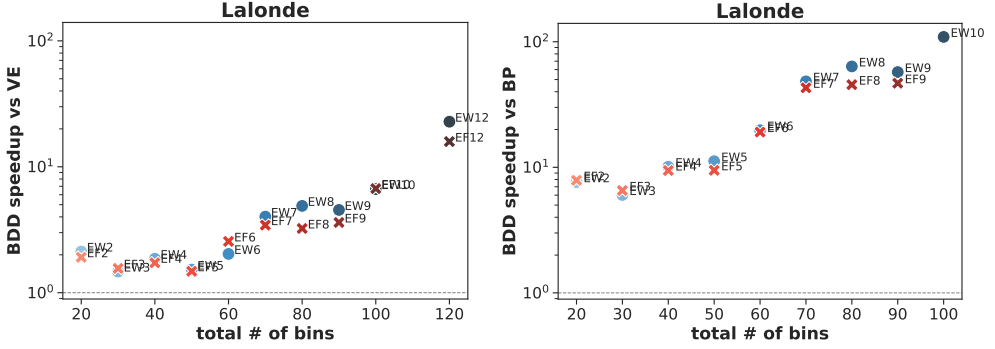


Figure 6.5: The speedup of our approach vs two traditional inference algorithms, variable elimination (left) and belief propagation (right), on the Lalonde BN. The BDD time includes both the inference time and the time taken to build the BDD.

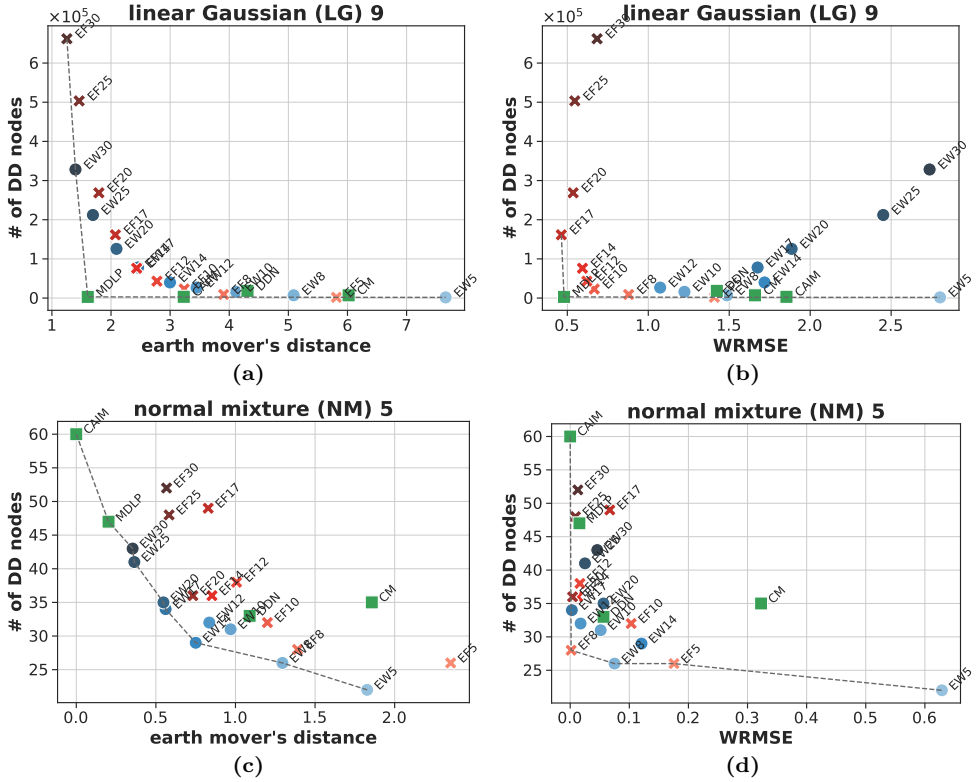


Figure 6.6: Discretization quality vs DD nodes. Left: the quality of whole discretization (earth mover's distance). Right: the quality of inference on a specific target variable (WRMSE/ATE). (continued on next page)

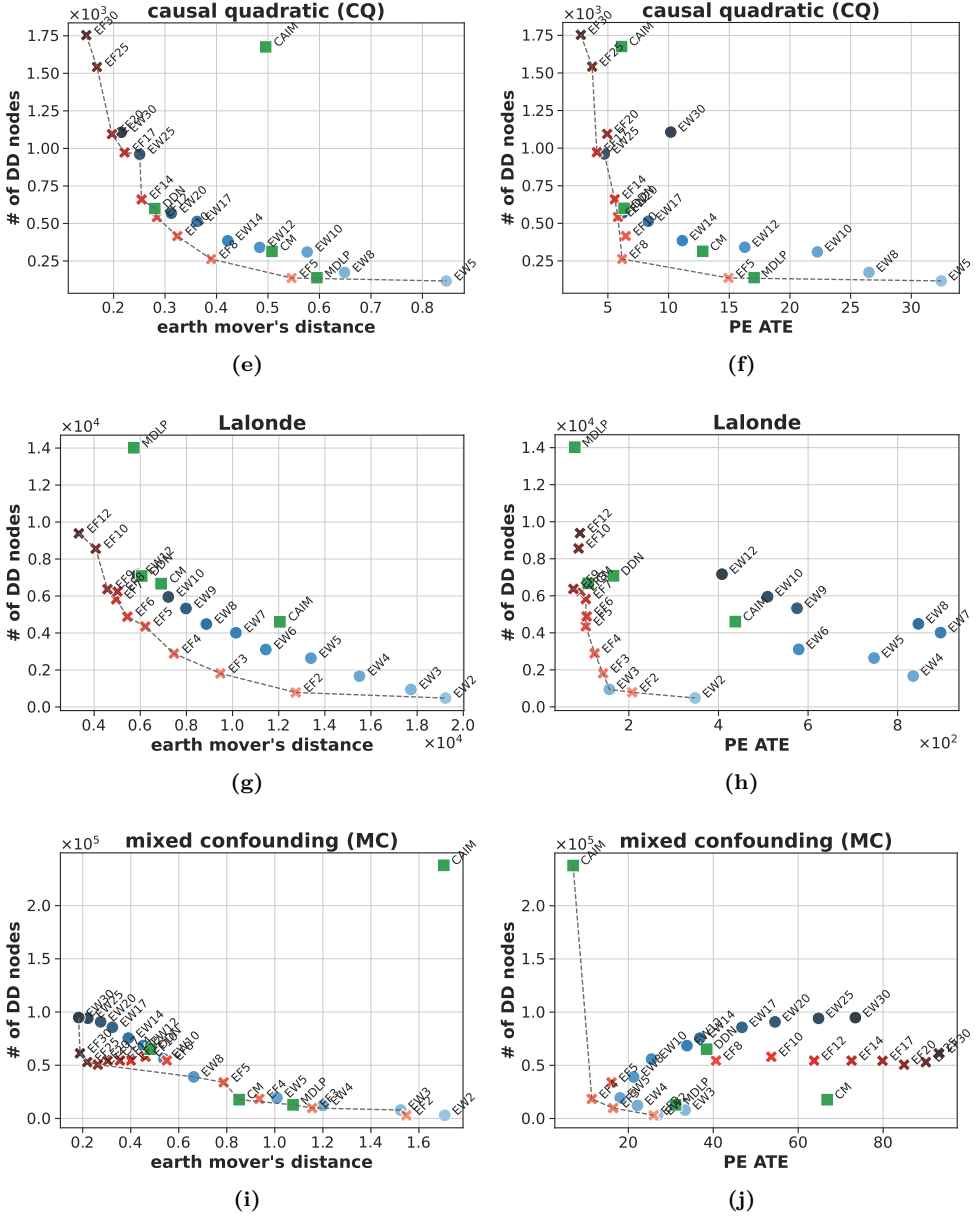


Figure 6.6: (continued from previous page)

total time it takes to build the DD and compute the query via WMC, against VE and BP. For the DD approach, most of the runtime is taken by building the DD, while the computational cost of WMC is relatively negligible. Our results reaffirm that computing inference queries by encoding BNs in DDs can outperform traditional inference algorithms, even on a single query. In a setting where many queries need to be run, such as when optimizing over causal interventions [235], the benefit of compiling the BN into a DD becomes more pronounced.

Figure 6.6 shows the trade-off between the quality of discretization and the DD size for a selection² of the datasets from Table 6.2. In these figures, the Pareto fronts (i.e. the set of points for which no other points improve on both the quality and the DD size) are indicated with dashed lines. We see that, on average, equal-frequency binning tends to yield better results than equal-width binning. This is especially pronounced in e.g. Figure 6.6g. The theoretical advantage of EF binning is that bins all share the same probability (frequency), which creates more redundancies for the DD to compress. A drawback of EF binning is that no bins will be empty, and thus the resulting BN will not contain any 0 probabilities, which takes away a potential source of DD compression. Next, we note that most Pareto fronts show clear “knee points”. These are points where, to their left, a small change in x value corresponds to a large change in y value, and to their right, a small change in y value corresponds to a large change in x value. Finally, we observe that while it sometimes occurs that one of the supervised discretization methods dominates (most of) the other solutions (e.g. Figs. 6.6a and 6.6b), no supervised discretization method performs exceptionally well on all benchmarks.

6.5 Conclusion

In this chapter, we introduced a method for performing inference on continuous Bayesian networks that combines discretization with the computational advantages of decision diagrams. In contrast to prior work on discretization techniques for BNs [174, 26], our approach evaluates a broad range of both supervised and unsupervised discretization methods across both causal and non-causal networks. This enables a thorough investigation into the trade-offs between computational efficiency and discretization quality.

Our results show that decision diagrams can significantly help with the scalability of inference in discretized BNs, achieving over an order-of-magnitude speedup relative to

²The full set of plots is available online at <https://doi.org/10.5281/zenodo.8211600>.

traditional inference techniques as discretization granularity increases. Regarding the quality of the discretization, we find that while finer discretization reduces the earth mover’s distance, the optimal bin count for minimizing inference error is dependent on both the sample size and the structural complexity of the BN.

Finally, we discuss a number of directions for future research. First, while our study focuses on binary decision diagrams, exploring the impact of discretization strategies on alternative decision diagrams, such as weighted positive binary decision diagrams (WPBDDs) [65] and affine algebraic decision diagrams (AADDs) [201], may show greater scalability or altogether different trade-offs between quality and cost. Second, the lack of standardized benchmark datasets for continuous BNs (especially those with established ground truth) limits systematic evaluation. We thus advocate for the development of such a benchmark set, building upon datasets like those in [208, 96]. And lastly, it would be valuable to compare our discretization-based method against other approximate inference methods for continuous BNs, including Monte Carlo sampling and variational inference techniques [138].

6.6 Appendix

This section contains details on the Bayesian network datasets that were generated for the empirical evaluation in Section 6.4. The various parameter configurations used for the linear Gaussian BNs are given in Tables 6.3 and 6.4, and the parameter configurations for the normal mixture model BNs are given in Table 6.5.

Table 6.3: Generated datasets from linear Gaussian BNs parameterized by Sobol sequences [176]. Each dataset consists of N samples from five correlated random variables A , B , C , D , and E . Each variable follows a normal distribution $\mathcal{N}(\mu, \sigma)$ with mean μ and standard deviation σ .

Dataset	N	$\Pr(A)$ $\mu = 20$ σ	$\Pr(B)$ $\mu = 20$ σ	$\Pr(C)$ $\mu = 15$ σ	$\Pr(D \mid A, B)$ $\mu = 2A + 3B$ σ	$\Pr(E \mid C, D)$ $\mu = 3C + 3D$ σ
LG101	100	1	1	1	1	1
LG102	1050	5.5	5.5	5.5	5.5	5.5
LG103	1525	3.25	3.25	3.25	3.25	3.25
LG104	575	7.75	7.75	7.75	7.75	7.75
LG105	813	4.38	4.38	4.38	4.38	4.38
LG106	1763	8.88	8.88	8.88	8.88	8.88
LG107	1288	2.13	2.13	2.13	2.13	2.13
LG108	338	6.63	6.63	6.63	6.63	6.63
LG109	456	3.81	3.81	3.81	3.81	3.81
LG110	1406	8.31	8.31	8.31	8.31	8.31
LG111	1881	1.56	1.56	1.56	1.56	1.56
LG112	931	6.06	6.06	6.06	6.06	6.06
LG113	694	2.69	2.69	2.69	2.69	2.69
LG114	1644	7.19	7.19	7.19	7.19	7.19
LG115	1169	4.94	4.94	4.94	4.94	4.94
LG116	219	9.44	9.44	9.44	9.44	9.44
LG117	278	5.22	5.22	5.22	5.22	5.22
LG118	1228	9.72	9.72	9.72	9.72	9.72
LG119	1703	2.97	2.97	2.97	2.97	2.97
LG120	754	7.47	7.47	7.47	7.47	7.47
LG121	991	1.84	1.84	1.84	1.84	1.84
LG122	1941	6.34	6.34	6.34	6.34	6.34
LG123	1466	4.09	4.09	4.09	4.09	4.09
LG124	516	8.59	8.59	8.59	8.59	8.59
LG125	397	2.41	2.41	2.41	2.41	2.41

Table 6.4: Generated datasets from linear Gaussian BNs parameterized by Sobol sequences [176]. Each dataset consists of N samples from five (correlated) random variables A , B , C , D , and E . Each variable follows a normal distribution $\mathcal{N}(\mu, \sigma)$ with mean μ and standard deviation σ . For these datasets, μ and σ are fixed for each random variable (a), and only the sample size is varied (b).

(a) Mean μ and standard deviation σ for each random variable.				
$\Pr(A)$	$\Pr(B)$	$\Pr(C)$	$\Pr(D \mid A, B)$	$\Pr(E \mid C, D)$
$\mathcal{N}(20, 2)$	$\mathcal{N}(20, 2)$	$\mathcal{N}(15, 2)$	$\mathcal{N}(2A + 3B, 2)$	$\mathcal{N}(3C + 3D, 2)$

(b) Sample size of each dataset.											
Dataset	LG9	LG10	LG11	LG12	LG13	LG14	LG15	LG16	LG17	LG18	LG19
N	5000	3000	2000	1000	800	600	500	400	300	200	100

Table 6.5: Generated datasets from normal mixture model BNs [166]. Each dataset consists of N samples from two (correlated) random variables A and B , where A follows a Bernoulli distribution $\text{Bern}(\Pr(A = 1))$, and B follows a normal distribution $\mathcal{N}(\mu, \sigma)$ with mean μ and standard deviation σ , conditioned on A .

Dataset	N	$\Pr(A)$	$\Pr(B \mid A = 1)$	$\Pr(B \mid A = 0)$
NM1	500	$\text{Bern}(0.5)$	$\mathcal{N}(21, 10)$	$\mathcal{N}(25, 1)$
NM2	500	$\text{Bern}(0.8)$	$\mathcal{N}(21, 10)$	$\mathcal{N}(25, 1)$
NM3	500	$\text{Bern}(0.5)$	$\mathcal{N}(6, 2)$	$\mathcal{N}(4, 2)$
NM4	500	$\text{Bern}(0.8)$	$\mathcal{N}(6, 2)$	$\mathcal{N}(4, 2)$
NM5	100	$\text{Bern}(0.5)$	$\mathcal{N}(21, 10)$	$\mathcal{N}(25, 1)$
NM6	100	$\text{Bern}(0.8)$	$\mathcal{N}(21, 10)$	$\mathcal{N}(25, 1)$
NM7	100	$\text{Bern}(0.5)$	$\mathcal{N}(6, 2)$	$\mathcal{N}(4, 2)$
NM8	100	$\text{Bern}(0.8)$	$\mathcal{N}(6, 2)$	$\mathcal{N}(4, 2)$

Chapter 7

Quantum graph-state synthesis with SAT

In quantum computing and quantum information processing, graph states are a specific type of quantum state that are commonly used in quantum networking and quantum error correction. A recurring problem is finding a transformation from a given source graph state to a desired target graph state using only local operations. Recently, it has been shown that deciding transformability is already **NP**-hard. In this chapter, we present a CNF encoding for both local and non-local graph state operations, corresponding to one- and two-qubit Clifford gates and single-qubit Pauli measurements. We use this encoding in a bounded-model-checking set-up to synthesize the desired transformation. Additionally, for a completeness threshold on local transformations, we provide an upper bound on the length of the transformation if it exists. We evaluate the approach in two settings: the synthesis of the ubiquitous GHZ state from a random graph state, for varying numbers of qubits, and GHZ-state synthesis for a proposed 14-node quantum network. We find that, when including two-qubit operations, the approach can synthesize transformations for graphs up to 17 qubits in under 30 minutes. In the setting with only single-qubit operations, significant performance improvements—up to 50 qubits in under 10 minutes—can be obtained by using domain-specific knowledge to add additional constraints.

This chapter is an extended version of work that has been published as: Sebastiaan Brand, Tim Coopmans, and Alfons Laarman. “Quantum graph-state synthesis with SAT”. in: *14th International Workshop on Pragmatics of SAT*. CEUR-WS. 2023. This chapter includes additional results (Sec. 7.3.3 + Fig. 7.7) that significantly improve on the published results under certain conditions.

7.1 Introduction

The creation, manipulation and transmission of quantum information brings into reach applications which are unfeasible or even impossible using classical computers, such as provably-secure communication [23, 84], more accurate clock synchronization [131], and chemistry applications [43]. Various questions regarding simulation, modeling and design of quantum computers and networks can be phrased using graph states, a subset of all possible states of a register of quantum bits (qubits) which can be described using graphs [109]. Additionally, graph states are crucial to a universal model of quantum computation called measurement-based quantum computing [186]. Furthermore, when augmented with a finite set of quantum operations called Clifford gates and single-qubit Pauli measurements, the graph-state formalism gives rise to efficient classical simulation of a large class of quantum circuits [14] and forms the basis for many quantum error correction schemes [100], a prerequisite for scaling up quantum computing with imperfect devices, as well as many quantum-networking applications [119, 109]. These applications have a focus on *local* quantum operations, i.e. on a single or few spatially-close qubits, for reasons regarding experimental implementation with imperfect devices.

Given this wide applicability, graph-state transformations have been extensively studied from a theoretical standpoint for various sets of allowed local quantum operations [168, 167, 110, 127, 109, 86]. In this chapter, we consider the following problem: given a source graph state, synthesize a desired target graph state using single-qubit Clifford gates, single-qubit Pauli measurement, and two-qubit Clifford gates between selected pairs of qubits. When only considering single-qubit Clifford gates and measurements, this problem was shown before [62] to be equivalent to transforming the associated graphs under two graph operations: an edge-toggling operation called *local complementation* (LC), corresponding to single-qubit Cliffords, and *vertex deletion* (VD), corresponding to measurements. Furthermore, two-qubit Clifford operations can be added to this graph description as *edge flips* (EF) (removing or adding arbitrary edges). The decision problem “Can a source graph be transformed to a target graph under LC+VD?” has been shown to be NP-complete [60], even when restricting the target graph to a practically-relevant scenario [61]. Although there exists an algorithm [62] (based on techniques from [56, 55]) which is fixed-parameter tractable (FPT) in the rank-width r of the graph, the authors of the algorithm themselves remark it is not useful in practice due to a giant FPT-prefactor equalling ten times repeated exponentiation with base 2 (i.e. $2^{2^{\dots 2^r}}$) [60]. Additionally, for determining LC+VD

transformability, Dahlberg et al. [62] reduce the search space based on a necessary and sufficient condition. The size of the search space however remains exponential in the number of nodes n , and their brute-force algorithm for traversing the space runs in $O(3^{n-k} \cdot \text{poly}(n, k))$ time, where k is the number of required vertex deletions.

We tackle the problem of graph-state synthesis under LC+VD+EF with bounded model checking (BMC) [50, 28]. To this end we present a Boolean encoding for graph states and the operations on them, and provide a completeness threshold for reachability under LC+VD. This approach can be applied to arbitrary graphs, in contrast to special cases for which poly-time algorithms have been found [60, 105] or unsatisfiability can be determined analytically [104, 130]. We evaluate this approach in two settings of particular interest [60, 130]: first, we synthesize the ubiquitous Greenberger–Horne–Zeilinger (GHZ) state [101] from random graphs with varying number of qubits. Next, we target a 14 node quantum network proposal [184]. We find that when searching for transformations under LC+VD+EF, where the pairs of nodes between which edge flips are allowed to take place can be selected, the BMC approach finds transformations for graphs up to 17 nodes (qubits) within 30 minutes. For transformations under LC+VD the BMC approach can be made significantly faster, finding transformations for graphs up to 50 nodes in under 10 minutes. In comparison, for transformations under single-qubit Clifford operations without measurements (a setting where deciding reachability is in P [36, 169] and counting reachable graphs is #P-complete [59]), various properties of equivalence classes have been explored up to 12 qubits [66, 42, 5]. Our approach can also find transformations under only LC+VD, although in this setting it is not faster than the brute-force algorithm from [62].

Aside from graph problems which have been tackled with SAT-based methods [117, 194, 206, 94, 152, 150, 151, 54], SAT has also been used on problems in quantum computing. For example, synthesizing optimal Clifford circuits without measurements (closely related to graph-state synthesis under LC + flipping arbitrary edges, but without VD) has been tackled with BMC [207]. Without the optimality constraint (i.e. shortest circuit) this problem is in P [2], while the complexity with the optimality constraint is unknown. SAT-based techniques have also been applied to quantum circuit equivalence checking for a limited selection of circuits [238]. BMC specifically has been applied to Clifford circuit (without measurements) equivalence checking [24] (a problem that is also in P [13]), and SMT and planning based approaches have been used to map logical quantum circuits to physical quantum-chips [223, 211]. Unlike much previous work we include measurements, which for our problem raises the complexity from P to NP-complete.

7.2 Preliminaries and problem definition

7.2.1 Quantum computing

We very briefly introduce quantum bits (qubits) and how to act on them with quantum gates and measurements (see [171] for a complete introduction). Where the state of a classical system can typically be described by a number of bits, the state of a quantum computer is described by the state of its quantum bits (qubits). Quantum states are usually denoted by $|\cdot\rangle$, which represents some vector. The state $|\psi\rangle$ of a single qubit is a complex 2-vector of unit norm, equalling the *computational-basis states* $|0\rangle = \begin{pmatrix} 1 & 0 \end{pmatrix}^\top$ or $|1\rangle = \begin{pmatrix} 0 & 1 \end{pmatrix}^\top$ or any linear combination of those, i.e. in general a single-qubit state is $|\psi\rangle = \alpha_0 |0\rangle + \alpha_1 |1\rangle = \begin{pmatrix} \alpha_0 & \alpha_1 \end{pmatrix}^\top$ for complex numbers α_0, α_1 satisfying $|\alpha_0|^2 + |\alpha_1|^2 = 1$ (here, $\top$ denotes vector transposition). A general n -qubit quantum state is represented as a complex vector of length 2^n with norm 1, e.g. $\begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{i}{\sqrt{2}} \end{pmatrix}^\top$ and $\begin{pmatrix} \frac{2}{\sqrt{13}} & 0 & 0 & -\frac{3}{\sqrt{13}} \end{pmatrix}^\top$ are quantum states. The joint state of two separate quantum registers in states $|\phi\rangle, |\psi\rangle$ is $|\phi\rangle \otimes |\psi\rangle$, where $\otimes$ denotes the tensor product: given $r_V \times c_V$ matrix V and $r_W \times c_W$ matrix W , the $r_V r_W \times c_V c_W$ matrix $V \otimes W$ is

$$V \otimes W = \begin{pmatrix} V_{00}W & V_{01}W & \dots & V_{0c_V}W \\ \vdots & \vdots & \ddots & \\ V_{r_V 0}W & V_{r_V 1}W & \dots & V_{r_V c_V}W \end{pmatrix}.$$

Given a bipartition $A \cup B = \{1, 2, \dots, n\}$, an n -qubit state $|\psi\rangle$ is called *separable over* A, B if we can write $|\psi\rangle = |\varphi\rangle_A \otimes |\phi\rangle_B$. It is *entangled* otherwise, a feature that has no classical analogue and is a prerequisite to many applications with a quantum advantage. For example $\begin{pmatrix} \frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} & 0 \end{pmatrix}^\top = \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix}^\top \oplus \begin{pmatrix} 1 & 0 \end{pmatrix}^\top$ is not entangled, but $\begin{pmatrix} \frac{1}{\sqrt{2}} & 0 & 0 & \frac{1}{\sqrt{2}} \end{pmatrix}^\top$ is.

A quantum gate (always reversible) on n qubits is given by a $2^n \times 2^n$ unitary matrix and the output state can be found by matrix-vector multiplication, for example H which maps input $\begin{pmatrix} 1 & 0 \end{pmatrix}^\top$ to output

$$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix}$$

An example universal gate set is shown below.

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix} \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad CZ = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}$$

The tensor product is used to apply gates in parallel to separate registers, e.g. $I \otimes H \otimes I$ is a 3-qubit gate performing a H on the second qubit and I on the first and third. The result of a two-qubit gate (e.g. controlled- Z (CZ), which maps e.g. $\begin{pmatrix} \frac{1}{\sqrt{2}} & 0 & 0 & \frac{1}{\sqrt{2}} \end{pmatrix}^T$ to $\begin{pmatrix} \frac{1}{\sqrt{2}} & 0 & 0 & -\frac{1}{\sqrt{2}} \end{pmatrix}^T$) between two non-adjacent qubits can be computed by swapping qubits: e.g. for qubits q_0, q_1, q_2 ,

$$CZ(q_0, q_2) = \text{SWAP}(q_1, q_2) CZ(q_0, q_1) \text{SWAP}(q_1, q_2),$$

where $\text{SWAP}(q_1, q_2)$ replaces $|a\rangle \otimes |b\rangle \otimes |c\rangle \rightarrow |a\rangle \otimes |c\rangle \otimes |b\rangle$ for $a, b, c \in \{0, 1\}$. The gates H, T^2 together generate (under matrix multiplication and tensoring with I) the group of *single-qubit Clifford gates*, and H, T^2, CZ together generate all Clifford gates.

A computational-basis measurement is a non-reversible operation which projects a single qubit state $\alpha_0 |0\rangle + \alpha_1 |1\rangle$ to one of $|0\rangle, |1\rangle$ with probability $|\alpha_0|^2$ or $|\alpha_1|^2$. For example, measuring a qubit $|\psi\rangle = \sqrt{\frac{1}{3}} |0\rangle + \sqrt{\frac{2}{3}} |1\rangle$ yields the state $|0\rangle$ with probability $1/3$ and the state $|1\rangle$ with probability $2/3$. Any n -qubit state $|\psi\rangle$ can be written as $|\psi\rangle = \alpha |0\rangle \otimes |\psi_0\rangle + \beta |1\rangle \otimes |\psi_1\rangle$ where $|\alpha|^2$ ($|\beta|^2$) is the probability of finding the first qubit in the $|0\rangle$ ($|1\rangle$) state after measuring it (for expressing measurement on the other qubits, swap qubits first). A Pauli measurement equals a computational-basis measurement preceded by a single-qubit Clifford gate. Sequences of quantum operations are typically visualized in a quantum circuit, an example of which is given in Figure 7.1.

7.2.2 Graph states

Graph states are a subset of all quantum states. An n -qubit graph state $|G\rangle$ is represented by an undirected simple graph $G = (V, E)$ with $|V| = n$ vertices and no self-loops (where V is the vertex set and $E \subseteq V \times V$ the edge set), constructed as starting from the state $H^{\otimes n} |0\rangle^{\otimes n}$, followed by a CZ gate on each pair of qubits $(u, v) \in E$. An example is given in Figure 7.2. From here on we say ‘graph’ to mean ‘undirected simple graph without self-loops’. Given a graph $G = (V, E)$, the

7.2. Preliminaries and problem definition

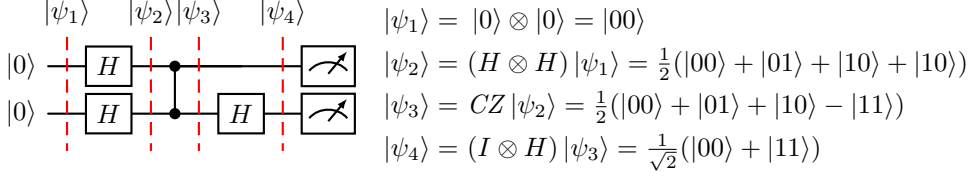


Figure 7.1: An example 2-qubit quantum circuit and corresponding intermediate states. Operations are applied from left to right. The controlled- Z (CZ) gate is visualized as $\bullet$. As is common, we write $|01\rangle$ as shorthand for $|0\rangle \otimes |1\rangle$, $|10\rangle = |1\rangle \otimes |0\rangle$, etc. Measuring both qubits at the end gives $|00\rangle$ or $|11\rangle$ with equal probability.

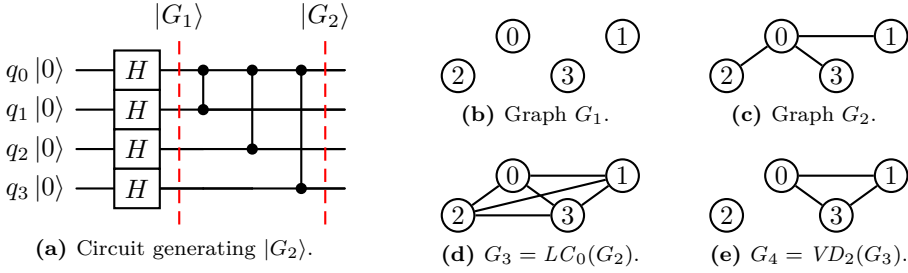


Figure 7.2: The circuit in (a) generates the state $|G_2\rangle$, corresponding to the graph in (c). Examples of local complementation and vertex deletion are shown in (d) and (e).

7

corresponding quantum state $|G\rangle$ is constructed as follows: starting from the state $|00\dots 0\rangle$ an H gate is applied on every qubit, followed by controlled- Z gates on all pairs of qubits $(u, v) \in E$. Intuitively, the graph G captures information about the entanglement between the qubits, where two qubits are entangled if they are (directly or indirectly) connected in the graph.

The two graph transformations corresponding to single-qubit quantum operations are:

- *Local complementation* LC_k on vertex $k \in V$ transforms $G = (V, E)$ into $LC_k(G) = (V, E')$ where E' is obtained from E by flipping the edges in the neighborhood of k , i.e. for all $u, v \in \mathcal{N}_k$, if $(u, v) \in E$ then $(u, v) \notin E'$ and if $(u, v) \notin E$ then $(u, v) \in E'$. Here, the neighborhood $\mathcal{N}_k$ is the set of all vertices adjacent to k , i.e. $\mathcal{N}_k = \{v \mid (k, v) \in E\}$. For any graphs G and G' , $|G'\rangle$ is reachable from $|G\rangle$ using only single-qubit Clifford operations if and only if G' is reachable from G using local complementations. More specifically, the graph

state $|LC_k(G)\rangle$ equals the resulting quantum state when applying a certain sequence of single-qubit Clifford operations to $|G\rangle$ (see [168] for details).

- *Vertex deletion* VD_k of vertex $k \in V$ transforms $G = (V, E)$ to $VD_k(G) = (V, E')$ with $E' = E \setminus \{(v, k) \mid v \in V\}$, i.e. k becomes *isolated* (all edges adjacent to k are removed). Vertex deletion of vertex k implements measurement on qubit k : for each graph G , the graph state $|VD_k(G)\rangle$ is single-qubit Clifford equivalent to $|G\rangle$ at which a computational-basis measurement has been performed on qubit k [110].

Additionally, we can also consider two-qubit operations, possibly on a selectable subset of all pairs of nodes:

- Given a subset of pairs of nodes $D \subseteq V \times V$ (for convenience $u < v$ for $(u, v) \in D$), G can be transformed into G' by *edge flips* among D and local complementations on vertices in V if and only if $|G\rangle$ can be transformed into $|G'\rangle$ using two-qubit Clifford operations on the qubit pairs in D and single-qubit Clifford on qubits in V [86; Th.1].

7.2.3 Graph state reachability

Rather than generating a graph state from scratch using CZ gates (as in Figs. 7.2a to 7.2c) a problem of interest for e.g. quantum networking is to obtain a particular graph from an existing graph *using only single-qubit operations* (LC+VD, and $D = \emptyset$). Below is a practical example.

Example 7.1. *Alice is part of a 6-node quantum network and wants to run a quantum secret sharing scheme [119] between herself and three other parties, each having one qubit. For this she needs a 4-qubit Greenberger–Horne–Zeilinger (GHZ) state [101], given by G_{GHZ} in Figure 7.3. However, generating $|G_{GHZ}\rangle$ using CZ-gates (as in Figs. 7.2a to 7.2c)) requires generating entanglement [83, 170; Fig.2.4], which is a time-consuming probabilistic process [200]. At some point in time the network is a state $|G_s\rangle$. Because single-qubit operations (LC+VD on the graph) are much easier to perform than entanglement generation, Alice wants to know whether a given G_s can be transformed into G_{GHZ} using only LC+VD.*

This motivates the problem we study in this chapter, posed before in [60] for single-qubit operations (LC+VD and $D = \emptyset$) and in [86] for multi-qubit operations (LC+VD and $D \neq \emptyset$).

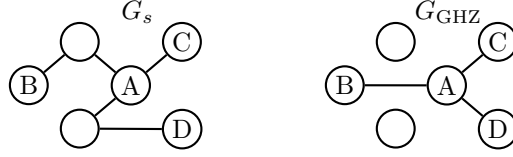


Figure 7.3: Example problem: can G_s be transformed into G_{GHZ} using LC+VD?

Definition 7.1 (Graph-state synthesis). *Given source and target graphs $G_s = (V, E_s)$ and $G_t = (V, E_t)$, find (if it exists) a sequence of local complementations and vertex deletions on any $v \in V$ (and also edge flips on $(u, v) \in D$ for some given $D \subseteq V \times V$ in case multi-qubit Clifford operations are allowed on D) which transforms G_s into G_t .*

We remark that if $D = V \times V$, any graph can be trivially synthesized because an edge may be added or removed between any pair of nodes (Figs. 7.2a to 7.2c). We also remark that we are not necessarily interested in the shortest sequence of graph transformations, as any sequence of LC+VD translates to at most one single-qubit Clifford and one measurement per qubit.

7.3 SAT encoding

As seen in the previous section, quantum operations on graph states can be expressed through graph transformations. In this section, we give Boolean encodings for these operations, as well as an encoding for the transition relation as a whole. In Section 7.5 we detail how these Boolean expressions are written in conjunctive normal form (CNF).

The encoding of a single transformation step from graph G to G' uses variables $\vec{x}$ for G and $\vec{x}'$ for G' . A particular graph G is encoded as a conjunction over variables $\vec{x}$ which specifies which edges are present in the graph.

Definition 7.2 (Graph encoding). *An undirected graph G of n vertices is encoded as a conjunction over $n(n-1)/2$ literals x_{uv} ($\bar{x}_{uv}$), for $(u, v) \in \mathbb{U} = \{(u, v) \in V \times V \mid u < v\}$, indicating there is (not) an edge between nodes u and v .*

7.3.1 Encoding of graph transformations

The Boolean encoding for deleting a vertex k , denoted VD_k , is given in Eq. (7.1). All edges (u, v) connected to k are set to false ($\bar{x}'_{uv}$) while all others remain unchanged ($x'_{uv} \Leftrightarrow x_{uv}$).

$$\text{VD}_k = \bigwedge_{(u,v) \in \mathbb{U}} \begin{cases} \bar{x}'_{uv} & \text{if } u = k \text{ or } v = k \\ x'_{uv} \Leftrightarrow x_{uv} & \text{otherwise.} \end{cases} \quad (7.1)$$

The encoding for performing a local complementation on vertex k , denoted LC_k , is given in Eq. (7.2) and can be read as follows: if vertices u, v are in the neighborhood of k ($x_{uk} \wedge x_{vk}$) then the value of the edge (u, v) is flipped ($x'_{uv} \Leftrightarrow \neg(1 \oplus \bar{x}_{uv})$).

$$\text{LC}_k = \bigwedge_{(u,v) \in \mathbb{U}} \begin{cases} x'_{uv} \Leftrightarrow \neg((x_{uk} \wedge x_{vk}) \oplus \bar{x}_{uv}) & \text{if } u \neq k \text{ and } v \neq k \\ x'_{uv} \Leftrightarrow x_{uv} & \text{otherwise.} \end{cases} \quad (7.2)$$

To encode edge flips on a selection of edges D (Definition 7.1), we take D to be an indexed set $D = \{(u_1, v_1), (u_2, v_2), \dots\}$ with $u_i < v_i$. Given this indexed set, the constraint in Eq. (7.3) encodes an edge flip of (u_i, v_i) .

$$\text{EF}_i = \bigwedge_{(u,v) \in \mathbb{U}} \begin{cases} x'_{uv} \oplus x_{uv} & \text{if } u = u_i \text{ and } v = v_i \\ x'_{uv} \Leftrightarrow x_{uv} & \text{otherwise} \end{cases} \quad (7.3)$$

In order to combine the transition relations LC_k , VD_k , and EF_i into a single CNF formula we use a construction similar to the BMC encoding of different concurrent threads in [185]: we add $\lceil \log_2(\max(|V|, |D|) + 1) \rceil$ variables $\vec{y}$ for the binary encoding of $k \in V$ or $i \in \{1, \dots, |D|\}$, and two variables $\vec{z}$ to indicate whether a given operation is a local complementation ($\vec{z} = 0$), a vertex deletion ($\vec{z} = 1$), or an edge flip ($\vec{z} = 2$). For example the constraint $\vec{y} = 3 \wedge \vec{z} = 1$ represents vertex deletion of node 3. Using these additional variables, we encode all local complementations, vertex deletions, and edge flips as in Equations (7.4) to (7.6).

$$R_{\text{LC}}(\vec{x}, \vec{x}') = \bigwedge_{k \in V} \left[(\vec{y} = k \wedge \vec{z} = 0) \Rightarrow \text{LC}_k(\vec{x}, \vec{x}') \right] \quad (7.4)$$

$$R_{\text{VD}}(\vec{x}, \vec{x}') = \bigwedge_{k \in V} \left[(\vec{y} = k \wedge \vec{z} = 1) \Rightarrow \text{VD}_k(\vec{x}, \vec{x}') \right] \quad (7.5)$$

$$R_{\text{EF}}(\vec{x}, \vec{x}') = \bigwedge_{i \in \{1, \dots, |D|\}} \left[(\vec{y} = i \wedge \vec{z} = 2) \Rightarrow \text{EF}_i(\vec{x}, \vec{x}') \right] \quad (7.6)$$

Additionally we add an identity transition $R_{\text{Id}}(\vec{x}, \vec{x}') = (\vec{z} = 3) \Rightarrow \text{Id}(\vec{x}, \vec{x}')$ to ensure that if a transformation of length d exists, a transformation of length $d' \geq d$ also exists (to avoid searching over all d), and we appropriately constrain the unused

7.3. SAT encoding

values of $\vec{y}$ and $\vec{z}$ by adding $C = (\vec{y} < |V| \vee z = 2) \wedge (\vec{y} < |D| \vee z \neq 2)$. Finally, we obtain the global transition relation in Eq. (7.7). When converted to CNF this formula has $m + n(n - 1)$ variables and $\leq 3.5n^3 + 2mn^2 + 0.5n^2 + 0.5|D|n^2$ clauses, where $n = |V|$ and $m = \lceil \log_2(\max(|V|, |D|) + 1) \rceil$.

$$R_{\text{global}}(\vec{x}, \vec{x}') = R_{\text{LC}} \wedge R_{\text{VD}} \wedge R_{\text{EF}} \wedge R_{\text{Id}} \wedge C \quad (7.7)$$

We use the transition relation specified in Eq. (7.7) in a bounded-model-checking set-up, i.e. we create Eq. (7.8) below, where $S(\vec{x}_1)$ encodes a source graph G_s , $T(\vec{x}_d)$ a target graph G_t , and d is the search depth.

$$\text{BMC}(G_s, G_t, d) = S(\vec{x}_1) \wedge \bigwedge_{i=1}^{d-1} R_{\text{global}}(\vec{x}_i, \vec{x}_{i+1}) \wedge T(\vec{x}_d) \quad (7.8)$$

The formula is satisfiable if and only if a sequence of operations of at most d steps exists that transforms G_s into G_t . In Sec. 7.3.2, we prove an upper bound on the required depth d for transformations under LC+VD.

7.3.2 Completeness threshold

To provide a completeness threshold for graph-state synthesis under LC+VD, we use the following observations to bound the search depth.

7

1. If G_s can be transformed to G'_s under LC, a transformation exists of at most M local complementations, where $M = 3(|V| - s)/2$ with $s = |V| \pmod{2}$ [36; §4].
2. If G_s can be transformed into G_t under LC+VD, then vertex deletion needs to be performed on exactly the Δ vertices that are isolated in G_t .¹
3. For $k \in V$, LC_k after VD_k leaves the graph unchanged, i.e. $LC_k(VD_k(G)) = VD_k(G)$.
4. For $j, k \in V$ and $j \neq k$, the transformations LC_j and VD_k commute, i.e. $LC_j(VD_k(G)) = VD_k(LC_j(G))$.

From points 3 and 4, it follows that all vertex deletions (measurements) can be postponed until after the local complementations (single-qubit Clifford gates). We then get that if G_s can be transformed into G_t under LC+VD, it can be transformed by a

¹Without loss of generality, we assume G_s has no isolated vertices. If G_s has isolated vertices that are not isolated in G_t , then G_t is trivially unreachable under LC+VD.

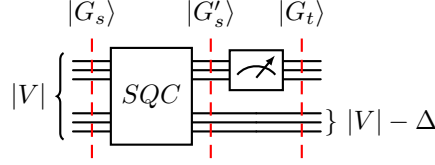


Figure 7.4: Graph-state transformation circuit under LC+VD.

circuit of the form given in Figure 7.4, taking at most M local complementations and Δ vertex deletions.

7.3.3 Improved constraint for LC+VD

While the encoding in Eq. (7.8) is sufficient for constraining the problem, we can add additional constraints that do not change whether the formula is satisfiable, but can make it easier for the solver to find solutions. Specifically, from the previous section, we know that for transformations under LC+VD, if a transformation exists, a transformation exists of the form of Figure 7.4. The constraint that enforces solutions to be of this form, i.e. that enforces vertex deletions to only happen at the end, can be encoded as

$$\text{VDs-end}(M, \Delta) = \bigwedge_{i=1}^M (\vec{z}_i = 0) \wedge \bigwedge_{i=M+1}^{M+\Delta} (\vec{z}_i = 1 \wedge \vec{y}_i = (i - (M + 1))), \quad (7.9)$$

with M and Δ as defined in Section 7.3.2. The first conjunction encodes that the first M operations must be local complementations ($\vec{z}_i = 0$, see Eq. (7.4)). The second conjunction encodes that the last Δ operations must be vertex deletions ($\vec{z}_i = 1$, see Eq. (7.5)), and that they need to happen on the isolated qubits that we assume (w.l.o.g.) have indices 0 through $\Delta - 1$ ($\vec{y}_i = (i - (M + 1))$, see Eq. (7.5)). We can add this constraint to the expression in Equation (7.8) and solve the following formula.

$$\text{BMC}(G_s, G_t, d) \wedge \text{VDs-end}(M, \Delta) \quad (7.10)$$

7.4 Empirical evaluation

We evaluate our approach in two settings: synthesizing a GHZ state from random graphs for an increasing number of qubits, and synthesis of graphs based on a proposal of a 14 node quantum network in the Netherlands [184]. For all experiments, we perform binary search over d up to the completeness threshold specified in Section 7.3.2.

7.5. Empirical evaluation

In our current setup, the solver is restarted for every different d . Experiments² were run on Ubuntu 18 with an AMD Ryzen 7 5800x CPU. Two different SAT solvers, Glucose 4 [17] and Kissat [29], have been used.

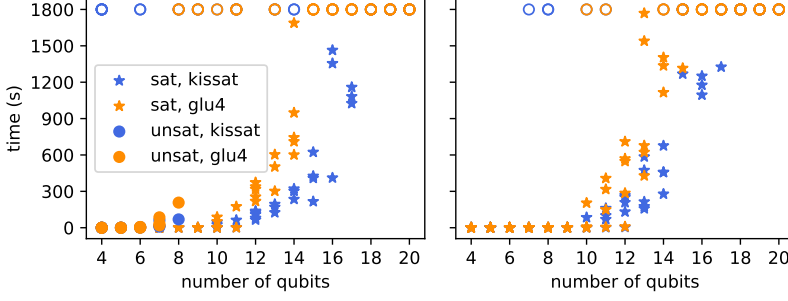
We first evaluate our approach in a setting where the target states are 4-qubit GHZ states (see Example 7.1), matching the target states in the empirical evaluation in [60]. Up to single-qubit Cliffords, a GHZ state is represented by a star graph (or equivalently a complete graph, since these only differ by a single local complementation). GHZ states are used in a large number of applications such as quantum secret sharing [119], anonymous transfer [46], and conference key agreement [188]. To evaluate our method we use Erdős-Rényi random graphs as source graphs, which have also been used in other work concerning graph-state synthesis [147, 145]. Results are shown in Figure 7.5.

We find that with a timeout of 30 minutes, Kissat can synthesize transformations under LC+VD+EF (with edge flips on a selectable subset of pairs of nodes) for graphs up to 17 qubits. Determining unreachability under LC+VD, which we do using the completeness threshold, can be done up to 8 qubits by Glucose within this timeout. When determining reachability under LC+VD, our approach using the encoding of Equation (7.8) performs similarly to the LC+VD+EF setting, although here it does not outperform the brute-force algorithm from Dahlberg et al. [62], which can find LC+VD transformations for 20 node graphs in under one minute.

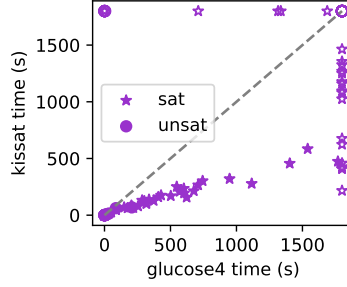
Next, we evaluate our approach on the specific quantum network architecture proposed in [184; Fig.3] (visualized in Figure 7.6a). As source states, we consider graphs with nodes from this network, and random edges as follows: $(u, v) \in E$ with probability p^d , where d is the distance (number of hops + 1) between the nodes, motivated by the fact that generating entanglement over larger distances is harder [200]. The target state is a GHZ state between the main network nodes (squares in Fig. 7.6a). Figure 7.6b shows the results for varying p . A higher p corresponds to a larger amount of entanglement in the network. We observe that for a fixed number of nodes, the time it takes to synthesize a transformation increases with the density of the source graph.

Finally, we evaluate the encoding that makes use of the additional constraint, given in Equation (7.10). The results are shown in Figure 7.7. We find that this additional constraint significantly speeds up the solve time. In fact, the speedup is so significant that now the time it takes to encode the problem in a CNF formula (which was relatively negligible in Figs. 7.5 and 7.6) starts to dominate the total runtime.

²Reproducible experiments are available online at <https://github.com/sebastianbrand/graph-state-synthesis>.



(a) Synthesis under LC+VD (left) and LC+VD+limited EF (right).



(b) Solver comparison.

Figure 7.5: The total SAT solver time for BMC (using Eq. (7.8)) with binary search over the depth up to the completeness threshold (see Section 7.3.2). For each number of qubits we run on three Erdős-Rényi random graphs with $p = 0.8$, with a 4-qubit GHZ state as target, with only LC+VD on the left, and LC+VD+EF on a random set D with $|D| = \frac{1}{2}|V|$ on the right. (b) shows the difference between the solvers for the data points from both the left and right plot in (a). Open symbols indicate timeouts. Solid spheres indicate unreachability at the depth of the completeness threshold. The largest solved instance is for 17 qubits at $d = 16$, which has a formula with ~ 2400 variables and $\sim 300,000$ clauses (see above Equation (7.7) for $d = 1$).

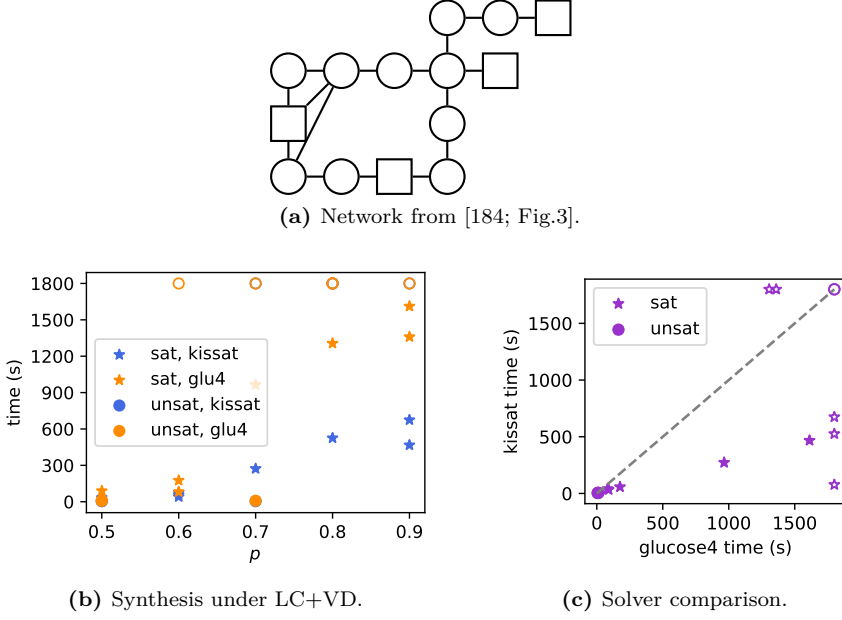


Figure 7.6: The 14-node quantum network proposed in [184; Fig.3], and the SAT solver time to synthesize a transformation into a GHZ state for different amounts of entanglement (p) in the network. Open circles indicate timeouts. Solid spheres indicate unreachability at the depth of the completeness threshold.

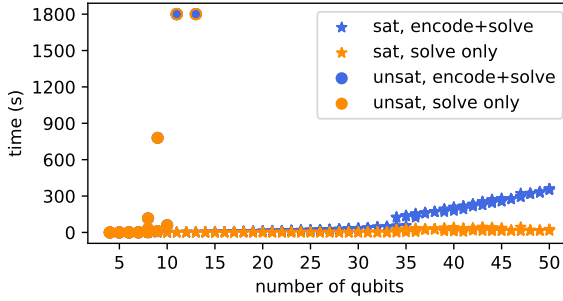


Figure 7.7: The total SAT solver time, using Kissat only, for BMC using Eq. (7.10) with binary search over the depth up to the completeness threshold (see Section 7.3.2). For each number of qubits, we run on three Erdős-Rényi random graphs with $p = 0.8$, with a 4-qubit GHZ state as target.

7.5 Appendix

In this section, we give full details on the encoding of the transition relation from Section 7.3 in conjunctive normal form (CNF), i.e. a conjunction over clauses which are each a disjunction over literals (a literal is variable x_{uv} or its negation $\bar{x}_{uv}$). For this we use standard identities such as distributivity ($x \vee (y \wedge z) = (x \vee y) \wedge (x \vee z)$) and De Morgan's law ($\neg(x \wedge y) = (\bar{x} \vee \bar{y})$).

We start with the expressions for VD_k , LC_k , and EF_i given in Eqs. (7.1) to (7.3).

$$\text{VD}_k = \bigwedge_{(u,v) \in \mathbb{U}} \begin{cases} \bar{x}'_{uv} & \text{if } u = k \text{ or } v = k \\ x'_{uv} \Leftrightarrow x_{uv} & \text{otherwise.} \end{cases} \quad (7.1)$$

$$\text{VD}_k^{\text{CNF}} = \bigwedge_{(u,v) \in E} \begin{cases} \bar{x}'_{uv} & \text{if } u = k \text{ or } v = k \\ (x'_{uv} \vee \bar{x}_{uv}) \wedge (\bar{x}'_{uv} \vee x_{uv}) & \text{otherwise.} \end{cases} \quad (7.11)$$

$$\text{LC}_k = \bigwedge_{(u,v) \in \mathbb{U}} \begin{cases} x'_{uv} \Leftrightarrow \neg((x_{uk} \wedge x_{vk}) \oplus \bar{x}_{uv}) & \text{if } u \neq k \text{ and } v \neq k \\ x'_{uv} \Leftrightarrow x_{uv} & \text{otherwise.} \end{cases} \quad (7.2)$$

$$\text{LC}_k^{\text{CNF}} = \bigwedge_{(u,v) \in E} \begin{cases} g & \text{if } u \neq k \text{ and } v \neq k \\ (x'_{uv} \vee \bar{x}_{uv}) \wedge (\bar{x}'_{uv} \vee x_{uv}) & \text{otherwise.} \end{cases} \quad (7.12)$$

with

$$g = (\bar{x}_{uk} \vee \bar{x}_{vk} \vee x'_{uv} \vee x_{uv}) \wedge (\bar{x}_{uk} \vee \bar{x}_{vk} \vee \bar{x}'_{uv} \vee \bar{x}_{uv}) \wedge (x_{uk} \vee x'_{uv} \vee \bar{x}_{uv}) \\ \wedge (x_{vk} \vee x'_{uv} \vee \bar{x}_{uv}) \wedge (x_{uk} \vee \bar{x}'_{uv} \vee x_{uv}) \wedge (x_{vk} \vee \bar{x}'_{uv} \vee x_{uv})$$

$$\text{EF}_i = \bigwedge_{(u,v) \in \mathbb{U}} \begin{cases} x'_{uv} \oplus x_{uv} & \text{if } u = u_i \text{ and } v = v_i \\ x'_{uv} \Leftrightarrow x_{uv} & \text{otherwise} \end{cases} \quad (7.3)$$

$$\text{EF}_i^{\text{CNF}} = \bigwedge_{(u,v) \in \mathbb{U}} \begin{cases} (x'_{uv} \vee x_{uv}) \wedge (\bar{x}'_{uv} \vee \bar{x}_{uv}) & \text{if } u = u_i \text{ and } v = v_i \\ (x'_{uv} \vee \bar{x}_{uv}) \wedge (\bar{x}'_{uv} \vee x_{uv}) & \text{otherwise} \end{cases} \quad (7.13)$$

Next we need expressions for $\vec{y} \neq b$ and $\vec{y} \leq b$. For n Boolean variables $\vec{y}$ and an integer $0 \leq b < 2^n$, with $\text{bin}(b)$ the binary encoding of b and b_i the i -th bit of $\text{bin}(b)$,

7.5. Appendix

we encode $\vec{y} \neq b$ and $\vec{y} \leq b$ as in Eqs. (7.14) and (7.15).

$$f_{\neq}(\vec{y}, b) = \bigvee_{\substack{b_i \in \text{bin}(b) \\ b_i = 0}} y_i \vee \bigvee_{\substack{b_i \in \text{bin}(b) \\ b_i = 1}} \bar{y}_i \quad (7.14)$$

$$f_{\leq}(\vec{y}, b) = \bigwedge_{\substack{b_i \in \text{bin}(b) \\ b_i = 0}} \left(\bar{y}_i \vee \bigvee_{\substack{b_j \in \text{bin}(b) \\ j > i, b_j = 1}} \bar{y}_j \right) \quad (7.15)$$

For the CNF encoding of R_{LC} , R_{VD} , and R_{EF} (Equations (7.4) to (7.6)), the construction $A \Rightarrow B$ can be written as $(\bar{A} \vee B)$. Additionally, in the following we let $c \in f$ denote a clause c in a CNF expression f .

$$R_{\text{LC}}(\vec{x}, \vec{x}') = \bigwedge_{k \in V} \left[(\vec{y} = k \wedge \vec{z} = 0) \Rightarrow \text{LC}_k(\vec{x}, \vec{x}') \right] \quad (7.4)$$

$$R_{\text{LC}}^{\text{CNF}}(\vec{x}, \vec{x}') = \bigwedge_{k \in V} \bigwedge_{c \in \text{LC}_k^{\text{CNF}}} \left(f_{\neq}(\vec{y}, k) \vee f_{\neq}(\vec{z}, 0) \vee c \right) \quad (7.16)$$

$$R_{\text{VD}}(\vec{x}, \vec{x}') = \bigwedge_{k \in V} \left[(\vec{y} = k \wedge \vec{z} = 1) \Rightarrow \text{VD}_k(\vec{x}, \vec{x}') \right] \quad (7.5)$$

$$R_{\text{VD}}^{\text{CNF}}(\vec{x}, \vec{x}') = \bigwedge_{k \in V} \bigwedge_{c \in \text{VD}_k^{\text{CNF}}} \left(f_{\neq}(\vec{y}, k) \vee f_{\neq}(\vec{z}, 1) \vee c \right) \quad (7.17)$$

$$R_{\text{EF}}(\vec{x}, \vec{x}') = \bigwedge_{i \in \{1, \dots, |D|\}} \left[(\vec{y} = i \wedge \vec{z} = 2) \Rightarrow \text{EF}_i(\vec{x}, \vec{x}') \right] \quad (7.6)$$

$$R_{\text{EF}}^{\text{CNF}}(\vec{x}, \vec{x}') = \bigwedge_{i \in \{1, \dots, |D|\}} \bigwedge_{c \in \text{EF}_i^{\text{CNF}}} \left(f_{\neq}(\vec{y}, i) \vee f_{\neq}(\vec{z}, 2) \vee c \right) \quad (7.18)$$

And finally, we write R_{ld} and C in CNF as follows.

$$R_{\text{ld}} = (\vec{z} = 3) \Rightarrow \text{ld}(\vec{x}, \vec{x}') \quad (7.19)$$

$$R_{\text{ld}}^{\text{CNF}} = \bigwedge_{(u,v) \in \mathbb{U}} (f_{\neq}(\vec{z}, 3) \vee x'_{uv} \vee \bar{x}_{uv}) \wedge (f_{\neq}(\vec{z}, 3) \vee \bar{x}'_{uv} \vee x_{uv}) \quad (7.20)$$

$$C = (\vec{y} \leq |V| + 1 \vee z = 2) \wedge (\vec{y} \leq |D| + 1 \vee z \neq 2) \quad (7.21)$$

$$C^{\text{CNF}} = \left(\bigwedge_{c \in f_{\leq}(\vec{y}, |V|+1)} (c \vee z_0) \wedge (c \vee z_1) \right) \wedge \left(\bigwedge_{c \in f_{\leq}(\vec{y}, |D|+1)} (c \vee \bar{z}_0 \vee \bar{z}_1) \right) \quad (7.22)$$

By using the CNF expressions for R_{LC} , R_{VD} , R_{EF} , R_{Id} , both Eqs. (7.7) and (7.8) are in CNF without any further modification.

Chapter 8

A divide-and-quantum algorithm for SAT

While much of this thesis focuses on the use of symbolic algorithms for the analysis of quantitative systems (including quantum systems), quantum algorithms themselves can also be used to speed up symbolic algorithms. Grover’s algorithm offers a quadratic speedup over a large class of unstructured search problems, including the Boolean satisfiability (SAT) problem. However, when applied naively, this quadratic speedup is obtained over the complexity of brute-force search, while the best-known worst-case complexity for classical SAT algorithms—given by the PPSZ algorithm—outperforms both brute-force search and Grover’s quadratic speedup. Subsequently, it is natural to ask whether Grover’s complexity for SAT solving (or in fact PPSZ’s complexity) can be improved. In this chapter, we provide the first quantum algorithm for PPSZ that combines Grover’s quadratic speedup with PPSZ’s computational advantages, and show that combining these is less trivial than previously suggested. To improve the practicality of this algorithm for small-scale quantum computers, we also give a hybrid version of the algorithm that can interpolate between the classical and the quantum runtime by scaling the size of the quantum computer, using an approach that we call divide-and-quantum.

The work in this chapter has been published as part of Mathys Rennela, Sebastiaan Brand, Alfons Laarman, and Vedran Dunjko. “Hybrid divide-and-conquer approach for tree search algorithms”. In: *Quantum* 7 (2023), p. 959.

8.1 Introduction

Over the past decades, tackling computationally hard problems by encoding them in Boolean satisfiability (SAT) formulas and letting these formulas be solved by highly optimized solvers has seen a lot of success across a wide array of problems, ranging from industrial formal verification [31] to resolving mathematical conjectures [116]. Thanks to steady algorithmic improvements, such as clause learning and effective branching heuristics, modern SAT-solvers can handle instances with hundreds of thousands of variables and millions of clauses [114].

With the growing interest in quantum computing and improvements that have been made with the physical realization of quantum computers in recent years, it is natural to ask whether (hybrid) quantum algorithms can provide computational speedups for SAT solving. Even though it is believed that quantum computers will never solve NP-complete problems (such as SAT) in polynomial time [1], more modest speedups might be possible. Grover’s algorithm [103], for example, provides a quadratic speedup on a wide class of unstructured search problems. For SAT-solving specifically, this means that, given a SAT formula over n variables, Grover’s algorithm can solve such formulas in $O(\sqrt{2^n}) = O(2^{0.5n})$ time. However, classical algorithms can do better than the brute-force $O(2^n)$ approach. In fact, the best-known classical approach to SAT solving—the PPSZ algorithm [178]—has a worst-case complexity of only $\sim O(2^{0.386n})$ [203] for 3-SAT, which beats a naive Grover-based approach.

While previous work has claimed that a Grover speedup can be trivially obtained over PPSZ [67], we show in this chapter that this is not so trivial, as the shape of PPSZ’s search space doesn’t immediately lend itself to be adapted into a Grover-based quantum algorithm. In this chapter, we provide a concrete method for obtaining a Grover speedup over PPSZ by reformulating PPSZ from a search over a non-complete binary tree with depth n and a branching factor γ_k , to a search over a complete binary tree with depth $n(\gamma_k + \varepsilon)$.

Additionally, we show in which settings hybrid quantum-classical algorithms—which we call *divide-and-quantum* methods—can provide speedups. The core idea of such hybrid algorithms is to classically split a problem into increasingly smaller sub-problems, until these sub-problems are small enough to be run on some limited-size quantum computer. While it is easy to divide a SAT problem into smaller problems and stitch the results of these back together, it is not trivial to guarantee that this still provides (smaller) quantum speedups. Difficulties lie, for example, in making sure the quantum subroutines do not use too much space.

The contributions in this chapter can be summarized as follows.

1. A concrete approach for achieving an $O(\sqrt{2^{\gamma_k n}})$ -time quantum version of PPSZ, where $O(2^{\gamma_k n})$ is PPSZ's classical complexity, with $\gamma_k \approx 0.386$ for 3-SAT.
2. Specifications of space-efficient subroutines that are required to achieve speedups over PPSZ with a hybrid quantum-classical approach, under some assumptions on the variable order.

The remainder of this chapter is structured as follows. In Section 8.2, we provide the necessary background on SAT, classical SAT-solving, and quantum approaches to this problem. In Section 8.3 we specify a fully quantum version of PPSZ, and outline the requirements for a hybrid divide-and-quantum approach to achieve speedups. Section 8.4 contains details on the space-efficient quantum circuits that are required for the hybrid approach. Finally, Section 8.5 concludes this chapter.

8.2 Preliminaries

This section introduces SAT and a general backtracking approach to SAT solving, which is instantiated for two algorithms: DPLL and PPSZ. While DPLL performs better in practice on many instances, PPSZ-based algorithms provide the best worst-case runtime guarantee at the time of writing [106]. The section ends with an explanation of the divide-and-quantum method.

8.2.1 Satisfiability

A Boolean formula F over n variables corresponds to a function $F : \{0, 1\}^n \rightarrow \{0, 1\}$ in the natural way. The Boolean satisfiability problem (SAT) is the constraint satisfaction problem of determining whether a given Boolean formula F in conjunctive normal form (CNF) has a satisfying assignment, i.e. a bit string $y \in \{0, 1\}^n$ such that $F(y) = 1$. A CNF formula is a conjunction (logical ‘and’) of disjunctions (logical ‘or’) over variables or their negations (jointly called literals). For ease of manipulation, we view CNF formulas as a set of clauses (a conjunction), where each clause is a set of k literals (a disjunction), with positive or negative polarity (i.e. a Boolean atom x , or its negation $\bar{x}$). In the k -SAT problem, the formula F is a k -CNF formula, meaning that all the clauses have k literals.¹

¹Without loss of generality, we will allow that the formula has clauses with fewer literals, but assume that at least one clause has k literals, and no clauses have more.

It is well known that solving SAT for 3-CNF formulas (3-SAT) is an NP-complete problem. As a canonical problem, it is highly relevant both in computer science [31] and outside, e.g. finding ground energies of classical systems can often be reduced to SAT (see e.g. [159]).

Since it is a disjunction, a clause $C \in F$ evaluates to true (1) on a (partial) assignment $\vec{x}$, if at least one of the literals in C attains the value true (1) according to $\vec{x}$. A formula F evaluates to true on an assignment $\vec{x}$, i.e. $F_{\vec{x}} = 1$, if all its clauses evaluate to true on this partial assignment. Note that to determine the formula evaluates to true on some (partial) assignment, the assignment has to fix of at least one variable per clause of F , i.e. the assignment should be linear in the number of variables. On the other hand, a formula can evaluate to false (0) on very sparse partial assignments: it suffices that the given partial assignment renders any of the clauses false by setting k variables, i.e. a constant amount. In such a case, we say that the partial assignment *establishes a contradiction*. This is the equivalent of saying that the constraint formula is *inconsistent*.

Given a partial assignment $\vec{x} \in \{0, 1, *\}^n$, we denote with $F_{|\vec{x}}$ the sub-function of F restricted to that assignment. This sub-function can also be obtained by setting the variables in the formula F to the values specified in the partial assignment. For CNF formulas, this means the following: for an assigned variable x_j , for every clause of F where x_j appears as a literal (of some polarity), and the setting of x_j renders the corresponding literal true, that clause is dropped in $F_{|\vec{x}}$. For every clause of F where x_j appears as a literal (of some polarity), and the setting of x_j renders the corresponding literal false, that literal is removed from the corresponding clause. Finally, an assignment $\vec{x}$ that establishes a contradiction introduces an *empty clause*, i.e. $\emptyset \in F_{|\vec{x}}$, whereas a satisfying assignment $\vec{y}$ yields an *empty formula*, i.e. $F_{|\vec{y}} = \emptyset$. We will call such formulas, where some variables have been fixed ($F_{|\vec{x}}$), *restricted formulas*, and say that $F_{|\vec{x}}$ is a restriction of F by the partial assignment $\vec{x}$. We will also use a similar notation for setting literals to true. Given a literal $l \in \{x_i, \bar{x}_i\}$, we write $F_{|l}$ for the restricted formula given by F with the value of the variable x_i set such as to render l true.

Additionally, we say a formula G is semantically entailed by a formula F if all satisfying assignments (models) of F also satisfy G , denoted $F \models G$. In general, G can be any formula over a subset of variables of F , although we only consider cases where G consists of a single literal $l \in \{x_i, \bar{x}_i\}$.

Finally, *resolution* is a logical inference rule used for satisfiability proving [72]. The basic resolution rule takes two clauses $(x \vee A)$ and $(\bar{x} \vee B)$, where A, B are clauses,

and derives a clause $(A \vee B)$, since both clauses are satisfiable if and only the inferred clause is. The clausal inference rule is refutation complete, meaning that a complete, recursive search over all inferred clauses will find a refutation if the original formula, i.e., a set of clauses, is unsatisfiable. In *unit resolution*, we have $A = \emptyset$ (or $B = \emptyset$) in the above, i.e., one of the input clauses is a *unit clause* (x) (or $(\bar{x})$).

8.2.2 The DPLL algorithm family

The algorithm of Davis Putnam Logemann Loveland (DPLL) is a backtracking algorithm that recursively explores the possible assignments for a given formula, applying (incomplete) reduction rules along the way to prune the search space [71]. Originally designed to resolve the memory intensity of solvers based only on resolution (the DP algorithm uses a complete resolution system [72]), this backtracking-based algorithm can now be found in some of the most competitive SAT solvers [115]. DPLL was also generalized to support various theories, including linear integer arithmetic and uninterpreted functions [173], and is used consequently in many combinatorial domains [31] and automated theorem proving [33].

DPLL branches on assignments to individual variables x and $\bar{x}$. It uses *reduction* rules based on resolution to prune the resulting search tree. In order to prune a branch, a reduction rule $\mathcal{R}$ (efficiently) *under-estimates* whether a literal l is entailed by a formula F , i.e.,

$$(F \models_{\mathcal{R}} l) \Rightarrow (F \models l).$$

We consider two rules:

- Unit resolution: if there exists a unit clause $\{x\}$ (or $\{\bar{x}\}$), then set value x to true (false).
- Pure literal: if variable x only appears positively (negatively), then set x to true (false).

Algorithm 8.1 corresponds to the DPLL algorithm in its classical form. It recursively assigns truth values to the variables of F in a heuristically chosen order (see line 4), while eagerly simplifying the formula (see $F_{[x]}$, $F_{[\bar{x}]}$ at lines 6 and 8). Branches are possibly pruned with the reduction rule at line 6. When the formula becomes (un)satisfiable, the recursion backtracks (lines 2 and 3). Note that a short circuiting or at line 8 will cause the algorithm to terminate early when the formula is found to be satisfiable.

Algorithm 8.1: The DPLL algorithm.

```

1 def DPLL( $F : \text{CNF}$ ) :
2   if  $F = \emptyset$  then return 1
3   if  $\emptyset \in F$  then return 0
4    $x \leftarrow$  next var according to branching heuristic
5   if  $F \models_h (x = c)$  with  $c \in \{0, 1\}$  then
6     return DPLL( $F|_{x=c}$ )
7   else then
8     return DPLL( $F|_x$ )  $\vee$  DPLL( $F|_{\bar{x}}$ )

```

The algorithm explores a sub-tree of the full binary search tree. We call the nodes where pruning happens the *forced nodes*, and the others *guessed nodes*.

A key element of the (heuristic) efficiency of backtracking algorithms is their ability to simplify subproblems as early as possible in order to prune the tree search. This is what DPLL achieves through its branching heuristic, which determines the order in which variables should be considered. Notice that the order may change in the different branches of the tree, for example, because different unit clauses appear under different assignments in the concrete heuristic discussed above. On the other hand, one can of course always implement a static branching heuristic that simply selects variables according to a predetermined order, as we do in the next section for PPSZ.

8.2.3 The PPSZ algorithm family

The best exact algorithms for the (unique) k -SAT problem have, for many years, been based on the PPSZ (Paturi, Pudlák, Saks, and Zane [178]) algorithm. The PPSZ algorithm is a Monte Carlo algorithm, i.e. it returns the correct answer with high probability using randomization, although a derandomized version also exists [191]. Like DPLL, PPSZ uses heuristics to force or aguess variables. The upper bound on the runtime is derived from the probability that a variable is guessed, as the expected number of guesses dictates the size of the search space. We first explain the intuition based on a simple version that achieves a runtime of $O^*(2^{(1-1/k+\varepsilon)n})$, for $\varepsilon > 0$.²

Assuming there is only a single satisfying assignment $\vec{u}$, it is easy to see that the unit resolution heuristic can force on average $1/k$ variables for a random variable

²In the remainder of the text, we will be using the standard notation for “lazy” scalings: with the superscript *, e.g. O^* , we denote scalings that ignore only polynomially contributing terms (relevant when the main costs scale exponentially), just as tilde, e.g. $\tilde{O}$, denotes we ignore logarithmically contributing terms (when the main costs are polynomial in the relevant parameters).

permutation $\pi \in S_n$: In the first place, for every variable x there has to be a clause C which forces the value of x , i.e., where the only true literal under assignment $\vec{u}$ is x or $\bar{x}$, otherwise there would be multiple satisfying assignments. Second, if the variables are assigned according to the order π and π places x last with respect to the other $k-1$ variables of C , then unit resolution will indeed identify the forced variable. As a result, a PPSZ algorithm merely has to search a space proportional to the number of guesses. For a k -SAT problem, we let γ_k denote the expected fraction of the n variables that are guessed by PPSZ, such that $\gamma_k n$ is the expected number of guessed variables. The number of variables that need to be guessed depends on the heuristic h used to check if $F \models_h l$. If the heuristic consists solely of unit resolution,³ then for unique⁴ k -SAT PPSZ needs to make at most $(1 - 1/k)n$ guesses. When unit resolution is replaced with the more general s -implication (discussed below), something equivalent to the original PPSZ algorithm is obtained.

There are currently various versions of PPSZ (see e.g. [192, 112, 113, 106]), but recent analysis [204] shows that the original PPSZ [178] still has the best worst-case runtime of $2^{(\gamma_k + \varepsilon)n}$, where for example $\gamma_3 \approx 0.386229$. The original PPSZ can be seen as using a reduction rule called s -implication during the tree search [112].

Definition 8.1 (s -implication). *A literal x_j ($\bar{x}_j$) is s -implied by F , written $F \models_s x_j$ ($F \models_s \bar{x}_j$), if and only if there is a subset of clauses $G \subseteq F$, with $|G| \leq s$, such that all satisfying assignments of G set x_j to 1 (0).*

For $s = 1$, s -implication is the same as unit resolution (Sec. 8.2.2).

8.2.4 Quantum algorithms for tree search

The extent to which quantum computing can help the exploration of trees, and more generally, graphs, is a long-standing question with infrequent but noticeable progress. In the context of SAT solving, up until relatively recently, the best methods involved Grover search [103] over all possible satisfying assignments (the leaves in the full search tree). In such an approach, we would introduce a separate register of length d equal to the number of variables and a *search predicate*, implementing the satisfiability criterion based on the input formula. This brute-force approach, however, yields a redundant search over branches that would be pruned away by the resolution rules, and, in the worst case, may force a quantum computer to search an exponentially larger space than the classical algorithm would.

³PPSZ with unit resolution as the only heuristic is in fact the PPZ algorithm [205].

⁴This approach can be generalized to a setting with multiple satisfying assignments [205].

Nonetheless, Grover provides an advantageous strategy whenever the classical search space is larger than $2^{n/2}$, hence this was the method used in previous work on hybrid divide-and-conquer strategies [95, 82]. One example is Schöning’s algorithm for SAT, which allows a full quadratic quantum speedup [8] (but was since surpassed by PPSZ). A particular advantage of Grover’s search is that it is frugal regarding time and space: beyond what is needed to implement the oracle (the search predicate), it requires at most one ancillary qubit, and very few other gates.

More recently, Montanaro [163], Ambainis & Kokainis [9], and Jarret & Wan [126] have given quantum-walk-based algorithms which can achieve essentially full quadratic speedups in the number of queries to the tree (when trees are exponentially sized). However, for the remainder of this chapter, we focus exclusively on the Grover-based approach. A more elaborate discussion that also includes quantum-walk-based algorithms can be found in the full paper [187] on which this chapter is based.

8.2.5 The hybrid divide-and-conquer method

The hybrid divide-and-conquer method was introduced to investigate the extent to which smaller quantum computers can help in solving larger instances of interesting problems. Divide-and-conquer strategies recursively break down an instance of a problem into smaller sub-instances, so at some point they become suitably small to be run on a quantum device of (almost) any size.

The idea of the hybrid quantum-classical divide-and-conquer method, which we call a *divide-and-quantum* (D&Q) method, is straightforward. Let us say we have a search tree of depth n (e.g., the search space generated by a Boolean formula of n variables), and a quantum computer of size κn , where $\kappa \in (0, 1]$. The top of the search tree is explored by a classical divide-and-conquer algorithm, up to the point where the subproblems are sufficiently small to be run on a quantum computer of size κn .

Intuitively, quantum speedups might still be obtained on these subproblems; however, such speedups are not guaranteed. For example, if the branching factor of the search tree is not uniform, with a lower branching factor at lower levels, then too little work might remain at the lower levels of the tree to benefit from quantum algorithms.

Under the assumption that the search trees are complete binary trees, it becomes easier to show that hybrid quantum speedups remain possible at arbitrarily small κn . Of course, the search trees of algorithms like DPPL and PPSZ are not complete, as it is exactly the pruning of branches that creates the advantage over brute-force search. In the next section, we show how (hybrid) quantum speedups can still be obtained.

Algorithm 8.2: PPSZ expressed as a divide-and-conquer algorithm.

```

1 def dncPPSZs(F, π) :
2   if F = ∅ then return 1
3   if ∅ ∈ F then return 0
4   if n - |π| > (γk + ε)n then return 0                                ▷ truncate search
5   x, π ← π[0], π[1...]                                                ▷ first var in π, postfix
6   if F ⊨s (x = c) with c ∈ {0, 1} then
7     return dncPPSZs(F|x=c, π)
8   else then
9     return dncPPSZs(F|x,π) ∨ dncPPSZs(F|¬x,π)
    
```

8.3 Divide-and-quantum for PPSZ

For a quantum version of PPSZ, we require the following. First, we need to reformulate the original PPSZ algorithm in a way that is compatible with quantum tree search, which we do in Section 8.3.1, and second, we need to take the representation of the search space into account, which we discuss in Section 8.3.2.

Additionally, for our divide-and-quantum algorithm to yield speedups, the space usage of the quantum subroutines must be restricted, which we discuss in Section 8.3.3. Furthermore, to be able to implement the required quantum subroutines given limited space, we are required to place constraints on the SAT instances. The consequences of these constraints on the theoretical speedups are discussed in Section 8.3.4.

8.3.1 Reformulation of PPSZ

The original PPSZ algorithm evaluates random assignments on random variable orders and is not immediately amenable to quantum tree search. Here we introduce a divide-and-conquer version of PPSZ that offers the same runtime guarantees as recent PPSZ versions, while allowing it to be translated into a quantum tree-search algorithm. Algorithm 8.2 shows dncPPSZ, which is similar to DPLL with s -implication as a reduction rule. It takes a fixed (random) variable order π and prunes branches after $(\gamma_k + \varepsilon)n$ guesses have been performed. Since the expected number of guesses over all variable orders equals $\gamma_k n$ for s -implication [205], Markov's inequality tells us that, for any $\varepsilon > 0 \in \Theta(1)$, with constant probability we can find a satisfying assignment with fewer than $(\gamma_k + \varepsilon)n$ guesses for a random π .

We now prove that dncPPSZ is correct and has a runtime of $2^{(\gamma_k + \varepsilon)n}$, i.e., it equals the best-known SAT algorithm, as recent analysis of PPSZ confirms [204]. We focus

8.3. Divide-and-quantum for PPSZ

here on unique- k -SAT, in which the input formula has a unique satisfying assignment if satisfiable. The results can be generalized to general k -SAT based on [204]. Let F be a k -SAT formula on n variables, $\vec{u}$ be the unique satisfying assignment of F (provided F is satisfiable).

Proving the runtime of dncPPSZ is easy, as shown by Theorem 8.1.

Theorem 8.1. *Algorithm 8.2 has a runtime of $2^{(\gamma_k + \varepsilon)n}$ with $\gamma_3 \approx 0.386229$ [204].*

Proof. For a given (fixed) permutation π , the algorithm only explores a tree truncated at $(\gamma_k + \varepsilon)n$ branches per path (Alg. 8.2, line 4). $\square$

We now show that dncPPSZ is a correct Monte-Carlo algorithm, i.e., it finds the unique satisfying assignment with constant probability if it exists (hence, if repeated a constant number of times, it will decide unique satisfiability). To this end, we first reiterate some results from the PPSZ algorithm studied in [204].

In contrast to the dncPPSZ algorithm, which tries all assignments with backtracking, recent versions of PPSZ [204] sample assignments using an *advice string* $\vec{a}$ and the *Decode* function shown in Algorithm 8.3. According to a random variable order π , the function only uses a bit of the advice string $\vec{a}$ when the next variable is guessed, i.e., when the heuristic fails to establish its truth value (see the **else** branch). It also does not terminate evaluation early, as in backtracking, i.e., it naively continues evaluation even when the formula is already (un)satisfied. This suffices because the only point of the *Decode* function is to study the expected number of guesses under s -implication and to demonstrate how the number of guesses dictates the expectation of finding the satisfying assignment.

Theorem 8.2 shows that *Decode* uses $\gamma_k n$ guesses to find the unique satisfying assignment of a k -SAT formula F . By Corollary 8.1, it can be found with high probability with a sub-exponential number of (random) calls to *Decode*. The PPSZ algorithm is therefore defined as the loop that calls *Decode* sufficiently many times.

Theorem 8.2 (Adapted from [204] Th.4). *Let formula F , advice $\vec{a}$ and order π be such that $\text{Decode}(F, \vec{a}, \pi) = \vec{u}$ (Algorithm 8.3). Then the expected number of guesses in *Decode* equals $\gamma_k n$.*

Corollary 8.1 ([204] Ob.5). *The unique satisfying assignment is found with high probability with $O^*(2^{(\gamma_k + \varepsilon)n})$ calls to *Decode* with random assignment $\vec{a}$ and order π .*

Instead of trying random advice strings, the dncPPSZ algorithm searches all advice strings of length $(\gamma_k + \varepsilon)n$. Since it implements the *Decode* function along each

Algorithm 8.3: $\text{Decode}(F, \vec{a}, \pi)$ where the value s of s -implication is a large constant or grows very slowly in n [204]. The function returns false or the unique satisfying assignment (as a conjunction of literals) if the advice $\vec{a}$ and order π ‘decode’ to it.

```

1 def  $\text{Decode}(F, \vec{a}, \pi)$  :
2   if  $|\pi| = 0$  then return  $F = \emptyset$  ▷ SAT?
3    $x \leftarrow \pi[0]$  ▷ first var in  $\pi$ 
4   if  $F \models_s (x = b)$  for  $b \in \{0, 1\}$  then ▷ forced
5      $c \leftarrow b$ 
6   else then ▷ guessed
7      $c \leftarrow a[0]$ 
8      $\vec{a} \leftarrow \vec{a}[1 \dots]$  ▷ postfix of array
9   return  $\text{Decode}(F|_{x=c}, \vec{a}, \pi[1 \dots]) \wedge (x = c)$ 

```

search branch, merely adding backtracking when a formula becomes (un)satisfiable on a partial assignment, it also inherits the probability of finding the unique satisfying assignment from PPSZ. Theorem 8.3 shows this.

Theorem 8.3. *Let F be a k -SAT formula defined on n variables and π a permutation in S_n . Then the probability that dncPPSZ_π returns a satisfying assignment is at least $1 - \frac{1}{1+\varepsilon/\gamma_k}$.*

Proof. Let $g(\pi)$ denote the number of guesses performed by $\text{Decode}(F, \vec{a}, \pi)$.

By applying Markov’s inequality, we obtain:

$$\begin{aligned}
 \Pr_\pi [g(\pi) > (\gamma_k + \varepsilon)n] &\leq \Pr_\pi [g(\pi) \geq (\gamma_k + \varepsilon)n] \\
 &\leq \frac{\mathbb{E}_\pi [g(\pi)]}{(\gamma_k + \varepsilon)n} = \frac{\gamma_k}{\gamma_k + \varepsilon},
 \end{aligned}$$

which is an upper bound on the probability that dncPPSZ_π fails to find a satisfying assignment, given it exists. Subsequently, $1 - \frac{\gamma_k}{\gamma_k + \varepsilon}$ is then a (constant) lower bound on the probability that dncPPSZ_π succeeds. $\square$

8.3.2 Branching representation

The way the search space is represented is important in order for the quantum algorithm to obtain an advantage over the best-known classical algorithm.

Given a SAT formula over n variables, the natural search space can be seen as a complete binary tree with n levels (excluding the leaves) (Fig. 8.1a). In this nat-

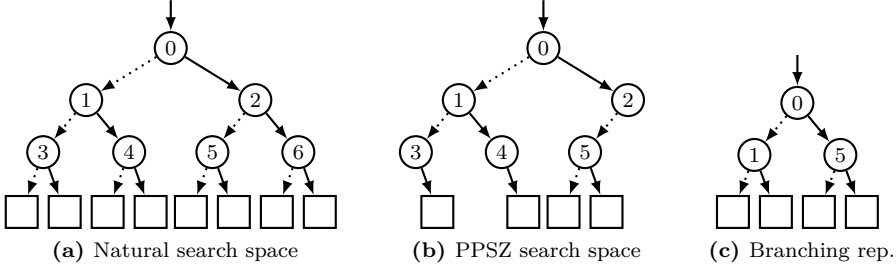


Figure 8.1: The natural search space (a), the search space traversed by PPSZ (b), and the PPSZ search space in the branching representation (c). Dashed (solid) lines correspond to setting the variable corresponding to that node to 0 (1).

ural representation, if the position of the leaves is unknown, we need to perform a brute force search over all possible partial assignments, leading to the quantum query complexity of $O(2^{n/2})$.

However, algorithms like DPLL and PPSZ prune this search space by recognizing forced variables (Alg. 8.2, lines 6 and 7), yielding a provably smaller search space (Fig. 8.1b). For PPSZ on 3-SAT formulas, the average worst-case branching factor is $\gamma_3 \approx 0.386229$, which yields a classical query complexity of $\sim O(2^{0.386n})$. Compared to this, the quantum approach on the natural representation is prohibitively slow.

A more efficient approach relies on the *branching representation*, in which instead of representing all nodes in the natural search space, only branching nodes are represented (Fig. 8.1c). In this case, the search space is 2^{br} . We note that this does not enumerate all the vertices or leaves, but possible paths in trees with no more than br branchings. In Section 8.3.1 we have shown that for dncPPSZ it suffices to only consider paths with up to $(\gamma_k + \varepsilon)n$ branches.

To implement this branching representation in quantum circuits, all that is required is a (reversible) subroutine that checks whether a given sequence of branch choices in this representation leads to a satisfying assignment. In other words, we require a reversible implementation of a search predicate P which identifies leaves and solutions, i.e.,

$$P(\vec{x}) = \begin{cases} b \in \{0, 1\} & \text{if } F|_{\vec{x}} = b \\ \perp & \text{otherwise} \end{cases} \quad (8.1)$$

In the natural representation, a reversible version of P for a node $\vec{x} \in \{0, 1, *\}^n$ can

be implemented by a circuit computing the number of satisfied clauses in $F|_{\vec{x}}$, using the values of the actual variable assignments in $\vec{x}$. So, in the branching representation, to evaluate P , we must in some (implicit) way first reconstruct the values of the variables that occur in the partial assignment corresponding to the vertex fixed by the branching representation. Such a modified P that evaluates the branching representation can be run as the Grover predicate to realize the search over all possible branch representation strings.

Now, to translate the branching representation to variable values, we must follow the path in the search tree, and for this, we will utilize the (reversible) implementations of the following operations $ch1(v)$, $ch2(v, b)$, $chNo(v)$, to trace the path.

- A function $ch2(v, b)$, which takes on input a vertex v , and returns one of the children, specified by the bit b .
- A function $ch1(v)$ which returns the single child if v is forced.
- A function $chNo(v)$, which returns whether v has zero, one or two children.

This is easily done reversibly if we are allowed to store each partial assignment along the path. However, due to space constraints that we discuss in Section 8.3.3, realizing an efficient implementation of the search predicate P (and thus also the functions above) is a non-trivial task.

What complicates the realization of these subroutines is that branching choices, i.e., the values of guessed variables alone, do not map trivially to individual variable values. For example, in the branching representation, node 5 (Fig. 8.1), should first be converted into a partial assignment to the variables (i.e., we need to deduce that there was a node (node 2) whose variable was forced to 0), before we can compute its children. To know which variable is the first guessed variable, we must compute the s -implications. An obvious solution is to compute all implications and store the corresponding partial assignment, but this violates the objective of using less memory than what the natural representation allows.

First, we simplify the problem by using eager evaluation of forced variables. This compresses all the line paths in the tree, resulting in a tree with only binary nodes. Algorithm 8.4 illustrates this using the tree node functions. The node v consists of an advice label (including a counter indicating its length). Implementing $ch2$ is trivial, e.g., $ch2(111, b) = 111b$. However, this moves the complications to the search function P , since it now has to deduce which variables were forced (and to which values).

8.3. Divide-and-quantum for PPSZ

Algorithm 8.4: $\text{dncPPSZ}(v)$ on the branching representation, using our functions, where $|v|$ is the length of node v 's advice label.

```

1 def  $\text{dncPPSZbr}(v)$  :
2   if  $P(v) = 1$  then return 1
3   if  $P(v) = 0$  or  $|v| > (\gamma_k + \varepsilon)n$  then return 0
4   return  $\text{dncPPSZbr}(\text{ch2}(v, 0)) \vee \text{dncPPSZbr}(\text{ch2}(v, 1))$ 

```

Next, we specify the problem that P has to solve. We call it *s-implication with advice* (SIA), which is intuitively and implicitly defined as follows (Section 8.4.2 provides a precise definition). Given a fixed formula F and variable order π , an algorithm solving SIA takes on input an advice string of size S_{adv} , specifying which branching choices will be made at the guessed variables. The algorithm should compute the path realized in an *s-implication-resolution*-based process in the tree specified by F by going over the variables in the specified order, setting them to either the forced value if *s*-implied, or to the next unused branching choice specified in the advice register if not. It should then output: $|00\rangle$, if more branches are encountered than the advice length (case 2 of Eq. (8.1)); $|1b\rangle$, if the path makes the formula (un)satisfied at some point, such that $F = b$ (case 1 of Eq. (8.1)).

The desired space-efficient implementation of SIA will utilize a S_{adv} -sized advice register for the choices, and ideally no more than $o(n)$ ancillas (although, low-prefactor linear scaling is also acceptable as explained in Section 8.3.3). While these criteria are easily met in an irreversible computation, critically, it must be realized reversibly under these conditions, as in this case, we achieve Grover-based search by searching over the advice register.

8

Obtaining $o(n)$ space is not trivial for the problems we consider. In particular, it can be proven that the routines required to implement PPSZ (e.g. repeated *s-implication*, unit resolution) are P-complete under log-space reductions (as proven for unit resolution in [129]). It is still an open problem whether $o(n)$ space, poly-time algorithms exist for P-complete problems [102; Section 5.4], but lower bounds on pebbling approaches [146] give a pessimistic outlook. Therefore, an approach that focuses on identifying classes of formulas that admit genuine hybrid speedups seems justified.

Accordingly, we focus on formulas where, for a given variable order, the formula has *bounded index width* (biw) as defined in Definition 8.2. In Section 8.4.2, we provide an $o(n)$ -space reversible algorithm for SIA that works for this class of formulas. Specifically, we provide an algorithm that has polynomial runtime, and space complexity

$S_{adv} + S_w$, where S_{adv} is the advice-string register, and

$$S_w = O(w \cdot \log(n/w)) \quad (8.2)$$

for any k -CNF formula of biw w . We do so by providing a frugal, reversible implementation of SIA, and then by splitting this (deterministic) computation into blocks that require only a partial assignment of w variables, applying Bennett's reversible pebbling strategy on those blocks [22].

Definition 8.2. For a given Boolean formula F and a variable ordering $\vec{x}$, we defined the index-width $\text{iw}(F)$ of a formula to be the largest difference between two indices of variables in a clause of the formula F , i.e.

$$\text{iw}(F) = \max_{C \in F} \max_{x_i, x_j \in C} |i - j|.$$

A formula F has bounded index width (biw) w if $\text{iw}(F) \leq w$.

However, we note that bounded index width formulas have specialized SAT-solving algorithms with best runtime $O(2^w \text{poly}(n))$ [149]. Thus, in the case that $w \in o(n)$, we can decide the satisfiability of the given formula in sub-exponential time. In Section 8.3.4, we discuss regimes in which our hybrid approach still provides an improvement over best-known classical algorithms.

8.3.3 Requirements of hybrid speedups

To understand the performance of the hybrid algorithm and what is required for it to yield speedups over purely classical algorithms, we introduce the search tree decomposition. Consider a search tree $\mathcal{T}$ generated by algorithm $\mathcal{A}$ on some instance of size n , where every vertex in the tree denotes a sub-problem which can (in principle) be delegated to a quantum computer, running the algorithm $\mathcal{A}_Q$ in a hybrid scheme.

In general, a κn -sized quantum computer does not provide enough space for $\mathcal{A}_Q$ to be run on the entire problem, i.e. on the root of the tree, but it can be run on some of the sub-instances represented by vertices deeper in the tree. This, of course, depends on the space efficiency features of $\mathcal{A}_Q$ – the *effective size* of the quantum computer we have, but we will not focus on this for the moment.

We merely assume that whether or not $\mathcal{A}_Q$ can be run on an instance v given a κn -sized device is monotonic with respect to the tree structure; that is, if it can be

8.3. Divide-and-quantum for PPSZ

run on v , it can be run on all descendants of v .⁵

With this in mind, we can identify the collection of J_c *cut-off vertices* $\{c_k\}_{k=1}^{J_c}$. That is, all the vertices that can be run on the quantum device, whose parents cannot be run on the same device due to size considerations. In principle, J_c may be zero for some trees. These vertices correspond to the largest instances in the search tree where we can use the quantum computer.

Now the *search tree decomposition* is characterized by the set of sub-trees $\{\mathcal{T}_j\}_{j=0}^{J_c}$, where the sub-tree $\mathcal{T}_j$ is the entire sub-tree rooted at the cut-off vertex c_j , and where $\mathcal{T}_0$ denotes the tree rooted at the root of $\mathcal{T}$, whose leaves are parents of the cut-points $\{c_j\}_j$. We refer to $\mathcal{T}_0$ as the “top of the tree”, and it is traversed by the classical algorithm alone. Note that by “gluing” all the sub-trees of $\{\mathcal{T}_j\}_j$ as the corresponding leaves of $\mathcal{T}_0$, we obtain the full tree $\mathcal{T}$.

With T_j we denote the size of the tree $\mathcal{T}_j$. The set $\{\mathcal{T}_j\}_j$ we call the search tree decomposition at cut-off c , and note that it holds

$$T = T_0 + \sum_{1 \leq j \leq J_c} T_j. \quad (8.3)$$

Next, we briefly illustrate, in terms of the search tree decomposition, on an intuitive level, in which cases speedups can be neatly characterized and achieved, and the cases where the speedup fails.

The classical algorithm will (in the worst case) explore the entire search tree requiring T_j steps, for each sub-tree $\mathcal{T}_j$. Note that T_j characterizes the upper bound on the query complexity of the classical algorithm. Let us, for the moment, assume that this is roughly equal to the overall runtime of the classical algorithm (we will discuss shortly when this is a justified assumption).

Further, assume that the quantum algorithm achieves a pure quadratic speedup in runtime over the classical algorithm. Then the hybrid algorithm will take $T_H = O^*(T_0 + \sum_{1 \leq j \leq J_c} \sqrt{T_j})$ time (queries), where $O^*(f(x)) \triangleq O(f(x) \cdot \text{poly}(x))$ to ignore polynomial factors. To achieve a genuine speedup in the query complexity (from which we will be able to discuss total complexity), it must hold that T_H is upper bounded by $T^{1-\epsilon}$ for some ϵ where $T = \sum_{k=0}^{J_c} T_k$ is the total tree size.

Now this actual speedup clearly depends on the cut-off points, and the structure of the original tree, as nothing a priori dictates the relative sizes of all the sub-trees.

⁵It is conceivable that a sub-instance, as defined by a classical backtracking algorithm, for some reason takes more quantum space than the overall problem, and our formalism can be adapted to treat this case. However, in the cases of all quantum algorithms which we consider here, this will not be the case, hence we focus on this more intuitive scenario.

We consider the case where the tree is a balanced, complete binary tree: The tree size is exactly $2^n - 1$ where n is the tree height. Further, assuming that the quantum algorithm can handle γn -sized instances (in terms of this natural instance size) on the κn -sized device, we get the following decomposition: $T_j = 2^{\lfloor \gamma n \rfloor}$, $j > 0$, $T_0 = 2^{n - \lfloor \gamma n \rfloor}$. For simplicity, we shall ignore rounding, with which we obtain:

$$\begin{aligned} T_H &= T_0 + J_c \sqrt{T_1} \\ &= 2^{n-\gamma n} - 1 + 2^{n-\gamma n} (2^{\gamma n/2} - 1) \\ &\approx 2^{(1-\gamma/2)n} \end{aligned} \tag{8.4}$$

The above example constitutes a genuine speedup with $\epsilon = \gamma/2$, with a full quadratic speedup when $\gamma = 1$, i.e. when we can run the entire tree on the quantum computer.

At this point, it becomes clear how the space efficiency of the quantum algorithm comes into play: If it is linear in the natural size n , then γ is a fraction of κ , and we obtain asymptotic speedups. However, if the space required is super-linear in n , γ becomes a decaying function of n , in which case all speedups vanish.

8.3.4 When D&Q PPSZ beats known classical algorithms

To defeat bounded-index-width specialized algorithms, we consider index widths that are linear in n . As we will show in Section 8.4.2, it is possible to implement s -implication with advice (SIA) reversibly using total space $S_{adv} + S_w$, where $S_w = O(w \cdot \log(n/w))$ (the ancillary space needed for reversible SIA), and where S_{adv} is the size of advice itself. For all w , the runtime of this subroutine is polynomial in n , and the runtime of the overall quantum algorithm, which solves the sub-tree by exploring the advice string space, is $O(2^{S_{adv}/2} \cdot \text{poly}(n))$.

In contrast, a classical algorithm can decide SAT on formulas of index width w in time $O(2^w \text{poly}(n))$ [149]. It is unlikely that one can do much better than this: in the case $w = n$, achieving a bound better than $O^*(2^{cn})$, for some constant $c > 0$, would violate the exponential time hypothesis (ETH).

Here we assume this also holds for index widths that are fractions of n .⁶ It will be convenient to fix $w = \zeta n$ (with ζ a constant). In this case, we can identify a regime in which the quantum search over the advice string is still faster than solving SAT on the formula. This is the case whenever $S_{adv}/2$ is less than w , as these are the exponents of the exponential part of the run time of the respective algorithms. However, we must

⁶Furthermore, under the strong ETH, c approaches 1 for large clause sizes (k in k -SAT).

8.3. Divide-and-quantum for PPSZ

still ensure that the quantum algorithm can be run at all. So, a part of the overall memory available must be split between the advice string and the memory required to solve SIA with advice.

We set $S_{adv} = \beta\kappa n$ for $\beta \in (0, 1)$, and the remainder of the memory of $(1 - \beta)\kappa n$ qubits is spent as ancillary memory S_w for the SIA subroutine. Note that when setting $\zeta = w/n$ as a constant, we have that $S_w \notin o(n)$, as w is then linear in n . However, in what follows, we show that we can always find a pair (β, ζ) such that at least $S_w < n$.

In summary, we have that

$$\log(1/\zeta)\zeta b \leq (1 - \beta)\kappa \quad (8.5)$$

since to process $w = \zeta n$ -index width formula we need $\log(1/\zeta)\zeta bn$ bits (for some constant b ; Eq. (8.2)), and since we allocated $(1 - \beta)\kappa n$ for this purpose. Moreover, the exponents of the runtimes of the quantum and classical algorithms are $\beta\kappa n/2$ (Grover speedup) and ζcn , respectively, so it must hold that

$$\beta\kappa < 2c\zeta. \quad (8.6)$$

For our purposes, it suffices to show that for every κ, b, c , there exists a (β, ζ) pair for which both conditions hold.

First, we fix a β , to some (small) value β' . Next, find ζ satisfying Eq. (8.5); note such a ζ exists as $\log(1/\zeta)\zeta$ decreases in ζ , when ζ is small enough, converging to zero. Note, if Eq. (8.5) holds for $\beta = \beta'$ it also holds for any smaller β . Then choose $\beta \leq \beta'$ such that Eq. (8.6) is satisfied, which can be done by choosing β small enough. Since these (β, ζ) exist for any b , the space overhead of S_w can be made an arbitrarily small (constant) fraction of n . This guarantees the existence of regions in the space set by the advice size (controlled by β) and index width (controlled by ζ) with polynomial speedups (for a given κ).

However, there are no guarantees that the PPSZ process generates formulas that remain in this region: PPSZ's upper bound on the number of guessed variables ($\gamma_k n$) requires the variable order to be randomly permuted, however, fully random permutations will destroy any bound on the index width. Limiting the permutations to w -sorted permutations⁷ would be sufficient to fulfill the $o(n)$ space requirement, but fails to retain PPSZ's guarantee on the expected number of guessed variables.

We therefore require the assumption that, for a formula F with biw w , at least

⁷A sequence $[a_1, \dots, a_n]$ is w -sorted iff $\forall i, j$ with $1 \leq i \leq j \leq n$, $i \leq j - w$, it holds that $a_i \leq a_j$.

$1/\text{poly}(n)$ of its w -sorted permutations are “good” variable orders. With a good variable order we mean a variable order for which the expected number of guessed variables during the PPSZ run is at most $\gamma_k n$.

8.4 Space-efficient quantum subroutines for PPSZ

As discussed in the previous section, in the hybrid setting it is important to limit the space usage of the quantum implementation of PPSZ. To this end, in Section 8.4.1, we explain how Grover’s algorithm for an n -variable SAT formula can be implemented using only $n + 2$ qubits. Furthermore, in Sections 8.4.2 and 8.4.3, we provide space-efficient quantum circuits that implement the search predicate P , which is given by the *s-implication with advice* task as explained in Section 8.3.2.

8.4.1 Space-efficient Grover search for k -SAT

To implement Grover search of Figure 8.2 for k -SAT, the oracle $\mathcal{O}_f$ should evaluate a formula F on an assignment of n variables. A naive oracle implementation uses $n + m + 1$ qubits for a formula of m clauses, including $p \equiv m + 1$ ancilla qubits to evaluate each clause and their conjunction. This oracle implementation evaluates each clause and stores the result in a dedicated ancilla qubit. Figure 8.3 shows an example which uses a 3-controlled Toffoli gate. After all clauses are evaluated, an m -controlled Toffoli gate can compute the final result from the dedicated clause qubits. The reversal is trivial.

More efficient oracle implementations are possible: previous work [221] has shown that multi-controlled incrementers can be used to reduce the ancilla count p from $m + 1$ to $\lceil \log(m) \rceil + 1$. Furthermore, using Fixed-Point Amplitude Amplification [240], the ancilla count can be further reduced to 2 [221].

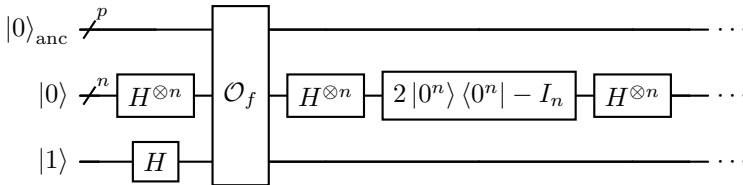


Figure 8.2: Grover’s algorithm: the oracle $\mathcal{O}_f$ xor’s the result on the phase wire (bottom).

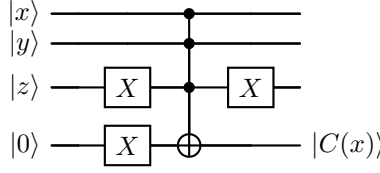


Figure 8.3: Naïve implementation of the evaluation of clause $C \equiv (\neg x \vee \neg y \vee z) \equiv \neg(x \wedge y \wedge \neg z)$

In the present chapter, we exploit another space-efficient quantum algorithm for formula evaluation, which only uses 1 ancilla qubit, relying on prior work [53] which defines a one-qubit program that computes any Boolean function exactly: this program takes x as input, and the output of the measurement is $f(x)$. This one-qubit model, which alternates single-qubit unitaries and controlled gates, can be seen as a quantum variant of Barrington’s theorem [20]. Note that k -CNF formulas (which are CNF formulas) correspond to depth-2 (AC) circuits, so the theorem (a statement on NC_1) applies to k -SAT input.

We can use the one-qubit model to implement the oracle $\mathcal{O}_f$, using one ancilla qubit to apply single-qubit unitaries and CNOT gates, controlled over the qubits of the input. We refer the interested reader to [53; Sec. 4] for the details of the implementations of the single-qubit unitaries of the one-qubit model.

Using the one-qubit model for formula evaluation to implement the oracle, we obtain a space-efficient implementation of Grover which uses 1 qubit for each variable of the formula, 1 qubit for the phase-flip oracle, and 1 qubit for formula evaluation, for a total space complexity of $n + 2$, as shown in Figure 8.2 with $p = 1$.

8.4.2 Space-efficient reversible SIA

This section implements the circuit SIA of Section 8.3.2 reversibly for formulas of bounded index width (biw; see Definition 8.2) in polynomial time and in a space-efficient way (as a function of w and n). We assume the input is a formula F defined on n variables with biw w .

For space efficiency, as discussed in Section 8.3.2, we will not manipulate the formula F , but instead work on partial assignments in the branching representation. The downside of this approach is, of course, that for a given node label, our circuits need to reconstruct the corresponding partial assignment. To avoid using $n \log(3)$ qubits for reconstructing the partial assignment, we exploit the bounded index width to split the

computation into blocks of w variables. We then use Bennett's pebbling strategy [22] to compose these blocks into a space-efficient reversible circuit which computes SIA.

The following three subsections treat the reconstruction of the partial assignments (Sec. 8.4.2.1), split the computation into blocks of w variables (Sec. 8.4.2.2), and combine these sub-computations in a space-efficient way (Sec. 8.4.2.3). At the end of this, we obtain Theorem 8.4.

Theorem 8.4. *There is a reversible circuit that, given an advice of size S_{adv} , computes s -SIA for a w -biw formula F with n variables in space*

$$S_w \triangleq O(w \cdot \log(n/w))$$

and time

$$O((n/w)^{\log(3)} \cdot w \cdot (2 + sw)^{ks} \cdot \text{polylog}(n)).$$

Note that the space complexity S_w excludes the space required to store the advice (S_{adv}). As a final remark, we also note that while in practice the size of s for s -implication can be considered a large constant, it should in fact grow (slowly) in n [178] (the inverse Ackermann function would suffice).

8.4.2.1 Reconstructing partial assignments

The simplifications in the search algorithm discussed in Section 8.3.2 (see Alg. 8.4) shift the difficulty to creating a space-efficient unitary implementing the search predicate P . It should take as input an advice string $\vec{a}$ of length $S_{adv} = |\vec{a}|$, representing the node in the search tree and determine whether this node is a solution leaf ($P(\vec{a}) = 1$), a normal leaf ($P(\vec{a}) = 0$) or a branch (such that we can query children $ch2(\vec{a}, 0)$ and $ch2(\vec{a}, 1)$). For conciseness, we will assume here that the advice length S_{adv} is fixed and no counter is required for recording the length of $\vec{a}$.

Ignoring space requirements for a moment, Algorithm 8.5 provides a specification of $P(\vec{a})$. We call this procedure s -implication with advice (SIA). It reconstructs the partial assignment on variables $x_1, \dots, x_n$ by testing s -implication on each variable x_i in order. If x_i is forced (s -implied), the variable is assigned accordingly. If x_i should be guessed, then the next value from the advice is used to assign it, using the advice pointer p . If at one point the formula becomes trivial under the partial assignment, i.e., $F|_{\vec{x}} = b$, for $b \in \{0, 1\}$, then the specification returns b (simplifying from the $|1b\rangle$ discussed in Section 8.3.2). If the advice runs out ($p = |\vec{a}|$) and the formula is not yet trivial after eagerly forcing variables, then the return value is $\perp$ ($|00\rangle$).

8.4. Space-efficient quantum subroutines for PPSZ

Algorithm 8.5: Specification for reconstructing the full partial assignment from the advice in branching representation. Indices start at 1.

```

1 def SIAF( $\vec{a}$ ) :
2    $p \leftarrow 1$  ▷ advice pointer
3   for  $i \in \{1, \dots, n\}$  do
4     if  $F_{x_1, \dots, x_{i-1}} = b$  then ▷ 0 children
5       return  $b$ 
6     elif  $F_{x_1, \dots, x_{i-1}} \models_s (x_i = b)$  then ▷ 1 child (forced var)
7        $x_i \leftarrow b$ 
8     elif  $p < |\vec{a}|$  then ▷ 2 children (use advice to guess)
9        $x_i \leftarrow \vec{a}[p]$ 
10       $p \leftarrow p + 1$ 
11    else then ▷ out of guesses
12      return  $\perp$ 

```

8.4.2.2 Splitting SIA into blocks

In order to create a space-efficient, reversible implementation of the entire SIA operation, we split SIA into a number of blocks, each of which acts on a subset of variables. The variables $x_1, \dots, x_n$ of F are split up into $l = n/w$ blocks of size w (for simplicity we assume w divides n). Each block B_i , for $i \in \{1, \dots, l\}$, contains variables

$$B_i \triangleq \{x_{j+1}, \dots, x_{j+w-1}\}, \text{ with } j = (i-1)w. \quad (8.7)$$

Note that having a bounded index width of w guarantees that computing the s -implication of x_i can only rely on $x_{i-w}, \dots, x_{i-1}$, and so in order to compute SIA for variables in B_i we only need the assignments on the previous w -sized block B_{i-1} . This is made more explicit in Section 8.4.3.1.

We define a sub-circuit SIAB_i (see Figure 8.4) which computes SIA (Alg. 8.5) for variables in B_i , unless input r (a flag) is set, in which case it is the identity function. Moreover, if SIAB_i encounters a contradiction or runs out of advice, it sets r . The first three registers contain the inputs: an assignment $\vec{y}_{i-1}$ on the variables in B_{i-1} , the advice pointer p (of $\log(n)$ qubits) which keeps track of the next unused advice bit, and the flag r (1 qubit). The outputs are written to the next three registers and contain an assignment $\vec{y}_i$ to the variables in B_i , and the updated values of p and r . Additionally, each SIAB_i block uses ancilla qubits $\vec{a}_b$. These ancillas are initialized in an all-zero state, and are reset to zero by means of uncomputation within the SIAB_i

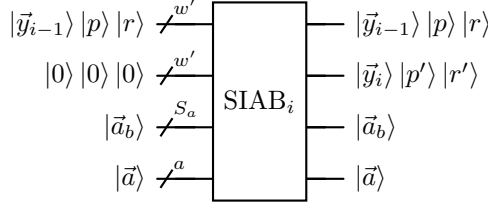


Figure 8.4: A SIAB_i block, for $i \geq 2$. SIAB_1 is a simpler version of SIAB_i without the top register. Here $w' = w + \log(n) + 1$, $S_a = w + O(\log(n))$, and $a = S_{adv}$.

block. As such, this ancilla register can be reused between all the blocks. Finally, SIAB_i needs read-only access to the advice $\vec{a}$, which is also shared between all SIAB_i blocks. Before SIAB_1 the registers for p , r , and $\vec{a}_b$ are initialized to all zeros.

A more detailed implementation of SIAB_i is given in Section 8.4.3.

8.4.2.3 Combining SIAB blocks

Having defined SIAB_i , the composition $\text{SIAB}_l \circ \dots \circ \text{SIAB}_1$ now computes the value we are interested in, still disregarding the low-space requirement. Note that in order to “compose” two blocks $\text{SIAB}_{i+1} \circ \text{SIAB}_i$, the second output register (Fig. 8.4) of SIAB_i needs to be connected to the first input register of SIAB_i . A circuit that composes the SIAB_i blocks in this manner is shown in Figure 8.5. In order to keep the computation unitary, the top output register of each SIAB_i block cannot simply be discarded, even though these registers are not used as inputs anymore.

Remark 8.1. *Naively composing all SIAB_i by simply allocating a new w -sized register for each block (see Fig. 8.5) requires $l \cdot w' + S_a$ qubits on top of the S_{adv} qubits storing the advice.*

To meet the space requirement of $o(n)$, we must reduce the $l \cdot w = n$ qubits used. To achieve this, we turn to Bennett’s strategy [22]. It relies on reducing the problem of turning a deterministic computation into a space-efficient reversible computation to a *reversible pebble game*. Such a pebble game is played on a rooted, directed graph, which corresponds to the dependency graph of inputs and outputs of different sub-computations. The rules of a reversible pebble game can be defined as follows: one can (un)pebble a leaf freely, whereas any other node can only be (un)pebbled if its predecessors are pebbled; one wins the game if one can put a pebble on the root node. After the game is won, the maximum number of pebbles used at any time during the game corresponds to the amount of space the reversible circuit requires.

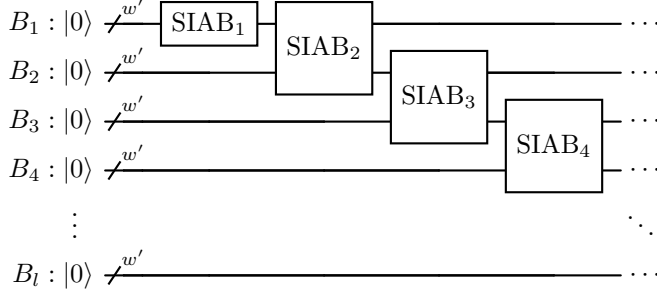


Figure 8.5: A (partial) visualization of the naive composition of all SIAB_i blocks. The bottom register of each SIAB_i block (see Fig. 8.4) has been omitted from this figure. This method requires $l \cdot w' + S_a$ qubits

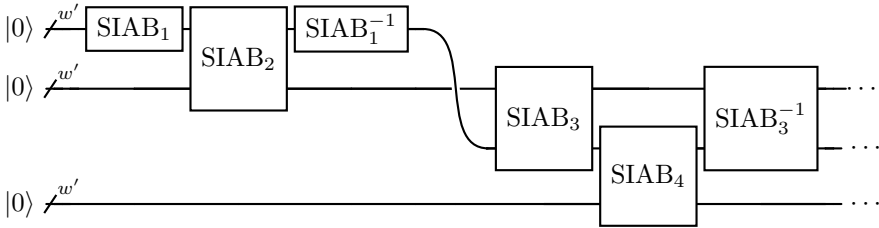


Figure 8.6: A (partial) visualization of SIAR; the composition of the SIAB_i blocks using Bennett's algorithm [22]. The bottom register of each SIAB_i block (see Fig. 8.4) has been omitted from this figure. Uncomputing SIAB_1 frees up a w' sized register which can be used by SIAB_3 . This method requires $w' \cdot (\log(l) + 1) + S_a$ qubits.

For our problem (composing $\text{SIAB}_l \circ \dots \circ \text{SIAB}_1$), the dependency graph is simply a line graph of length l . Bennett's algorithm constructs a concrete pebbling strategy for the line graph corresponding to a deterministic computation. The strategy shows that a line graph of length l can be reversibly pebbled with only $\lfloor \log(l) \rfloor + 1$ pebbles. A clear visualization of this algorithm is shown in [22; Table 2]. Figure 8.6 shows the SIAB_i blocks composed using Bennett's algorithm, and illustrates how this saves on the space requirements: For example, by temporarily uncomputing SIAB_1 , a w' sized register is freed up, which can be used by SIAB_3 . We will refer to this specific reversible composition of the SIAB_i blocks as SIAR.

Lemma 8.1 (Rephrased from Bennett [22]). *The composition of (irreversible) operations $f_t \circ \dots \circ f_1$, each taking T time and S space, can be implemented reversibly in $O(t^{\log(3)} \cdot T)$ time and $(\lfloor \log(t) \rfloor + 1) \cdot S$ space.*

Theorem 8.5. *There is a reversible circuit, SIAR, that computes SIA for a w -bit formula F using only*

$$T_r \triangleq O(l^{\log(3)} \cdot w \cdot (2 + sw)^{ks} \cdot \text{polylog}(n))$$

time and

$$S_r \triangleq (\lfloor \log(l) \rfloor + 1) \cdot w' + S_a = O(w \cdot \log(l))$$

space (wires), where $w' = (w + \log(n) + 1)$ and $S_a = w + O(\log(n))$.

Proof. By Theorem 8.6, SIAB can be reversibly implemented in $T_b \triangleq O(w \cdot (2 + sw)^{ks} \cdot \text{polylog}(n))$ time. The time for SIAR follows then from Lemma 8.1. In terms of space, all SIAB blocks share a $S_a = O(w + \log(n))$ sized register, on top of which Bennet's strategy requires $(\lfloor \log(l) \rfloor + 1) \cdot w'$ space to reversibly compose all SIAB_i (Lemma 8.1). $\square$

Theorem 8.4 simplifies this statement.

8.4.3 SIAB

Next, we implement the circuit SIAB_i defined in Section 8.4.2, which realizes a w -sized block of computation of s-SIA, by computing the values of the variables in B_i (Eq. (8.7)) given access to the variables in B_{i-1} . First, the dependency of SIAB_i on only the previous w assigned variables is proven in Section 8.4.3.1. After that, the specification of the circuit SIAB_i (given in Sections 8.4.3.2 and 8.4.3.3 ultimately leads to a proof of the space and time complexity (Section 8.4.3.4) stated in the following theorem.

8.4. Space-efficient quantum subroutines for PPSZ

Theorem 8.6. *Each circuit SIAB_i can be implemented reversibly in $T_b \triangleq O(w \cdot (2 + sw)^{ks} \cdot \text{polylog}(n))$ time, with $S_b \triangleq 2(w + \log(n) + 1)$ wires and $S_a \triangleq w + O(\log(n))$ ancillas, omitting the space for the advice $\vec{a}$.*

8.4.3.1 S-implication under bounded index width

In this subsection, we prove that in order to do s -implication in the dncPPSZ algorithm for a variable x_j , only two sets of variables need to be considered: the previous w assigned variables, and any of the unassigned variables x_k , with $j \leq k \leq n$.

We recall the definition of s -implication.

Definition 8.1 (s -implication). *A literal x_j ($\bar{x}_j$) is s -implied by F , written $F \models_s x_j$ ($F \models_s \bar{x}_j$), if and only if there is a subset of clauses $G \subseteq F$, with $|G| \leq s$, such that all satisfying assignments of G set x_j to 1 (0).*

The following lemma specifies that it is not necessary to remember a partial assignment in its entirety in order to determine s -implications.

Lemma 8.2. *Consider an n -variable formula F of bounded index width w , with a fixed variable order $x_1 \prec \dots \prec x_n$. Let $F|_\alpha$ be the formula obtained by assigning $\alpha = [\alpha_1, \dots, \alpha_{j-1}]$ to variables $[x_1, \dots, x_{j-1}]$, and assume that $F|_\alpha$ is not yet trivial. Then, in order to determine whether $F|_\alpha \models_s x_j$, it is sufficient to consider the assignment $\beta = [\alpha_{j-w}, \dots, \alpha_{j-1}]$ to $[x_{j-w}, \dots, x_{j-1}]$.*

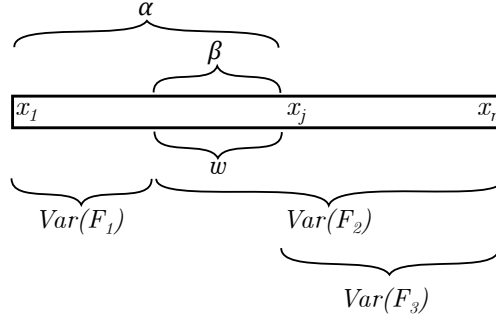
Proof. Let us define F_1 , F_2 , and F_3 as sub-formulas of F , such that each only contains clauses with the variables as visualized in Figure 8.7. By the property of index width, we have that $F|_\beta$ consists of two independent formulae F_1 and F_3 . Hence, we have $F_3 = F_{2|\beta} = F|_\alpha$. It immediately follows then that $F|_\alpha \models_s x_j \Leftrightarrow F_{2|\beta} \models_s x_j$. $\square$

Generally, in order to check if $F|_\alpha \models_s x_j$, all s -sized $G \subseteq F$ need to be considered, where each G can take variables from $\{x_1, \dots, x_n\}$. However, since F has bounded index width w , from Lemma 8.2 we have that $F_{2|\beta} = F|_\alpha$ and so checking all $G \subseteq F_2$ is sufficient.

8.4.3.2 Implementing SIAB_i

Algorithm 8.6 gives a high-level (non-reversible⁸) presentation of the implementation of SIAB_i , with references to the corresponding reversible subroutines (see Sec-

⁸For a reversible version of this procedure, non-reversible assignments $a \leftarrow b$ are replaced with “XOR assignments” $a \leftarrow a \oplus b$.


 Figure 8.7: Visualization of subsets of variables of F .

Algorithm 8.6: Pseudocode for SIAB_i .

```

1  def  $\text{SIAB}_i(\vec{y}_{i-1}, p, r)$  :
2      for  $j \in \{iw, \dots, iw + w - 1\}$  do ▷ Figure 8.8
3          if  $r = 0$  then
4               $c \leftarrow 0$ 
5              foreach  $s$ -sized  $G \subseteq F_{2|\vec{y}_{i-1}}$  do ▷ Fig. 8.9
6                  if  $c = 0$  then
7                       $(b', b'_j) \leftarrow \text{check if } G \models x_j, \bar{x}_j$  ▷ Fig. 8.10
8                      if  $b' = 1$  then ▷  $x_j$  is implied
9                           $c \leftarrow 1$ 
10                     if  $G$  is unsat then
11                          $c \leftarrow 1$ 
12                          $r \leftarrow 1$ 
13                 if  $b' = 1$  then ▷  $x_j$  is implied
14                      $x_j \leftarrow b'$ 
15                 else then ▷  $\mathcal{A}$  in Fig. 8.9
16                     if  $p < |\vec{a}|$  then
17                          $x_j \leftarrow \vec{a}[p]$ 
18                          $p \leftarrow p + 1$ 
19                     else then
20                          $r \leftarrow 1$ 
21      $o1, o2, o3 \leftarrow [x_{i \cdot w}, \dots, x_{(i+1) \cdot w}], p, r$ 
22     uncompute workspace ▷  $\mathcal{L}^\dagger$  in Fig. 8.8
    
```

8.4. Space-efficient quantum subroutines for PPSZ

tion 8.4.3.3). We omit SIAB_1 , which is a simpler version of SIAB_i without an input block.

Each circuit SIAB_i has the following input registers: register 1 contains an assignment $\vec{y}_{i-1}$ to previous w -sized block B_{i-1} , register 2 contains the pointer p to the next unused advice index (defined as a counter from 0 to $S_{adv} - 1 = |\vec{a}| - 1$, using $\lceil \log(S_{adv}) \rceil$ bits), register 3 contains a one-bit flag r which is raised whenever we encounter a contradiction or the advice is fully used. The registers $o1, o2, o3$ are used to output the next values $\vec{y}_i, p'$, and r' respectively. We also allocate ancilla registers $a0, \dots, a11$.

$$\begin{aligned} \text{SIAB}_i : & |\vec{y}_{i-1}\rangle_1 |p\rangle_2 |r\rangle_3 |0\rangle_{o1} |0\rangle_{o2} |0\rangle_{o3} |\vec{a}\rangle |\vec{a}_b\rangle \mapsto \\ & |\vec{y}_{i-1}\rangle_1 |p\rangle_2 |r\rangle_3 |y_i\rangle_{o1} |p'\rangle_{o2} |r'\rangle_{o3} |\vec{a}\rangle |\vec{a}_b\rangle \end{aligned}$$

The ancilla registers $a0, \dots, a8$ are the ones primarily manipulated by the subroutines $\mathcal{H}$ and $\mathcal{H}_G$, as defined in Section 8.4.3.3. Ancilla registers $a9$ and $a10$ are used to store c' and r' (see Alg. 8.6), such that c and r can be modified within the loop-body, without immediately affecting the control qubits c' and r' . Finally, ancilla register $a11$ is a buffer of size w in which one stores the values of the current w -block B_i .

The circuit in Figure 8.9 corresponds to the inner loop of SIAB_i and the subsequent instructions which fix the value of x_j . The unitary $\mathcal{A}$ corresponds to the instructions in the pseudocode which set x_j to the next advice bit and increase the advice pointer. Figure 8.8 gives the implementation of the whole circuit SIAB_i (note that running $\mathcal{L}_j$ is conditional on $r' = 0$, and the circuit flips r when it finds that the remainder of the formula is unsatisfiable). To simplify our representation, we omit ancilla wires that are only manipulated in the sub-components of the circuit, but not the overall circuit.

8

8.4.3.3 Reversible subroutines of SIAB_i

For each s -sized sub-formula $G \subseteq F$, we define a unitary

$$\begin{aligned} \mathcal{G} : & |\vec{y}\rangle_1 |\vec{z}\rangle_{a0} |j\rangle_{a1} |0\rangle_{a2} |0\rangle_{a3} \mapsto \\ & |\vec{y}\rangle_1 |\vec{z}\rangle_{a0} |j\rangle_{a1} |b\rangle_{a2} |b_j\rangle_{a3} \end{aligned}$$

which takes the following inputs: an index j , an assignment $\vec{y}$ to the w variables preceding x_j , an assignment $\vec{z}$ of length (at most) ks to variables of indices greater or equal to j , and two ancilla qubits to write the output to.

Each unitary $\mathcal{G}$ hardcodes the Boolean formula G , and encodes the following re-

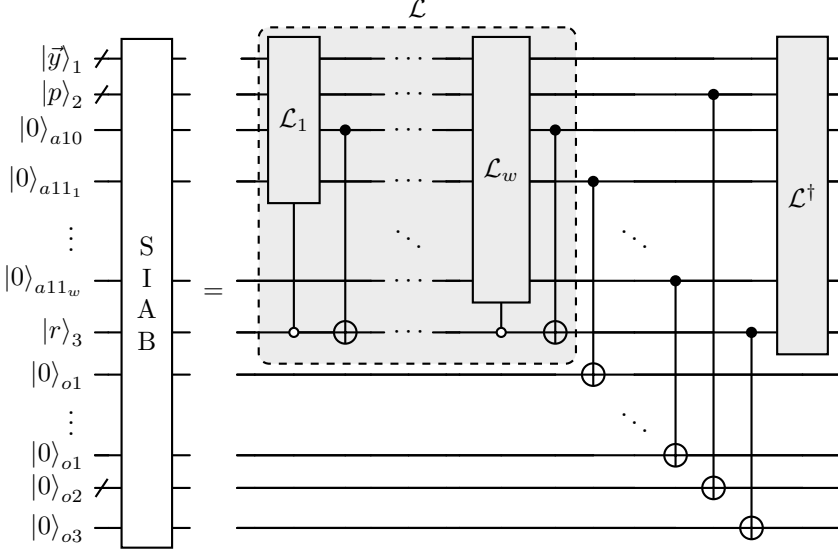


Figure 8.8: Circuit for SIAB_i , running the inner loop for every variable in the current w -block, provided that the remaining formula has not been found to be unsatisfiable (unsatisfiability would be indicated by $r > 0$). For ease of visualization, wires in $\mathcal{L}_j$ (Fig. 8.9) that are not directly relevant to this high-level overview have been omitted.

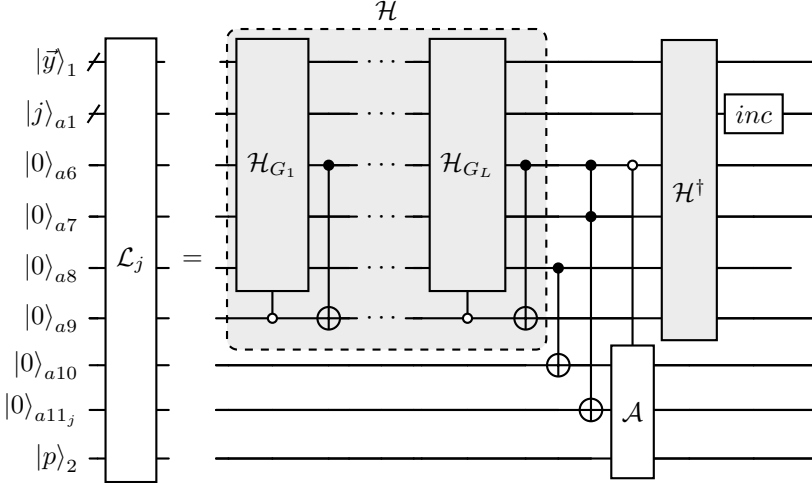


Figure 8.9: Inner loop $\mathcal{L}_j$ of SIAB_i , checking all s -sized subsets of clauses and fixing the value of the current variable. For ease of visualization, wires in $\mathcal{H}_G$ (Fig. 8.10) that are not directly relevant in $\mathcal{L}_j$ have been omitted.

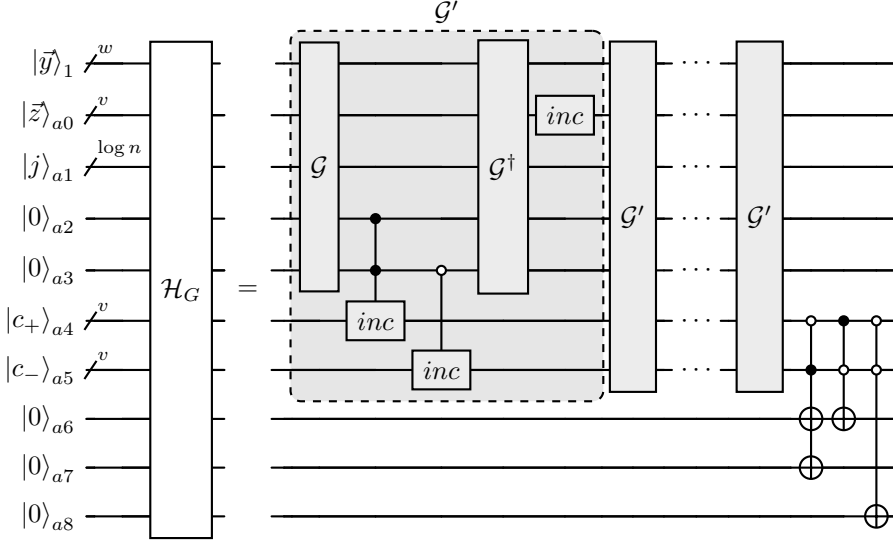


Figure 8.10: Routine $\mathcal{H}_G$, used to check whether x_j is s -implied by a given subset of clauses G . Each $\mathcal{G}'$ checks a single assignment $\vec{y}$ to the variables in G . The $\mathcal{G}'$ block is repeated $|V'|$ times. The registers for $\vec{z}$, c_+ , and c_- need $v = \lceil \log(|V'|) \rceil \leq ks$ qubits. The controls on $|c_+>_{a4}$ and $|c_->_{a5}$ check whether $c_\pm=0$ (open dot) or $c_\pm \neq 0$ (solid dot).

versible procedure: if variable x_j does not appear in G , output $(0,0)$; else if G is satisfied by the partial assignment specified by $\vec{y}$ on $x_{j-w}, \dots, x_{j-1}$ and $\vec{z}$ on

$$V' = \text{Var}(G) \setminus \{x_{j-w}, \dots, x_{j-1}\},$$

8

output $(1, b_j)$, where b_j is the assignment to variable x_j , and else output $(0,0)$.

Next, we define a unitary

$$\begin{aligned} \mathcal{G}' : |\vec{y}\rangle_1 |\vec{z}\rangle_{a0} |j\rangle_{a1} |c_+\rangle_{a4} |c_-\rangle_{a5} &\mapsto \\ |\vec{y}\rangle_1 |\vec{z} \oplus 1\rangle_{a0} |j\rangle_{a1} |c'_+\rangle_{a4} |c'_-\rangle_{a5} \end{aligned}$$

which applies $\mathcal{G}$, increase the counter c_+ (c_-) if $\mathcal{G}$ outputs $(1,1)$ ($(1,0)$), and then uncomputes the ancilla registers. Additionally, $\mathcal{G}'$ increased the value of the $|\vec{z}\rangle_{a0}$ register, such that the next $\mathcal{G}'$ is applied to the next partial assignment. This is shown in Figure 8.10. The registers $|c_+>_{a4}$ and $|c_->_{a5}$ (as well as $|\vec{z}\rangle_{a0}$) require $\lceil \log(|V'|) \rceil \leq ks$ qubits to count up to $2^{|V'|}$, the total number of possible assignments to G .

For each sub-formula $G \subseteq F$, we implement an unitary

$$\mathcal{H}_G : |\bar{x}\rangle_1 |j\rangle_{a1} |0\rangle_{a6} |0\rangle_{a7} |0\rangle_{a8} \mapsto |\bar{x}\rangle_1 |j\rangle_{a1} |b'\rangle_{a6} |b'_j\rangle_{a7} |r''\rangle_{a8}$$

which evaluates G for all its possible partial assignments, to determine whether the satisfying assignments of G agree on the value assigned to x_j , and if yes ($b' = 1$), outputs that value (b'_j). If G is unsatisfiable, $\mathcal{H}_G$ outputs $r'' = 1$.

Our implementation of $\mathcal{H}_G$ initializes the ancilla registers $a0, a2, \dots, a5$ to 0, and repeatedly applies $\mathcal{G}'$ and increments $\vec{y}$ (see Figure 8.10) up until $\vec{y} = 2^{|V'|} - 1$. After applying $\mathcal{G}'$ for all assignments $\vec{y}$, the following can be concluded based on the counters c_+ and c_- :

$$\begin{cases} x_j \text{ not implied, } G \text{ is sat} & \text{if } c_+ > 0, c_- > 0 \\ G \text{ implies } x_j & \text{if } c_+ > 0, c_- = 0 \\ G \text{ implies } \bar{x}_j & \text{if } c_+ = 0, c_- > 0 \\ G \text{ is unsat} & \text{if } c_+ = 0, c_- = 0 \end{cases}$$

If both c_+ and c_- are non-zero, G has satisfying assignments, but they do not all agree on x_j . In this case $\mathcal{H}_G$ outputs $(b', b'_j) = (0, 0)$. If exactly one of c_+ and c_- is non-zero, all satisfying assignments of G agree on x_j , and either x_j or $\bar{x}_j$ is implied by G . Here $\mathcal{H}_G$ outputs $(1, 1)$ or $(1, 0)$ respectively. If both c_+ and c_- are zero, G is not satisfiable at all given the variables assigned before x_j , and so neither is F . In this case, $\mathcal{H}_G$ outputs $r'' = 1$, which ultimately leads the circuit to raise the flag r .

8.4.3.4 Complexity of the implementation of SIAB

First, let us analyze the time complexity of SIAB_i . To simplify the analysis, we drop any $\text{polylog}(n)$ factor by writing $\tilde{O}(f(n)) = O(\text{polylog}(n) \cdot f(n))$. By doing so, the increment operations, as well as the $\mathcal{A}$ operation, can be done in $\tilde{O}(1)$ time.

The unitary $\mathcal{G}$ hardcodes a Boolean formula G which has (at most) s clauses and k variables per clause, taking $T_{\mathcal{G}} = O(ks) = \tilde{O}(1)$ time. The subroutine $\mathcal{G}'$ runs $\mathcal{G}$ and $\mathcal{G}^\dagger$, and a number of $\tilde{O}(1)$ operations, which gives $T_{\mathcal{G}'} = \tilde{O}(1)$. Next, $\mathcal{H}_G$ repeats $\mathcal{G}'$ for all assignments to $\vec{z}$, which are at most 2^{ks} , giving $T_{\mathcal{H}_G} = \tilde{O}(2^{ks})$.

The inner loop $\mathcal{L}_i$ loops over all s -sized formulae G . Since we consider formulas with bounded index width w , for any variable x_j , any sub-formula $G \subseteq F$ with s clauses can

either be split into independent formulae or contains only unassigned variables from $\{x_j, \dots, x_{j+sw}\}$ (a chain of s clauses each with index width w). Each s -sized G has at most ks variables, and which sw options per variable this results in at most $O((sw)^{ks})$ formulae G . This yields a time complexity of $T_{\mathcal{L}_j} = \tilde{O}((sw)^{ks} \cdot 2^{ks}) = \tilde{O}((2+sw)^{ks})$.

Finally, the outer loop of SIAB_i loops over w variables. Giving $T_b = \tilde{O}(w \cdot (2+sw)^{ks}) = O(w \cdot (2+sw)^{ks} \cdot \text{polylog}(n))$.

Including the input registers 1, 2, 3 and output registers o1, o2, o3 the space requirement for SIAB_i is $2(w + \log(n) + 1) + S_a$, where S_a is the space for the ancilla registers a0 through a11, which adds up to $S_a = w + \log(n) + 3ks + O(1) = w + O(\log(n))$.

8.5 Conclusion

Summary. In this chapter, we outlined a concrete approach for obtaining quantum speedups over the classical PPSZ SAT-solving algorithm, both in a purely quantum setting and in a hybrid setting.

To achieve this, we reformulated PPSZ as a divide-and-conquer tree-search algorithm and have proved that it suffices to explore the resulting search tree only up to depth $(\gamma_k + \varepsilon)n$. This resulted in an algorithm that retains PPSZ's classical runtime, while also being amenable to Grover-based quantum speedups.

To obtain speedups in the hybrid divide-and-quantum setting, we showed that it is crucial to limit the space usage of the quantum subroutines and have given detailed implementations of these subroutines. Our construction relies on limiting the approach to formulas with bounded index width.

Quantum backtracking. To achieve a full quadratic Grover-based speedup, the construction of the search tree in Section 8.3.1 requires knowing the branching factor beforehand. While the branching factor is known for PPSZ [204], not all pruning heuristics have (tight) theoretical bounds on the branching factor they yield.

As an alternative approach to Grover, *quantum backtracking* [163, 9, 126] approaches can achieve essentially full quadratic speed-ups in the number of queries to the tree (when trees are exponentially sized), rather than Grover's $O(\sqrt{2^d})$ speedup for depth d trees (regardless of how sparse they are). In the paper this chapter is based on [187], we also discuss the divide-and-quantum approach for these types of algorithms.

Quantum speedups over CDCL? Let us conclude by discussing a SAT-solving algorithm that is, in practice, favored over PPSZ, and why making a quantum version of this algorithm is not trivial using the same techniques that work for DPLL and PPSZ.

Even though PPSZ currently has the best-known worst-case asymptotic complexity for SAT-solving, it is typically not the algorithm that performs best in practice. While SAT solvers can employ many different heuristics and pre-processing steps, at the core of many modern SAT solvers is the *conflict-driven clause learning* (CDCL) algorithm [164, 153].

Similar to DPLL and PPSZ, the CDCL algorithm also explores a search tree of depth n , pruning branches when it detects they do not lead to solutions. However, there is a crucial difference. In algorithms like DPLL and our dncPPSZ formulation of PPSZ (Alg. 8.2), once the variable order is fixed, the exact structure of the search tree is fixed as well. In contrast, CDCL adds clauses to the original formula during the search. The goal of these clauses is so that later in the search, non-satisfying or “no-good” [218] branches can be recognized and pruned earlier. Crucially, the order in which the search space is explored, i.e., the choices made at guessed variables, affects what clauses are added, and thus also affects the overall structure of the search tree. On the other hand, with a quantum-based search, the search typically happens over a superposition of guesses, which means there is no order in which the guesses are explored. This leaves whether a quantum version of CDCL can be constructed as an open question.

Chapter 9

Conclusions

In this final chapter, we reflect back on the research questions posed in Chapter 1 and summarize the contributions made in this thesis. Finally, we discuss several directions for future work.

9.1 Contributions

Scaling of symbolic algorithms. A recurring problem throughout various areas of computer science is the need or desire to tackle increasingly large problems. While decision diagrams themselves are already a data structure designed to analyze large systems, there is still room to improve the effectiveness of DD-based methods and allow them to handle even larger systems. This led us to pose Research question 1.

Research question 1. *How can we scale up symbolic algorithms for the analysis of both (non)deterministic and quantitative systems?*

In Chapter 3, we presented a new DD algorithm for computing fixpoints in a transition system. A major hurdle in this task is the size of the DDs during the computation: while the DD that represents the fixpoint might be compact, DDs that are produced in the intermediate steps of the computation can often be much larger. For over two decades, the saturation algorithm [48] has been the state-of-the-art approach to tackling this problem. However, saturation is only applicable when the transition relation of a system can be partitioned into multiple more-local transition relations. In contrast, our new algorithm, which we implemented for BDDs and MDDs, can act on monolithic transition relations. In an empirical evaluation on a large set of

benchmarks, we have shown that our algorithm often performs faster than saturation, especially on instances that have low locality. Additionally, we found that, in a comparison with the state-of-the-art model checking tool ITS-tools [228], our new algorithm outperforms ITS-tools on 29% of models, suggesting that it can be useful as a complementary method in a portfolio approach.

In Chapter 4, we presented an efficient parallel implementation of edge-valued decision diagrams, which can be used for the analysis of quantum circuits. Due to diminishing gains in sequential computational speed in the past decade [227], the ability to parallelize computations has become increasingly important. Parallelizing DD computations is generally difficult, mostly for the same reasons that make the DD compact: because DDs merge equivalent subgraphs, splitting the work up between multiple threads without doing redundant work requires some amount of coordination, and determining whether subgraphs are equivalent requires a single table that stores unique nodes, to which concurrent access can become a bottleneck for the different threads. Despite this, several types of DDs have been efficiently parallelized in the DD package Sylvan [81]. However, attempts to parallelize EVDDs have yielded limited success, obtaining speedups of up to $\times 3$ on 32 cores [120, 148]. We evaluated our parallel EVDD implementation on a large set of benchmarks across two use cases: quantum circuit simulation and equivalence checking. We found that our new tool, Q-Sylvan,¹ shows a single-core performance that is competitive with the state-of-the-art quantum DD tool MQT DDSIM on large instances, and also achieves parallel speedups of up to $\times 18$ on 64 cores.

In Chapter 7, we tackled the problem of finding transformations of quantum states that can be described by graphs, a problem that has direct applications in quantum networking and can be related to problems in quantum error correction and measurement-based quantum computing. As work done in a Bachelor’s project [18] has shown, a DD-based approach to this problem struggles to solve instances beyond 10-node graphs. We hypothesize that this is in part due to the transition relation that describes this problem being highly non-local. As discussed in Chapter 3, the effectiveness of decision diagrams can greatly depend on finding a good variable order. However, for problems that contain highly non-local correlations, it can be the case that all variable orders yield large decision diagrams [232; Thm.2]. To be able to solve this problem for larger instances, we turned to a SAT-based approach. Our initial SAT-based approach was able to solve instances for up to 17-node graphs in under 30 minutes, performing significantly better than the DD-based approach. Additionally,

¹Which is available online: <https://github.com/System-Verification-Lab/Q-Sylvan>.

we showed that by over-constraining the formulas using domain-specific knowledge, further improvements can be made, with our approach ultimately being able to solve instances of up to 50 nodes in under 10 minutes.

In Chapter 8, instead of using symbolic algorithms for the analysis of quantitative systems, we investigated to what extent quantum computing can help symbolic algorithms. Specifically, we looked at how Grover’s quantum search algorithm can speed up PPSZ, the SAT-solving algorithm with the best-known asymptotic complexity. This task turned out to be less trivial than previously suggested [67], as the shape of PPSZ’s search space doesn’t directly lend itself to a naive application of Grover’s algorithm. In this chapter, we presented a quantum version of PPSZ that combines the quadratic speedup of Grover’s quantum search algorithm with the advantages of PPSZ, resulting in a quantum algorithm for 3-SAT with a complexity of $\sim O(2^{0.193n})$, compared to PPSZ’s classical complexity of $\sim O(2^{0.386n})$. We also specified a hybrid quantum-classical version of PPSZ, and the space-efficient quantum subroutines that it requires.

Handling floating-point values. The effectiveness of decision diagrams relies on recognizing equivalent sub-functions or subgraphs. For DDs that include floating-point values this can cause issues, as floating-point computations are subject to small rounding errors. While previous works have acknowledged that floating-point errors can cause issues in DDs [201, 241, 172, 214], a rigorous analysis of the effects of these errors has not been performed. To this end, we posed Research question 2.

Research question 2. *How do we handle floating-point values in decision diagrams?*

In Chapter 4, we have looked at the effect of floating-point errors in EVDDs when different edge-weight normalization strategies are used. We found here that normalization strategies that put small values high up in the DD appear to yield significantly numerical errors than strategies that either do the opposite (put small values low in the DD) or keep the magnitudes of edge values similar throughout the DD. However, as this was not the main focus of this chapter, we left a more thorough analysis for Chapter 5.

In Chapter 5, we performed a theoretical analysis of the propagation of numerical errors in MTBDDs. One of our main findings here is that for the errors to grow at most polynomially, MTBDDs that are used to analyze probabilistic and quantum systems require a floating-point precision that grows linearly with the number of variables in these DDs. Of course, most DD implementations that make use of floating-point

values use fixed-precision 64-bit floats, and thus do not satisfy this requirement. In an empirical evaluation, where we tested the simulation of various quantum circuits using MTBDDs, we found that the behavior of numerical errors can greatly depend on the specific class of instances. In line with the theoretical results, the empirical evaluation also found the need for increasing the numerical precision when larger systems are analyzed.

Discretization. While we considered discrete and finite domains throughout most of this thesis (either Boolean or pseudo-Boolean), in Chapter 6 we have taken a detour to consider the discretization of continuous domains. When using traditional methods, the computational costs of analyzing discretized domains can greatly increase as the discretization granularity increases. The compactness offered by DDs has been of great use when analyzing discrete systems, and might also be useful in compactly representing discretized continuous domains. This leaves us with the final question we posed.

Research question 3. *How can decision diagrams aid in the efficient discretization and analysis of continuous domains?*

In Chapter 6, we investigated the effectiveness of BDDs when encoding continuous Bayesian networks that have been discretized using different discretization methods and granularities. We find that when computing inference queries on these discretized Bayesian networks, a BDD-based approach can significantly outperform traditional inference algorithms, especially for higher discretization granularities.

9.2 Future work

In previous chapters, we discussed potential future research directions relating to the research in those individual chapters. In this section, we reiterate some of the ideas that look most promising, as well as discuss some ideas not discussed previously.

Extend ReachDD to the quantitative domain. The REACHDD algorithm presented in Chapter 3 computes fixpoints for Boolean or integer transition systems. A natural next step, especially in the context of this thesis, is to extend this algorithm to quantitative transition systems. REACHDD computes $\vec{x}.R^*$, which, for an initial set states $\vec{x}$ and a transition relation R , gives all reachable states from $\vec{x}$ under R . The notation $\vec{x}.R^*$ can be thought of as “repeatedly apply R until no new states are found”.

In Markov chains, steady states can be found in a similar way: for a Markov chain M and an initial probability distribution over states $\vec{x}$, repeatedly applying M will eventually converge to a steady state (for “well-behaved” Markov chains). However, it is not trivially clear that REACHDD can be used to compute steady states for Markov chains. In the Boolean case, $\vec{x}$ changes monotonically, i.e., states are only ever added to $\vec{x}$, never removed. On the other hand, when computing $\vec{x}M^*$ for a Markov chain, the probabilities in $\vec{x}$ can either increase or decrease.

Adapt ReachDD to handle local transition relations more efficiently. While our REACHDD algorithm performs quite well on average, we see in Figure 3.5 that the saturation algorithm still outperforms REACHDD on instances with high locality—exactly the instances where saturation is expected to perform well by its design. It might be possible to integrate the ideas of ReachDD in saturation, or vice versa, to obtain a single algorithm that combines both their strengths.

Numerical error analysis for other DDs and operations. In Chapter 5, we analyzed numerical errors in the matrix-vector multiplication algorithm for MTBDDs. However, other types of DDs exist that offer significantly more conciseness, such as the EVDDs we use in Chapter 4, as well as LIMDDs and CFLOBDDs. It would be valuable to understand how numerical errors behave in these decision diagrams as well. This is, however, not without its challenges: these other DDs are not only more concise, but also more complicated, which increases the complexity of a theoretical error analysis. However, a thorough empirical analysis, such as we give in Section 5.4, is not hindered by the complexity of the decision diagrams.

In a similar direction, we can also analyze the behavior of numerical errors in other DD algorithms. As mentioned in Section 5.2.4, for matrix multiplication without DDs, it has been shown that while the traditional $O(n^3)$ algorithm is componentwise numerically stable [118], faster algorithms such as the Strassen algorithm are not [161]. Analyzing numerical errors in various DD algorithms might also lead to insights into how to design DD algorithms that are more robust against these errors.

Heuristic improvements to numerical errors in DDs. In Chapter 5, we discussed that, due to floating-point rounding errors, it is necessary to introduce some merging threshold δ , which says that when two values a and a' occur in the DD, they should be merged if $|a - a'| \leq \delta$. While the theoretical analysis in Chapter 5 shows that, in the worst case, this δ -merging can cause errors that are significantly worse than

the floating-point errors by themselves, it might also be possible to take advantage of δ -merging in order to actually *reduce* numerical errors. In practical DD implementations, this “merging” means that whichever value was inserted first is preserved and overwrites the other value. Knowing this, we could start by populating the DD with correct versions of values that we know are going to occur during the computation. For example, at the beginning of the calculation we insert $1/\sqrt{2} \approx 0.7071067811865475$, and then when during the calculation 0.7071067811865476 appears, it is automatically replaced with the correct approximation inserted earlier. Empirically, we see the positive effect of this in Section 5.4.2, where the output of the calculation becomes more correct when δ is set to a small non-zero value, as it allows the slightly incorrect $\cos(\pi/4)$ values (which *should* equal $1/\sqrt{2}$) to be merged with (i.e. replaced by) the more precise approximation of $1/\sqrt{2}$.

Apply numerical analysis results from WMC to DDs. Recent work has been done on the analysis of numerical errors in weighted model counting (WMC) [38]. Weighted model counting is a generalization of model counting (i.e. counting the number of satisfying assignments of a Boolean formula) in which each assignment is associated with a weight, and the sum of the weights of all satisfying assignments is to be computed. Many problems regarding the analysis of quantitative systems, such as quantum circuit simulation and equivalence checking (Chapter 4) or computing inference queries on Bayesian networks (Chapter 6), can be reduced to WMC [156, 157, 199]. In [38], the numerical errors of WMC computations are analyzed, and from this analysis conclusions are made about the required numerical precision to guarantee accurate outputs. Although the problems for DDs are a bit different from those for weighted model counting in general—for DDs, aside from the numerical correctness of the output, we also want the DDs to be compact—the results from [38] can be applied to DDs to improve the numerical correctness of their outputs, and possibly aid their compactness as well.

Extend graph state SAT approach to related problems. In Chapter 7, we introduced a SAT approach to find transformations from given source to target graphs under local complementations (LC), vertex deletions (VD), and edge flips (EF). However, several related problems also appear in recent literature, such as “Given a graph G , find the smallest graph that is reachable from G under LC” [42, 212] and “Given a graph G , construct G from an empty graph using LC, VD, and EF, using the lowest possible number of edge flips” [141]. Since our approach is quite general, it is

straightforward to extend it to cover these related problems. However, challenges still remain, as previous work [189] has shown that the minimization involved in finding the smallest graph rather than a specific graph comes with a significant computational overhead.

Bibliography

- [1] Scott Aaronson. “BQP and the polynomial hierarchy”. In: *Proceedings of the forty-second ACM symposium on Theory of computing*. 2010, pp. 141–150.
- [2] Scott Aaronson and Daniel Gottesman. “Improved simulation of stabilizer circuits”. In: *Physical Review A* 70.5 (2004), p. 052328.
- [3] Karim Abbas, Joost Berkhout, and Bernd Heidergott. “A critical account of perturbation analysis of Markov chains”. In: *Markov Processes and Related Fields* 22 (2016), pp. 227–265.
- [4] Abbas Acar, Hidayet Aksu, A Selcuk Uluagac, and Mauro Conti. “A survey on homomorphic encryption schemes: theory and implementation”. In: *ACM Computing Surveys (Csur)* 51.4 (2018), pp. 1–35.
- [5] Jeremy C Adcock, Sam Morley-Short, Axel Dahlberg, and Joshua W Silverstone. “Mapping graph state orbits under local complementation”. In: *Quantum* 4 (2020), p. 305.
- [6] Fadi A Aloul, Igor L Markov, and Karem A Sakallah. “Faster SAT and smaller BDDs via common function structure”. In: *ICCAD 2001*. IEEE. 2001, pp. 443–448.
- [7] Fadi A Aloul, Igor L Markov, and Karem A Sakallah. “FORCE: a fast and easy-to-implement variable-ordering heuristic”. In: *ACM VLSI*. 2003, pp. 116–119.
- [8] Andris Ambainis. “Quantum search algorithms”. In: *ACM SIGACT News* 35.2 (2004), pp. 22–35. DOI: 10.1145/992287.992296.
- [9] Andris Ambainis and Martins Kokainis. “Quantum algorithm for tree size estimation, with applications to backtracking and 2-player games”. In: *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing*. 2017, pp. 989–1002. DOI: 10.1145/3055399.3055444.

- [10] Jérôme Amilhastre, Hélène Fargier, Alexandre Niveau, and Cédric Pralet. “Compiling CSPs: a complexity map of (non-deterministic) multivalued decision diagrams”. In: *International Journal on Artificial Intelligence Tools* 23.04 (2014), p. 1460015.
- [11] Elvio Gilberto Amparore, Marco Beccuti, and Susanna Donatelli. “Gradient-based variable ordering of decision diagrams for systems with structural units”. In: *ATVA*. Springer. 2017, pp. 184–200.
- [12] Elvio Gilberto Amparore, Susanna Donatelli, Marco Beccuti, Giulio Garbi, and Andrew Miner. “Decision diagrams for Petri nets: a comparison of variable ordering algorithms”. In: *Transactions on Petri Nets and Other Models of Concurrency XIII*. Springer, 2018, pp. 73–92.
- [13] Matthew Amy. “Towards large-scale functional verification of universal quantum circuits”. In: *arXiv preprint arXiv:1805.06908* (2018).
- [14] Simon Anders and Hans J Briegel. “Fast simulation of stabilizer circuits using a graph-state representation”. In: *Physical Review A* 73.2 (2006), p. 022334.
- [15] Boran Apak, Medina Bandic, Aritra Sarkar, and Sebastian Feld. “KetGPT–Dataset augmentation of quantum circuits using transformers”. In: *International Conference on Computational Science*. Springer. 2024, pp. 235–251.
- [16] Robert St-Aubin, Jesse Hoey, and Craig Boutilier. “APRICODD: approximate policy construction using decision diagrams”. In: *Advances in Neural Information Processing Systems* 13 (2000).
- [17] Gilles Audemard and Laurent Simon. “On the Glucose SAT solver”. In: *International Journal on Artificial Intelligence Tools* 27.01 (2018), p. 1840001.
- [18] Stan van Baarsen. “Using binary decision diagrams for exploring graph state orbits”. Bachelor’s Thesis. 2024. URL: <https://theses.liacs.nl/2918>.
- [19] R Iris Bahar, Erica A Frohm, Charles M Gaona, Gary D Hachtel, Enrico Macii, Abelardo Pardo, and Fabio Somenzi. “Algebraic decision diagrams and their applications”. In: *FMSD* 10.2 (1997), pp. 171–206.
- [20] David A Barrington. “Bounded-width polynomial-size branching programs recognize exactly those languages in NC1”. In: *Journal of Computer and System Sciences* 38.1 (1989), pp. 150–164. DOI: 10.1016/0022-0000(89)90037-8.

- [21] Maurice H ter Beek, Arne Borälv, Alessandro Fantechi, Alessio Ferrari, Stefania Gnesi, Christer Löfving, and Franco Mazzanti. “Adopting formal methods in an industrial setting: the railways case”. In: *Formal Methods—The Next 30 Years: Third World Congress, FM 2019, Porto, Portugal, October 7–11, 2019, Proceedings 3*. Springer. 2019, pp. 762–772.
- [22] Charles H Bennett. “Time/space trade-offs for reversible computation”. In: *SIAM Journal on Computing* 18.4 (1989), pp. 766–776. DOI: 10.1137/0218053.
- [23] Charles H Bennett and Gilles Brassard. “Quantum cryptography: Public key distribution and coin tossing”. In: *Proceedings of IEEE International Conference on Computers, Systems and Signal Processing* 175 (1984). DOI: 10.1016/j.tcs.2014.05.025.
- [24] Lucas Berent, Lukas Burgholzer, and Robert Wille. “Towards a SAT encoding for quantum circuits: a journey from classical circuits to Clifford circuits and beyond”. In: *arXiv:2203.00698* (2022).
- [25] Freark van der Berg and Alfons Laarman. “SpinS: extending LTSmin with Promela through SpinJa”. In: *ENTCS* 296 (2013), pp. 95–105.
- [26] Tomas Beuzen, Lucy Marshall, and Kristen D Splinter. “A comparison of methods for discretizing continuous variables in Bayesian networks”. In: *Environmental modelling & software* 108 (2018), pp. 61–66.
- [27] Armin Biere, Cyrille Artho, and Viktor Schuppan. “Liveness checking as safety checking”. In: *ENTCS* 66.2 (2002), pp. 160–177.
- [28] Armin Biere, Alessandro Cimatti, Edmund M Clarke, Ofer Strichman, and Yunshan Zhu. “Bounded model checking.” In: *Handbook of satisfiability*. IOS press, 2009, pp. 457–481.
- [29] Armin Biere and Mathias Fleury. “Gimsatul, IsaSAT and Kissat entering the SAT Competition 2022”. In: *Proc. of SAT Competition 2022 – Solver and Benchmark Descriptions*. Ed. by Tomas Balyo, Marijn Heule, Markus Iser, Matti Järvisalo, and Martin Suda. Vol. B-2022-1. Department of Computer Science Series of Publications B. University of Helsinki, 2022, pp. 10–11.
- [30] Armin Biere, Mathias Fleury, Nils Froleyks, and Marijn JH Heule. “The SAT museum.” In: *14th International Workshop on Pragmatics of SAT*. 2023, pp. 72–87.
- [31] Armin Biere, Marijn Heule, and Hans van Maaren. *Handbook of satisfiability*. Vol. 185. IOS press, 2009.

- [32] Magnus Björk. “Successful SAT encoding techniques”. In: *Journal on Satisfiability, Boolean Modelling and Computation* 7.4 (2011), pp. 189–201.
- [33] Jasmin Christian Blanchette, Sascha Böhme, and Lawrence C Paulson. “Extending Sledgehammer with SMT solvers”. In: *Journal of automated reasoning* 51.1 (2013), pp. 109–128. DOI: 10.1007/s10817-013-9278-5.
- [34] Stefan Blom and Jaco van De Pol. “Symbolic reachability for process algebras with recursive data types”. In: *ICTAC*. Springer. 2008, pp. 81–95.
- [35] Beate Bollig and Ingo Wegener. “Improving the variable ordering of OBDDs is NP-complete”. In: *Transactions on computers* 45.9 (1996), pp. 993–1002.
- [36] André Bouchet. “An efficient algorithm to recognize locally equivalent graphs”. In: *Combinatorica* 11 (1991), pp. 315–329.
- [37] Randal E Bryant. “Graph-based algorithms for Boolean function manipulation”. In: *IEEE Transactions on Computers* 100.8 (1986), pp. 677–691.
- [38] Randal E Bryant. “Numerical considerations in weighted model counting”. In: *arXiv preprint arXiv:2508.06264* (2025).
- [39] Randal E Bryant. “Symbolic Boolean manipulation with ordered binary-decision diagrams”. In: *CSUR* 24.3 (1992), pp. 293–318.
- [40] Jerry R Burch, Edmund M Clarke, Kenneth L McMillan, David L Dill, and Lain-Jinn Hwang. “Symbolic model checking: 10^{20} states and beyond”. In: *Information and computation* 98.2 (1992), pp. 142–170.
- [41] Lukas Burgholzer and Robert Wille. “Advanced equivalence checking for quantum circuits”. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 40.9 (2020), pp. 1810–1824.
- [42] Adán Cabello, Lars Eirik Danielsen, Antonio J López-Tarrida, and José R Portillo. “Optimal preparation of graph states”. In: *Physical Review A* 83.4 (2011), p. 042314.
- [43] Yudong Cao, Jonathan Romero, Jonathan P. Olson, Matthias Degroote, Peter D. Johnson, Mária Kieferová, Ian D. Kivlichan, Tim Menke, Borja Peropadre, Nicolas P. D. Sawaya, Sukin Sim, Libor Veis, and Alán Aspuru-Guzik. “Quantum chemistry in the age of quantum computing”. In: *Chemical Reviews* 119.19 (Oct. 2019), pp. 10856–10915. ISSN: 0009-2665. DOI: 10.1021/acs.chemrev.8b00803. URL: <https://doi.org/10.1021/acs.chemrev.8b00803>.

- [44] Roberto Cavada, Alessandro Cimatti, Michele Dorigatti, Alberto Griggio, Alessandro Mariotti, Andrea Micheli, Sergio Mover, Marco Roveri, and Stefano Tonetta. “The nuXmv symbolic model checker”. In: *CAV*. Springer. 2014, pp. 334–342.
- [45] Mark Chavira and Adnan Darwiche. “On probabilistic inference by weighted model counting”. In: *Artificial Intelligence* 172.6-7 (2008), pp. 772–799.
- [46] Matthias Christandl and Stephanie Wehner. “Quantum anonymous transmissions”. In: *Advances in Cryptology-ASIACRYPT 2005: 11th International Conference on the Theory and Application of Cryptology and Information Security, Chennai, India, December 4-8, 2005. Proceedings 11*. Springer. 2005, pp. 217–235.
- [47] Søren Christensen, Lars Michael Kristensen, and Thomas Mailund. “A sweep-line method for state space exploration”. In: *TACAS*. Springer. 2001, pp. 450–464.
- [48] Gianfranco Ciardo, Gerald Lüttgen, and Radu Siminiceanu. “Saturation: an efficient iteration strategy for symbolic state generation”. In: *Tools and Algorithms for the Construction and Analysis of Systems*. Springer. 2001, pp. 328–342.
- [49] Gianfranco Ciardo, Robert Marmorstein, and Radu Siminiceanu. “Saturation unbound”. In: *TACAS*. Ed. by Hubert Garavel and John Hatcliff. Springer, 2003, pp. 379–393. ISBN: 978-3-540-36577-8.
- [50] Edmund Clarke, Armin Biere, Richard Raimi, and Yunshan Zhu. “Bounded model checking using satisfiability solving”. In: *Formal methods in system design* 19 (2001), pp. 7–34.
- [51] Byron Cook, Andreas Podelski, and Andrey Rybalchenko. “Terminator: beyond safety”. In: *CAV*. Springer. 2006, pp. 415–418.
- [52] Gregory F Cooper. “The computational complexity of probabilistic inference using Bayesian belief networks”. In: *Artificial intelligence* 42.2-3 (1990), pp. 393–405.
- [53] Alessandro Cosentino, Robin Kothari, and Adam Paetznick. “Dequantizing Read-once Quantum Formulas”. In: *8th Conference on the Theory of Quantum Computation, Communication and Cryptography (TQC 2013)*. Vol. 22. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2013, pp. 80–92. DOI: 10.4230/LIPIcs.TQC.2013.80.

- [54] B Courcelle and Irène A Durand. “On using SAT solvers for graph computations”. In: *HAL preprint hal-03668884* (2022).
- [55] Bruno Courcelle and Joost Engelfriet. *Graph structure and monadic second-order logic: a language-theoretic approach*. Encyclopedia of Mathematics and its Applications. Cambridge University Press, 2012.
- [56] Bruno Courcelle and Sang-il Oum. “Vertex-minors, monadic second-order logic, and a conjecture by seese”. In: *Journal of Combinatorial Theory, Series B* 97.1 (2007), pp. 91–126.
- [57] Jean-Michel Couvreur and Yann Thierry-Mieg. “Hierarchical decision diagrams to exploit model structure”. In: *International Conference on Formal Techniques for Networked and Distributed Systems*. Springer. 2005, pp. 443–457.
- [58] Andrew W Cross, Lev S Bishop, John A Smolin, and Jay M Gambetta. “Open quantum assembly language”. In: *arXiv preprint arXiv:1707.03429* (2017).
- [59] Axel Dahlberg, Jonas Helsen, and Stephanie Wehner. “Counting single-qubit Clifford equivalent graph states is $\#P$ -complete”. In: *Journal of Mathematical Physics* 61.2 (2020), p. 022202.
- [60] Axel Dahlberg, Jonas Helsen, and Stephanie Wehner. “How to transform graph states using single-qubit operations: computational complexity and algorithms”. In: *Quantum Science and Technology* 5.4 (2020), p. 045016.
- [61] Axel Dahlberg, Jonas Helsen, and Stephanie Wehner. “Transforming graph states to Bell-pairs is NP-complete”. In: *Quantum* 4 (2020), p. 348.
- [62] Axel Dahlberg and Stephanie Wehner. “Transforming graph states using single-qubit operations”. In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 376.2123 (2018), p. 20170325.
- [63] Giso H Dal, Alfons W Laarman, Arjen Hommersom, and Peter JF Lucas. “A compositional approach to probabilistic knowledge compilation”. In: *International Journal of Approximate Reasoning* 138 (2021), pp. 38–66.
- [64] Giso H Dal, Alfons W Laarman, and Peter JF Lucas. “Parallel probabilistic inference by weighted model counting”. In: *International Conference on Probabilistic Graphical Models*. PMLR. 2018, pp. 97–108.
- [65] Giso H Dal and Peter JF Lucas. “Weighted positive binary decision diagrams for exact probabilistic inference”. In: *International Journal of Approximate Reasoning* 90 (2017), pp. 411–432.

-
- [66] Lars Eirik Danielsen and Matthew G Parker. “On the classification of all self-dual additive codes over $gf(4)$ of length up to 12”. In: *Journal of Combinatorial Theory, Series A* 113.7 (2006), pp. 1351–1367.
 - [67] Evgeny Dantsin, Vladik Kreinovich, and Alexander Wolpert. “On quantum versions of record-breaking algorithms for SAT”. In: *ACM SIGACT News* 36.4 (2005), pp. 103–108.
 - [68] Adnan Darwiche. “A logical approach to factoring belief networks”. In: *KR* 2 (2002), pp. 409–420.
 - [69] Adnan Darwiche. “SDD: a new canonical representation of propositional knowledge bases”. In: *IJCAI*. 2011.
 - [70] Adnan Darwiche and Pierre Marquis. “A knowledge compilation map”. In: *Journal of Artificial Intelligence Research* 17 (2002), pp. 229–264.
 - [71] Martin Davis, George Logemann, and Donald W Loveland. *A machine program for theorem-proving*. New York University, Institute of Mathematical Sciences, 1961. DOI: 10.1145/368273.368557.
 - [72] Martin Davis and Hilary Putnam. “A Computing Procedure for Quantification Theory”. In: *J. ACM* 7.3 (July 1960), pp. 201–215. ISSN: 0004-5411. DOI: 10.1145/321033.321034. URL: <https://doi.org/10.1145/321033.321034>.
 - [73] Christopher M Dawson and Michael A Nielsen. “The Solovay-Kitaev algorithm”. In: *arXiv preprint quant-ph/0505030* (2005).
 - [74] James Demmel, Ioana Dumitriu, and Olga Holtz. “Fast linear algebra is stable”. In: *Numerische Mathematik* 108.1 (2007), pp. 59–91.
 - [75] James Demmel, Ioana Dumitriu, Olga Holtz, and Robert Kleinberg. “Fast matrix multiplication is stable”. In: *Numerische Mathematik* 106.2 (2007), pp. 199–224.
 - [76] Tom van Dijk, Alfons Laarman, and Jaco van de Pol. “Multi-core BDD operations for symbolic reachability”. In: *ENTCS* 296 (2013), pp. 127–143.
 - [77] Tom van Dijk, Alfons Laarman, and Jaco Van De Pol. “Multi-core BDD operations for symbolic reachability”. In: *Electronic Notes in Theoretical Computer Science* 296 (2013), pp. 127–143.
 - [78] Tom van Dijk, Jeroen Meijer, and Jaco van de Pol. “Multi-core on-the-fly saturation”. In: *TACAS*. Springer International Publishing, 2019, pp. 58–75.

- [79] Tom van Dijk and Jaco van de Pol. “Sylvan: multi-core decision diagrams”. In: *TACAS*. Springer. 2015, pp. 677–691.
- [80] Tom van Dijk and Jaco C van de Pol. “Lace: non-blocking split deque for work-stealing”. In: *Euro-Par 2014: Parallel Processing Workshops: Euro-Par 2014 International Workshops, Porto, Portugal, August 25-26, 2014, Revised Selected Papers, Part II 20*. Springer. 2014, pp. 206–217.
- [81] Tom van Dijk and Jaco Van de Pol. “Sylvan: multi-core framework for decision diagrams”. In: *International Journal on Software Tools for Technology Transfer* 19.6 (2017), pp. 675–696.
- [82] Vedran Dunjko, Yimin Ge, and J Ignacio Cirac. “Computational speedups using small quantum devices”. In: *Physical review letters* 121.25 (2018), p. 250501. DOI: 10.1103/PhysRevLett.121.250501.
- [83] Jens Eisert, Kurt Jacobs, Polykarpos Papadopoulos, and Martin B Plenio. “Optimal local implementation of nonlocal quantum gates”. In: *Physical Review A* 62.5 (2000), p. 052317.
- [84] Artur K. Ekert. “Quantum cryptography based on Bell’s theorem”. In: *Phys. Rev. Lett.* 67 (6 Aug. 1991), pp. 661–663. DOI: 10.1103/PhysRevLett.67.661. URL: <http://link.aps.org/doi/10.1103/PhysRevLett.67.661>.
- [85] Andreas Enge, Mickaël Gastineau, Philippe Théveny, and Paul Zimmermann. *GNU MPC*. <https://www.multiprecision.org/>. 2022.
- [86] Matthias Englbrecht, Tristan Kraft, and Barbara Kraus. “Transformations of stabilizer states in quantum networks”. In: *Quantum* 6 (2022), p. 846.
- [87] Hélène Fargier, Pierre Marquis, Alexandre Niveau, and Nicolas Schmidt. “A knowledge compilation map for ordered real-valued decision diagrams”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 28.1 (2014).
- [88] Usama M Fayyad and Keki B Irani. “Multi-interval discretization of continuous-valued attributes for classification learning”. In: *International Joint Conference on Artificial Intelligence*. 1993, pp. 1022–1029.
- [89] Joan Feigenbaum, Sampath Kannan, Moshe Y Vardi, and Mahesh Viswanathan. “Complexity of problems on graphs represented as OBDDs”. In: *STACS*. Springer. 1998, pp. 216–226.
- [90] Alessio Ferrari and Maurice H Ter Beek. “Formal methods in railways: a systematic mapping study”. In: *ACM Computing Surveys* 55.4 (2022), pp. 1–37.

- [91] R Iris Bahar Erica A Frohm and Charles M Gaona Gary D Hachtel. “Algebraic decision diagrams and their applications”. In: *International Conference on Computer Aided Design*. 1993.
- [92] Masahiro Fujita, Patrick C. McGeer, and JC-Y Yang. “Multi-terminal binary decision diagrams: an efficient data structure for matrix representation”. In: *Formal methods in system design* 10.2-3 (1997), pp. 149–169.
- [93] Francois Gagnon, J Ch Grégoire, and D Zampunieris. “Sharing trees for “on-the-fly” verification”. In: *International Conference on Formal Techniques for Distributed Objects, Components, and Systems*. Springer. 1995, pp. 417–432.
- [94] Robert Ganian, Neha Lodha, Sebastian Ordyniak, and Stefan Szeider. “SAT-encodings for treecut width and treedepth”. In: *2019 Proceedings of the Twenty-First Workshop on Algorithm Engineering and Experiments*. SIAM. 2019, pp. 117–129.
- [95] Yimin Ge and Vedran Dunjko. “A hybrid algorithm framework for small quantum computers with application to finding hamiltonian cycles”. In: *Journal of Mathematical Physics* 61.1 (2020), p. 012201. DOI: 10.1063/1.5119235.
- [96] Tomas Geffner, Javier Antoran, Adam Foster, Wenbo Gong, Chao Ma, Emre Kiciman, Amit Sharma, Angus Lamb, Martin Kukla, Nick Pawlowski, et al. “Deep end-to-end causal inference”. In: *arXiv preprint arXiv:2202.02195* (2022).
- [97] Jaco Geldenhuys and Antti Valmari. “Techniques for smaller intermediary BDDs”. In: *CONCUR*. Ed. by Kim G. Larsen and Mogens Nielsen. Berlin, Heidelberg: Springer, 2001, pp. 233–247. ISBN: 978-3-540-44685-9.
- [98] Irving John Good. “Some history of the hierarchical Bayesian methodology”. In: *Trabajos de estadística y de investigación operativa* 31 (1980), pp. 489–519.
- [99] Martijn Goorden, Joanna van de Mortel-Fronczak, Koen van Eldik, Wan Fokkink, and Jacobus Rooda. “Lessons learned in the application of formal methods to the design of a storm surge barrier control system”. In: *IFAC-PapersOnLine* 55.28 (2022), pp. 93–99.
- [100] Daniel Gottesman. *Stabilizer codes and quantum error correction*. California Institute of Technology, 1997.
- [101] Daniel M Greenberger, Michael A Horne, and Anton Zeilinger. “Going beyond bell’s theorem”. In: *Bell’s theorem, quantum theory and conceptions of the universe*. Springer, 1989, pp. 69–72.

- [102] Raymond Greenlaw, H James Hoover, Walter L Ruzzo, et al. *Limits to parallel computation: p-completeness theory*. Oxford University Press on Demand, 1995. URL: <https://dl.acm.org/doi/book/10.5555/203244>.
- [103] Lov K Grover. “A fast quantum mechanical algorithm for database search”. In: *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*. 1996, pp. 212–219.
- [104] Frederik Hahn, Axel Dahlberg, Jens Eisert, and Anna Pappa. “Limitations of nearest-neighbor quantum networks”. In: *Physical Review A* 106.1 (2022), p. L010401.
- [105] Frederik Hahn, A Pappa, and Jens Eisert. “Quantum network routing and local complementation”. In: *npj Quantum Information* 5.1 (2019), p. 76.
- [106] Thomas Dueholm Hansen, Haim Kaplan, Or Zamir, and Uri Zwick. “Faster k-SAT algorithms using biased-PPSZ”. In: *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*. STOC 2019. Phoenix, AZ, USA: ACM, 2019, pp. 578–589. ISBN: 978-1-4503-6705-9. DOI: 10.1145/3313276.3316359. URL: <http://doi.acm.org/10.1145/3313276.3316359>.
- [107] Arnd Hartmanns. “Correct probabilistic model checking with floating-point arithmetic”. In: *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer. 2022, pp. 41–59.
- [108] Stuart Harwood, Claudio Gambella, Dimitar Tenev, Andrea Simonetto, David Bernal, and Donny Greenberg. “Formulating and solving routing problems on quantum computers”. In: *IEEE Transactions on Quantum Engineering* 2 (2021), pp. 1–17.
- [109] Marc Hein, Wolfgang Dür, Jens Eisert, Robert Raussendorf, M Nest, and H-J Briegel. “Entanglement in graph states and its applications”. In: *arXiv preprint quant-ph/0602096* (2006).
- [110] Marc Hein, Jens Eisert, and Hans J Briegel. “Multiparty entanglement in graph states”. In: *Physical Review A* 69.6 (2004), p. 062311.
- [111] Christian Hensel, Sebastian Junges, Joost-Pieter Katoen, Tim Quatmann, and Matthias Volk. “The probabilistic model checker Storm”. In: *International Journal on Software Tools for Technology Transfer* (2022), pp. 1–22.
- [112] Timon Hertli. “3-SAT faster and simpler—unique-SAT bounds for PPSZ hold in general”. In: *SIAM Journal on Computing* 43.2 (2014), pp. 718–729. DOI: 10.1137/120868177.

- [113] Timon Hertli. “Breaking the PPSZ barrier for unique 3-SAT”. In: *Automata, Languages, and Programming: 41st International Colloquium, ICALP 2014, Copenhagen, Denmark, July 8-11, 2014, Proceedings, Part I 41*. Springer. 2014, pp. 600–611. DOI: 10.1007/978-3-662-43948-7_50.
- [114] Marijn JH Heule, Markus Iser, Matti Järvisalo, and Martin Suda. *Proceedings of SAT competition 2024: solver, benchmark and proof checker descriptions*. 2024.
- [115] Marijn JH Heule, Matti Juhani Järvisalo, Martin Suda, et al. “Solver and benchmark descriptions”. In: *Proceedings of SAT competition 2018*. Department of Computer Science, University of Helsinki, 2018. URL: <http://hdl.handle.net/10138/237063>.
- [116] Marijn JH Heule, Oliver Kullmann, and Victor W Marek. “Solving and verifying the Boolean Pythagorean triples problem via cube-and-conquer”. In: *International Conference on Theory and Applications of Satisfiability Testing*. Springer. 2016, pp. 228–245.
- [117] Marijn JH Heule and Stefan Szeider. “A SAT approach to clique-width”. In: *ACM Transactions on Computational Logic (TOCL)* 16.3 (2015), pp. 1–27.
- [118] Nicholas J Higham. *Accuracy and stability of numerical algorithms*. SIAM, 2002.
- [119] Mark Hillery, Vladimír Bužek, and André Berthiaume. “Quantum secret sharing”. In: *Physical Review A* 59.3 (1999), p. 1829.
- [120] Stefan Hillmich, Alwin Zulehner, and Robert Wille. “Concurrency in DD-based quantum circuit simulation”. In: *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE. 2020, pp. 115–120.
- [121] Gerard J. Holzmann. “The model checker SPIN”. In: *IEEE Transactions on software engineering* 23.5 (1997), pp. 279–295.
- [122] Xin Hong, Aochu Dai, Dingchao Gao, Sanjiang Li, Zhengfeng Ji, and Mingsheng Ying. “LimTDD: a compact decision diagram integrating tensor and local invertible map representations”. In: *arXiv preprint arXiv:2504.01168* (2025).
- [123] Hsin-Yuan Huang, Richard Kueng, Giacomo Torlai, Victor V Albert, and John Preskill. “Provably efficient machine learning for quantum many-body problems”. In: *Science* 377.6613 (2022), eabk3333.
- [124] Jaehee Jang, Byungook Na, and Sungroh Yoon. “Homomorphic parameter compression for distributed deep learning training”. In: *arXiv:1711.10123* (2017).

Bibliography

- [125] Dominik Janzing, Pawel Wocjan, and Thomas Beth. “Non-identity-check is QMA-complete”. In: *International Journal of Quantum Information* 3.03 (2005), pp. 463–473.
- [126] Michael Jarret and Kianna Wan. “Improved quantum backtracking algorithms using effective resistance estimates”. In: *Physical Review A* 97.2 (2018), p. 022337. DOI: 10.1103/PhysRevA.97.022337.
- [127] Zhengfeng Ji, Jianxin Chen, Zhaohui Wei, and Mingsheng Ying. “The LU-LC conjecture is false”. In: *arXiv preprint arXiv:0709.1266* (2007).
- [128] Zhengfeng Ji and Xiaodi Wu. “Non-identity check remains QMA-complete for short circuits”. In: *arXiv preprint arXiv:0906.5416* (2009).
- [129] Neil D. Jones and William T. Laaser. “Complete problems for deterministic polynomial time”. In: *Theoretical Computer Science* 3.1 (1976), pp. 105–117. ISSN: 0304-3975. DOI: [https://doi.org/10.1016/0304-3975\(76\)90068-2](https://doi.org/10.1016/0304-3975(76)90068-2). URL: <http://www.sciencedirect.com/science/article/pii/0304397576900682>.
- [130] Jarn de Jong, Frederik Hahn, Nikolay Tcholtchev, Manfred Hauswirth, and Anna Pappa. “Extracting maximal entanglement from linear cluster states”. In: *arXiv preprint arXiv:2211.16758* (2022).
- [131] Richard Jozsa, Daniel S Abrams, Jonathan P Dowling, and Colin P Williams. “Quantum clock synchronization based on shared prior entanglement”. In: *Physical Review Letters* 85.9 (2000), p. 2010.
- [132] Timothy Kam. “Multi-valued decision diagrams: theory and applications”. In: *Multiple-Valued Logic* 4.1 (1998), pp. 9–62.
- [133] Gijs Kant, Alfons Laarman, Jeroen Meijer, Jaco van de Pol, Stefan Blom, and Tom van Dijk. “LTSmin: high-performance language-independent model checking”. In: *TACAS*. Springer. 2015, pp. 692–707.
- [134] Joost-Pieter Katoen. “The probabilistic model checking landscape”. In: *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science*. 2016, pp. 31–45.
- [135] Randy Kerber. “Chimerge: discretization of numeric attributes”. In: *Proceedings of the tenth national conference on Artificial intelligence*. 1992, pp. 123–128.
- [136] A Yu Kitaev. “Quantum computations: algorithms and error correction”. In: *Russian Mathematical Surveys* 52.6 (1997), p. 1191.

-
- [137] Donald E Knuth. *The art of computer programming, volume 4a: combinatorial algorithms, part 1*. Pearson Education India, 2011.
 - [138] Daphne Koller and Nir Friedman. *Probabilistic graphical models: principles and techniques*. MIT press, 2009.
 - [139] F. Kordon, P. Bouvier, H. Garavel, L. M. Hillah, F. Hulin-Hubard, N. Amat., E. Amparore, B. Berthomieu, S. Biswal, D. Donatelli, F. Galla, S. Dal Zilio, P. G. Jensen, C. He, D. Le Botlan, S. Li, J. Srba, Y. Thierry-Mieg, A. Walner, and K. Wolf. *Complete Results for the 2021 Edition of the Model Checking Contest*. <http://mcc.lip6.fr/2021/results.php>. 2021.
 - [140] F. Kordon, H. Garavel, L. M. Hillah, F. Hulin-Hubard, G. Chiardo, A. Hamez, L. Jezequel, A. Miner, J. Meijer, E. Paviot-Adet, D. Racordon, C. Rodriguez, C. Rohr, J. Srba, Y. Thierry-Mieg, G. Trinh, and K. Wolf. *Complete results for the 2016 edition of the model checking contest*. <https://mcc.lip6.fr/2016/results.php>. 2016.
 - [141] Soh Kumabe, Ryuhei Mori, and Yusei Yoshimura. “Complexity of graph-state preparation by Clifford circuits”. In: *arXiv preprint arXiv:2402.05874* (2024).
 - [142] Lukasz A Kurgan and Krzysztof J Cios. “CAIM discretization algorithm”. In: *IEEE transactions on Knowledge and Data Engineering* 16.2 (2004), pp. 145–153.
 - [143] Alfons Laarman, Jaco van de Pol, and Michael Weber. “Boosting multi-core reachability performance with shared hash tables”. In: *Formal Methods in Computer Aided Design*. IEEE. 2010, pp. 247–255.
 - [144] Robert J LaLonde. “Evaluating the econometric evaluations of training programs with experimental data”. In: *The American economic review* (1986), pp. 604–620.
 - [145] Seok-Hyung Lee and Hyunseok Jeong. “Graph-theoretical optimization of fusion-based graph state generation”. In: *arXiv preprint arXiv:2304.11988* (2023).
 - [146] Thomas Lengauer and Robert E Tarjan. “Asymptotically tight bounds on time-space trade-offs in a pebble game”. In: *Journal of the ACM (JACM)* 29.4 (1982), pp. 1087–1130. DOI: 10.1145/322344.322354.
 - [147] Bikun Li, Sophia E Economou, and Edwin Barnes. “Photonic resource state generation from a minimal number of quantum emitters”. In: *npj Quantum Information* 8.1 (2022), p. 11.

- [148] Shaowen Li, Yusuke Kimura, Hiroyuki Sato, and Masahiro Fujita. “Parallelizing quantum simulation with decision diagrams”. In: *IEEE Transactions on Quantum Engineering* (2024).
- [149] Hong-Yu Liang and Jing He. “Satisfiability with index dependency”. In: *Journal of Computer Science and Technology* 27.4 (2012), pp. 668–677. DOI: 10.1007/978-3-642-17517-6_7.
- [150] Neha Lodha, Sebastian Ordyniak, and Stefan Szeider. “A SAT approach to branchwidth”. In: *Theory and Applications of Satisfiability Testing–SAT 2016: 19th International Conference, Bordeaux, France, July 5-8, 2016, Proceedings 19*. Springer. 2016, pp. 179–195.
- [151] Neha Lodha, Sebastian Ordyniak, and Stefan Szeider. “A SAT approach to branchwidth”. In: *ACM Transactions on Computational Logic (TOCL)* 20.3 (2019), pp. 1–24.
- [152] Neha Lodha, Sebastian Ordyniak, and Stefan Szeider. “SAT-encodings for special treewidth and pathwidth”. In: *Theory and Applications of Satisfiability Testing–SAT 2017: 20th International Conference, Melbourne, VIC, Australia, August 28–September 1, 2017, Proceedings 20*. Springer. 2017, pp. 429–445.
- [153] Joao Marques-Silva, Inês Lynce, and Sharad Malik. “Conflict-driven clause learning SAT solvers”. In: *Handbook of satisfiability* (2009), pp. 131–153.
- [154] Yusuke Matsunaga, Patrick C McGeer, and Robert K Brayton. “On computing the transitive closure of a state transition relation”. In: *Proceedings of the 30th international design automation conference*. 1993, pp. 260–265.
- [155] Kenneth Lauchlin McMillan. “Symbolic model checking: an approach to the state explosion problem”. PhD thesis. Carnegie Mellon University, 1992.
- [156] Jingyi Mei, Marcello Bonsangue, and Alfons Laarman. “Simulating quantum circuits by model counting”. In: *International Conference on Computer Aided Verification*. Springer. 2024, pp. 555–578.
- [157] Jingyi Mei, Tim Coopmans, Marcello Bonsangue, and Alfons Laarman. “Equivalence checking of quantum circuits by model counting”. In: *International Joint Conference on Automated Reasoning*. Springer. 2024, pp. 401–421.
- [158] Jeroen Meijer and Jaco van de Pol. “Bandwidth and wavefront reduction for static variable ordering in symbolic reachability analysis”. In: *NFM*. Springer. 2016, pp. 255–271.

- [159] Marc Mezard, Marc Mezard, and Andrea Montanari. *Information, physics, and computation*. Oxford University Press, 2009. DOI: 10.1063/1.3397047.
- [160] D Michael Miller and Mitchell A Thornton. “QMDD: a decision diagram structure for reversible and quantum circuits”. In: *36th International Symposium on Multiple-Valued Logic (ISMVL’06)*. IEEE. 2006, pp. 30–30.
- [161] Webb Miller. “Computational complexity and numerical stability”. In: *Proceedings of the sixth annual ACM symposium on Theory of computing*. 1974, pp. 317–322.
- [162] A Yu Mitrophanov. “Stability and exponential convergence of continuous-time Markov chains”. In: *Journal of applied probability* 40.4 (2003), pp. 970–979.
- [163] Ashley Montanaro. “Quantum-Walk Speedup of Backtracking Algorithms”. In: *Theory of Computing* 14.15 (2018), pp. 1–24. DOI: 10.4086/toc.2018.v014a015.
- [164] Matthew W Moskewicz, Conor F Madigan, Ying Zhao, Lintao Zhang, and Sharad Malik. “Chaff: engineering an efficient SAT solver”. In: *Proceedings of the 38th annual Design Automation Conference*. 2001, pp. 530–535.
- [165] Jeffrey Negrea and Jeffrey S Rosenthal. “Approximations of geometrically ergodic reversible Markov chains”. In: *Advances in Applied Probability* 53.4 (2021), pp. 981–1022.
- [166] Martin Neil, Manesh Tailor, and David Marquez. “Inference in hybrid Bayesian networks using dynamic discretization”. In: *Statistics and Computing* 17 (2007), pp. 219–233.
- [167] Maarten van den Nest. “Local equivalence of stabilizer states and codes”. PhD thesis. Katholieke Universiteit Leuven, 2005.
- [168] Maarten van den Nest, Jeroen Dehaene, and Bart De Moor. “Graphical description of the action of local Clifford transformations on graph states”. In: *Phys. Rev. A* 69 (2 Feb. 2004), p. 022316. DOI: 10.1103/PhysRevA.69.022316.
- [169] Maarten van den Nest, Jeroen Dehaene, and Bart de Moor. “Efficient algorithm to recognize the local Clifford equivalence of graph states”. In: *Physical Review A* 70.3 (2004), p. 034302.
- [170] Naomi Nickerson. “Practical fault-tolerant quantum computing.” PhD thesis. Imperial College London, 2015.

- [171] Michael A Nielsen and Isaac L Chuang. *Quantum computation and quantum information*. Cambridge university press, 2010.
- [172] Philipp Niemann, Alwin Zulehner, Rolf Drechsler, and Robert Wille. “Overcoming the tradeoff between accuracy and compactness in decision diagrams for quantum computation”. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39.12 (2020), pp. 4657–4668.
- [173] Robert Nieuwenhuis, Albert Oliveras, and Cesare Tinelli. “Abstract DPLL and abstract DPLL modulo theories”. In: *International Conference on Logic for Programming Artificial Intelligence and Reasoning*. Springer. 2005, pp. 36–50. DOI: 10.1007/978-3-540-32275-7_3.
- [174] Farnaz Nojavan, Song S Qian, and Craig A Stow. “Comparative analysis of discretization methods in Bayesian networks”. In: *Environmental Modelling & Software* 87 (2017), pp. 64–71.
- [175] Román Orús, Samuel Mugel, and Enrique Lizaso. “Quantum computing for finance: overview and prospects”. In: *Reviews in Physics* 4 (2019), p. 100028.
- [176] Prasad Ostwal. *Ostwalprasad/lgnpy: v1.0.0*. 2020. DOI: 10.5281/ZENODO.3902122. URL: <https://zenodo.org/record/3902122>.
- [177] Harsh Parikh, Carlos Varjao, Louise Xu, and Eric Tchetgen Tchetgen. “Validating causal inference methods”. In: *International Conference on Machine Learning*. PMLR. 2022, pp. 17346–17358.
- [178] Ramamohan Paturi, Pavel Pudlák, Michael E Saks, and Francis Zane. “An improved exponential-time algorithm for k-SAT”. In: *Journal of the ACM (JACM)* 52.3 (2005), pp. 337–364. DOI: 10.1145/1066100.1066101.
- [179] Radek Pelánek. “BEEM: benchmarks for explicit model checkers”. In: *SPIN*. Springer. 2007, pp. 263–267.
- [180] Martin L Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 1994.
- [181] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. “MQT Bench: benchmarking software and design automation tools for quantum computing”. In: *Quantum* (2023). MQT Bench is available at <https://www.cda.cit.tum.de/mqtbench/>.
- [182] Arend-Jan Quist, Tim Coopmans, and Alfons Laarman. “Exact quantum decision diagrams scaling in the T -count”. In: *In preparation* (2025).

Bibliography

- [195] Vasilis Samoladas. “Improved BDD algorithms for the simulation of quantum circuits”. In: *European Symposium on Algorithms*. Springer. 2008, pp. 720–731.
- [196] Tian Sang, Fahiem Bacchus, Paul Beame, Henry A Kautz, and Toniann Pitassi. “Combining component caching and clause learning for effective model counting”. In: *SAT 4 (2004)*, 7th.
- [197] Tian Sang, Paul Beame, and Henry Kautz. “Heuristics for fast exact model counting”. In: *Theory and Applications of Satisfiability Testing: 8th International Conference, SAT 2005, St Andrews, UK, June 19-23, 2005. Proceedings 8*. Springer. 2005, pp. 226–240.
- [198] Tian Sang, Paul Beame, and Henry Kautz. “Solving Bayesian networks by weighted model counting”. In: *Proceedings of the Twentieth National Conference on Artificial Intelligence (AAAI-05)*. Vol. 1. AAAI Press. 2005, pp. 475–482.
- [199] Tian Sang, Paul Beame, and Henry A Kautz. “Performing bayesian inference by weighted model counting”. In: *AAAI*. Vol. 5. 2005, pp. 475–481.
- [200] Nicolas Sangouard, Christoph Simon, Hugues de Riedmatten, and Nicolas Gisin. “Quantum repeaters based on atomic ensembles and linear optics”. In: *Rev. Mod. Phys.* 83 (1 Mar. 2011), pp. 33–80. DOI: 10.1103/RevModPhys.83.33. URL: <https://link.aps.org/doi/10.1103/RevModPhys.83.33>.
- [201] Scott Sanner and David McAllester. “Affine algebraic decision diagrams (AADDs) and their application to structured probabilistic inference”. In: *IJCAI*. 2005, pp. 1384–1390.
- [202] Scott Sanner, William TB Uther, Karina Valdivia Delgado, et al. “Approximate dynamic programming with affine ADDs”. In: *AAMAS*. 2010, pp. 1349–1356.
- [203] Dominik Scheder. “PPSZ for $k \geq 5$: more is better”. In: *Electronic Colloquium on Computational Complexity (ECCC)* 25 (2018), p. 113. URL: <https://eccc.weizmann.ac.il/report/2018/113>.
- [204] Dominik Scheder. “PPSZ is better than you think”. In: *2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS)*. 2022, pp. 205–216. DOI: 10.1109/FOCS52979.2021.00028. URL: <https://eccc.weizmann.ac.il/report/2021/069>.

-
- [205] Dominik Scheder and John P Steinberger. “PPSZ for general k-SAT-making Hertli’s analysis simpler and 3-SAT faster”. In: *32nd Computational Complexity Conference (CCC 2017)*. Vol. 79. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik. 2017, 9:1–9:15. DOI: 10.4230/LIPIcs.CCC.2017.9.
- [206] André Schidler and Stefan Szeider. “A SAT approach to twin-width”. In: *2022 Proceedings of the Symposium on Algorithm Engineering and Experiments*. SIAM. 2022, pp. 67–77.
- [207] Sarah Schneider, Lukas Burgholzer, and Robert Wille. “A SAT encoding for optimal Clifford circuit synthesis”. In: *Proceedings of the 28th Asia and South Pacific Design Automation Conference*. 2023, pp. 190–195.
- [208] Marco Scutari. “Learning Bayesian networks with the bnlearn R package”. In: *Journal of statistical software* 35 (2010), pp. 1–22.
- [209] Peter Selinger. “Efficient Clifford+T approximation of single-qubit operators”. In: *arXiv preprint arXiv:1212.6253* (2012).
- [210] Aziz Sfar, David Carral, Dina Irofti, and Madalina Croitoru. “Testing logical diagrams in power plants: a tale of LTL model checking”. In: *International Conference on Formal Methods for Industrial Critical Systems*. Springer. 2023, pp. 189–204.
- [211] Irfansha Shaik and Jaco van de Pol. “Optimal layout synthesis for quantum circuits as classical planning”. In: *arXiv preprint arXiv:2304.12014* (2023).
- [212] Hemant Sharma, Kenneth Goodenough, Johannes Borregaard, Jonas Helsen, et al. “Minimizing the number of edges in LC-equivalent graph states”. In: *arXiv preprint arXiv:2506.00292* (2025).
- [213] Peter W Shor. “Algorithms for quantum computation: discrete logarithms and factoring”. In: *Proceedings 35th annual symposium on foundations of computer science*. IEEE. 1994, pp. 124–134.
- [214] Meghana Sistla, Swarat Chaudhuri, and Thomas Reps. “Symbolic quantum simulation with Quasimodo”. In: *International Conference on Computer Aided Verification*. Springer. 2023, pp. 213–225.
- [215] Reza Soltani, Matthias Volk, Leonardo Diamonte, Milan Lopuhaä-Zwakenberg, and Mariëlle Stoelinga. “Optimal spare management via statistical model checking: a case study in research reactors”. In: *International Conference on Formal Methods for Industrial Critical Systems*. 2023, pp. 205–223.

Bibliography

- [216] Fabio Somenzi. “Binary decision diagrams”. In: *Nato ASI Subseries F CSS* 173 (1999), pp. 303–368.
- [217] Fabio Somenzi. *CUDD: CU decision diagram package release 3.0.0*. <https://web.archive.org/web/20171208230728/http://vlsi.colorado.edu/~fabio/CUDD/cudd.pdf>. 2015.
- [218] Richard M Stallman and Gerald J Sussman. “Forward reasoning and dependency-directed backtracking in a system for computer-aided circuit analysis”. In: *Artificial intelligence* 9.2 (1977), pp. 135–196.
- [219] Volker Strassen. “Gaussian elimination is not optimal”. In: *Numerische mathematik* 13.4 (1969), pp. 354–356.
- [220] Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: an introduction*. Vol. 1. 1. MIT press Cambridge, 1998.
- [221] Martijn Swenne. *Solving SAT on noisy quantum computers*. <https://theses.liacs.nl/1725>. Bachelor thesis. 2019.
- [222] Paul Tafertshofer and Massoud Pedram. “Factored edge-valued binary decision diagrams”. In: *Formal Methods in System Design* 10 (1997), pp. 243–270.
- [223] Bochen Tan and Jason Cong. “Optimal layout synthesis for quantum computing”. In: *Proceedings of the 39th International Conference on Computer-Aided Design*. 2020, pp. 1–9.
- [224] Yu Tanaka. “Exact non-identity check is NQP-complete”. In: *International Journal of Quantum Information* 8.05 (2010), pp. 807–819.
- [225] Alfred Tarski. “A lattice-theoretical fixpoint theorem and its applications.” In: *Pacific journal of Mathematics* 5.2 (1955), pp. 285–309.
- [226] Dimitrios Thanos, Tim Coopmans, and Alfons Laarman. “Fast equivalence checking of quantum circuits of Clifford gates”. In: *International Symposium on Automated Technology for Verification and Analysis*. Springer. 2023, pp. 199–216.
- [227] Thomas N Theis and H-S Philip Wong. “The end of Moore’s law: a new beginning for information technology”. In: *Computing in science & engineering* 19.2 (2017), pp. 41–50.
- [228] Yann Thierry-Mieg. “Symbolic model-checking using ITS-tools”. In: *TACAS*. Springer. 2015, pp. 231–237.

-
- [229] Jan Tretmans, Klaas Wijbrans, and Michel Chaudron. “Software engineering with formal methods: the development of a storm surge barrier control system”. In: *Formal Methods in System Design* 19 (2001), pp. 195–215.
 - [230] Yuan-Hung Tsai, Jie-Hong R Jiang, and Chiao-Shan Jhang. “Bit-slicing the Hilbert space: Scaling up accurate quantum circuit simulation”. In: *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE. 2021, pp. 439–444.
 - [231] George F Viamontes, Igor L Markov, and John P Hayes. “High-performance QuIDD-based simulation of quantum circuits”. In: *Proceedings Design, Automation and Test in Europe Conference and Exhibition*. Vol. 2. IEEE. 2004, pp. 1354–1355.
 - [232] Lieuwe Vinkhuijzen, Tim Coopmans, David Elkouss, Vedran Dunjko, and Alfons Laarman. “LIMDD: a decision diagram for simulation of quantum computing including stabilizer states”. In: *Quantum* 7 (2023), p. 1108.
 - [233] Lieuwe Vinkhuijzen, Thomas Grurl, Stefan Hillmich, Sebastiaan Brand, Robert Wille, and Alfons Laarman. “Efficient implementation of LIMDDs for quantum circuit simulation”. In: *International Symposium on Model Checking Software*. Springer. 2023, pp. 3–21.
 - [234] Lieuwe Vinkhuijzen and Alfons Laarman. “Symbolic model checking with sentential decision diagrams”. In: *International Symposium on Dependable Software Engineering: Theories, Tools, and Applications*. Springer. 2020, pp. 124–142.
 - [235] Maarten C Vonk, Diederick Vermetten, Jacob de Nobel, Sebastiaan Brand, Ninoslav Malekovic, Thomas Bäck, Alfons Laarman, and Anna V Kononova. “Optimizing causal interventions in hybrid Bayesian networks: a discretization, knowledge compilation, and heuristic optimization approach”. In: *International Conference on Information Processing and Management of Uncertainty in Knowledge-Based Systems*. Springer. 2024, pp. 245–256.
 - [236] Chun-Yu Wei, Yuan-Hung Tsai, Chiao-Shan Jhang, and Jie-Hong R Jiang. “Accurate BDD-based unitary operator manipulation for scalable and robust quantum circuit verification”. In: *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 2022, pp. 523–528.

- [237] Ming Xu, Jingyi Mei, Ji Guan, and Nengkun Yu. “Model checking quantum continuous-time Markov chains”. In: *32nd International Conference on Concurrency Theory (CONCUR 2021)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik. 2021, 13:1–13:17.
- [238] Shigeru Yamashita and Igor L Markov. “Fast equivalence-checking for quantum circuits”. In: *2010 IEEE/ACM International Symposium on Nanoscale Architectures*. IEEE. 2010, pp. 23–28.
- [239] George Yin and Hanqin Zhang. “Singularly perturbed Markov chains: limit results and applications”. In: *The Annals of Applied Probability* 17 (2007), pp. 207–229.
- [240] Theodore J Yoder, Guang Hao Low, and Isaac L Chuang. “Fixed-point quantum search with an optimal number of queries”. In: *Physical review letters* 113.21 (2014). DOI: 10.1103/PhysRevLett.113.210501.
- [241] Alwin Zulehner, Stefan Hillmich, and Robert Wille. “How to efficiently handle complex values? Implementing decision diagrams for quantum computing”. In: *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE. 2019, pp. 1–7.
- [242] Alwin Zulehner and Robert Wille. “Advanced simulation of quantum computations”. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38.5 (2018), pp. 848–859.

Summary

Modern infrastructure is increasingly governed by complex, software-driven systems whose failure can have severe consequences. Formal methods offer a rigorous framework for analyzing the reliability and correctness of such systems.

In this thesis, we focus on quantitative transition systems, where states and transitions carry values (e.g., probabilities), enabling the analysis of not only whether certain outcomes are possible but also, for example, how likely they are. This includes applications in probabilistic safety analysis and quantum computing.

However, analyzing such systems is computationally difficult due to the exponential growth of the state space. To tackle this hurdle, people have turned to symbolic algorithms, which aim to represent and manipulate large state spaces efficiently without explicitly enumerating them. We particularly focus on decision diagrams (DDs) and satisfiability solving (SAT).

We investigate three challenges. First is improving the scalability of DD and SAT-based approaches. We make four contributions towards addressing this challenge. We present a novel DD algorithm for analyzing reachability in transition systems that avoids large intermediate DDs. Unlike the state-of-the-art algorithm, which only works for partitioned transition systems, our new algorithm can analyze monolithic transition systems. We also develop a parallel implementation of edge-valued DDs (that are used for analyzing quantum systems), which shows significant performance gains compared to previous work. When using a SAT-based approach to tackle the analysis of quantum graph states, we show that over-constraining the formulas using domain-specific knowledge leads to orders-of-magnitude improvement. We also show how Grover’s quantum search algorithm can be used to speed up PPSZ—the best-known asymptotic algorithm for SAT-solving—in both a purely quantum setting as well as a hybrid-quantum classical setting.

The second challenge is handling floating-point imprecision in DDs, which can

Summary

compromise both performance and correctness in DD computations. Towards this challenge, we present an in-depth study on the effect of numerical imprecision in computations that use multi-terminal binary DDs. This study includes theoretical bounds, an empirical case study, and practical mitigation strategies.

In the third challenge, we investigate how DDs can contribute to the efficient discretization of continuous domains. To this end, we evaluate the encoding of continuous Bayesian networks into DDs using various discretization methods. We demonstrate that high-quality discretizations are achievable with manageable computational costs, and that our DD-based approach often outperforms traditional inference methods.

Samenvatting

Moderne infrastructuur wordt steeds meer beheerst door complexe, softwaregestuurde systemen waarvan het falen ernstige gevolgen kan hebben. Formele methoden bieden een rigoureuus framework voor het analyseren van de betrouwbaarheid en juistheid van dergelijke systemen.

In dit proefschrift richten we ons op kwantitatieve transitiesystemen, waarbij toestanden en transities waarden (bijvoorbeeld kansen) kunnen hebben, waardoor niet alleen kan worden geanalyseerd of bepaalde uitkomsten mogelijk zijn, maar bijvoorbeeld ook hoe waarschijnlijk ze zijn. Dit omvat toepassingen in probabilistische veiligheidsanalyse en quantumcomputing.

Echter is het analyseren van zulke systemen computationeel moeilijk vanwege de exponentiële groei van de toestandsruimte. Om deze hindernis te overwinnen, kan gebruik gemaakt worden van symbolische algoritmen, die als doel hebben grote toestandsruimten efficiënt weer te geven en te manipuleren zonder ze expliciet op te sommen. We richten ons in het bijzonder op decision diagrams (DD's) en satisfiability solving (SAT).

We onderzoeken drie uitdagingen. De eerste is het verbeteren van de schaalbaarheid van DD- en SAT-gebaseerde aanpakken. We leveren vier bijdragen om deze uitdaging aan te pakken. We presenteren een nieuw DD-algoritme voor het analyseren van bereikbaarheid in transitiesystemen dat grote tussenliggende DD's vermijdt. In tegenstelling tot het state-of-the-art algoritme, dat alleen werkt voor gepartitioneerde transitiesystemen, kan ons nieuwe algoritme monolithische transitiesystemen analyseren. We ontwikkelen ook een parallelle implementatie van edge-valued DD's (die gebruikt worden bij het analyseren van quantumsystemen), die aanzienlijke prestatieverbeteringen laat zien in vergelijking met eerder werk. Bij het gebruik van een op SAT-gebaseerde aanpak voor de analyse van quantum graph states, laten we zien dat het toevoegen van extra constraints aan de formules met behulp van domeinspeci-

fieke kennis leidt tot verbeteringen van ordes van grootte. We laten ook zien hoe Grover's quantumzoekalgoritme kan worden gebruikt om PPSZ—het zover bekend beste asymptotische algoritme voor het oplossen van SAT problemen—te versnellen, zowel in een pure quantum setting als in een hybride quantum-klassieke setting.

De tweede uitdaging is het omgaan met floating-point nauwkeurigheid in DD's, wat zowel de prestaties als de corretheid van DD-berekeningen kan aantasten. Als een stap richting deze uitdaging, presenteren we een studie naar het effect van numerieke onnauwkeurigheid in berekeningen die gebruikmaken van multi-terminal binaire DD's. Deze studie omvat theoretische resultaten, een empirische casestudy en praktische mitigatiestrategieën.

In de derde uitdaging onderzoeken we hoe DD's kunnen bijdragen aan de efficiënte discretisatie van continue domeinen. Daartoe evalueren we de codering van continue Bayesiaanse netwerken in DD's met behulp van verschillende discretisatiemethoden. We tonen aan dat goede discretisaties haalbaar zijn met redelijke rekenkosten en dat onze op DD's gebaseerde aanpak vaak beter presteert dan traditionele inferentiemethoden.

List of publications

Included in this thesis:

1. Sebastiaan Brand, Thomas Bäck, and Alfons Laarman. “A decision diagram operation for reachability”. In: *International Symposium on Formal Methods*. Springer. 2023, pp. 514–532
2. Mathys Rennela, Sebastiaan Brand, Alfons Laarman, and Vedran Dunjko. “Hybrid divide-and-conquer approach for tree search algorithms”. In: *Quantum* 7 (2023), p. 959
3. Sebastiaan Brand, Tim Coopmans, and Alfons Laarman. “Quantum graph-state synthesis with SAT”. in: *14th International Workshop on Pragmatics of SAT*. CEUR-WS. 2023
4. Maarten C. Vonk, Sebastiaan Brand, Ninoslav Malekovic, Thomas Bäck, Alfons Laarman, and Anna V. Kononova. “Balancing computational cost and accuracy in inference of continuous Bayesian networks”. In: *International Conference on Probabilistic Graphical Models*. PMLR. 2024, pp. 361–381
5. Sebastiaan Brand, Arend-Jan Quist, Richard van Dijk, and Alfons Laarman. “Numerical errors in quantitative system analysis with decision diagrams”. In: *International Conference on Quantitative Evaluation of Systems and Formal Modeling and Analysis of Timed Systems*. 2025
6. Sebastiaan Brand and Alfons Laarman. “Q-Sylvan: a parallel decision diagram package for quantum computing”. In: *International Symposium on Automated Technology for Verification and Analysis*. 2025

Not included in this thesis:

7. Sebastiaan Brand, Tim Coopmans, and David Elkouss. “Efficient computation of the waiting time and fidelity in quantum repeater chains”. In: *IEEE Journal on Selected Areas in Communications* 38.3 (2020), pp. 619–639
8. Tim Coopmans, Sebastiaan Brand, and David Elkouss. “Improved analytical bounds on delivery times of long-distance entanglement”. In: *Physical Review A* 105.1 (2022), p. 012608
9. Lieuwe Vinkhuijzen, Thomas Grurl, Stefan Hillmich, Sebastiaan Brand, Robert Wille, and Alfons Laarman. “Efficient implementation of LIMDDs for quantum circuit simulation”. In: *International Symposium on Model Checking Software*. Springer. 2023, pp. 3–21
10. Dimitrios Thanos, Alejandro Villoria, Sebastiaan Brand, Arend-Jan Quist, Jingyi Mei, Tim Coopmans, and Alfons Laarman. “Automated reasoning in quantum circuit compilation”. In: *International Symposium on Model Checking Software*. Springer. 2024, pp. 106–134
11. Maarten C. Vonk, Diederick Vermetten, Jacob de Nobel, Sebastiaan Brand, Ninoslav Malekovic, Thomas Bäck, Alfons Laarman, and Anna V. Kononova. “Optimizing causal interventions in hybrid Bayesian networks”. In: *20th International Conference on Information Processing and Management of Uncertainty in Knowledge-Based Systems*. Springer. 2024

Curriculum Vitae

Sebastiaan Brand was born on May 25th, 1994, in Oegstgeest. He received his BSc. in Computer Science and Engineering at the TU Delft in 2016, and his MSc. in Computer Science at Leiden University in 2019. During the final year of his master's he did an external thesis project at QuTech, TU Delft, on the analysis of waiting time in quantum repeater chains. He started his PhD in September 2020 at Leiden University, where he was part of the Theory (theoretical computer science) group and the Applied Quantum Algorithms group, and worked on symbolic algorithms for quantum computing. During his PhD studies, he took courses in academic writing, communication in science, and scientific conduct, among others. He also assisted with teaching several courses and was the primary lecturer for the Concepts of Programming Languages course in 2023.

Titles in the IPA Dissertation Series since 2023

S. Kochanthara. *A Changing Landscape: On Safety & Open Source in Automated and Connected Driving.* Faculty of Mathematics and Computer Science, TU/e. 2023-01

L.M. Ochoa Venegas. *Break the Code? Breaking Changes and Their Impact on Software Evolution.* Faculty of Mathematics and Computer Science, TU/e. 2023-02

N. Yang. *Logs and models in engineering complex embedded production software systems.* Faculty of Mathematics and Computer Science, TU/e. 2023-03

J. Cao. *An Independent Timing Analysis for Credit-Based Shaping in Ethernet TSN.* Faculty of Mathematics and Computer Science, TU/e. 2023-04

K. Dokter. *Scheduled Protocol Programming.* Faculty of Mathematics and Natural Sciences, UL. 2023-05

J. Smits. *Strategic Language Workbench Improvements.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2023-06

A. Arslanagić. *Minimal Structures for Program Analysis and Verification.* Faculty of Science and Engineering, RUG. 2023-07

M.S. Bouwman. *Supporting Railway Standardisation with Formal Veri-*

fication. Faculty of Mathematics and Computer Science, TU/e. 2023-08

S.A.M. Lathouwers. *Exploring Annotations for Deductive Verification.* Faculty of Electrical Engineering, Mathematics & Computer Science, UT. 2023-09

J.H. Stoel. *Solving the Bank, Lightweight Specification and Verification Techniques for Enterprise Software.* Faculty of Mathematics and Computer Science, TU/e. 2023-10

D.M. Groenewegen. *WebDSL: Linguistic Abstractions for Web Programming.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2023-11

D.R. do Vale. *On Semantical Methods for Higher-Order Complexity Analysis.* Faculty of Science, Mathematics and Computer Science, RU. 2024-01

M.J.G. Olsthoorn. *More Effective Test Case Generation with Multiple Tribes of AI.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2024-02

B. van den Heuvel. *Correctly Communicating Software: Distributed, Asynchronous, and Beyond.* Faculty of Science and Engineering, RUG. 2024-03

H.A. Hiep. *New Foundations for Separation Logic.* Faculty of Mathematics

and Natural Sciences, UL. 2024-04

C.E. Brandt. *Test Amplification For and With Developers.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2024-05

J.I. Hejderup. *Fine-Grained Analysis of Software Supply Chains.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2024-06

J. Jacobs. *Guarantees by construction.* Faculty of Science, Mathematics and Computer Science, RU. 2024-07

O. Bunte. *Cracking OIL: A Formal Perspective on an Industrial DSL for Modelling Control Software.* Faculty of Mathematics and Computer Science, TU/e. 2024-08

R.J.A. Erkens. *Automaton-based Techniques for Optimized Term Rewriting.* Faculty of Mathematics and Computer Science, TU/e. 2024-09

J.J.M. Martens. *The Complexity of Bisimilarity by Partition Refinement.* Faculty of Mathematics and Computer Science, TU/e. 2024-10

L.J. Edixhoven. *Expressive Specification and Verification of Choreographies.* Faculty of Science, OU. 2024-11

J.W.N. Paulus. *On the Expressivity of Typed Concurrent Calculi.* Faculty of Science and Engineering, RUG. 2024-12

J. Denkers. *Domain-Specific Languages for Digital Printing Systems.*

Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2024-13

L.H. Applis. *Tool-Driven Quality Assurance for Functional Programming and Machine Learning.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2024-14

P. Karkhanis. *Driving the Future: Facilitating C-ITS Service Deployment for Connected and Smart Roadways.* Faculty of Mathematics and Computer Science, TU/e. 2024-15

N.W. Cassee. *Sentiment in Software Engineering.* Faculty of Mathematics and Computer Science, TU/e. 2024-16

H. van Antwerpen. *Declarative Name Binding for Type System Specifications.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2025-01

I.N. Mulder. *Proof Automation for Fine-Grained Concurrent Separation Logic.* Faculty of Science, Mathematics and Computer Science, RU. 2025-02

T.S. Badings. *Robust Verification of Stochastic Systems: Guarantees in the Presence of Uncertainty.* Faculty of Science, Mathematics and Computer Science, RU. 2025-03

A.M. Mir. *Machine Learning-assisted Software Analysis.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2025-04

L.T. Vinkhuijzen. *Data Structures for Quantum Circuit Verification and How To Compare Them.* Faculty of Mathematics and Natural Sciences, UL. 2025-05

D. van der Wal. *What is the Point? Single-Input-Change Testing a EULYNX Controller.* Faculty of Electrical Engineering, Mathematics & Computer Science, UT. 2025-06

A. Rosset. *Uniform Monad Presentations and Graph Quasitoposes.* Faculty of Sciences, Department of Computer Science, VU. 2025-07

L. Guo. *Higher-Order Termination with Logical Constraints.* Faculty of Science, Mathematics and Computer Science, RU. 2025-08

D.A. Manrique Negrin. *A Model Orchestra in Digital Twins: A Model-Driven Approach to Integration and Orchestration.* Faculty of Mathematics and Computer Science, TU/e. 2025-09

C.A. Esterhuysen. *Specification-Centric Multi-Agent Systems.* Faculty of Science, UvA. 2025-10

H.M. Muctadir. *Consistency Matters: Building Consistent Digital Twin Virtual Entities.* Faculty of Mathematics and Computer Science, TU/e. 2025-11

A. Stramaglia. *Model Checking Machine-Control Applications.* Faculty of Mathematics and Computer Science, TU/e. 2025-12

M. Saeedi Nikoo. *Supporting business process management: clone detection and recommendation techniques.* Faculty of Mathematics and Computer Science, TU/e. 2025-13

R.B. Rubbens. *Bridging the Implementation Gap: Advances in Model-Based Concurrent Program Verification.* Faculty of Electrical Engineering, Mathematics & Computer Science, UT. 2025-14

F.I. van der Berg. *DMC Model Checker: Delta-Driven Variable-Length Next-State Generation via Recursive Compression.* Faculty of Electrical Engineering, Mathematics & Computer Science, UT. 2025-15

D.G. Sprokholt. *Correct Translation between Weak Memory Model Architectures.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2025-16

B. Kohlen. *Verified & Fast Probabilistic Verification: Stepwise Refinement of Probabilistic Model Checking into an IEEE 754 Floating-Point Implementation.* Faculty of Electrical Engineering, Mathematics & Computer Science, UT. 2026-01

D. Frolich. *Exploratory Language Development.* Faculty of Science, UvA. 2026-02

L.B. van den Haak. *Verifying Optimised Parallel Code.* Faculty of

Mathematics and Computer Science,
TU/e. 2026-03

A. Papotti. *Human Acceptance of Machine Generated Code Security Analysis.* Faculty of Sciences, Department of Computer Science, VU. 2026-04

N. Doorn. *O bug, where art thou? Improving software testing education.* Faculty of Science, OU. 2026-05

C.R. van der Rest. *Reusable Programming Language Components.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2026-06

T.T.P. Franken. *A Parallel Programming Perspective – Designing AuDaLa and AuDaLogic.* Faculty of Mathematics and Computer Science, TU/e. 2026-07

T.J. Smeding. *Reverse-mode Automatic Differentiation for Functional Array Languages.* Faculty of Science, UU. 2026-08

S.A. Rukmono. *Towards Automated Software Architecture Explanation: An AI-enabled method and industrial evaluation.*

Faculty of Mathematics and Computer Science, TU/e. 2026-09

R.T.C. Turkenburg. *Comparing and Constructing (Co)Inductive Predicates.* Faculty of Science, Mathematics and Computer Science, RU. 2026-10

P.B.H. Tasche. *Deductive Verification Techniques for Embedded Systems.* Faculty of Electrical Engineering, Mathematics & Computer Science, UT. 2026-11

W.B. Mbaka. *An Empirical Investigation Into Human Judgments of Security Threats.* Faculty of Sciences, Department of Computer Science, VU. 2026-12

S. Srinivasan. *Taming Uncertainty, Enabling Predictability: Systematic Timing Analysis of Real-Time Systems Across Computation and Communication.* Faculty of Mathematics and Computer Science, TU/e. 2026-13

S.O. Brand. *Symbolic Algorithms for the Analysis of Quantitative Systems.* Faculty of Mathematics and Natural Sciences, UL. 2026-14